L1 (3.1 – 3.25)

- Basic logic gates: AND, OR, NOT, buffer
- Truth table
- Logic or Boolean expression
- Timing waveform: rise time, fall time and propagation delay
- Logic circuit diagram

3. Logic Gates and Boolean Algebra

Digital circuits can be found in smart phones, computers, washing machines, cars, etc.

Logic gates are the basic building blocks of digital circuits.

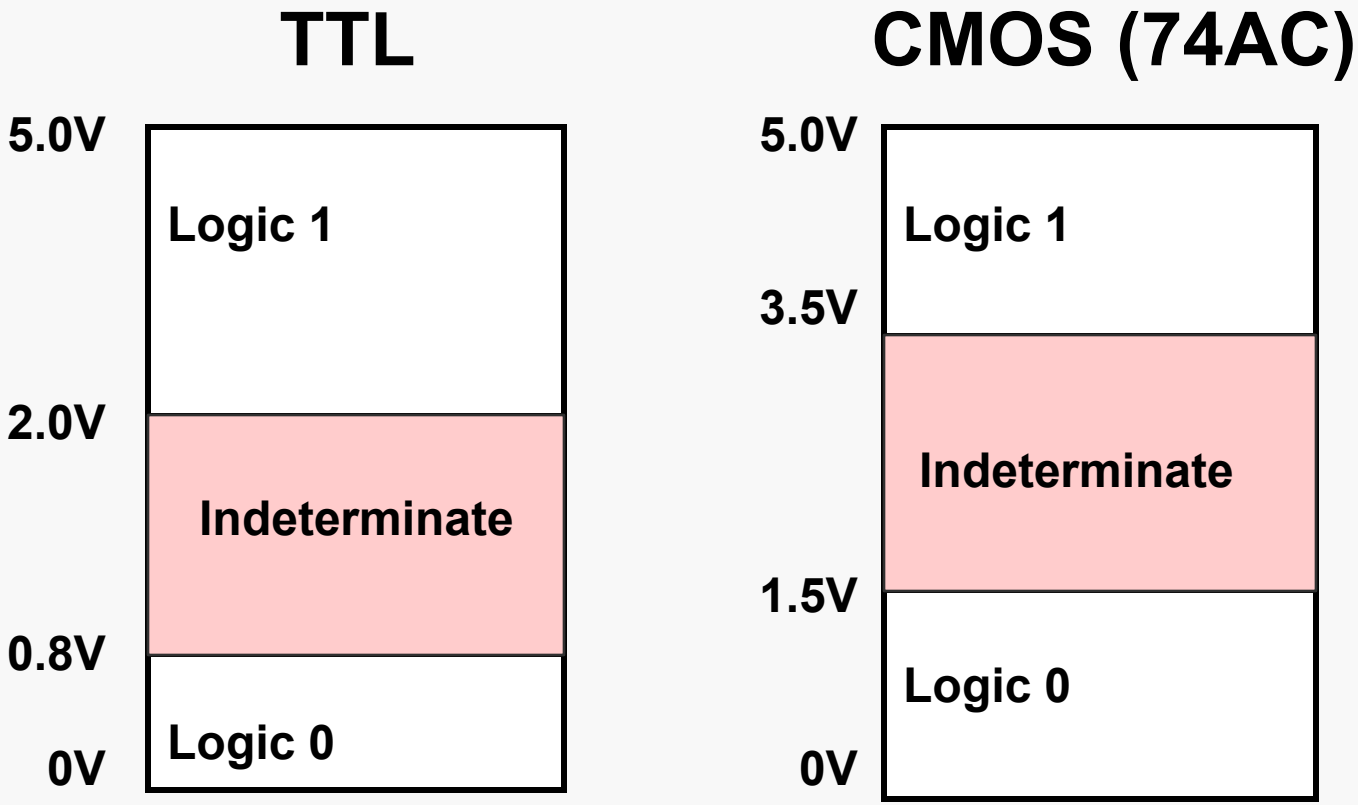
Boolean algebra is used to describe, design and simplify digital circuits.

Boolean Constants – only 2 values

- **TRUE, FALSE**
- **Logic HIGH, Logic LOW**
- **HI, LO**
- **1, 0**

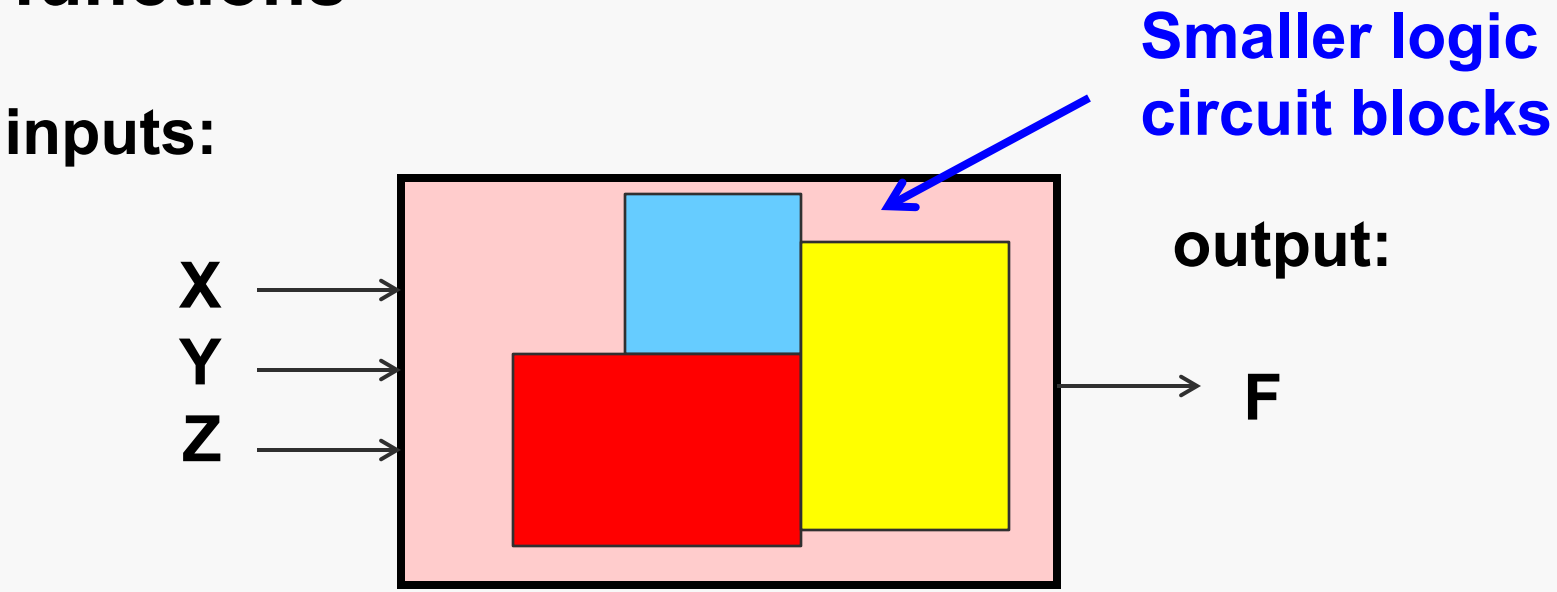
Boolean (logic) variables can only assume one of the two values

Common Logic-level voltage ranges



A typical logic circuit

- 1 or more logic inputs
- 1 or more logic outputs
- Outputs are related to inputs by logic functions



Truth table

- Logic function can be fully described by a Truth Table.
- The Truth table
  - shows how a logic circuit's output responds to various combinations of logic inputs
  - has 2^N number of input combinations for N inputs
  - lists all possible input combinations in the binary counting sequence

A typical N-input truth table

2^N rows

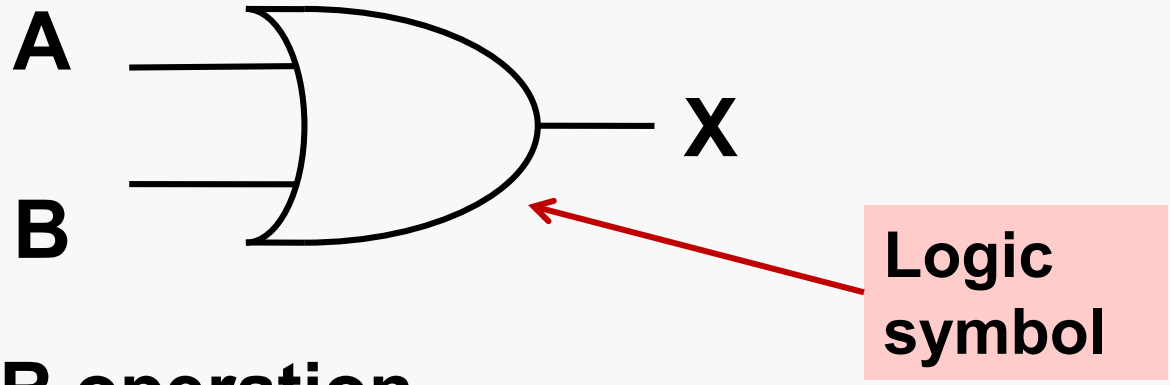
Input 1	Input 2	…	Input N-1	Input N	Output
0	0	…	0	0	1
0	0	…	0	1	0
0	.	…	1	0	0
0	.	…	1	1	.
.	.	…	.	.	.
.	.	…	.	.	.
1	.	…	0	0	.
1	.	…	0	1	.
1	1	…	1	0	.
1	1	…	1	1	.

3 Basic Logic Operations

- Logical addition - OR
- Logical multiplication - AND
- Logical complement or inversion - NOT

In digital circuits, these are realised by electronic devices called logic gates.

Logical OR Operation



- Logical OR operation
  - X = A + B
  - X = A OR B

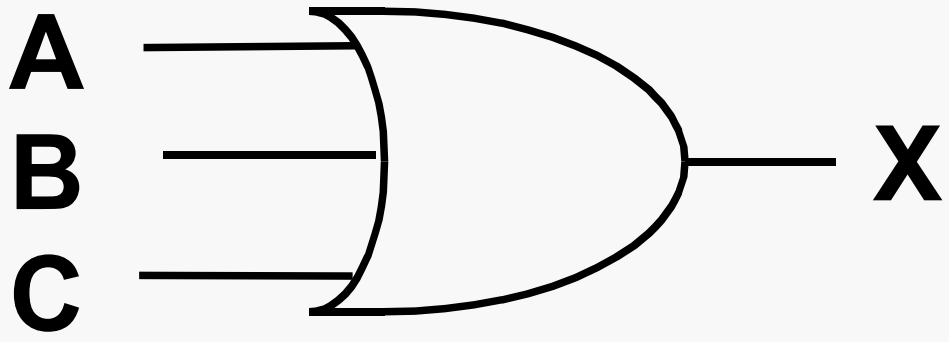
Logic expression

- 2-input OR gate: X = A + B

Truth table for a 2-input OR gate (X = A+B)

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

X=1 if at least one input is = 1



3-input OR gate:

$$X = A + B + C$$

OR operation result will be 1 if at least one input is 1

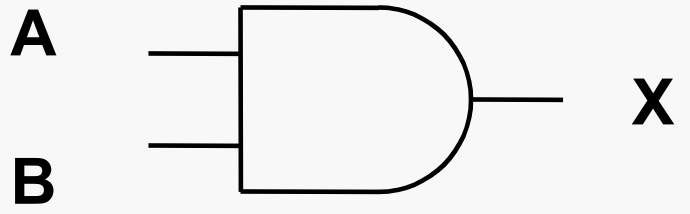
Truth Table for a 3-input OR gate (X = A+B+C)

Inputs			Output
A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Logical AND operation

Logical AND operation

- X = A • B
- X = A AND B
- X = AB

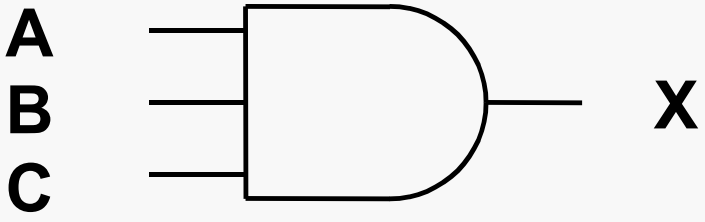


2-input AND gate:

- X = AB

3-input AND gate:

- X = ABC



AND operation result will be 1 only if all inputs are 1

Truth table for 2-input AND gate (X = AB)

Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

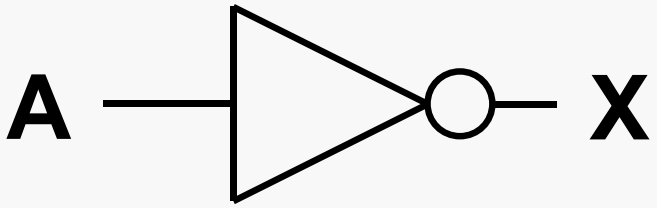
Output X=1 only if all inputs are 1

The Truth Table for a 3-input AND gate (X = ABC)

Inputs			Output
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Logical NOT operation

- NOT gate only has 1 input and it is commonly known as an inverter
- the output is the complement/inverse of the input
- $X = \overline{A}$



- X = A' ⟵ we will use this notation

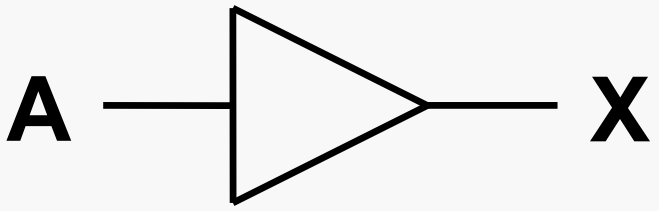
Logical NOT operation

- Its truth table is very simple

input	output
A	X = A’
0	1
1	0

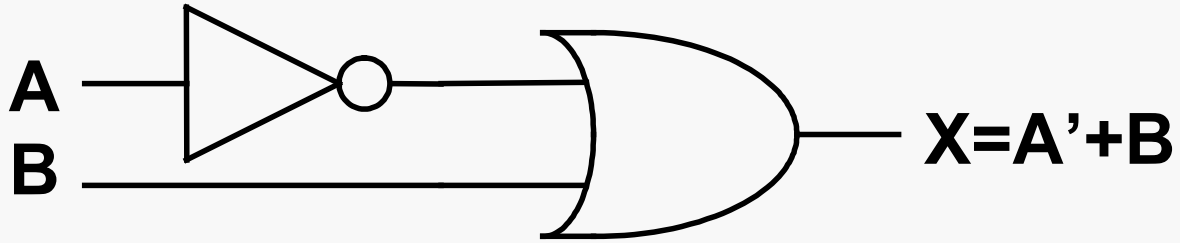
Buffer

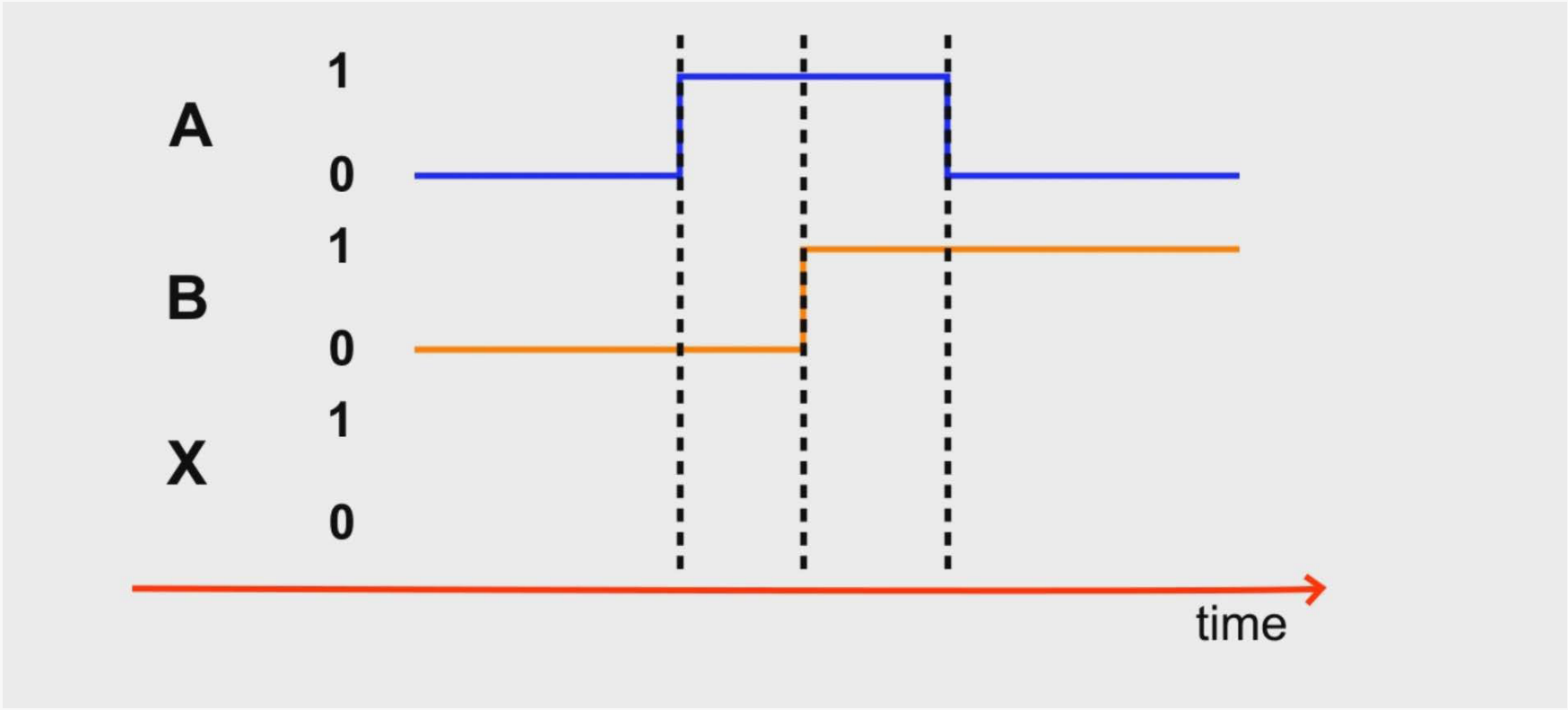
- Its truth table is also very simple
- No change in logic



input	output
A	X = A
0	0
1	1

- **Example: Sketch the logic waveform of X**





- **Timing diagram with more realistic appearance, showing propagation delays, rise time and fall time**

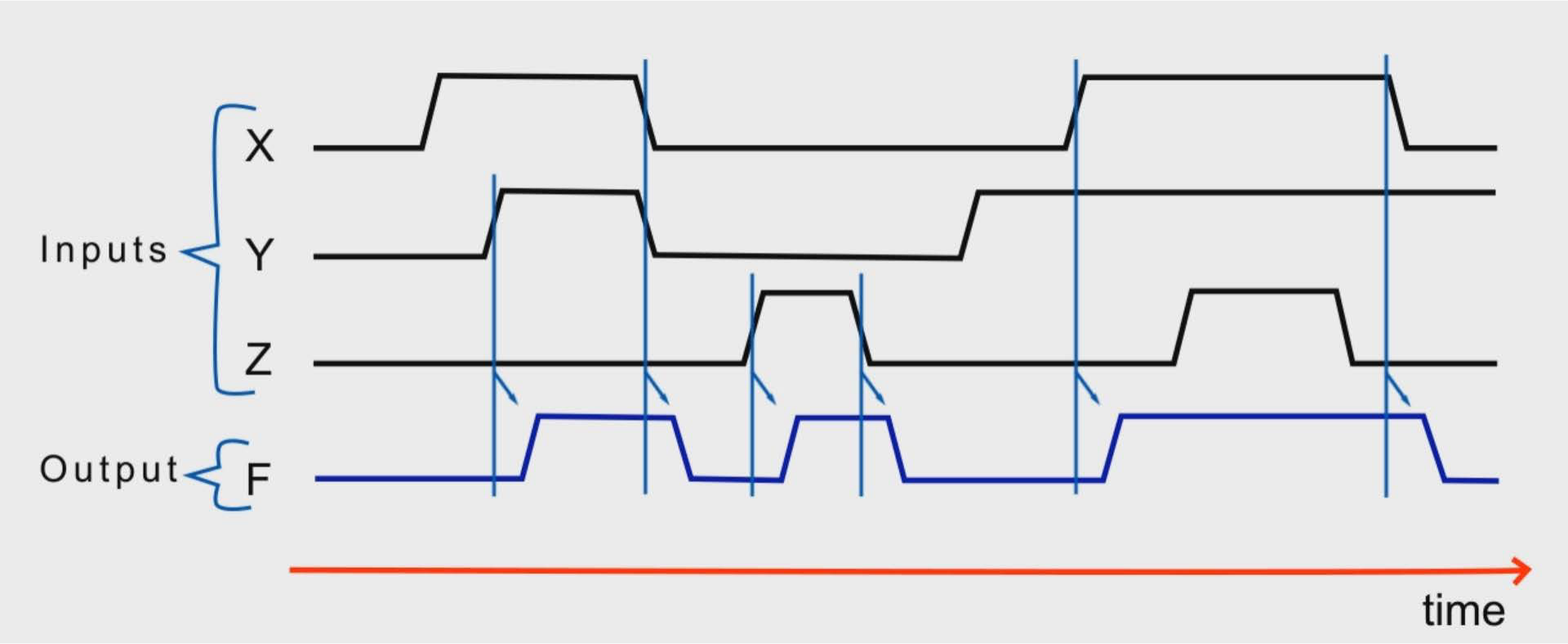


Fig. 3.17 (taken from Wakerly)

Boolean Algebra

- Helps to analyse logic circuits.
- Express operations mathematically.
- Similar to normal algebra but much simpler.
- It does not have fraction or negative number.

Order of Precedence in Boolean Algebra:

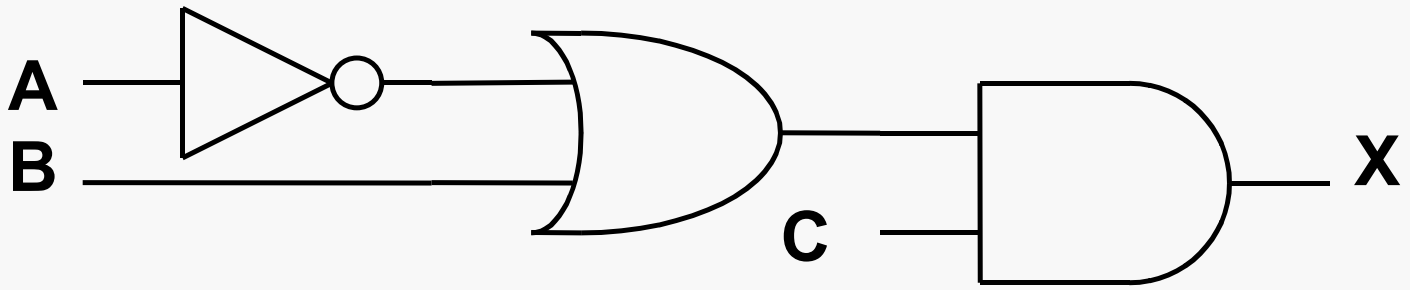
- Complement over a single variable (inversion)
- Expression within parentheses
- AND
- OR

Examples:

1. Y = A + B C’
2. Y = (A + B) C’
3. Y = A + (B C)’

Describing logic circuits algebraically

- AND, OR and NOT operations
  - are basic building blocks of digital system
  - can completely describe any logic circuit
- Example: express output X in terms of inputs A, B and C



Evaluating logic circuit outputs

From a Boolean expression, the logic level of an output can be determined for any values of the circuit inputs.

Example

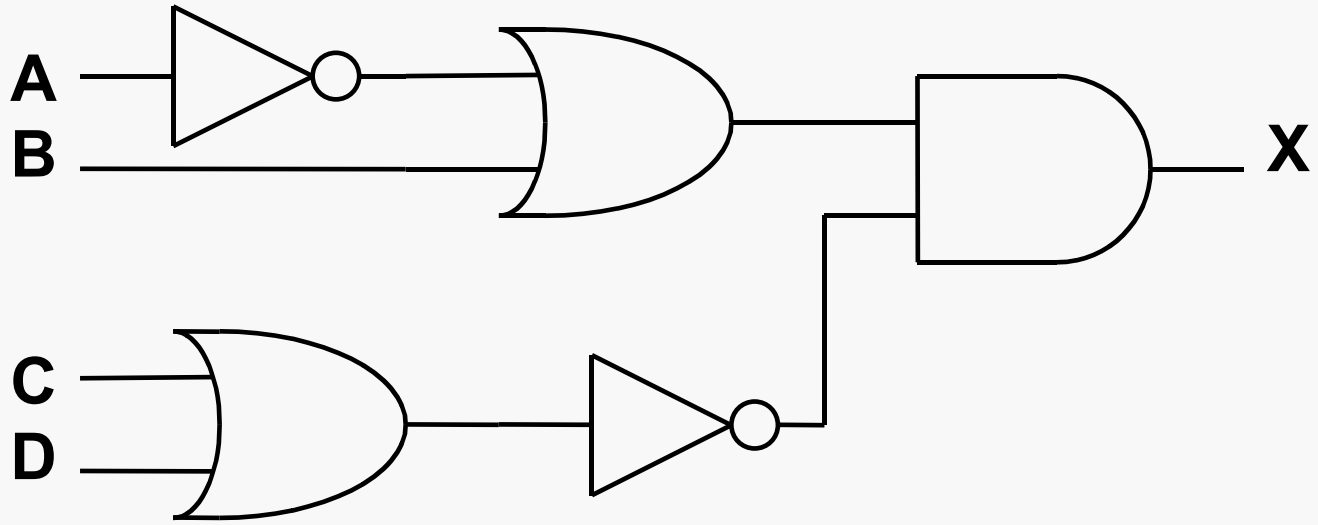
X = A'(B+C)(A+D)'

If inputs A,B,C,D = 0,1,1,0
X = ?

If inputs A,B,C,D = 1,1,1,1
X = ?

If inputs A,B,C,D = 0,0,0,0
X = ?

Determining Instantaneous Output Level from a Logic Circuit Diagram



e.g. if A=1, B=1, C=0, D=0, then X=?

Implementing Circuits from Boolean expressions

Example Y = AC + BC’ + A’BC

L2 (3.26 – 3.46)

- Single and Multi-variable Boolean theorems
- NAND, NOR gates
- NAND-only and NOR-only implementations

Boolean Theorems

- Many of the theorems are similar to those in normal algebra.
- The theorems can be used to simplify logic expressions and therefore can help to simplify logic circuits.
- Simpler circuits cost less to build and are less prone to failure.

Boolean Theorems

Axioms:

$X = 0$ if $X \neq 1$

$X = 1$ if $X \neq 0$

$0 \cdot 0 = 0$

$1 \cdot 1 = 1$

$0 \cdot 1 = 1 \cdot 0 = 0$

$1 + 1 = 1$

$0 + 0 = 0$

$1 + 0 = 0 + 1 = 1$

Single variable theorems:

$X \bullet 0 = 0$	$X + 1 = 1$
$X \bullet 1 = X$	$X + 0 = X$
$X \bullet X = X$	$X + X = X$
$X \bullet X' = 0$	$X + X' = 1$
	$(X')' = X$

Duality: any theorem or identity in switching algebra remains true if 0 and 1 are swapped and · and + are swapped throughout

Single variable theorems:

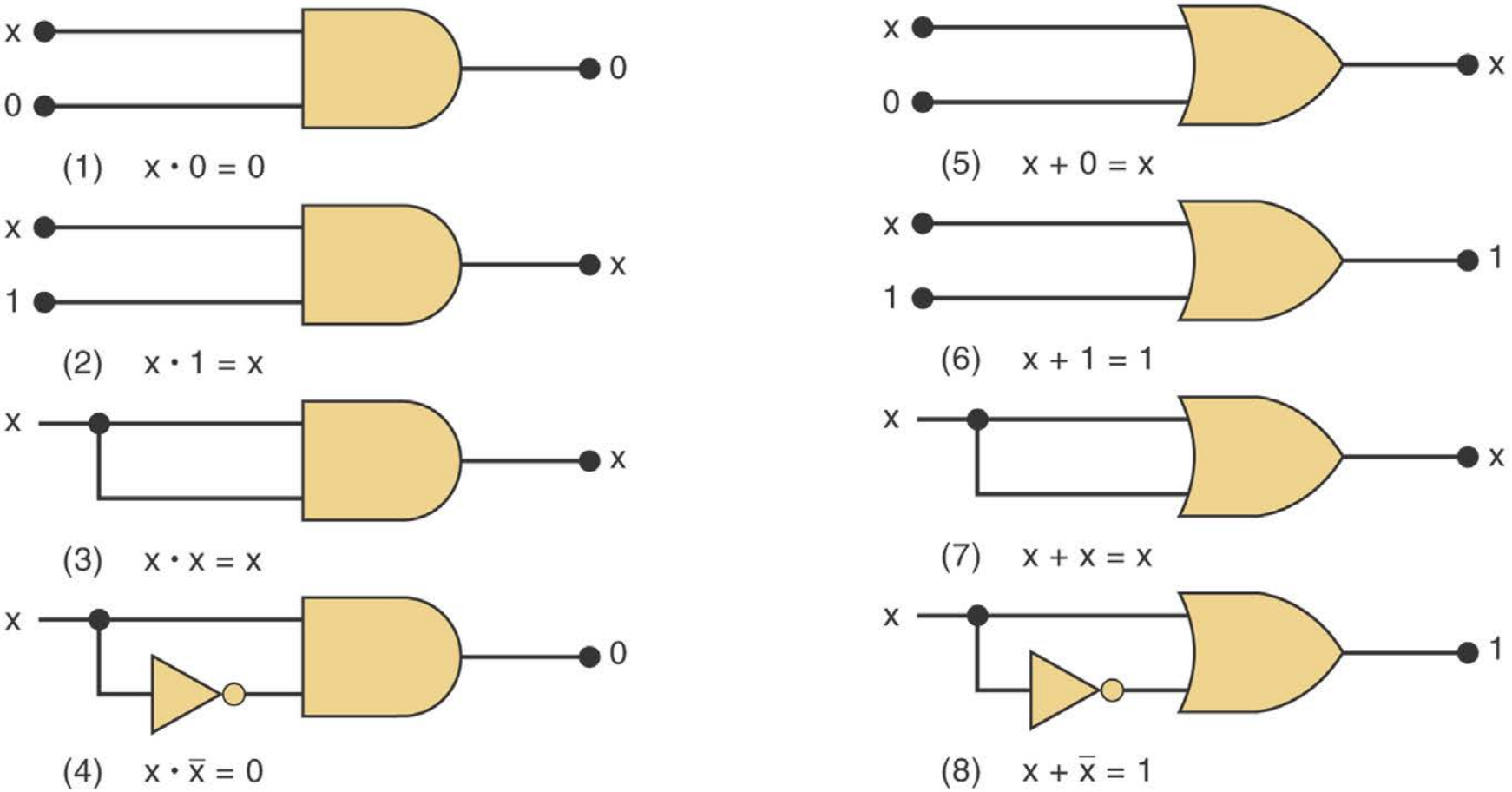


Fig. 3-25 (Tocci, 10th ed. Pg. 77)

Multivariable theorems:

- **Commutative laws:**

  $A + B = B + A$

  $A \bullet B = B \bullet A$

- **Associative laws:**

  $A + (B + C) = (A + B) + C = A + B + C$

  $A(BC) = (AB)C = ABC$

- **Distributive laws:**

  $A(B + C) = AB + AC$

  $(A + B)(C + D) = AC + BC + AD + BD$

Absorption laws

A + AB = A

proof:

A + AB = A(1 + B)

= A • 1 = A

A + A'B = A + B

proof:

A + A'B = A + AB + A'B

= A + (A + A')B

= A + (1) B

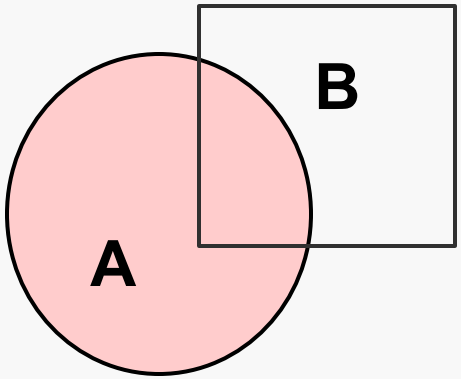
= A + B

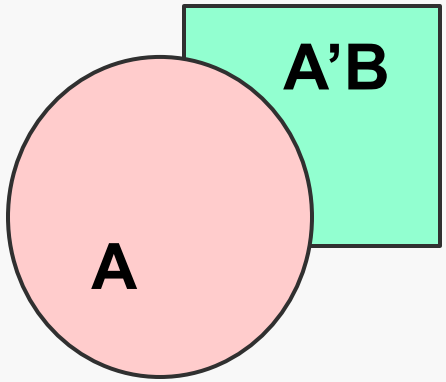
A technique commonly used in algebraic simplification

Absorption laws (Venn diagrams)

A + AB = A

A + A'B = A + B





Consensus

AB + A'C + BC = AB + A'C

proof:

BC = ABC + A'BC

Thus AB + A'C + BC = AB + A'C + ABC + A'BC

= AB + ABC + A'C + A'BC

= AB (1+C) + A'C(1+B)

= AB + A'C

DeMorgan's Theorems

- **(A + B)' = A' • B'**

proof:

A	B	(A+B)'	A' • B'
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

DeMorgan's Theorems

- **(AB)' = A' + B'**

proof:

A	B	(AB)'	A' + B'
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

DeMorgan's Theorems generalise to many variables

$$(A+B+C+D+\ldots)' = A' \bullet B' \bullet C' \bullet D' \bullet \ldots$$

$$(ABCD\ldots)' = A'+B'+C'+D'+ \ldots$$

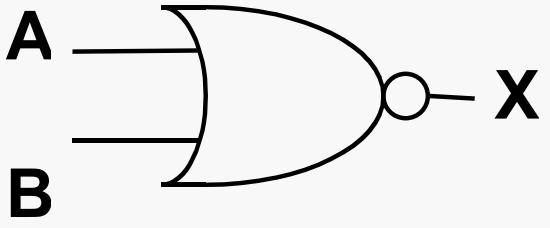
- Add or remove inverter to each variable
- Interchange AND with OR

Example
Simplify [A (B + C')' D]'

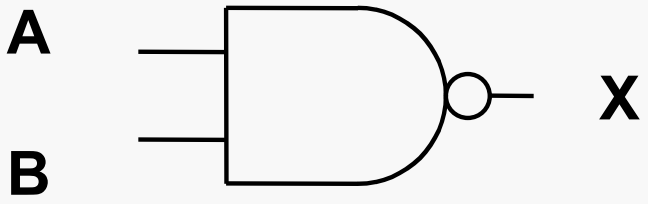
NOR gate & NAND gate

- Combines basic operations of OR & AND with NOT

- NOR: X = (A+B)'



- NAND: X = (AB)'



Truth table for 2-input NOR gate
X = A NOR B

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

Output X=1 only when all inputs are 0

Truth table for 2-input NAND gate
X = A NAND B

Inputs		Output
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

Output X=0 only when all inputs are 1

Truth table for 3-input NOR gate
X = (A+B+C)'

Inputs			Output
A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Truth table for 3-input NAND gate
X = (ABC)'

Inputs			Output
A	B	C	X
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Summary

- **OR**: output is 1 when any of the inputs is 1
- **AND**: output is 1 when all the inputs are 1
- **NOT**: output is 0 when input is 1 and vice versa
- **NOR**: output is 0 when any of the inputs is 1
- **NAND**: output is 0 when all the inputs are 1

Universality of NAND gates and NOR gates

- NAND gates can be used to form AND gate, OR gate and NOT gate
- Therefore, NAND gates can be used to implement any Boolean function
- Similarly for NOR gates
- Equivalence can be proved by DeMorgan's theorems

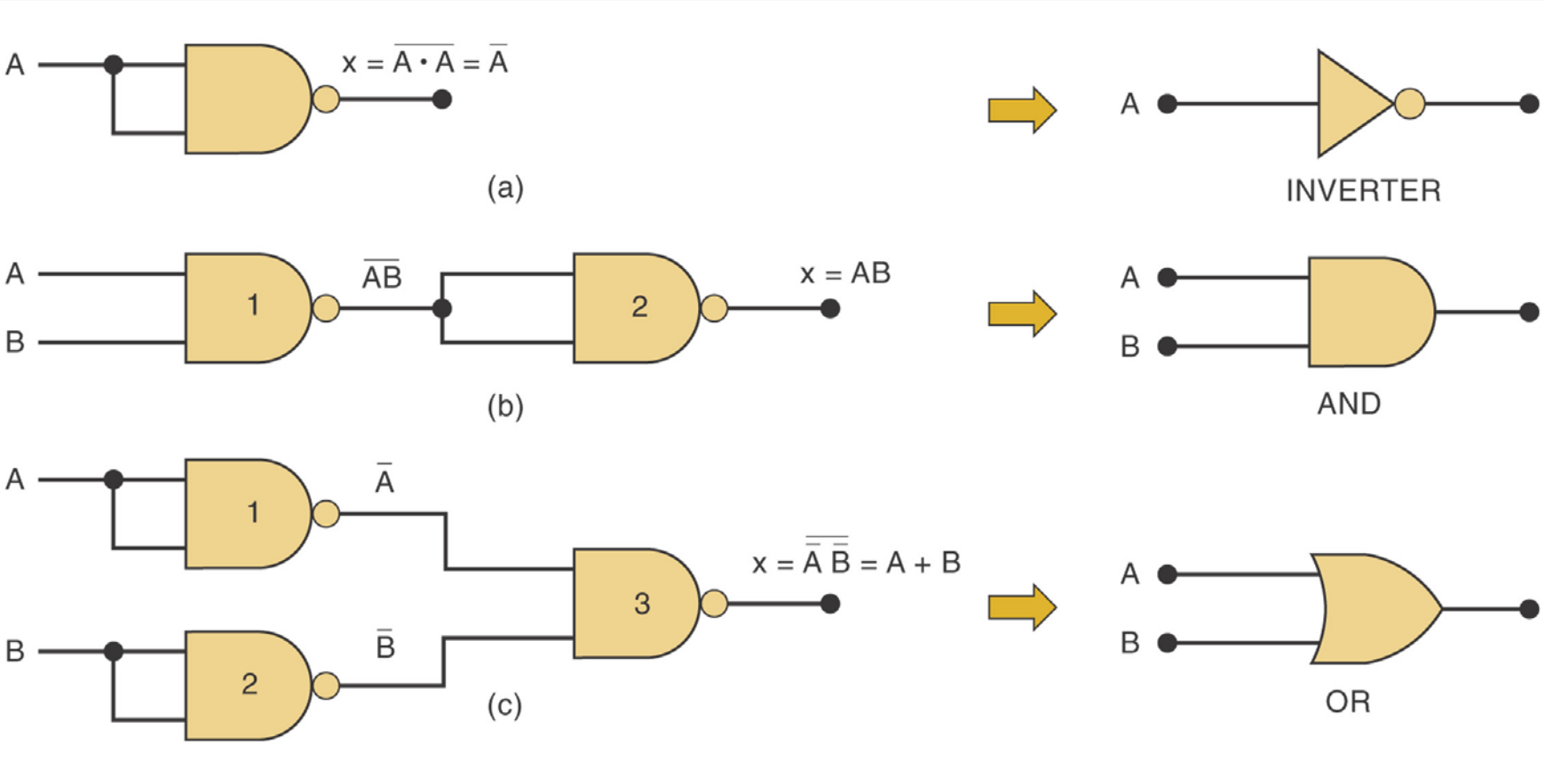


Fig. 3-29 Basic gates from NAND

(Tocci, 10th ed. Pg. 84)

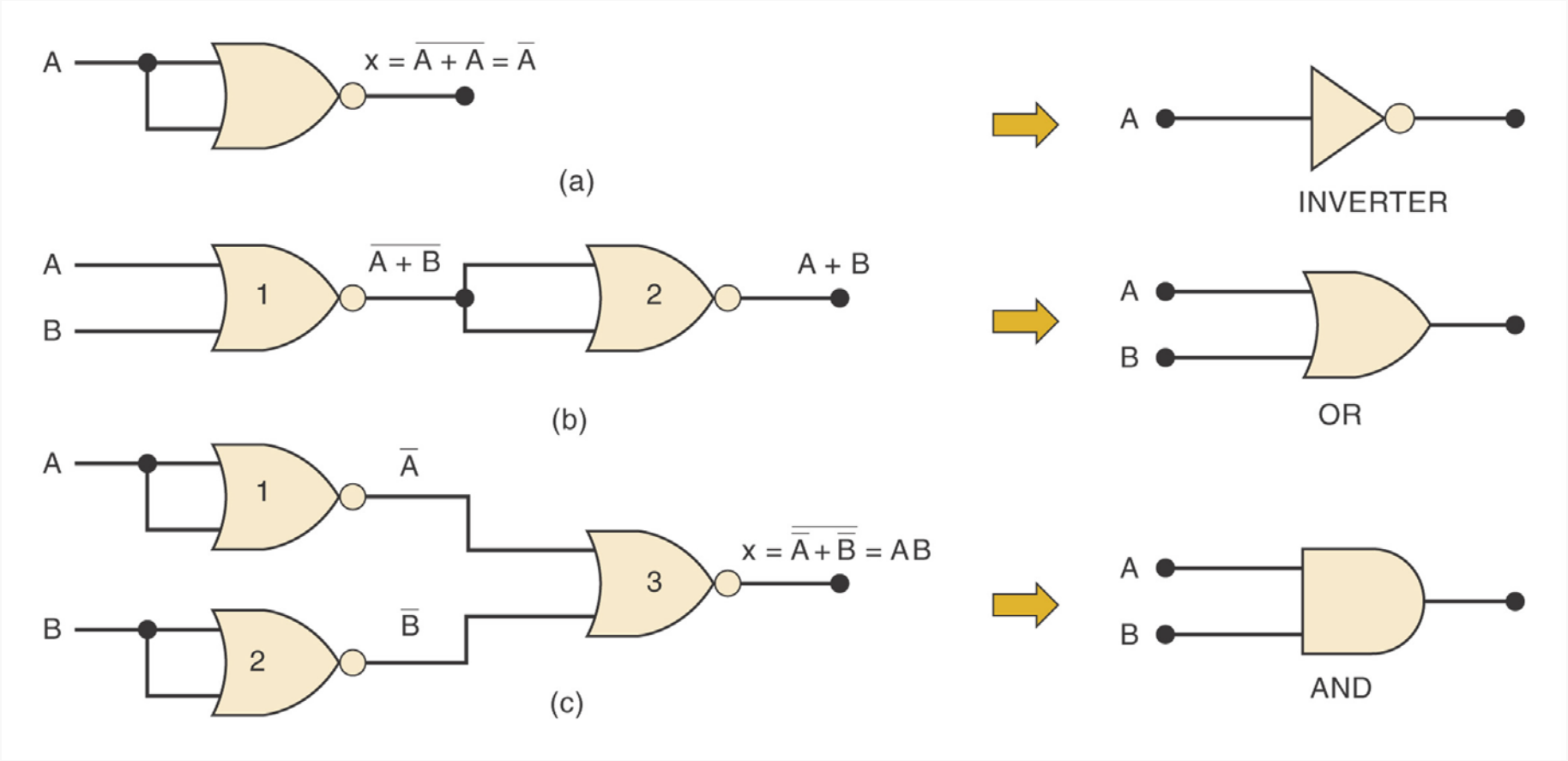


Fig. 3-30 Basic gates from NOR

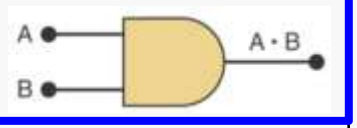
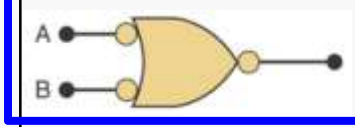
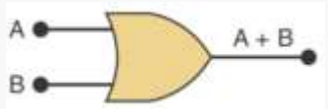
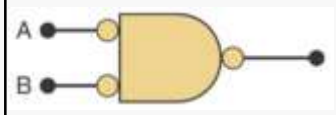
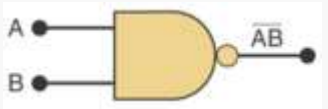
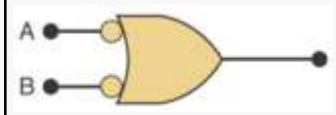
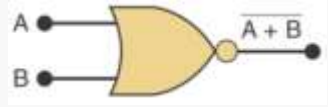
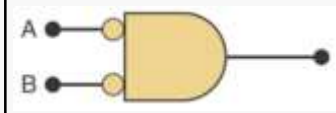
(Tocci, 10th ed. Pg. 84)

L3 (3.47 – 3.61)

- Alternate logic symbols
- XOR, XNOR gates
- Parity generator/checker
- Logic components connection diagram

Alternate Logic gate representations

- The alternate symbol is obtained from the standard symbol by applying DeMorgan's theorems

Logic operator	Standard symbol	Logic expression	Alternate Symbol
AND		A.B = (A' + B')'	
OR		A+B = (A'.B')'	
NAND		(A.B)' = A'+B'	
NOR		(A+B)' = A'.B'	

- Modification of standard to alternate symbol (and vice-versa) for the **same logic operator**:

	standard ⟷ **alternate**	
Bubble at input or output	No	Add
	Yes	Remove
Symbol shape	“and”	Replace with “or”
	“or”	Replace with “and”

Note that a pair of standard and alternate symbols describes the **same logic gate** with the same truth table.

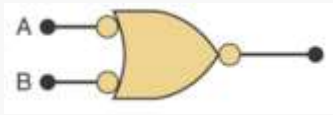
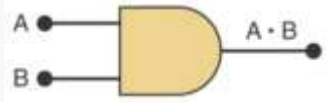
- **The standard symbol of a NOT gate is similarly modified to its alternate symbol (and vice-versa) but there is no change in symbol shape:**



We interpret the above symbol this way:
Output=0
when input=1

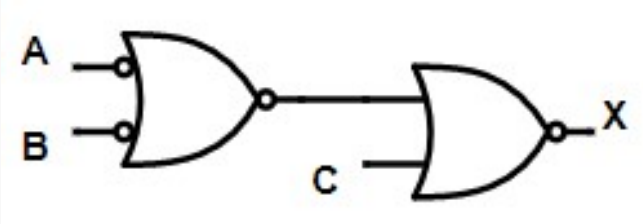
NOT gate truth table	
Input A	Output A'
0	1
1	0

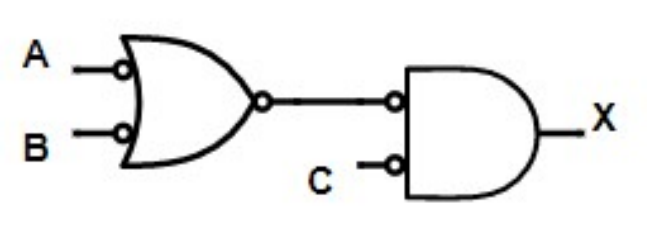
We interpret the above symbol this way:
Output=1
when input=0

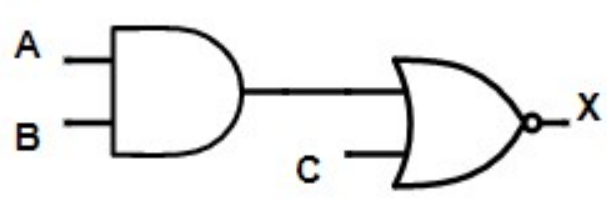
A	B	A AND B	symbol
0	0	0	
0	1	0	
1	0	0	
1	1	1	

- Both standard symbols and alternate symbols may be used in the same diagram to help describe logic flow
- Comply with bubble-to-bubble matching
- Say "0" when there is a bubble, otherwise say "1"

Example 1:



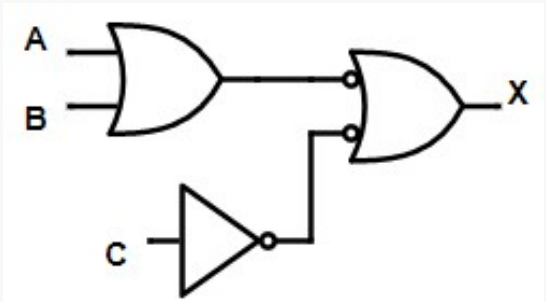


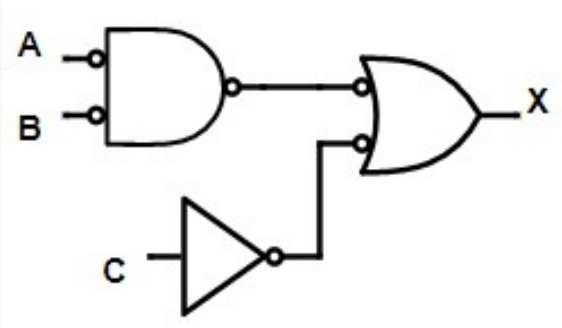


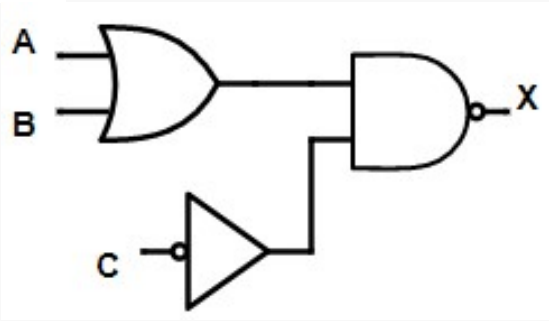
We say:
Output X=1
when inputs
C=0 and at the
same time either
A=0 or B=0

We say:
Output X=0
when inputs
C=1 or
A and B are
both=1

Example 2:







We say:
Output X=1
when inputs
C=1 or both A and B=0

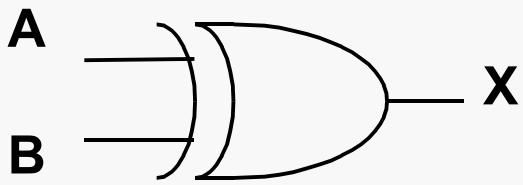
We say:
Output X=0
when inputs
C=0 and at the same time either A=1 or B=1

Exclusive-OR gate

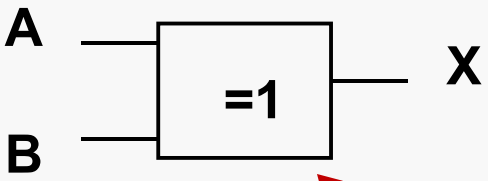
Ex-OR (XOR)

$X = AB' + A'B$

$X = A \oplus B$



A	B	X
0	0	0
0	1	1
1	0	1
1	1	0



IEEE symbol

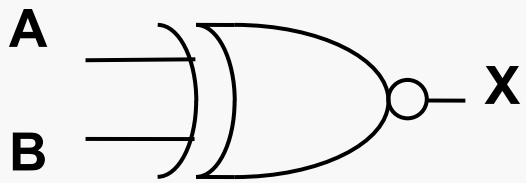
Different from OR

Exclusive-NOR gate

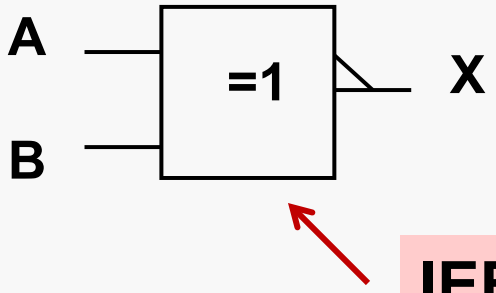
Ex-NOR (XNOR)

$X = AB + A'B'$

$X = (A \oplus B)'$



A	B	X
0	0	1
0	1	0
1	0	0
1	1	1



IEEE symbol

Application of XOR

- **Bit-wise comparator**
  - **output is 1 if the two multi-bit inputs are different**

XOR with multiple inputs

- Essentially an odd-function generator
- Output is 1 if there is an odd number of 1's among all the inputs
- E.g. for 3-input XOR, the output is 1 if there are 1 or 3 bits of 1 among the inputs

- $A \oplus B \oplus C = (A \oplus B) \oplus C$
- $= A \oplus (B \oplus C)$

Logic devices

- Different ways to create a physically functioning logic circuit
- Examples: use standard logic integrated circuits (ICs), application-specific ICs (ASICs), programmable logic devices
- Small-scale integrated logic devices: AND, OR, NOT, NAND, NOR, XOR, XNOR
- We will use some of these in lab experiment 1

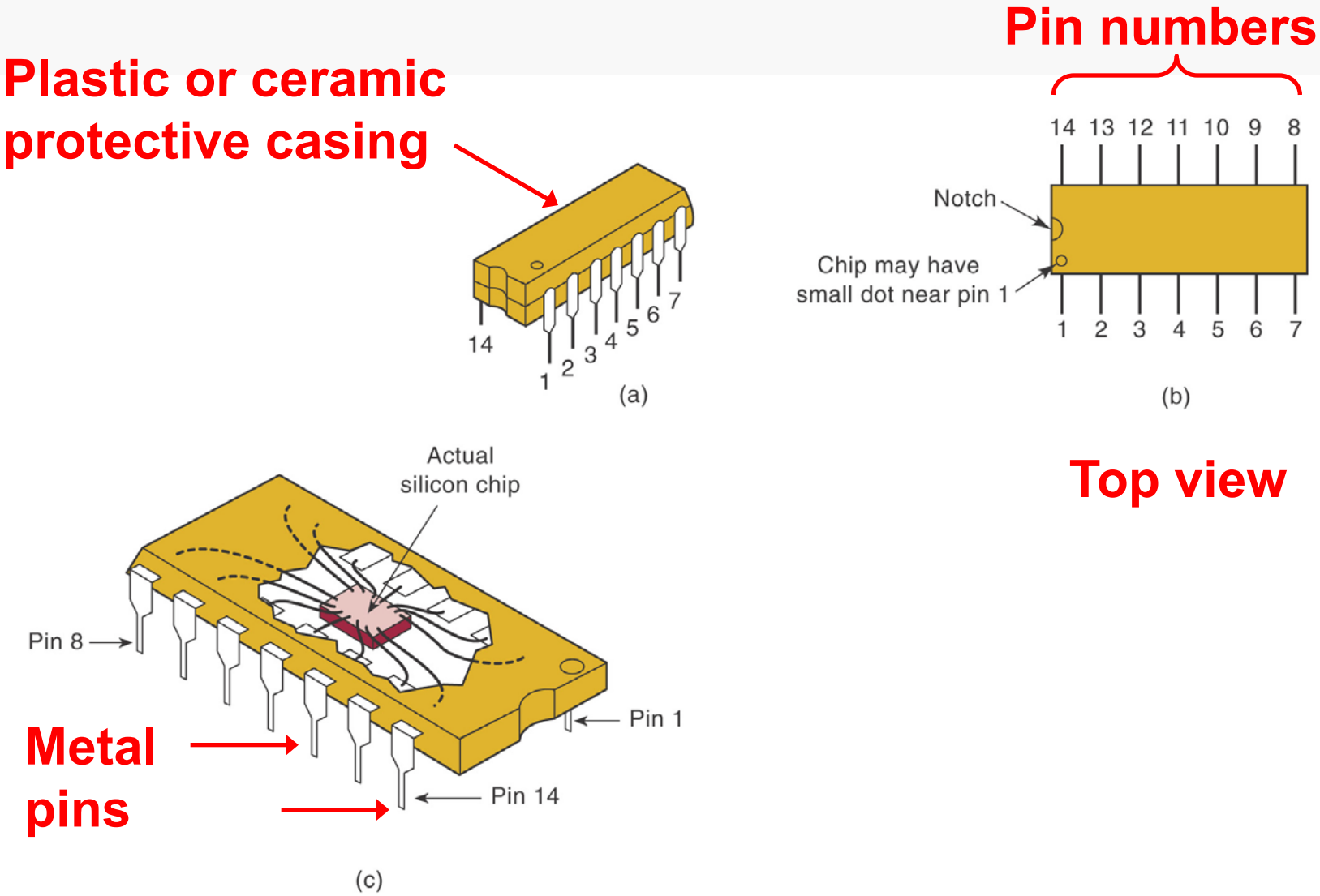
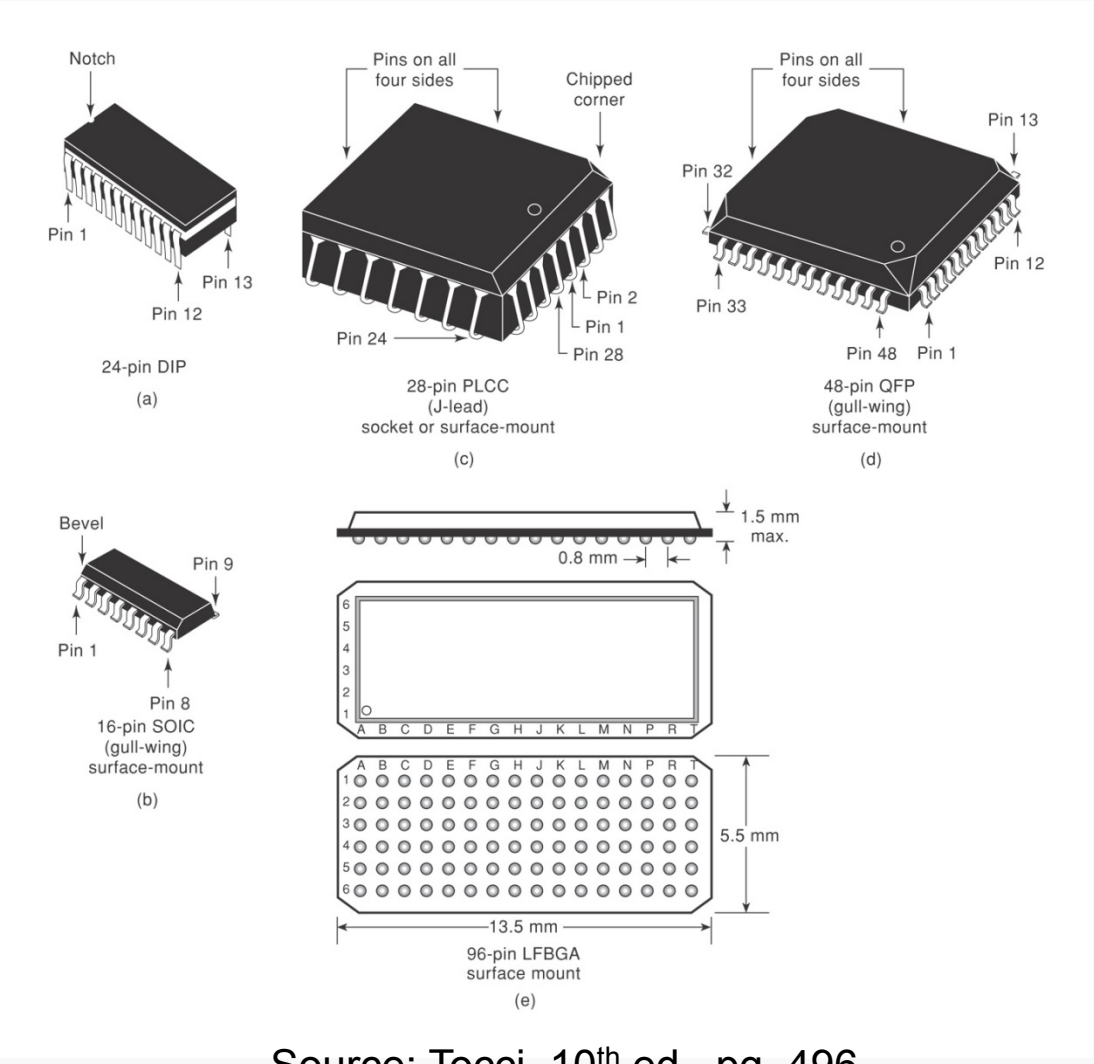


Figure 4.29: (Tocci 10th Ed) Dual-in-line Package

Common IC packaging

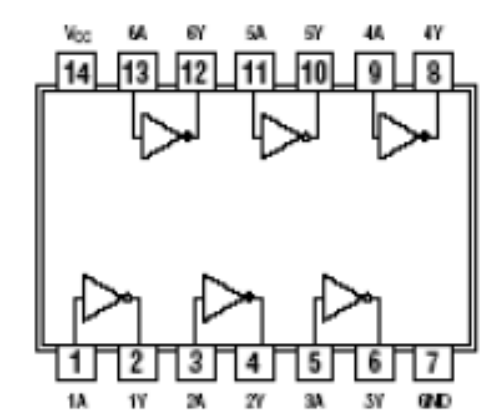


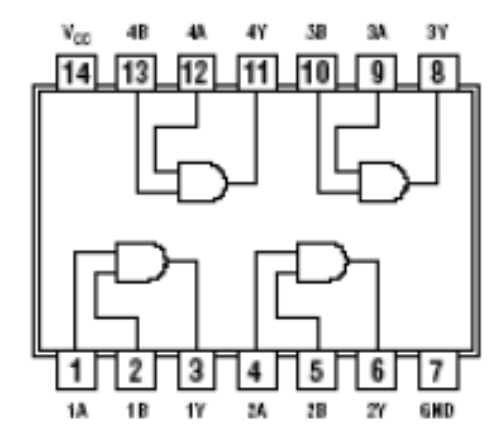
Source: Tocci, 10th ed., pg. 496

Logic circuit connections

- **Implement Y = AB’**

7404
Hex-NOT





7408
Quad-AND

Circuit connection diagram

L4 (4.1 – 4.22)

- Half adder and full adder
- Parallel adder
- Carry propagation
- Sign-magnitude representation of signed numbers
- 2’s complement representation of signed numbers

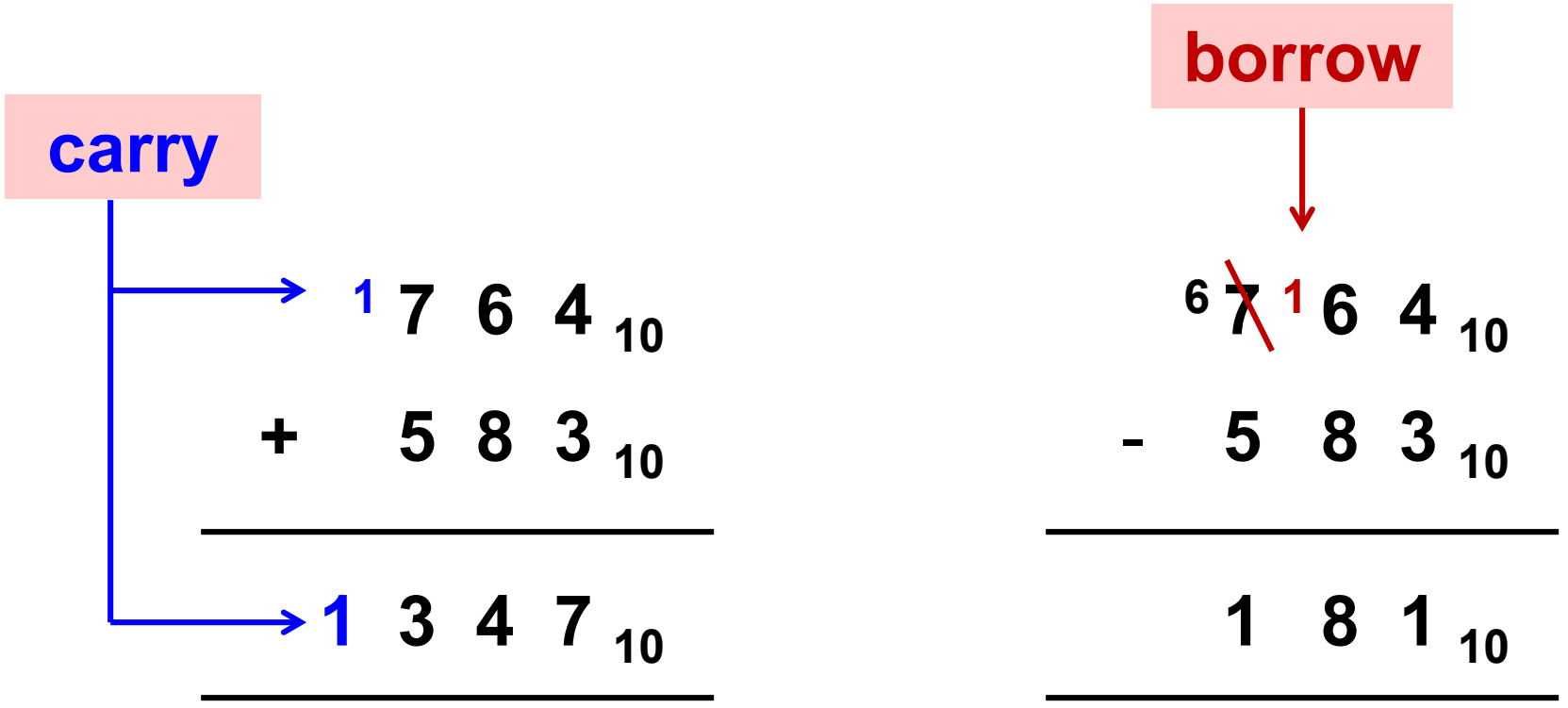
4. Digital Arithmetic

In digital circuits, e.g. digital computers and electronic calculators, arithmetic operations are carried out on binary numbers:

- **Addition, subtraction, multiplication and division**

Binary Addition and Subtraction both begin with the LSB (least significant bit)

Decimal Addition and Subtraction both begin with the **LSD:**



Binary addition example:

$10111_2 + 1010_2 = ?$ $(23_{10} + 10_{10} = 33_{10})$

$$\begin{array}{r} 1\ 0\ 1\ 1\ 1_{\,2} \\ +\quad 1\ 0\ 1\ 0_{\,2} \\ \hline \\ \hline \end{array}$$

Adder

Half adder (HA)

- combination logic circuit that performs addition of 2 bits

$Carry = A \cdot B$

$Sum = A \oplus B$

Inputs		Outputs	
A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Full adder (FA)

- combination logic circuit that performs addition of 3 bits

Inputs			Outputs	
A	B	Cin	Cout	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Since the truth table is rather simple, we can obtain the Boolean expressions by observation,

Sum = $A \oplus B \oplus Cin$

Cout = A•B + B•Cin + A•Cin

Fig 6.81 shows the circuit implementation.

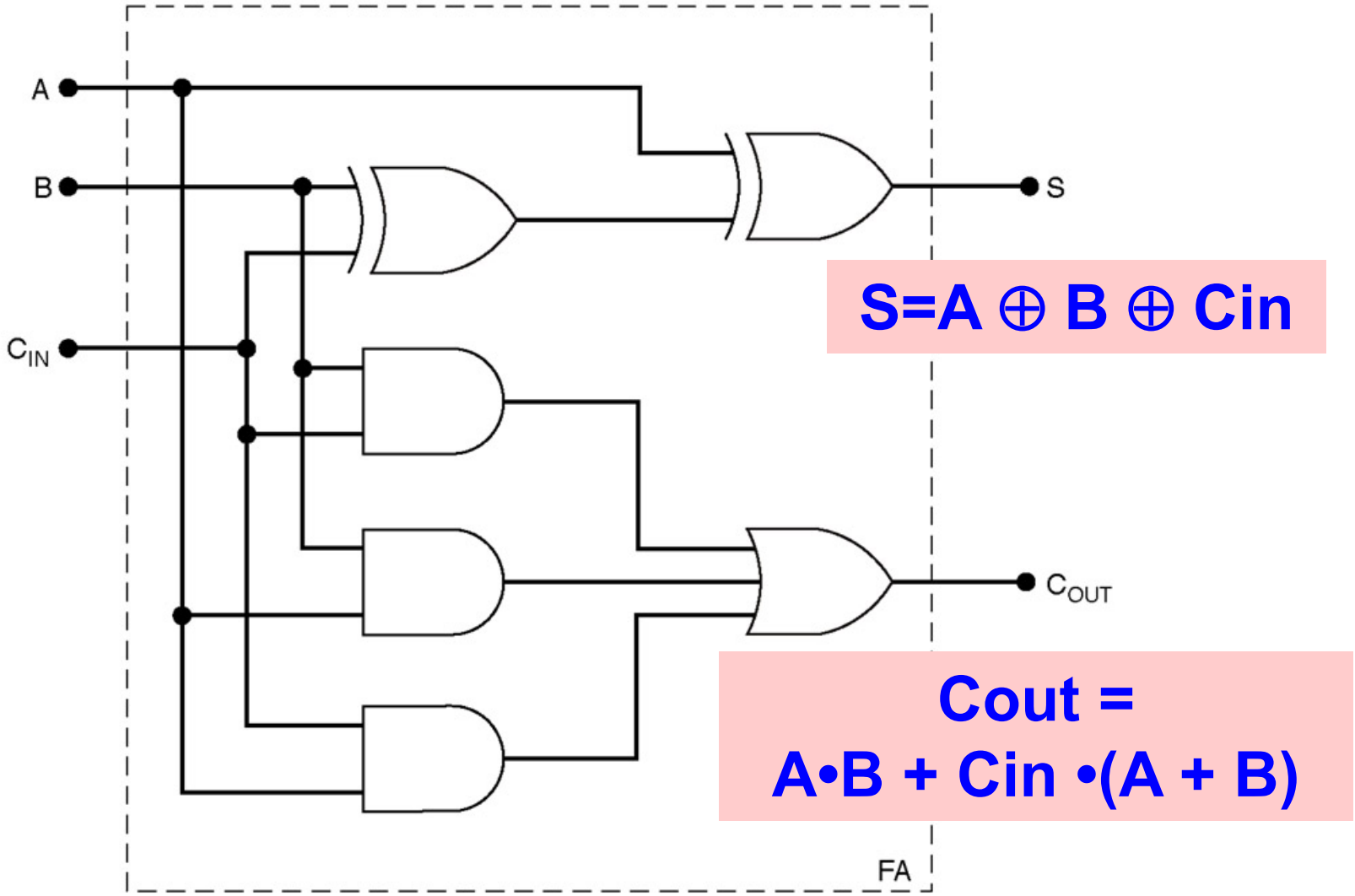


Fig. 6-81 Full adder circuit

If we re-arrange the Boolean expression,

$$Cout = A \cdot B + B \cdot Cin + A \cdot Cin$$

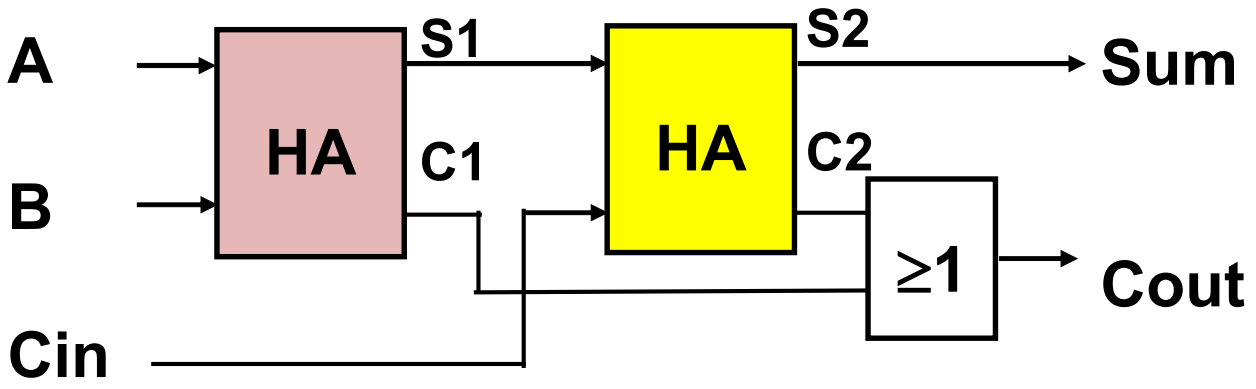
$$= A \cdot B + Cin\,(A'B + AB) + Cin\,(AB' + AB)$$

$$= A \cdot B + Cin\,(AB) + Cin\,(A'B + AB')$$

$$= A \cdot B + Cin \cdot (A \oplus B)$$

- An alternate circuit implementation using Half-adders is obtained

Full adder circuit implemented using 2 half-adders and an OR gate



$S1 = A \oplus B$

$C1 = A \cdot B$

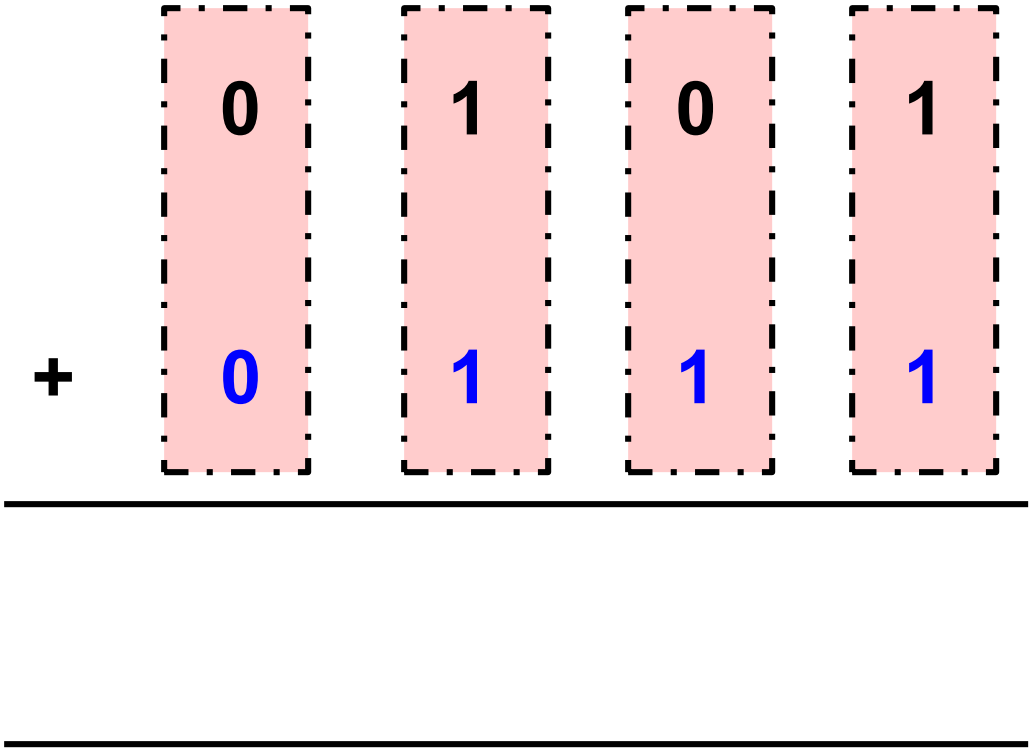
$S2 = (A \oplus B) \oplus Cin$

$C2 = Cin \cdot (A \oplus B)$

$Cout = A \cdot B + Cin \cdot (A \oplus B)$

Binary addition example: $0101_2 + 0111_2 = ?$

Need 4 FAs



Parallel Adder

- N full-adders can be cascaded to form an N-bit parallel adder
- also known as ripple adder
- all the bits of the augend and addend are fed into the adder circuits simultaneously
- addition is very fast
- addition speed is limited by propagation delays of FAs - carry propagation
- Fig. 6-82 shows a 4-bit adder

Example: 0011_2 + 0001_2 = 0100_2

(In decimal: 3 + 1 = 4)

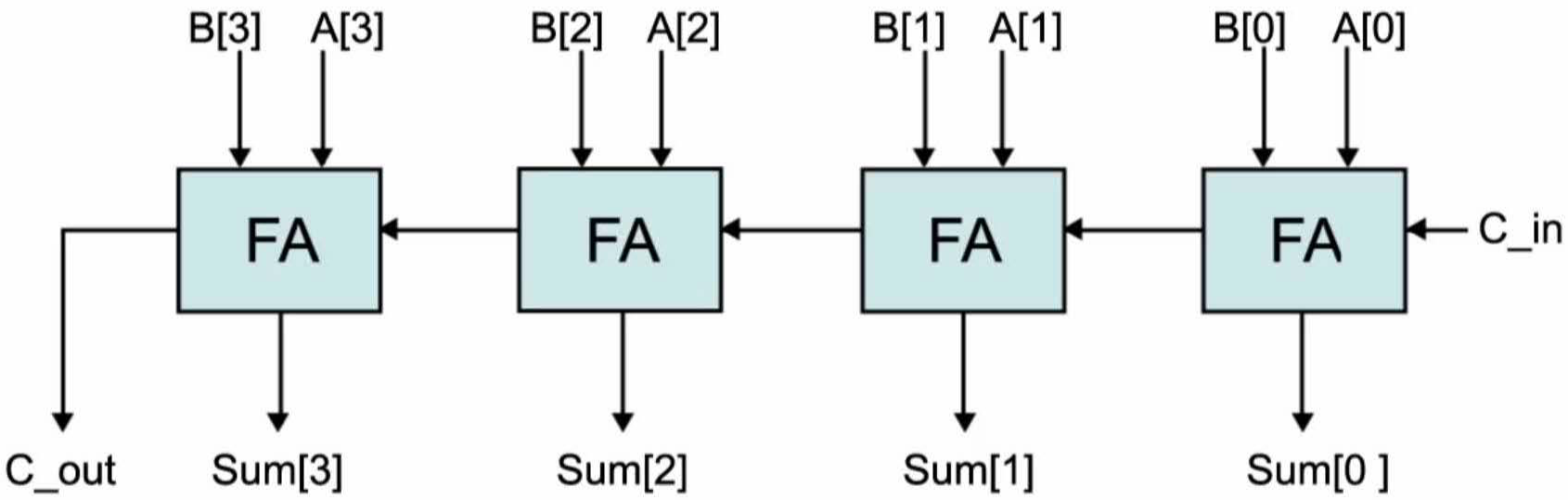


Fig. 6-82 : 4-bit ripple adder

Representing Signed Numbers

Signed-Magnitude system

- sign bit = MSB : 0 for positive numbers
- sign bit = MSB: 1 for negative numbers

$14_{10} = 1\ 1\ 1\ 0_2$

	sign	magnitude
+14	0	1 1 1 0
-14	1	1 1 1 0

Signed-Magnitude system

- There are equal numbers of positive and negative values
- An N-bit value lies in the range

$$\{ -(2^{N-1} - 1) \text{ to } +(2^{N-1} - 1) \}$$

Examples:

$0101\ 0101_2 = +85_{10}$ $\quad$ $1101\ 0101_2 = -85_{10}$

$0000\ 0000_2 = +0_{10}$ $\quad$ $1000\ 0000_2 = -0_{10}$

2's complement system

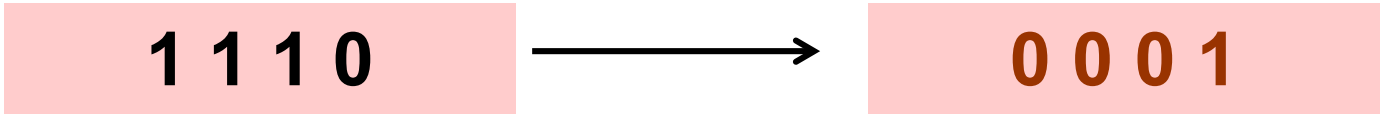
- sign bit = MSB: 0 for zero and positive numbers
- sign bit = MSB: 1 for negative numbers

$14_{10} = 1\ 1\ 1\ 0_2$

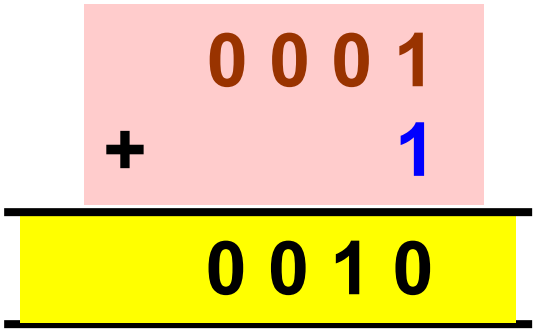
	sign	
+14	0	1 1 1 0
-14	1	???

2's complement operation:

Step 1: invert every bit of a binary number (i.e. perform 1's complement)



Step 2: add (arithmetic addition) 1 to it



The 2's complement of 1110 is 0010

2's complement system

$14_{10} = 1\ 1\ 1\ 0_2$

	sign	
+14	0	1 1 1 0
-14	1	0 0 1 0

A short-cut method for 2's complement operation:

- starting from LSB, copy the bit if it is ‘0’ and repeat process with remaining bits
- copy the bit if it is the first ‘1’, then invert all the remaining bits
- a sequential process

This method works on all binary bit patterns

Examples:

$0\ 1\ 0\ 1\ 0\ 1\ 0\ 1_2 = +85_{10}$



$1\ 0\ 1\ 0\ 1\ 0\ 1\ 1_2 = -85_{10}$

Invert the remaining bits

Copy the bits from the right until it reaches the first 1

$0\ 1\ 0\ 0\ 0\ 0\ 0\ 0_2 = +64_{10}$



$1\ 1\ 0\ 0\ 0\ 0\ 0\ 0_2 = -64_{10}$

2's complement system

- There is 1 more negative value than positive ones
- An N-bit value lies in the range

$$\{ -(2^{N-1}) \text{ to } +(2^{N-1} - 1) \}$$

- Eg. 4 bits: {-8, 7}
- 8 bits: {-128, 127}

Summary: representing signed numbers in 2's complement system:

If the number is zero or positive

- represent its magnitude in binary
- append a sign bit (0) in front of the MSB if the MSB is 1

If the number is negative

- represent its magnitude in binary
- obtain the 2's complement
- append a sign bit (1) in front of the MSB if the MSB is 0

Examples: Represent the following signed decimal numbers in 2's complement

(a) 5

- binary of 5 is 101
- Positive 5 is 0101

(b) -7

- binary of 7 is 111
- 2's complement of 111 is 001
- Negative 7 is 1001

L5 (4.23 – 4.41)

- Sign-extension for 2’s complement numbers
- 2’s complement add/subtract
- Arithmetic overflow

E.g. A 4-bit 2's complement number system:

Decimal	Binary	
	magnitude	2's comp
-8	1000	1000
-7	0111	1001
-6	0110	1010
-5	0101	1011
-4	0100	1100
-3	0011	1101
-2	0010	1110
-1	0001	1111
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111

Generalised to an N-bit 2's complement number system:

Decimal	2's comp	
		Most negative
$-(2^{N-1})$	1000…00	←
$-(2^{N-1}) + 1$	1000…01	
$-(2^{N-1}) + 2$	1000…10	
.	.	
.	.	-1
-1	1111…11	←
0	0000…00	← zero
1	0000…01	
2	0000…10	
.	.	
.	.	Most positive
$(2^{N-1}) - 2$	0111…10	
$(2^{N-1}) - 1$	0111…11	←

Some special bit patterns:

- 100….000 (with N-1 zeros)

  the decimal value is $-(2^{N-1})$

- 111….111 (all ones)

  the decimal value is -1

- 000….000 (all zeroes)

  the decimal value is 0

- 011….111 (with N-1 ones)

  the decimal value is $(2^{N-1}-1)$

Obtaining the decimal value of a 2's complement number

If the number is positive:

- perform a binary-to-decimal conversion on the number

If the number is negative:

- First perform 2's complement to convert it to positive
- Next perform a binary-to-decimal conversion on the positive number

Examples: Obtain the decimal equivalent of the following 2's complement numbers

(a) 01001

- The magnitude 1001 is 9 in decimal
- 01001 is positive 9 in decimal

(b) 10011

- 2's complement of 10011 is 01101
- The magnitude 1101 is 13 in decimal
- 10011 is negative 13 in decimal

Some observations on the 2's complement number system

- The 2's complement of an N-bit binary number is also of N bits.
- To represent a binary number with N significant bits in its magnitude using the 2's complement system, we need (N+1) bits. The extra bit is the sign bit. *

* note exception

- If there are more bits than necessary to represent a binary number in the 2's complement system, the more significant bits are filled with the sign bit, which is 0 for positive and 1 for negative – known as **sign extension**

101 = 1101 = 11101

011 = 0011 = 00011

- A 2's complement operation will change a positive number to negative and vice-versa, with no change in the magnitude. *

* note exception

* Exception arises when dealing with the most negative number that can be represented given a number of bits.

e.g.

-8 in a 4-bit system,

-16 in a 5-bit system

-128 in an 8-bit system

$-(2^{N-1})$ in an N-bit system

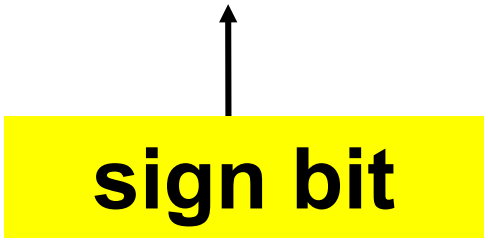
Reasons for using the 2's complement system:

- Using the 2's complement representation for signed numbers, subtraction of numbers can be carried out in the same way as addition
- Therefore, the same set of hardware circuits can be used for both subtraction and addition

Addition in the 2's complement system

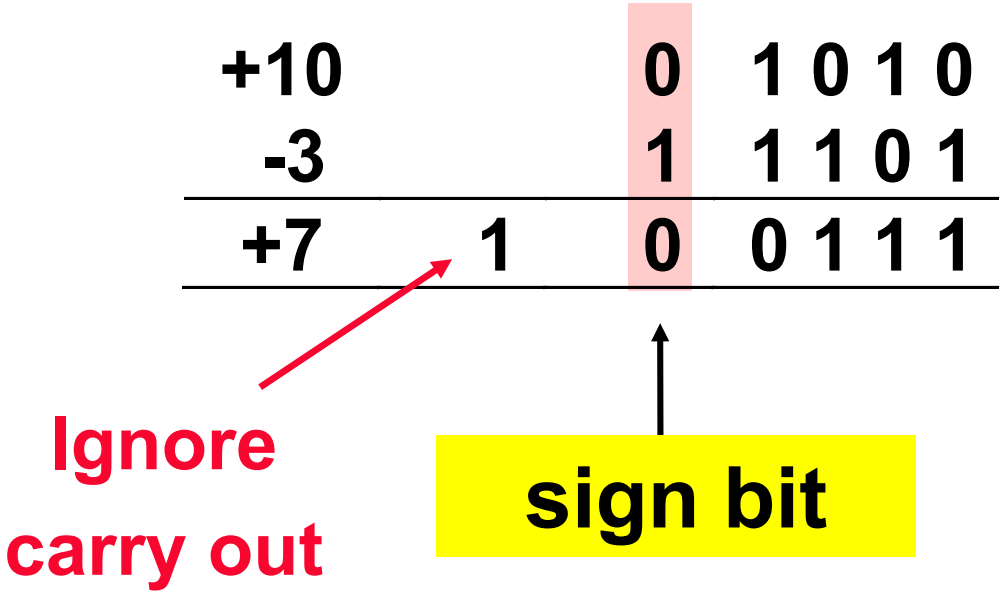
two positive numbers

+10	0	1 0 1 0	(augend)
+3	0	0 0 1 1	(addend)
+13	0	1 1 0 1	(sum)

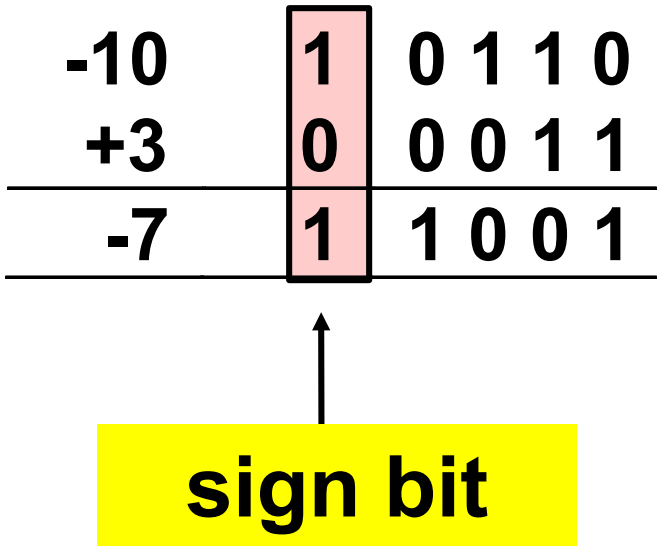


The sign bits are added like the other bits

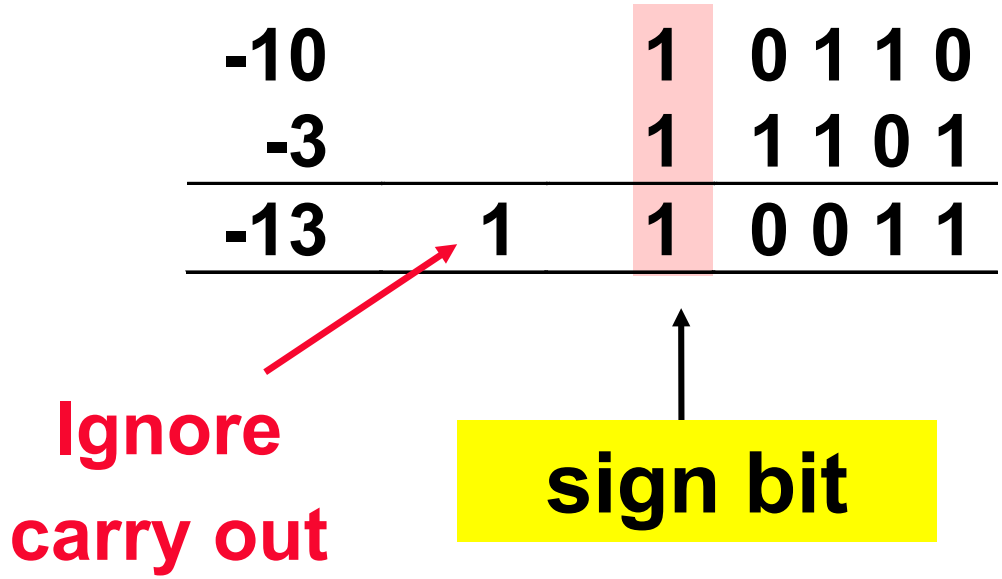
positive number and smaller negative number



positive number and larger negative number



two negative numbers



Subtraction in the 2's complement system

Subtraction can be carried out using 2's complement and addition

A - B = A + (-B)

-B = 2's complement of B

B may be positive or negative

Examples:

10 - 3 = 10 + (-3) = 7

-3 is 2's complement of 3

$$\begin{array}{r} {}^{1}0\ 1\ 0\ 1\ 0 \\ +\ 1\ 1\ 1\ 0\ 1 \\ \hline 1\ 0\ 0\ 1\ 1\ 1 \end{array}$$

Ignore carry out

-10 - (-3) = -10 + 3 = -7

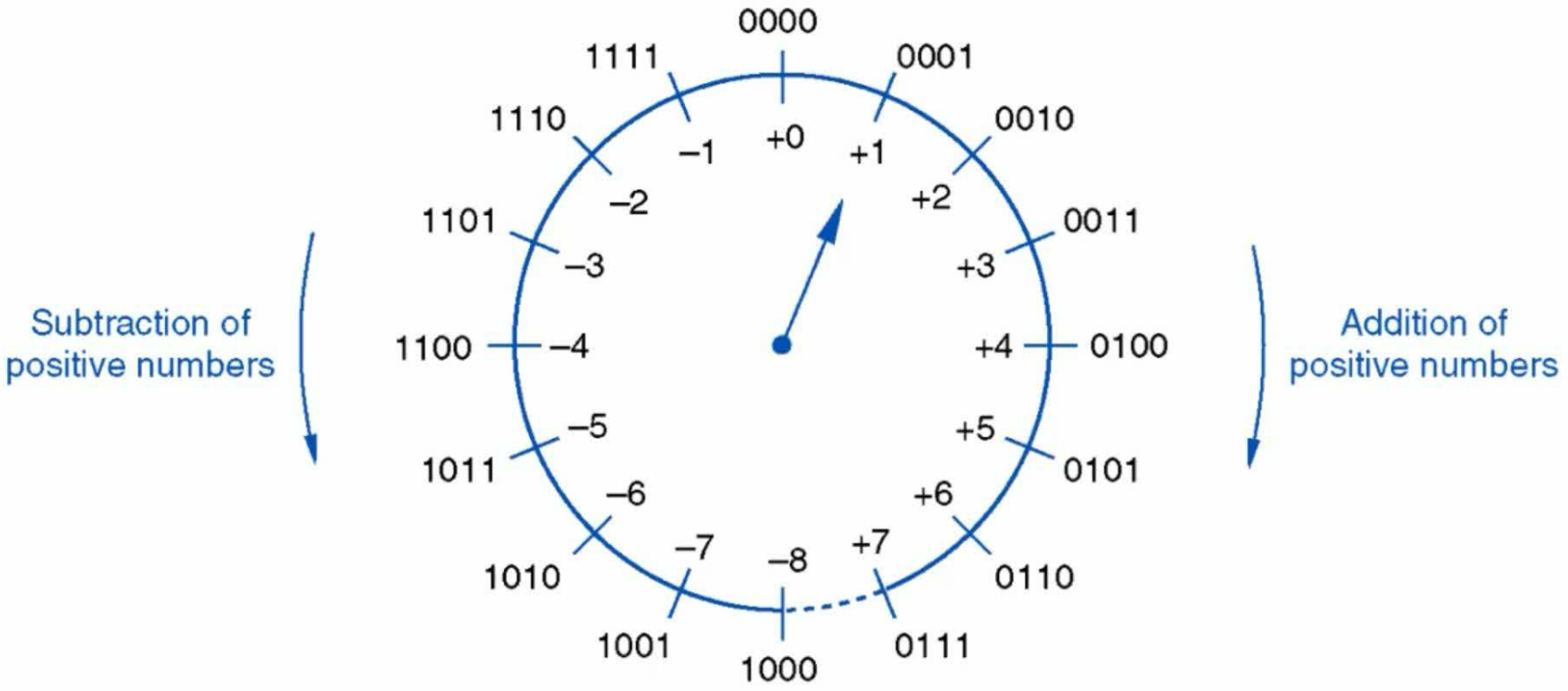
3 is 2's complement of -3

$$\begin{array}{r} 1\ {}^{1}0\ {}^{1}1\ 1\ 0 \\ +\ 0\ 0\ 0\ 1\ 1 \\ \hline 1\ 1\ 0\ 0\ 1 \end{array}$$

Remember:

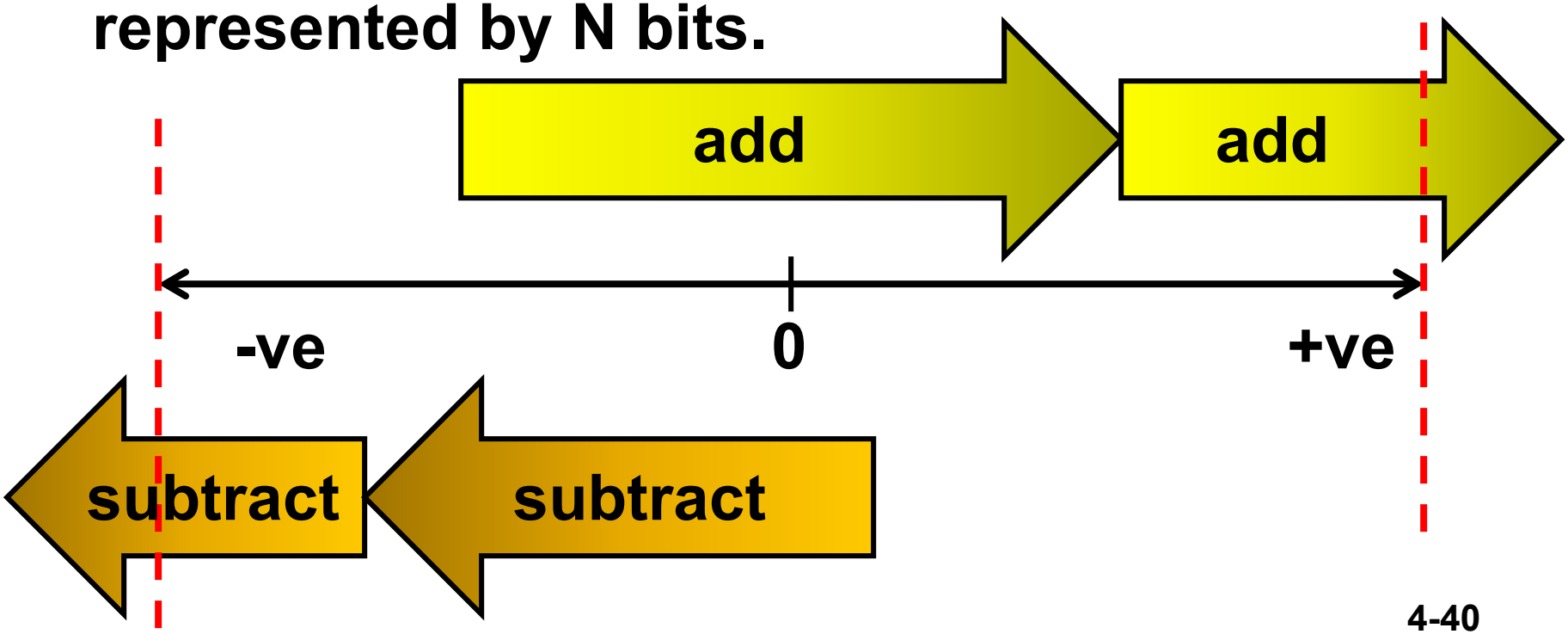
The pair of numbers to be added/subtracted must have the same size, i.e. same number of bits. This ensures that the sign bits are aligned. The resulting sum must also be of the same size.

Addition and Subtraction in the 2's complement system



Arithmetic Overflow

It occurs when an arithmetic operation between two N-bit operands produces a result that cannot be sufficiently represented by N bits.



Rules to detect overflow in 2's complement addition:

No overflow occurs if the operands have opposite signs. Overflow is detected when operands have the same sign, but the arithmetic sum has an opposite sign.

$$\begin{array}{r} {}^{1}0\ 1\ 0\ 1\ 0 \\ +\ 0\ 1\ 0\ 0\ 0 \\ \hline 1\ 0\ 0\ 1\ 0 \end{array}$$

10+8=-14?

For subtraction, no overflow occurs if the operands have the same sign.

L6 (4.42 – 4.58)

- Circuit for 2’s complement add/subtract
- Parallel addition with registers
- Multiplication
- BCD addition

Combined circuit for addition and subtraction

Fig 6-12 (X0 and Y0 are LSB)

- To perform the addition: X + Y
- Make inputs Add/Sub = 0, C0 = 0
- XOR gates do not invert Y
- Inputs to FAs are X and Y, with C0 = 0
- E.g. 0001 + 0011 = 0100
- i.e. 1 + 3 = 4 (in decimal)

Combined circuit for addition and subtraction

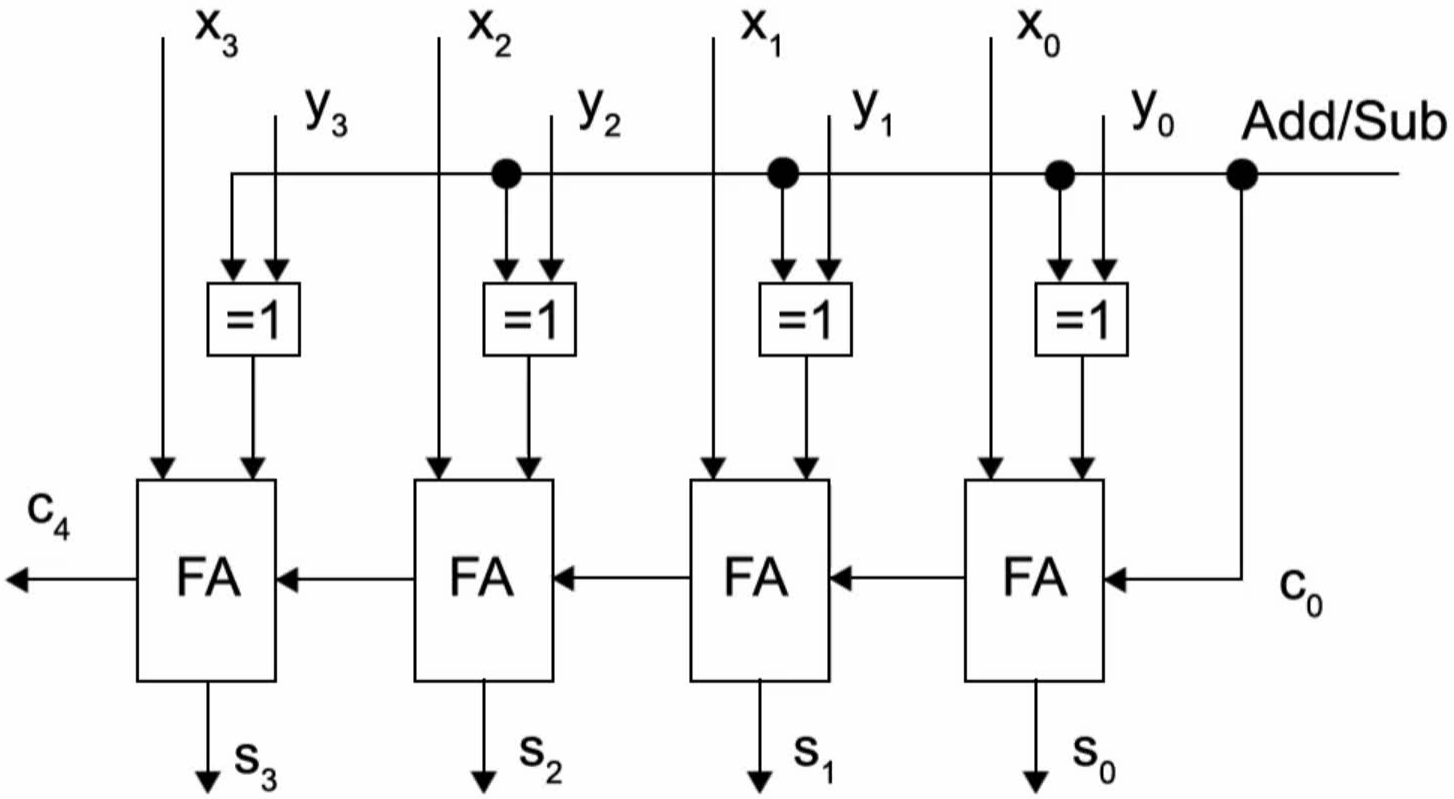


Fig 6-12 (add)

Fig 6-12 (subtraction)

- To perform the subtraction: X - Y
- Recall: -Y is 2's complement of Y
- Make inputs Add/Sub = 1, C0 = 1
- XOR gates invert Y to give Y'
- Inputs to FAs are X and Y', with C0 = 1
- E.g. 0001 - 0100 = 0001 + 1011 + 1 = 1101
- i.e. 1 - 4 = -3 (in decimal)

Combined circuit for addition and subtraction

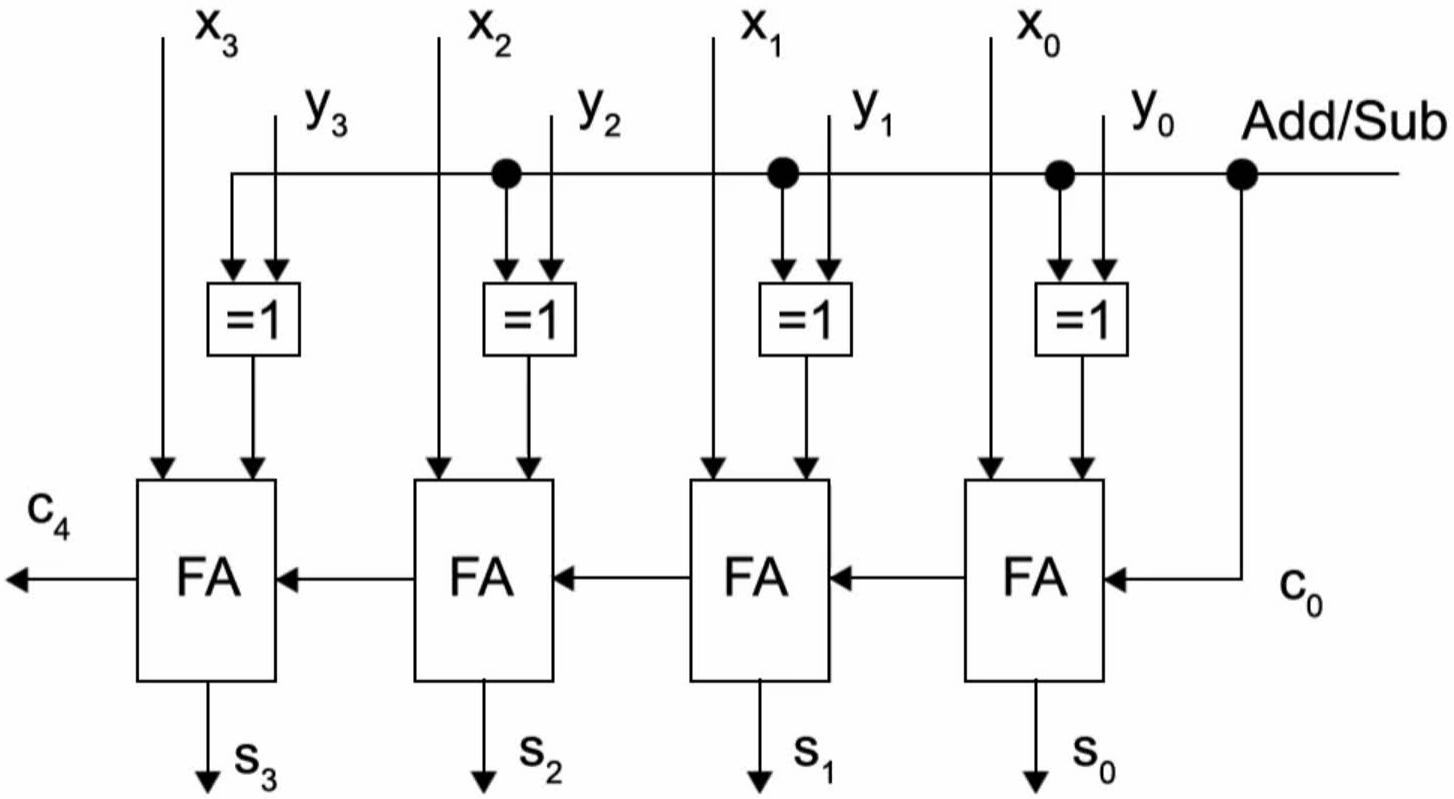


Fig 6-12 (subtract)

Fig. 6-9: Complete 4-bit Parallel Adder with Registers

t1: CLEAR* clears the contents of A register to 0's

t2: PGT of first LOAD pulse transfers operand X from memory into B register

t3: PGT of first TRANSFER pulse transfers FA output (=X) into A register

t4: PGT of second LOAD pulse transfers operand Y from memory into B register

t5: PGT of second TRANSFER pulse transfers FA output (=X+Y) into A register

- Note that sufficient time (between load and transfer) must be given to FAs to complete addition
- E.g. 0001 + 0010 = 0011

Thus A register holds the result of X+Y

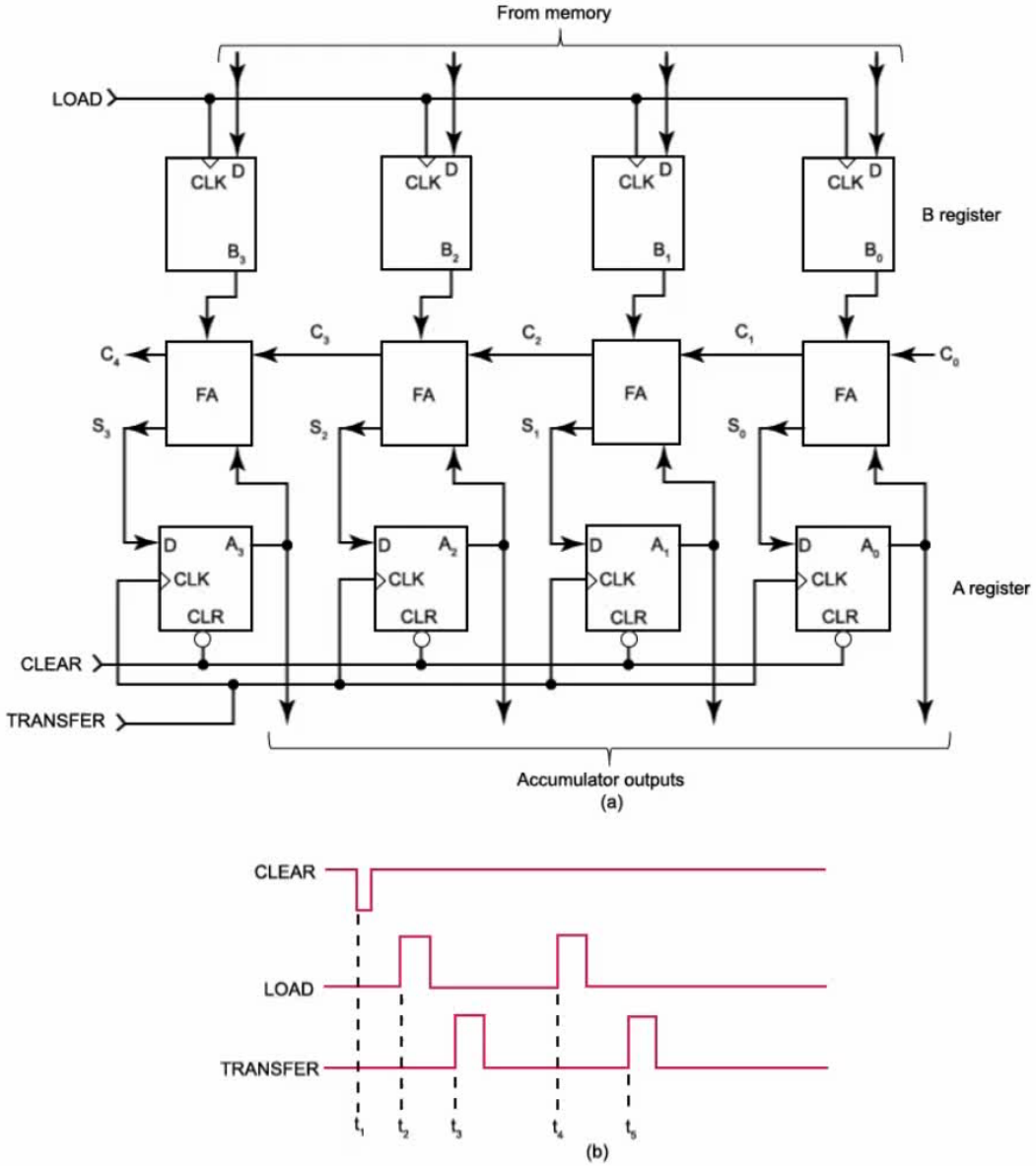

From memory
LOAD
CLK
D
B3
B2
B1
B0
B register
C4
C3
C2
C1
C0
FA
S3
S2
S1
S0
A3
A2
A1
A0
CLR
CLEAR
TRANSFER
A register
Accumulator outputs
(a)
LOAD
t1
t2
t3
t4
t5
(b)

Fig.6-9

Carry propagation can be reduced (hence increasing speed of addition) by a special-purposed logic circuit

- **carry-look-ahead circuit**

Many IC parallel adders have built-in carry look-ahead circuit to speed up the addition.

Binary Multiplication

Unsigned multiplication:

similar to decimal multiplication

```
   0111    (7)
x  1100    (12)
-------
```

2's complement multiplication:

- If the multiplier is positive, same as unsigned multiplication
- If the multiplier is negative, need to take care of negative weight of MSB (i.e. the sign bit)
- Treat multiplier as a sum of 2 parts:
  - MSB – negative part
  - Remaining bits – positive part

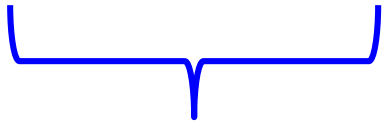
Example of 2's complement multiplication:

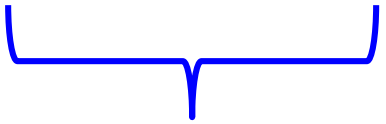
$$\begin{array}{rr} & -5 \\ \times & -3 \\ \hline & 15 \\ \hline \end{array}$$

Explanation:

$$1011 \times 1101 = 1011 \times (1000 + 0101)$$

$$= (1011 \times 1000) + (1011 \times 0101)$$





This is done by shift and 2's complement

This is done by shift and sign-extension

Binary Division

Unsigned division:

similar to the long division method in decimal arithmetic

$9 \div 3 = 3$

Signed division:

convert the signed numbers to unsigned, divide them as above, then convert the result using the appropriate sign representation

BCD addition

When the sum of two BCD digits does not exceed 9_{10}, the operation is the same as binary addition

If the sum of two BCD digits is more than 9_{10} , a correction needs to be made by adding 6 (0110) to skip over the six invalid codes

The correction involves two steps:

- **a carry of decimal value 1 is brought forward and added to the next higher digit**
- **the decimal value 6 is added to the sum to obtain the correct BCD digit**

Example:

$$24_{10} + 47_{10} = 71_{10}$$

L7 (5.1 – 5.23)

- Minterm and maxterm
- Canonical Boolean expressions
- SOP and POS expressions
- Active-high and active-low logic signals

5. Combinational Logic Circuits

- The most common type of digital logic circuits
- Made up of combinations of logic gates
- At any point in time, the output logic level only depends on the present combination of logic levels at the N inputs
    - $O(t) = f [I_1(t), I_2(t), I_3(t), \ldots, I_N(t)]$
- It has no memory characteristics, unlike sequential circuits
- Relatively easy to analyse and design compared to sequential circuits

Designing and Implementing a Combinational Logic Circuit

- The function of a required logic circuit can be fully described by a truth table
- To design the circuit, we obtain the Boolean expression from the truth table
- The Boolean expression can then be implemented using a proper choice of logic gates

A Boolean expression is also known as

- a Boolean equation
- a logic function

It fully describes, algebraically,
the logic circuit's output in response
to every possible input condition.

For simple circuits, the expression can usually be obtained by observation.

Forms of Boolean Expressions

- **Canonical Form**
  - **Sum of minterms expression (SOm)**
  - **Product of maxterms expression (POM)**
- **Standard Form**
  - **Sum of products expression (SOP)**
  - **Product of sums expression (POS)**

- **minterms**:
  - All possible combinations of a given set of Boolean variables formed by the **AND** operation
- **maxterms**:
  - All possible combinations of a given set of Boolean variables formed by the **OR** operation

A logic circuit with 2 inputs X and Y will have these 4 minterms and maxterms:

inputs		minterms		maxterms	
X	Y				
0	0	X' • Y'	m0	X + Y	M0
0	1	X' • Y	m1	X + Y'	M1
1	0	X • Y'	m2	X' + Y	M2
1	1	X • Y	m3	X' + Y'	M3

A minterm or maxterm uniquely describes the input combination at a given time instant

A logic circuit with 3 inputs X, Y and Z will have these 8 minterms and maxterms:

inputs			minterms		maxterms	
X	Y	Z				
0	0	0	X' • Y' • Z'	m0	X + Y + Z	M0
0	0	1	X' • Y' • Z	m1	X + Y + Z'	M1
0	1	0	X' • Y • Z'	m2	X + Y' + Z	M2
0	1	1	X' • Y • Z	m3	X + Y' + Z'	M3
1	0	0	X • Y' • Z'	m4	X' + Y + Z	M4
1	0	1	X • Y' • Z	m5	X' + Y + Z'	M5
1	1	0	X • Y • Z'	m6	X' + Y' + Z	M6
1	1	1	X • Y • Z	m7	X' + Y' + Z'	M7

For N-inputs, there will be 2^N minterms

e.g. 4 inputs: a, b, c, d

- 13 in decimal = 1101 in binary
- Then minterm m13 = a b c' d
- maxterm M13 = a' + b' + c + d'
- 2 in decimal = 0010 in binary
- Then minterm m2 = a' b' c d'
- maxterm M2 = a + b + c' + d

- **Minterms are formed such that given a set of input conditions, only the corresponding minterm (but not the other minterms) will yield a logic 1.**

- **Eg. x=0, y=1, z=1 the corresponding minterm is m3, i.e. x'yz**

- **Substituting the values of x, y and z will result in m3 = x'yz = 0' • 1 • 1 = 1**

- **Notice that by arranging x, y, z together to form a 3-bit binary number (MSB=x, LSB=z), the decimal equivalent is used to denote the minterm number ($011_2 = 3_{10}$ in this example)**

- **Maxterms** are formed such that given a set of input conditions, only the corresponding maxterm (but not the other maxterms) will **yield a logic 0**.

- eg. x=1, y=0, z=1 the corresponding maxterm is M5, i.e. x' + y + z'

- Substituting the values of x, y and z will result in M5 = x' + y + z' = 1' + 0 + 1' = 0

- Notice that arranging x, y, z together to form a 3-bit binary number (MSB=x, LSB=z), the decimal equivalent is used to denote the maxterm number ($101_2 = 5_{10}$ in this example)

The Sum of minterms expression

To write the sum-of-minterms Boolean expression from a truth table:

- **For each combination of the input variables that produces a logic 1 in the output, collect the corresponding minterms and OR them together**

The Product of maxterms Expression

- For each combination of the input variables that produces a logic 0 in the output, collect the corresponding maxterms and AND them together
- Conversion between the two forms is easy.
- Sum of minterms expression is associated with active HIGH output.
- Product of maxterms expression is associated with active LOW output.

Example: given the truth table, obtain the SOm and POM expressions for output F

inputs			Output F
X	Y	Z	
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

inputs			minterms		maxterms		F
X	Y	Z					
0	0	0	X' • Y' • Z'	m0	X + Y + Z	M0	0
0	0	1	X' • Y' • Z	m1	X + Y + Z'	M1	1
0	1	0	X' • Y • Z'	m2	X + Y' + Z	M2	1
0	1	1	X' • Y • Z	m3	X + Y' + Z'	M3	0
1	0	0	X • Y' • Z'	m4	X' + Y + Z	M4	1
1	0	1	X • Y' • Z	m5	X' + Y + Z'	M5	0
1	1	0	X • Y • Z'	m6	X' + Y' + Z	M6	0
1	1	1	X • Y • Z	m7	X' + Y' + Z'	M7	1

$F = X'Y'Z + X'YZ' + XY'Z' + XYZ$

$= \Sigma_{XYZ}(1, 2, 4, 7)$

SOm

$F = (X+Y+Z)(X+Y'+Z')(X'+Y+Z')(X'+Y'+Z)$

$= \pi_{XYZ}(0, 3, 5, 6)$

POM

Other ways of writing canonical expressions:

$F = \sum_{XYZ} (1, 2, 4, 7)$ SOm

$F(X,Y,Z) = \sum m\ (1, 2, 4, 7)$

$F(X,Y,Z) = m1 + m2 + m4 + m7$

$F = \Pi_{XYZ} (0, 3, 5, 6)$ POM

$F(X,Y,Z) = \Pi M\ (0, 3, 5, 6)$

$F(X,Y,Z) = M0 \bullet M3 \bullet M5 \bullet M6$

Interpretation of active High and active Low for the above example:

Value of Output F	Interpretation of output F	
	There is an **odd** number of 1's among the 3 inputs X, Y, Z	There is an **even** number of 1's among the 3 inputs X, Y, Z
1 (High)	**TRUE**	FALSE
0 (Low)	FALSE	**TRUE**

- **E.g. rename F as ODD, which is active High** (usually write Som expression)
- **E.g. rename F as EVEN*, which is active Low** (usually write PoM expression)

Standard Form of Boolean Expressions

- SOP and POS
- Simplified expressions from the canonical forms
- Leads to simpler logic circuits
- Known as combinational circuit minimisation
- Minimise the number of gates (minumum number of product terms or sum terms)
- Minimise the number of inputs on each gate (minimum number of input variables in each product term and sum term)

Sum of products (SOP) expression

example:

This is a sum-of-minterms expression:

$$f(x, y, z) = xyz' + xyz + x'y'z + xy'z$$

Simplifying, we get

$$f(x, y, z) = xy\,(z' + z) + (x' + x)\,y'z$$
$$= xy + y'z$$

This is now a sum-of-products expression.

A product term need not contain all the input variables, unlike a minterm.

Product of sums (POS) expression

example:
This is a product-of-maxterms expression:

f(x, y, z) = (x+y'+z')(x+y'+z)(x'+y'+z)(x'+y+z)

Simplifying, we get

f(x, y, z) = (x + y')(x' + z)

This is now a product-of-sums expression.

A sum term need not contain all the input variables, unlike a maxterm.

These are neither SOP nor POS expressions:

$f = (xy)'z + xz'$ not a product term

$f = xy(x' + z)'$ not a sum term

$f = (xy + z)(x' + y)$ not a sum term

Use the standard form (i.e. SOP or POS) wherever possible.

Obtaining Simplified Standard Expressions from the Canonical Form

- **Different methods for Boolean expression simplification**
  - **Algebraic method**
  - **Karnaugh map (K-map)**
  - **Quine-McCluskey method (Q-M method or tabulation method)**
  - **Heuristic methods, e.g. Espresso-II**

Simplified Boolean expressions yield

- Simpler circuits with fewer logic gates
- Fewer connections
- Lower cost
- Improved reliability

Algebraic method

- Use Boolean theorems
- Requires experience and skills

Examples:

simplify $Z = ABC + AB'(A'C')'$



$Z = AB' + AC$

simplify $X = (A' + B)(A + B + D)D'$



$X = BD'$

L8 (5.24 – 5.48)

- Karnaugh map to simplify Boolean expression
- What to do with “Don’t cares”
- How to enable or disable a circuit

Karnaugh Map

- Graphical method
- Easier to use than algebraic method
- Based on the Boolean theorems

  AB + AB' = A(B + B') = A (for SOP)

  (A+B)(A+B') = A (for POS)

- Truth table gives value of output X for each combination of input values. K-map gives the same info in a different format.

- K-map squares are labelled such that adjacent squares differ only in one variable.
- SOP expression for output X can be obtained by ORing together those squares that contain a 1.
- Can also obtain POS expression by ANDing together those squares that contain a 0.
- Note the correspondence with SOm (think 1) and POM (think 0).

Truth Table to K-Map Conversion [2 Inputs]

$X = A'B' + AB$

A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

K-map

X	B=0	B=1
A=0	1	0
A=1	0	1

Truth Table to K-Map Conversion [3 Inputs]

$$X = A'B'C' + A'B'C + A'BC' + ABC'$$

A	B	C	X
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

K-map

X	C=0	C=1
A=0,B=0	1	1
A=0,B=1	1	0
A=1,B=1	1	0
A=1,B=0	0	0

Truth Table to K Map Conversion [4 Inputs]

A	B	C	D	X
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

$$X = A'B'C'D + A'BC'D + ABC'D + ABCD$$

X	C, D: 0 0	0 1	1 1	1 0
A, B: 0 0	0	1	0	0
0 1	0	1	0	0
1 1	0	1	1	0
1 0	0	0	0	0

Truth Table to K Map Conversion [4 Inputs]

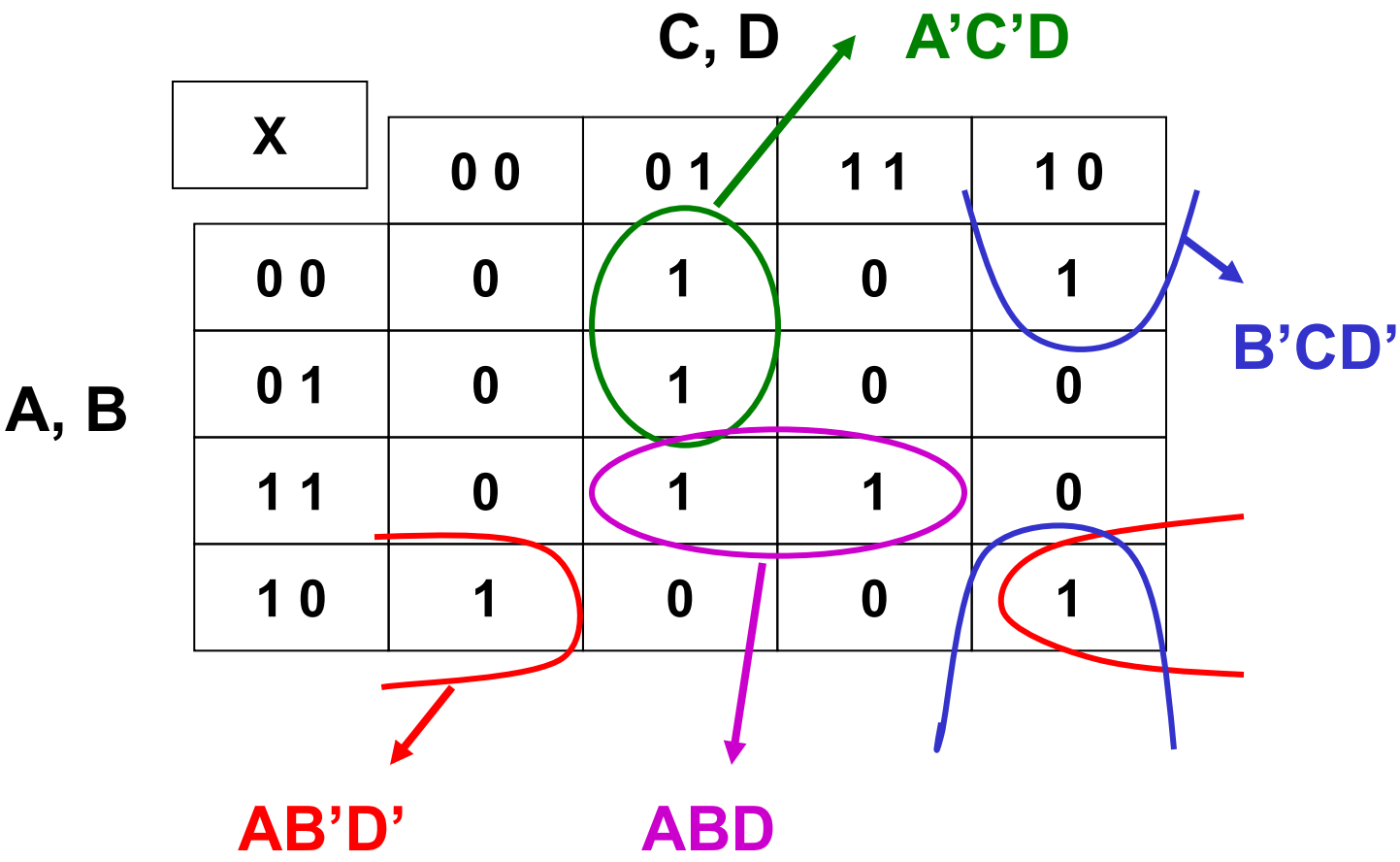
A	B	C	D	dec
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	10
1	0	1	1	11
1	1	0	0	12
1	1	0	1	13
1	1	1	0	14
1	1	1	1	15

C, D

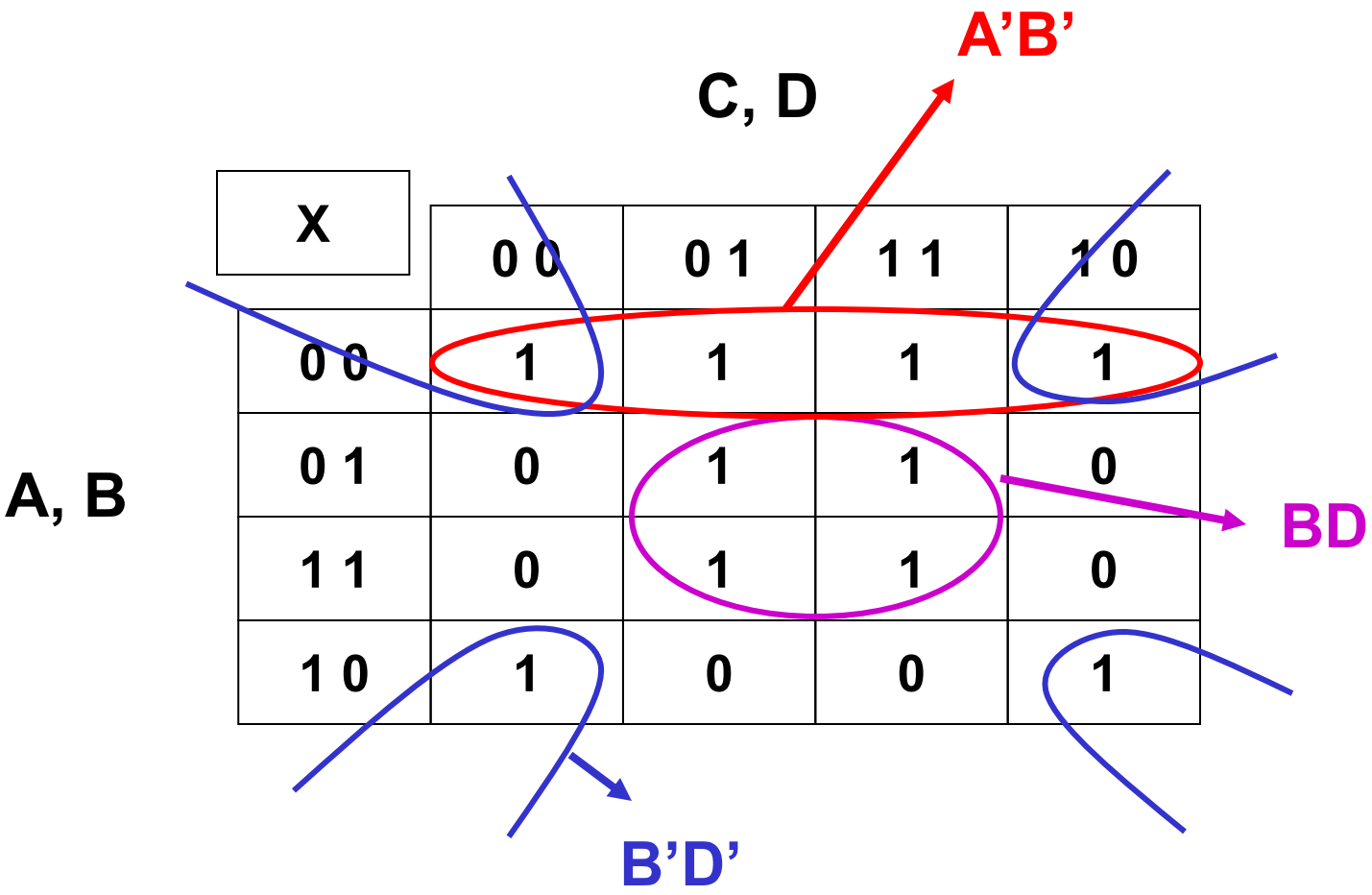
X	0 0	0 1	1 1	1 0
0 0	0	1	3	2
0 1	4	5	7	6
1 1	12	13	15	14
1 0	8	9	11	10

A, B

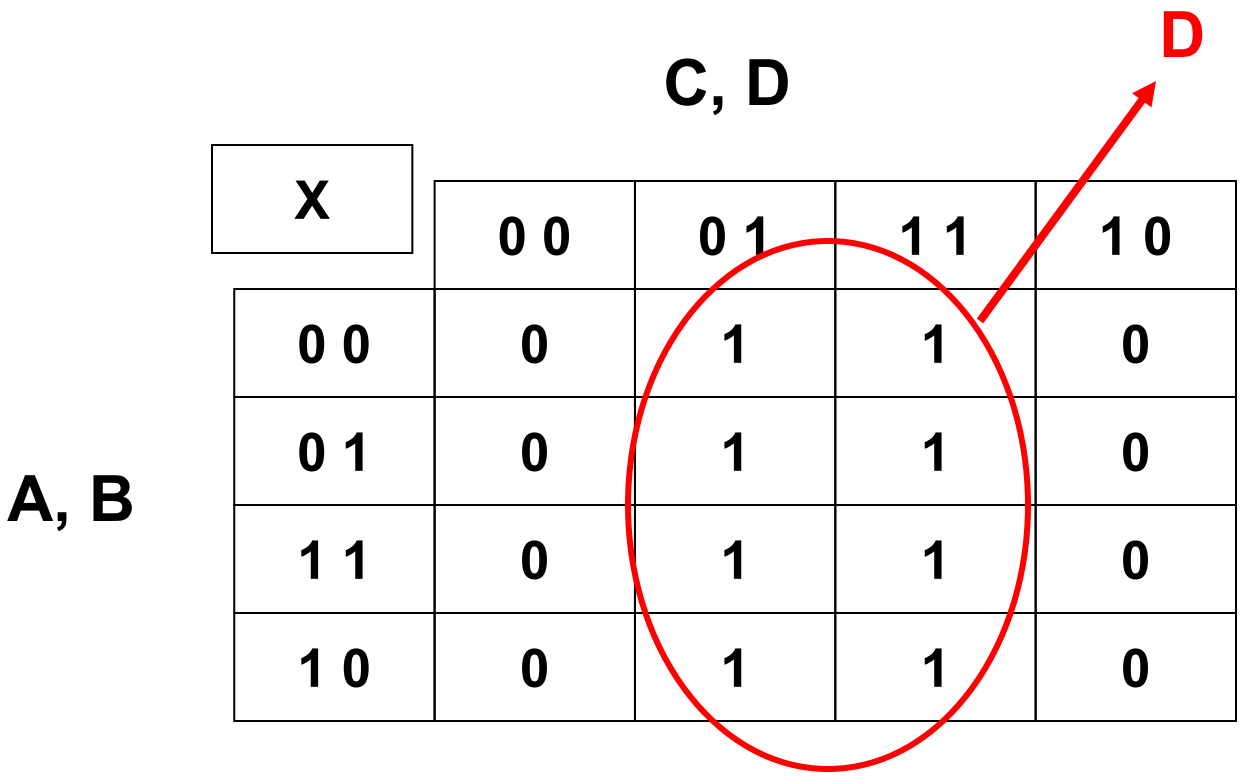
Kmap: Looping 2 “neighbouring” 1’s



Kmap: Looping 4 “neighbouring” 1’s



Kmap: Looping 8 “neighbouring” 1’s



Note these rules on Kmap for simplification:

- Loop 1's to obtain SOP expression.
- Only 2^N number of "neighbouring" 1's can be looped together.
- No looping along diagonal.
- All 1's must be looped.
- Use the biggest loops and the fewest loops.
- A square(s) may be looped more than once.
- Each loop of 1's will yield a product term.

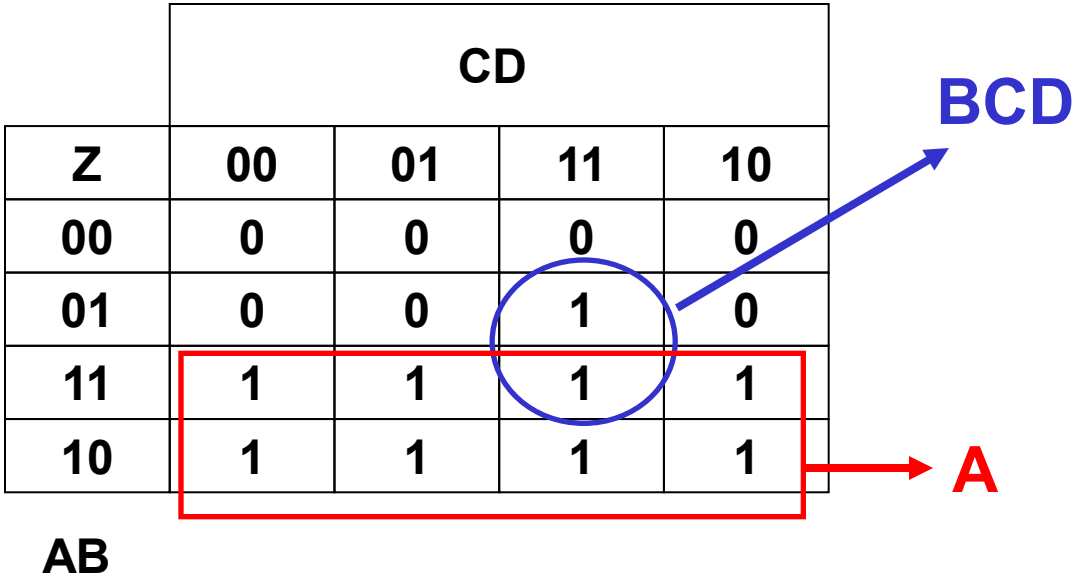
- 0's can also be looped in a similar manner to obtain POS expression.
- Each loop of 0's will yield a sum term.

Example: Simplify the Boolean expression for output Z using K-map

A	B	C	D	Z
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Example (cont)

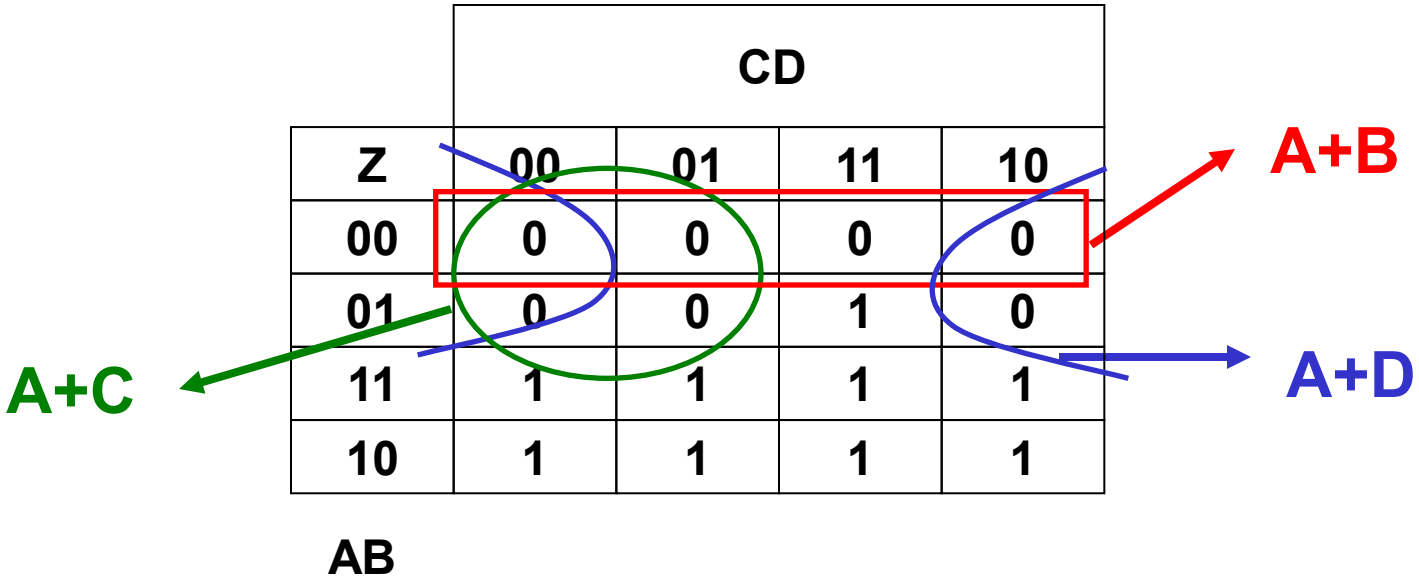
Step 1 : Convert truth table to K-map
Step 2 : Loop adjacent 1's to get SOP



$$Z = A + BCD$$

Example (cont)

Alternatively
Step 2 : Loop adjacent 0's to get POS



$$Z = (A + B)(A+C)(A+D)$$

Don’t care conditions

- Some logic circuits can be designed such that there are certain input conditions for which there are no specified output levels.

- This is possible because
  - These input conditions will not occur
  - It does not matter whether the output is 0 or 1

- The designer is free to make the output for any “don’t care” condition to be 0 or 1 in order to produce the simplest output expression.

Example: Design a logic circuit whose input is a BCD digit, and whose output goes HIGH if the input is smaller than 6_{10}

A	B	C	D	Z
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

X denotes “don’t care”

Example (cont):

Design 1: SOP

$$Z = A'B' + A'C'$$

Z	CD 00	01	11	10
00	1	1	1	1
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

AB

X denotes "don't care"

Example (cont):

Design 2: SOP

$$Z = A'B' + BC'$$

	CD			
Z	**00**	**01**	**11**	**10**
00	1	1	1	1
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

AB

X denotes "don't care"

Example (cont):

Design 3: SOP

$$Z = B'C + A'C'$$

	CD			
Z	00	01	11	10
00	1	1	1	1
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

AB

X denotes “don’t care”

Example (cont):

Design 4: POS

$$Z = (A'+B+C)(A+B'+C')$$

	CD			
Z	00	01	11	10
00	1	1	1	1
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

AB

X denotes “don’t care”

Example (cont):

Design 5: POS

$$Z = A' (B'+C')$$

	CD			
Z	00	01	11	10
00	1	1	1	1
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

AB

X denotes “don’t care”

Don’t cares

- “Don’t cares” can be looped in a similar way on the Karnaugh map.
- When looped with 1’s to write SOP expression, the “don’t care” is treated as 1. Those “don’t cares” not looped are treated as 0.
- Conversely, when looped with 0’s to write POS expression, the “don’t care” is treated as 0. Those “don’t cares” not looped are treated as 1.
- “Don’t cares” should only be looped if it helps to simplify a Boolean expression (i.e. helps to form a bigger loop).

Summary: Designing a Combinational Logic Circuit

1. **From problem specifications, derive the relationship between the output(s) and the inputs. This can be expressed in the form of a truth table.**

2. **Obtain the Boolean expression that relates the desired output to the inputs. It can either be in SOP or POS form, although SOP is usually used.**

3. **Simplify the expression using either algebraic, K-map or QM method.**

4. **Implement the circuit from the simplified expression. Certain restrictions may need to be taken into account, eg. use only 2-input NAND gates.**

Enable/Disable Circuits

Enable

A logic circuit is said to be enabled if the output is allowed to change in response to changes in the inputs.

Disable/Inhibit

A logic circuit is said to be disabled/inhibited if the output is not allowed to change in response to changes in the inputs.
The output is either fixed at 0 (typically for active High output) or 1 (typically for active Low output).

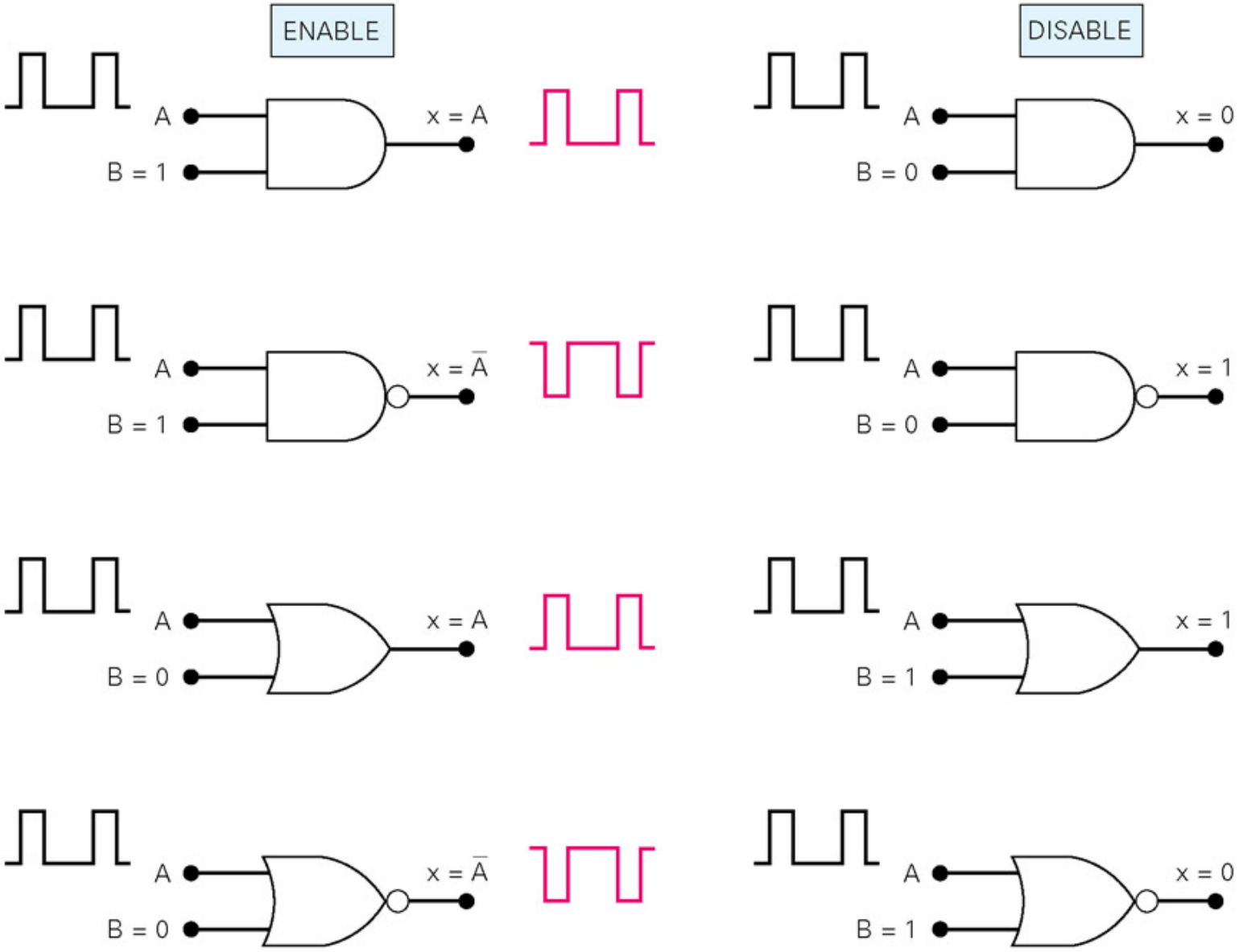


Fig. 4-26 (Tocci 10th ed.)

Enable/Disable Example

Inputs		Output	Effect of output
Child Safe	Unlock	Open	
0	X	0	Door closed
1	0	0	Door closed
1	1	1	Door opened

X: “don’t care”, i.e. can be 0 or 1

- **Circuit is enabled when ChildSafe=1; the output Open changes with input Unlock.**
- **Circuit is disabled when ChildSafe=0; Open is stuck at 0.**

L9 (6.1 – 6.22)

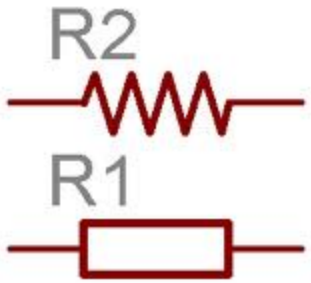
- Difference between TTL & CMOS
- Transistor on/off
- Active-high, active-low, asserted, negated

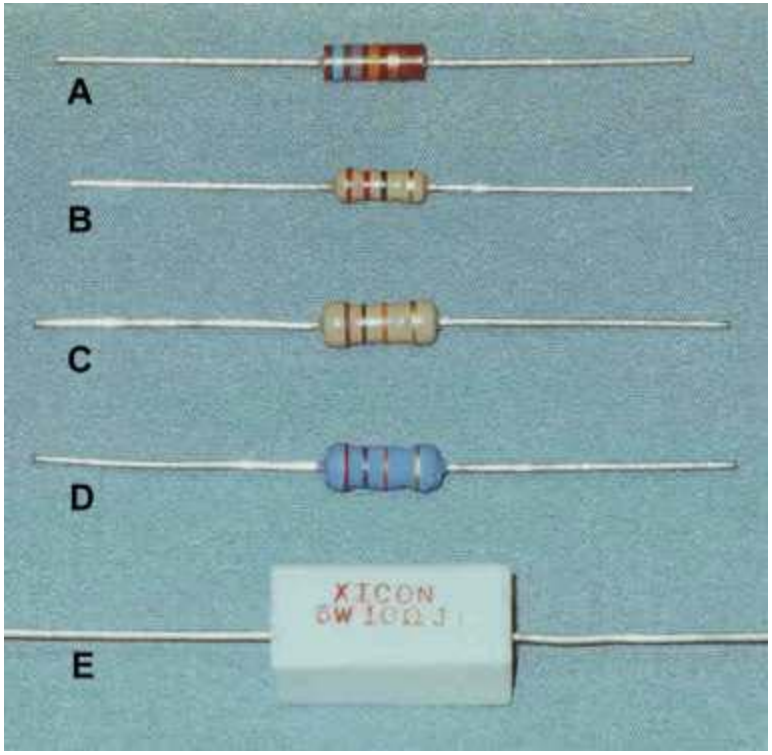
6. Digital Circuits

Integrated digital circuits are

- **made up of a collection of resistors, diodes and transistors**
- **fabricated on a single piece of semiconductor material (e.g. silicon) called substrate - also called die, or chip**

Discrete components: Resistors





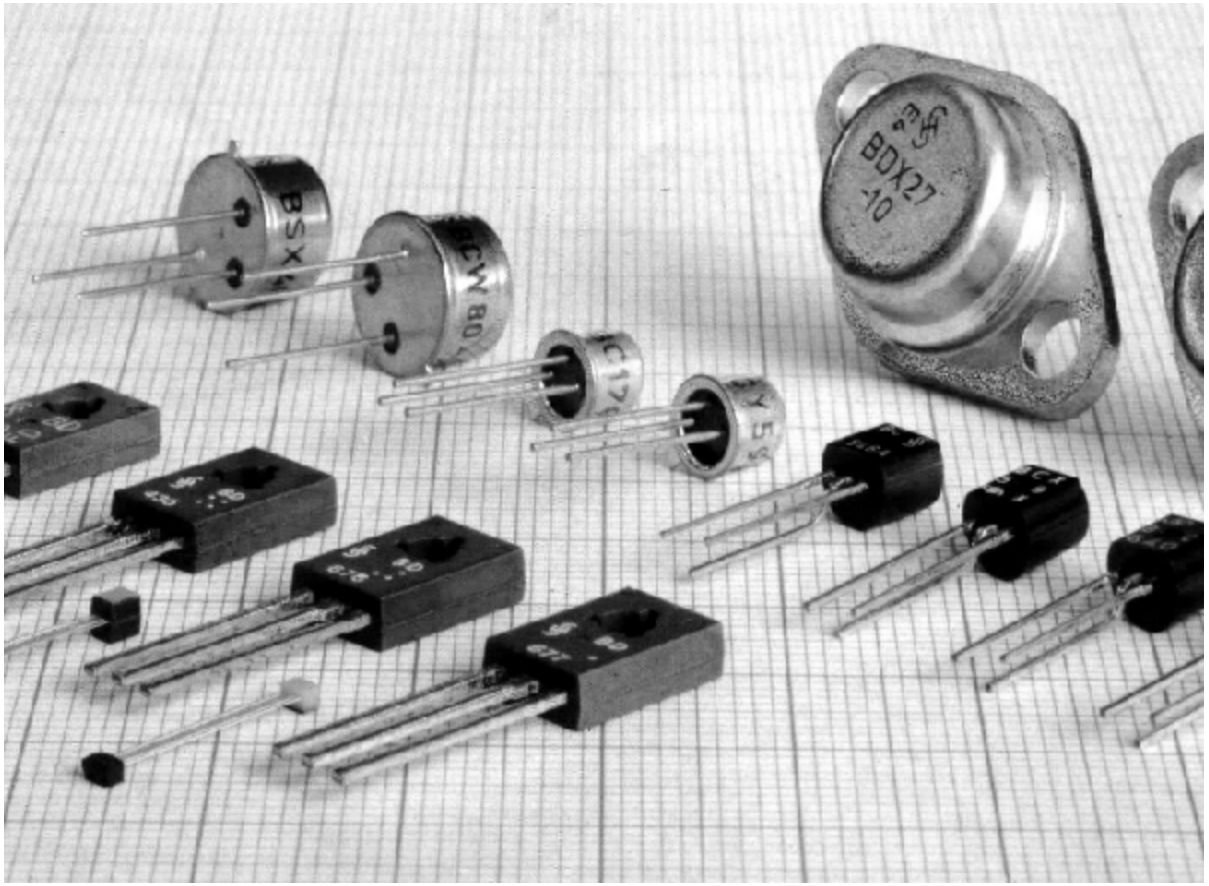
Discrete components: Diodes



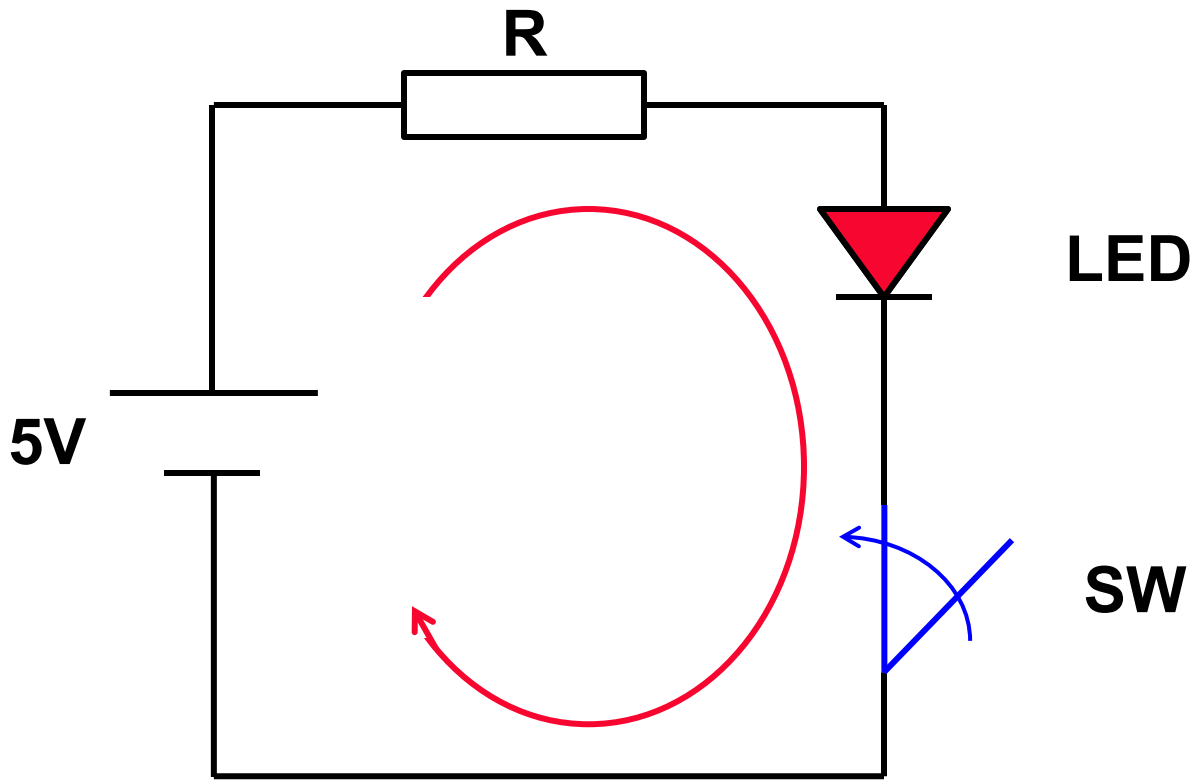




Discrete components: transistors



Example of a simple circuit



Technology	State Representing Bit	
	0	1
Pneumatic logic	Fluid at low pressure	Fluid at high pressure
Relay logic	Circuit open	Circuit closed
Complementary metal-oxide semiconductor (CMOS) logic	0–1.5 V	3 .5–5.0 V
Transistor-transistor logic (TTL)	0–0.8 V	2 .0–5.0 V
Dynamic memory	Capacitor discharged	Capacitor charged
Nonvolatile, erasable memory	Electrons trapped	Electrons released
Microprocessor on-chip serial number	Fuse blown	Fuse intact
Polymer memory	Molecule in state A	Molecule in state B
Fiber optics	Light off	Light on
Magnetic disk or tape	Flux direction “north”	Flux direction “south”
Compact disc (CD)	No pit	Pit
Writeable compact disc (CD-R)	Dye in crystalline state	Dye in noncrystalline state

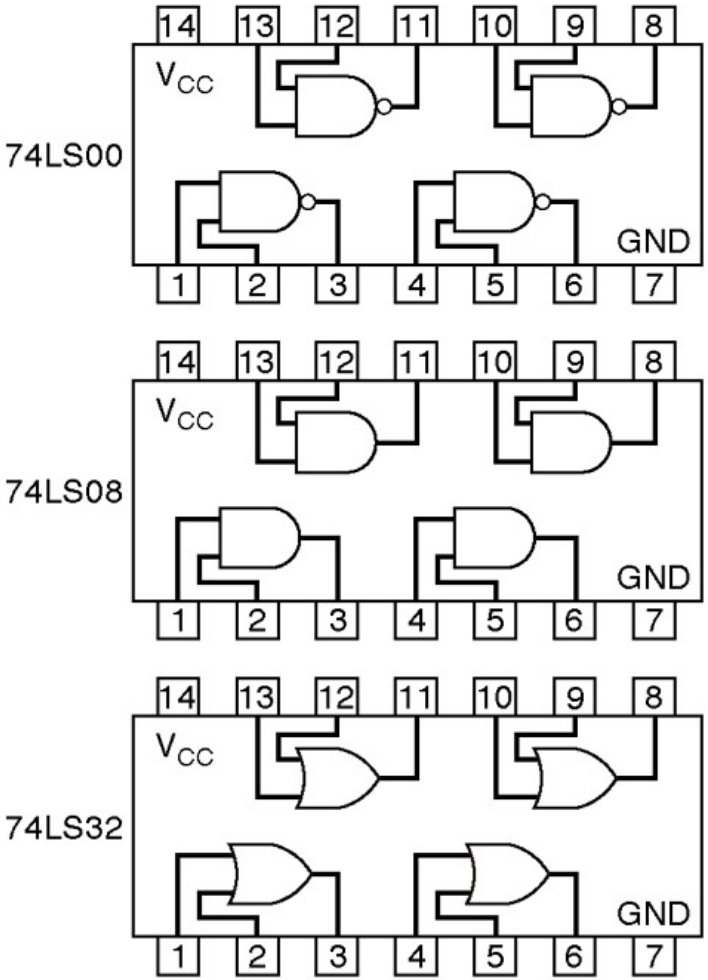
Table 3-1

Physical states representing bits in different logic and memory technologies.

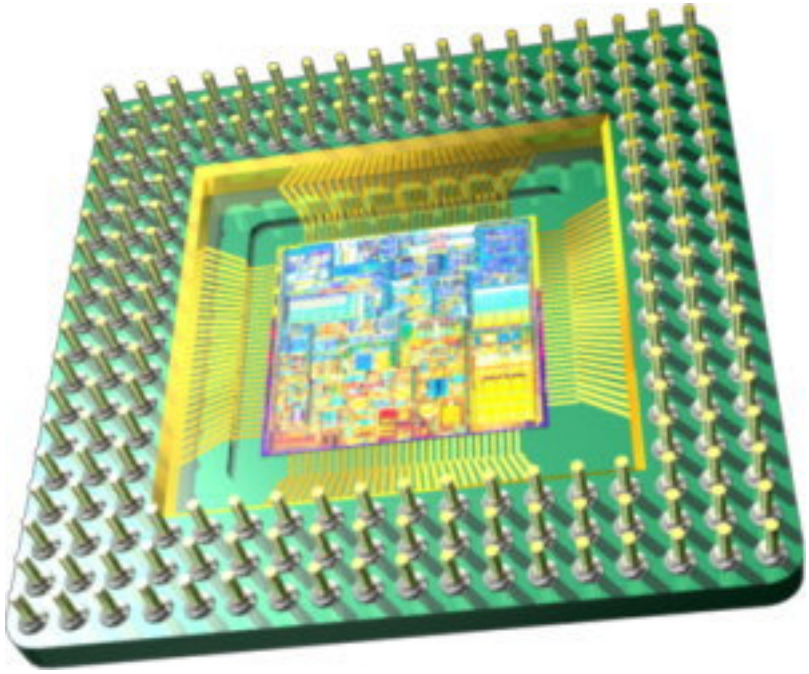
Scale of integration

Complexity	No. of gates
Small-scale (SSI)	< 12
Medium-scale (MSI)	12 - 99
Large-scale (LSI)	100 - 9999
Very large-scale (VLSI)	10,000 – 99,999
Ultra large-scale (ULSI)	100,000 – 999,999
Giga-scale (GSI)	≥ 1,000,000

Examples of SSI logic devices



Example of GSI device:
A microprocessor, MPU, CPU



To make logic outputs switch between High and Low (1 and 0), transistors are used.

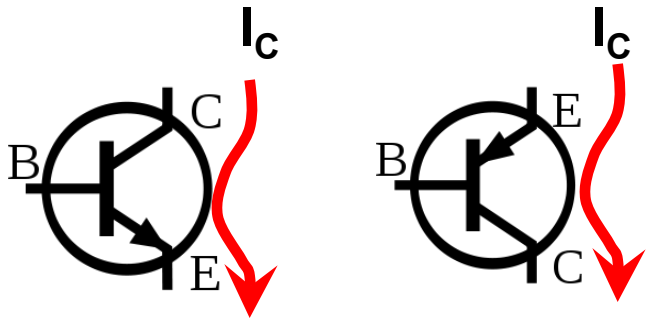
2 broad families:

- **use bipolar junction transistors**
- **e.g. TTL (transistor-transistor logic) circuits**

- **use unipolar field-effect transistors**
- **e.g. CMOS (complementary metal-oxide semiconductor) circuits**

Bipolar junction transistors (BJT)

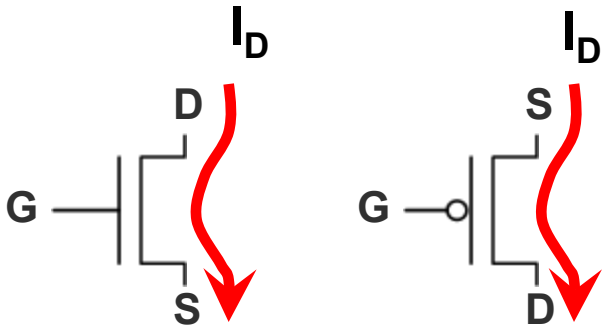
- Base, emitter, collector
- With correct voltage at B, current will flow between C and E



npn pnp

MOS Field-effect transistors (MOSFET)

- Gate, drain, source
- With correct voltage at G, current will flow between S and D



NMOS PMOS

TTL family

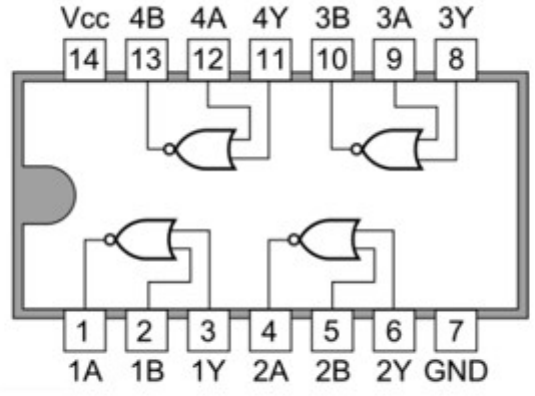
- different prefixes
- different in electrical characteristics such as power dissipation, delay time, switching speed
- do not differ in pin layout or logic function

- examples (hex inverters)

TTL series	Prefix	E.g.
Standard	74	7404
High-speed	74H	74H04
Low-power	74L	74L04
Advanced low-power Schottky	74ALS	74ALS04

CMOS family

- old series not compatible with TTL
- HC series pin-compatible with TTL: share same pin layout



examples (quad 2-input NOR)

CMOS series	Prefix	Example
Old (seldom used)	**40/140**	**4001** **14001**
Metal gate	**74C**	**74C02**
High speed	**74HC**	**74HC02**
Electrically compatible with TTL	**74HCT**	**74HCT02**

Logic-level Voltage Ranges

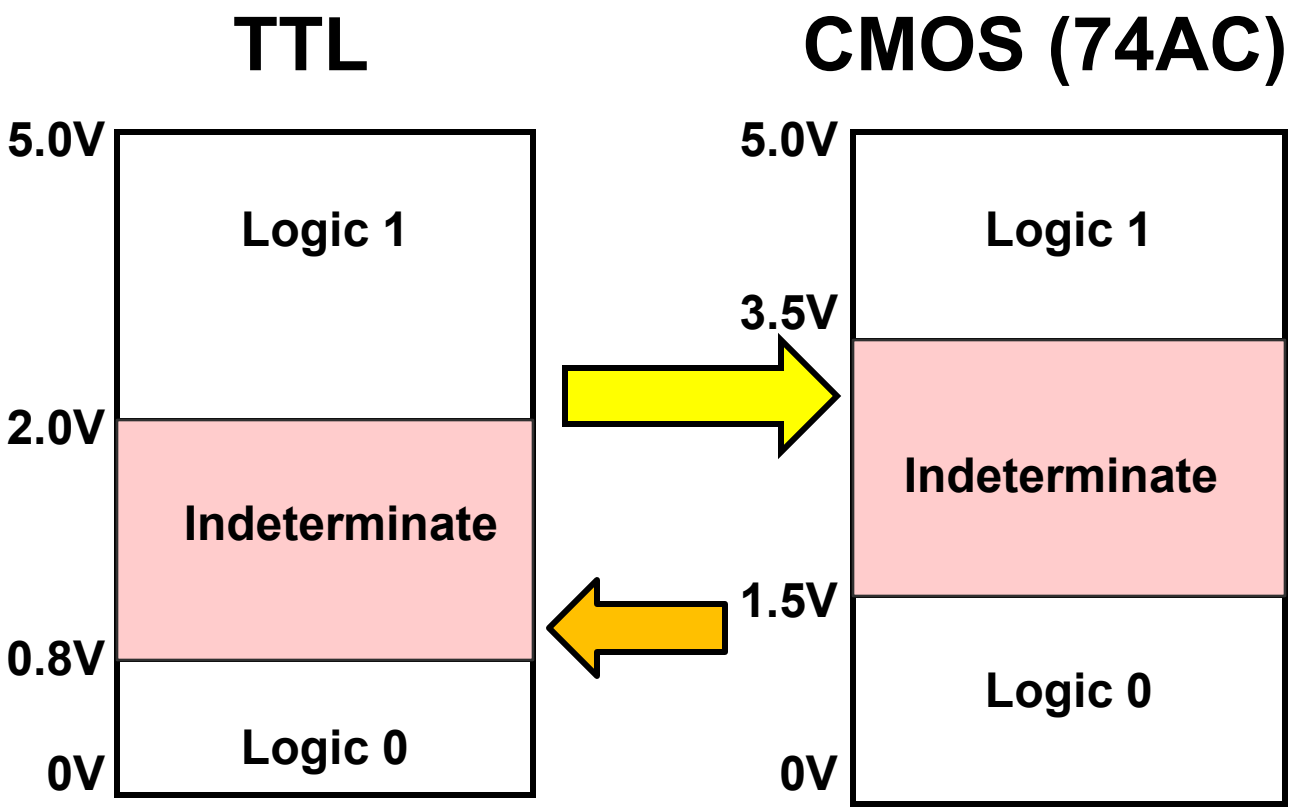
TTL

- **Vcc nominally +5v**

CMOS

- **V_{DD} ranges from +3 to +18v**
- **+5v is most often used when CMOS ICs are used in same circuit with TTL ICs**

Incompatible voltage ranges



Unconnected (floating) inputs

TTL

- **floating input acts like logic 1**
- **measures dc level between 1.4 to 1.8 volts**
- **not recommended due to noise pick-up**

CMOS

- **disastrous result**
- **IC may become overheated**
- **all unused input pins must be connected to V_{DD}, GND, or another input**

Active levels

- Recall 5.16: ODD and EVEN* example

A signal is said to be **active high** if	it produces the named effect or result when it is logic 1 **TRUE (asserted) when it is 1**
A signal is said to be **active low** if	it produces the named effect or result when it is logic 0 **TRUE (asserted) when it is 0**

Most signals that we encounter are active high

Active levels (examples)

Signal name	Effect/result when signal=0	Effect/result when signal=1	Active high/low
Subtract	Add Y to X	**Subtract** Y from X	Active high
Add*	**Add** Y to X	Subtract Y from X	Active low
Unmute	Mute a speaker	**Unmute** a speaker	Active high
Unmute*	**Unmute** a speaker	Mute a speaker	Active low
Read	Write data	**Read** data	Active high
Write*	**Write** data	Read data	Active low

Active high or active low depends on the signal name and effect/result

Active Low	*Active High*
READY–	READY+
ERROR.L	ERROR.H
ADDR15(L)	ADDR15(H)
RESET*	RESET
ENABLE~	ENABLE
~GO	GO
/RECEIVE	RECEIVE
TRANSMIT_L	TRANSMIT

RESET*

RESET

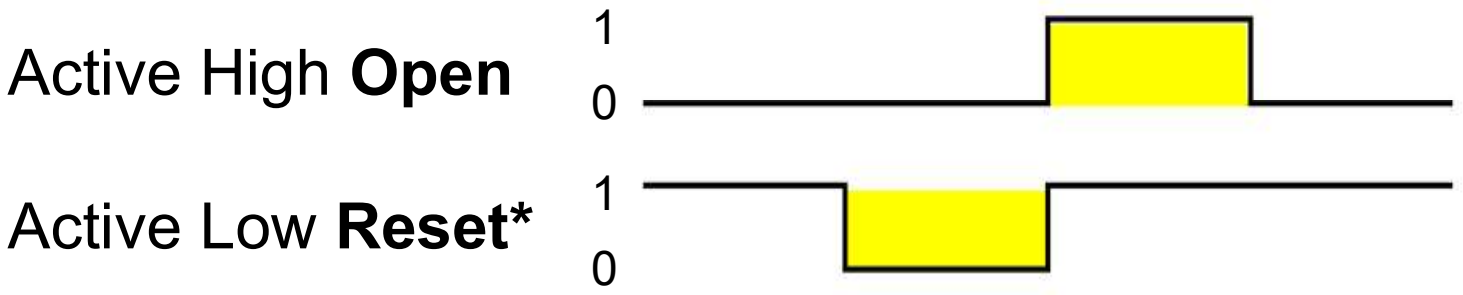
Each row shows a different naming convention for active levels.

Active low signals, being less common, are usually highlighted to draw our attention.

Asserted and negated

Asserted: when an active high signal is =1; or
when an active low signal is =0;
Negated: otherwise

Example: the timing diagram below shows that the signals are **asserted** for a while and negated (or de-asserted) for the rest of the time



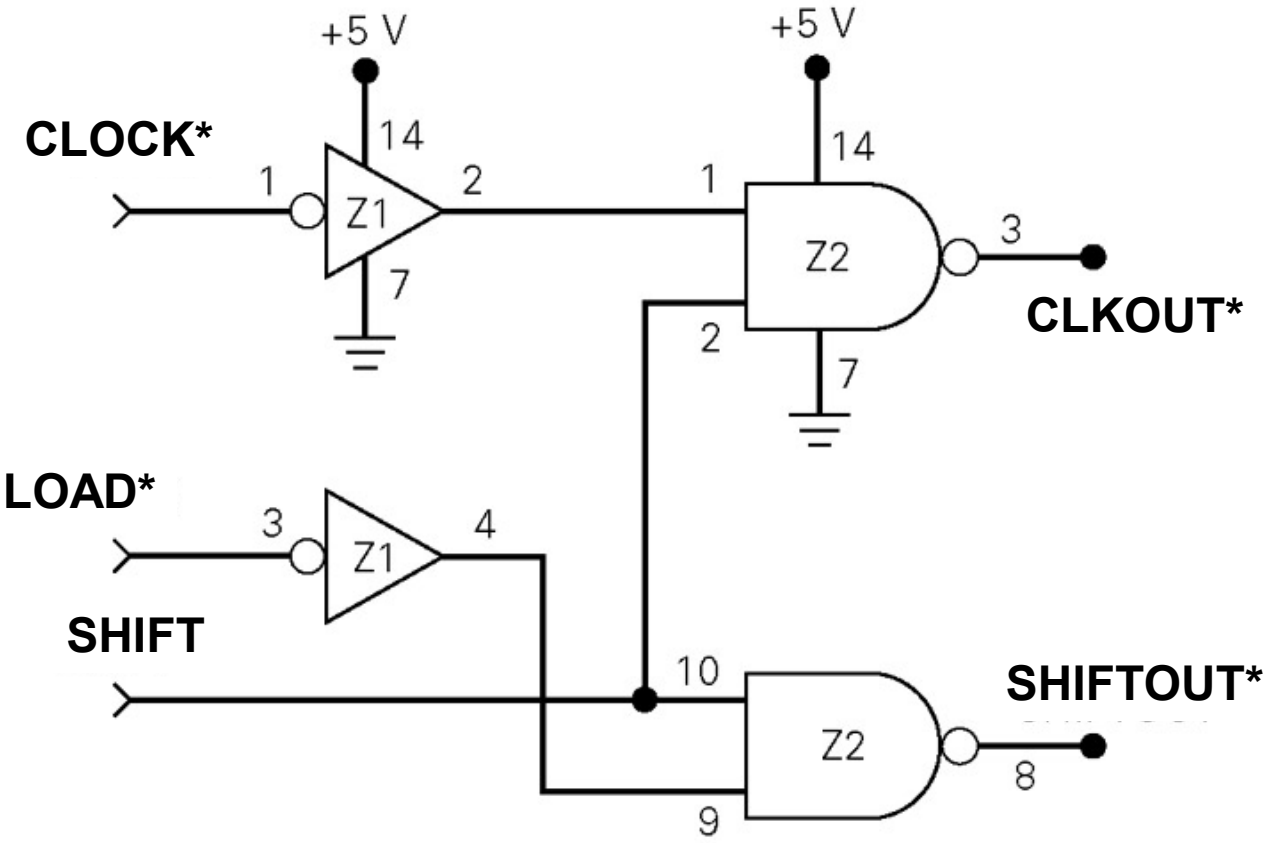
L10 (6.23 – 6.39)

- Bubble-to-bubble matching in logic circuit diagrams
- Functional analysis of simple CMOS logic circuits

Logic circuit connection diagrams

Fig. 4-32 contains more information than the usual circuit diagram to facilitate circuit connection and troubleshooting

- **pin numbers, IC numbers, component values, signal names and power supply voltages are clearly indicated**
- **conform to bubble-to-bubble design rules**
- **active levels are clearly indicated**



IC	Type
Z1	74HC04 hex inverter
Z2	74HC00 quad nand

Fig. 4-32 Typical logic-circuit connection diagram

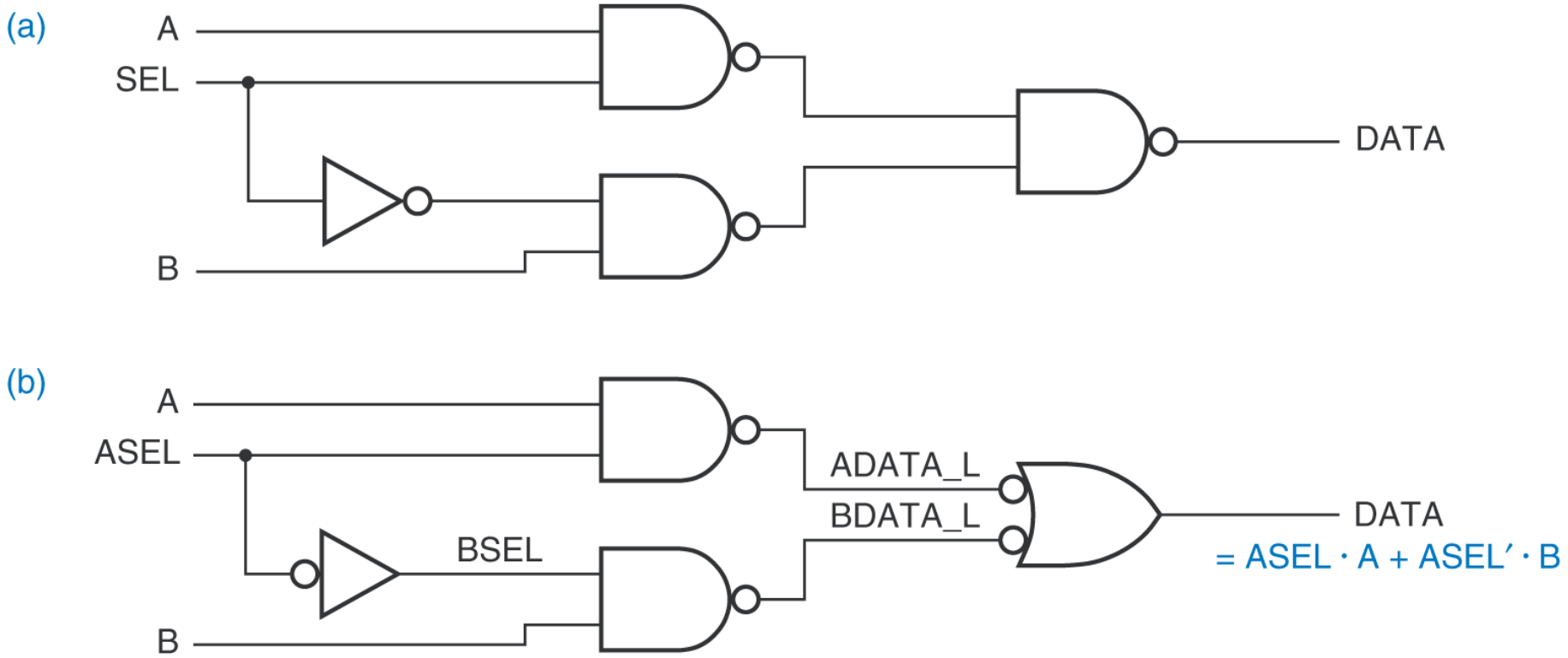


Figure 6-11

A 2-input multiplexer (you're not expected to know what that is yet): (a) cryptic logic diagram; (b) proper logic diagram using active-level designators and alternate logic symbols.

Fig. 6-11 Bubble-to-bubble logic diagram (Wakerly, 4th ed)

Troubleshooting Digital Systems

Read supplementary lab manual

- **Fault detection**
- **Fault isolation**
- **Fault correction**

Internal IC faults

- **malfunction in internal circuitry**
- **short circuit**
- **open circuit**
- **short between two pins**

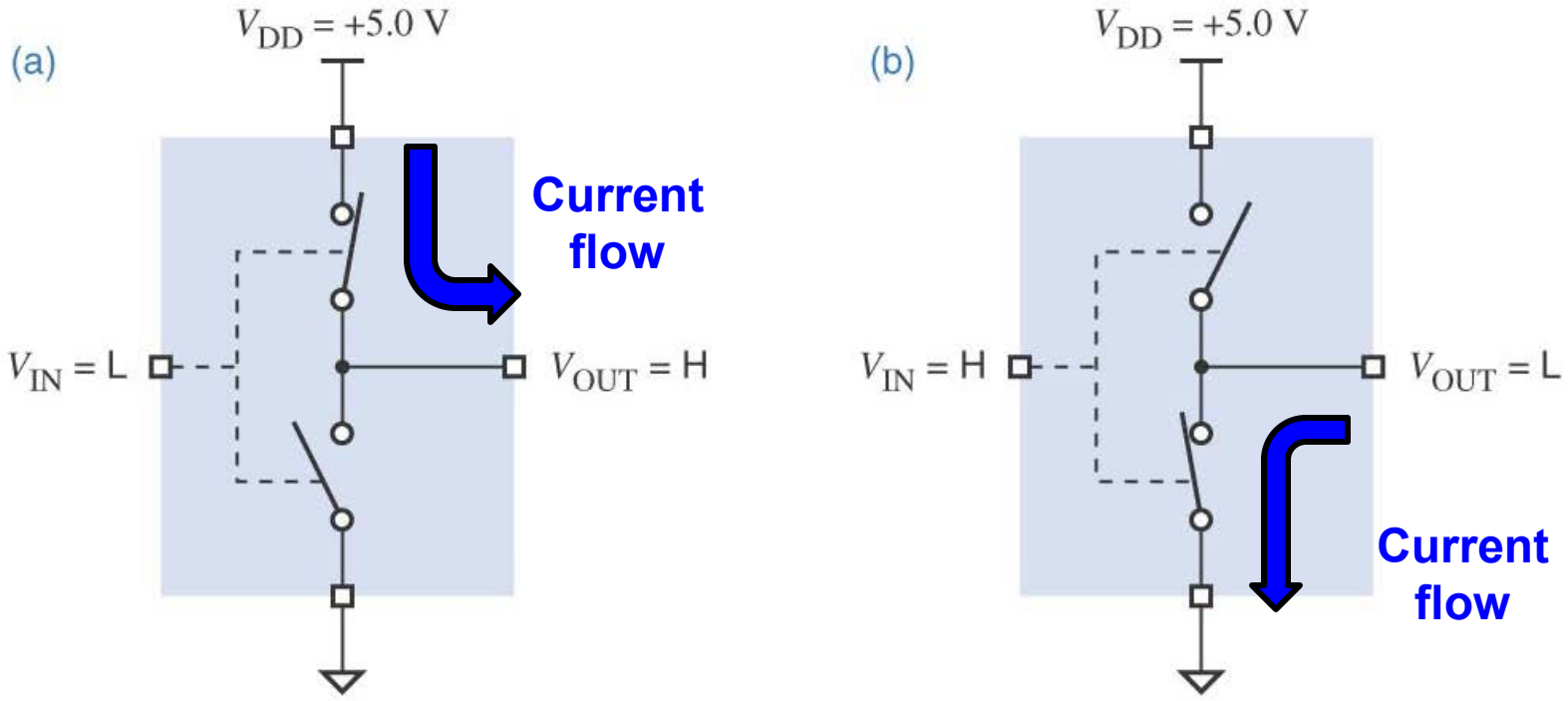
External IC faults

- open signal line
- shorted signal lines
- faulty power supply
- wrong connection

NEVER tie any circuit output to:

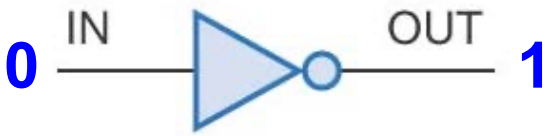
- another output unless you are very sure
- Vcc , GND or any other fixed voltage level

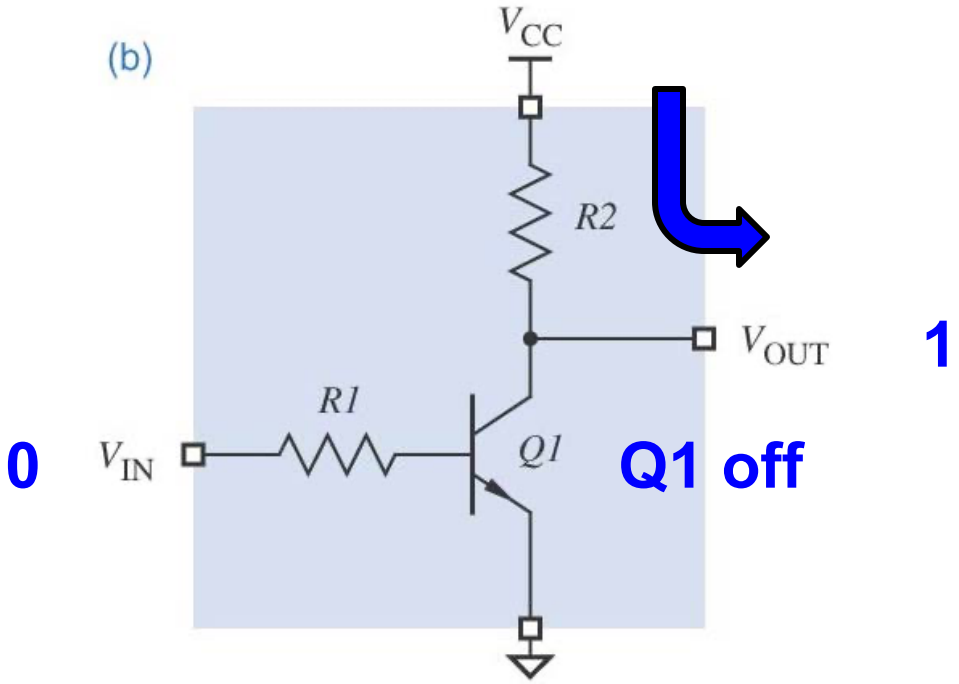
The transistor as a Switch



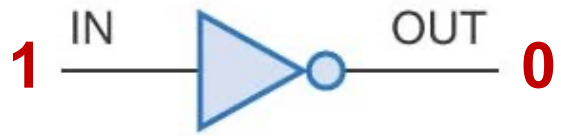
From *Digital Design*: *Principles and Practices*, Fourth Edition, John F. Wakerly, ISBN 0-13-186389-4.
©2006, Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

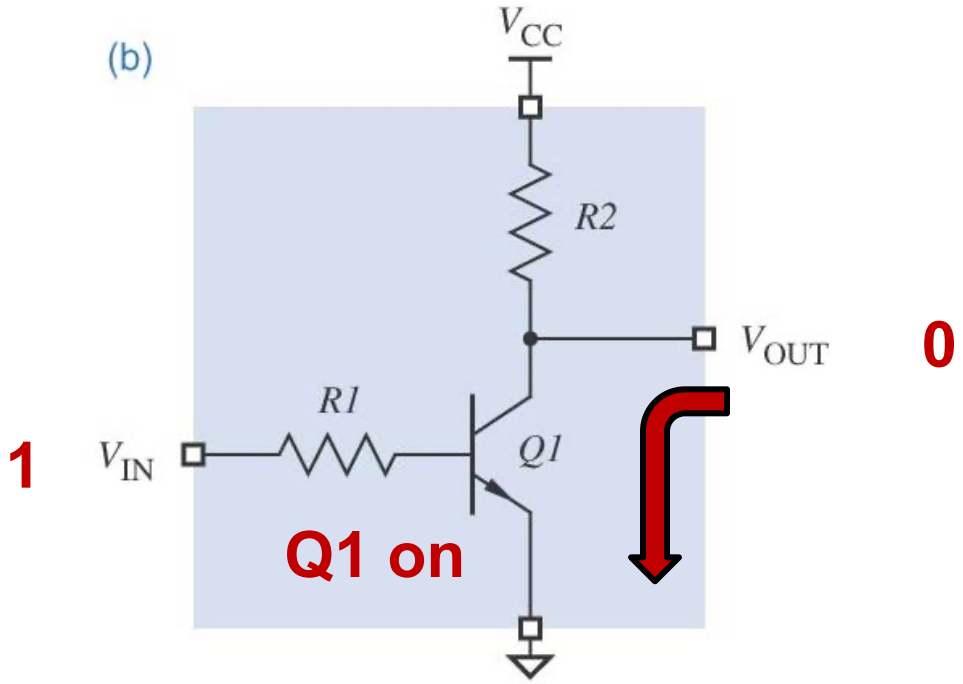
A simple BJT logic inverter



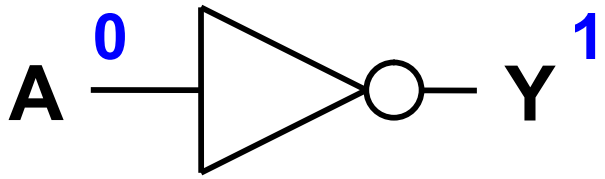


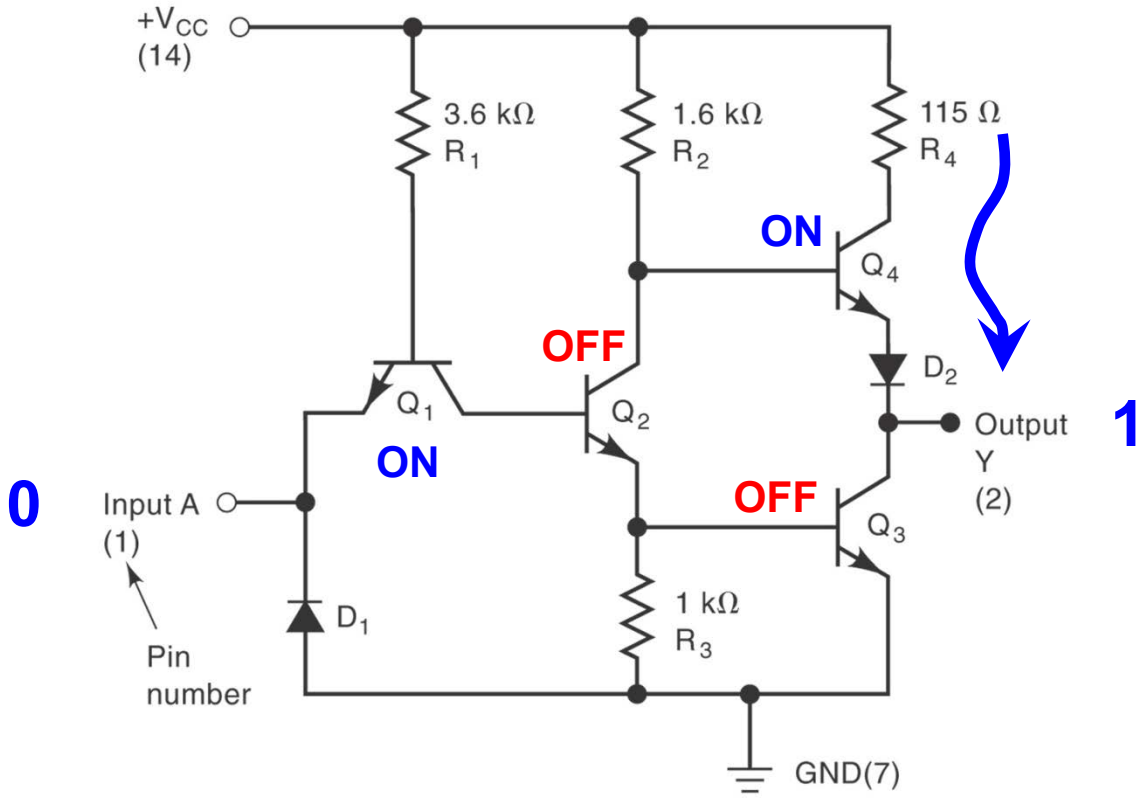
A simple BJT logic inverter



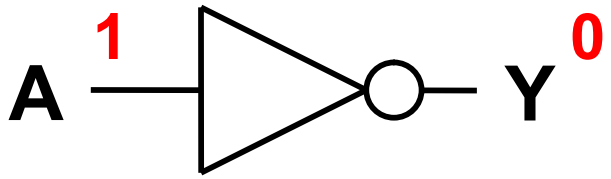


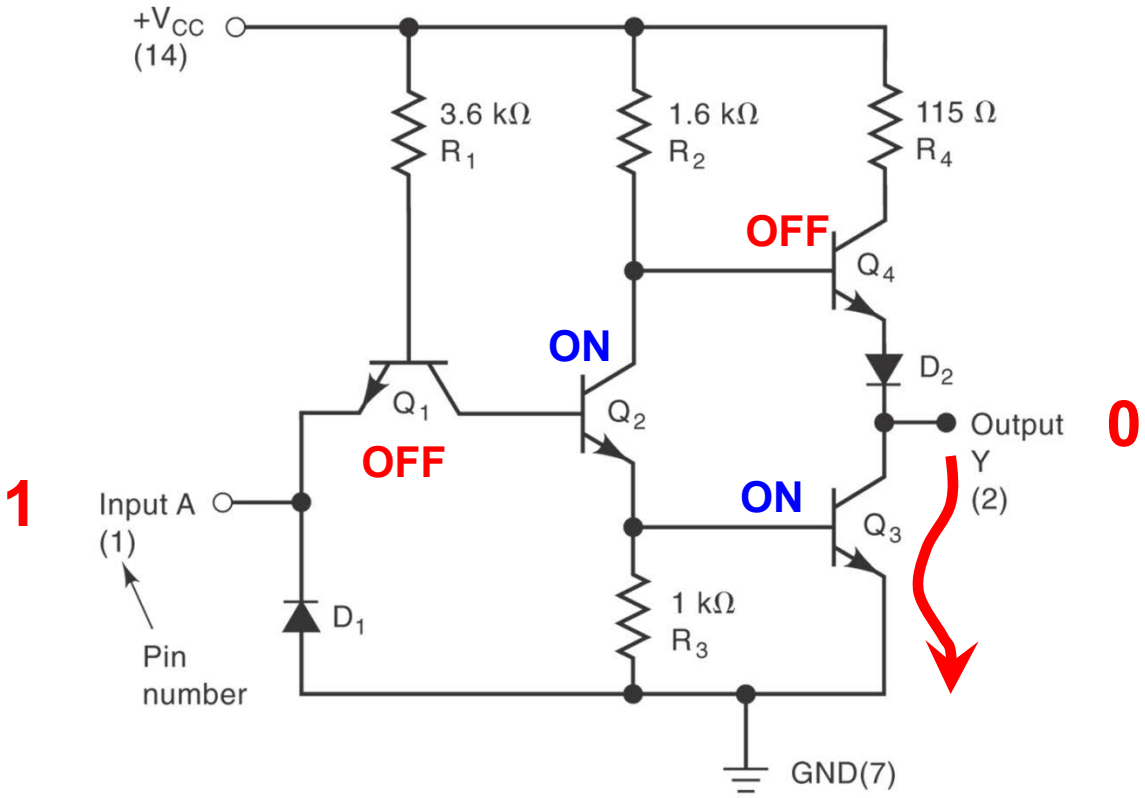
TTL inverter



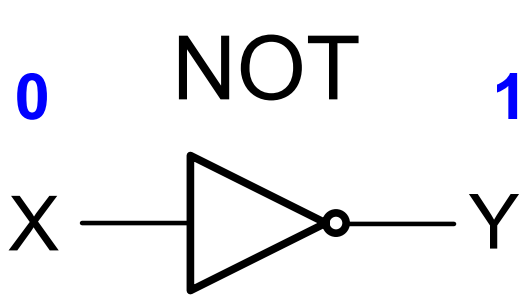


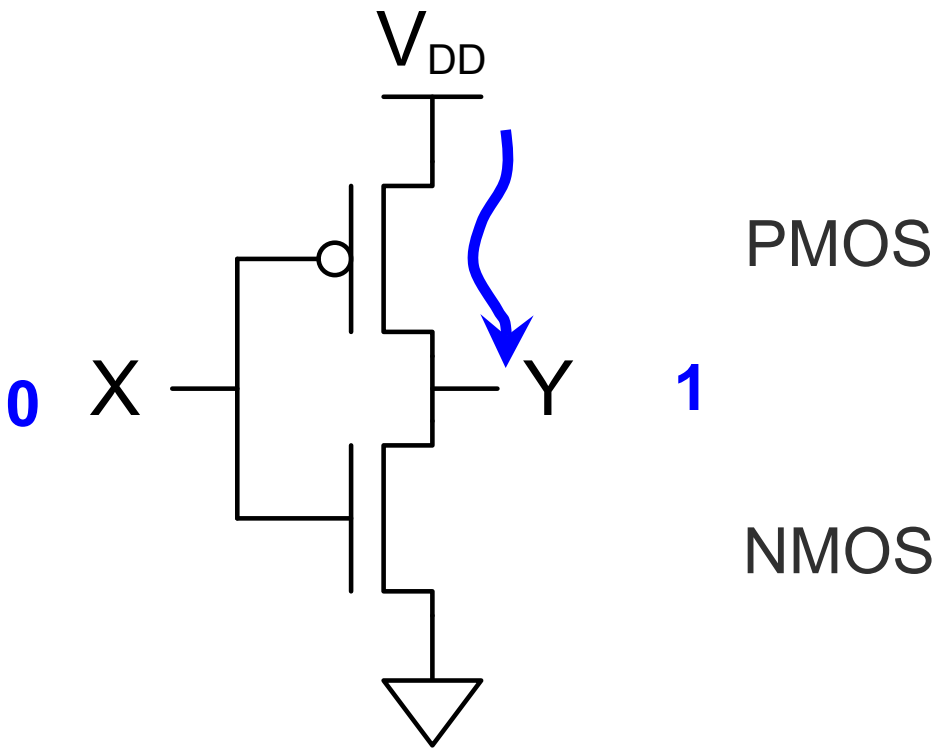
TTL inverter



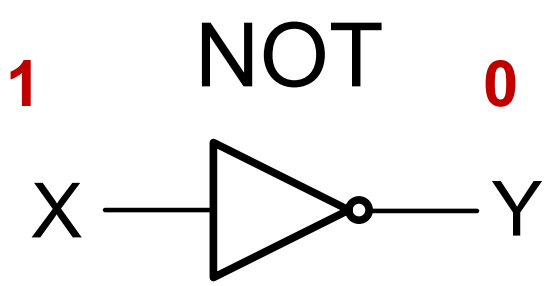


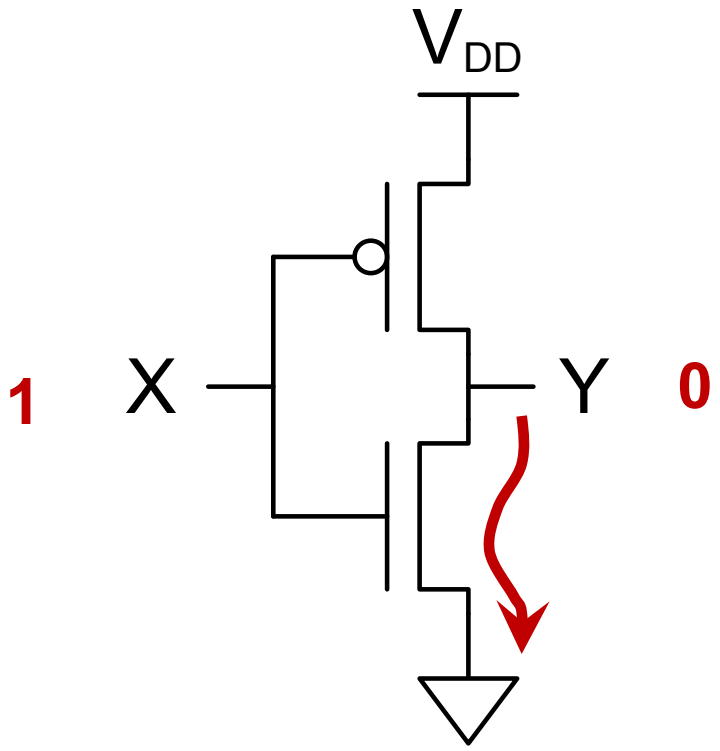
CMOS inverter



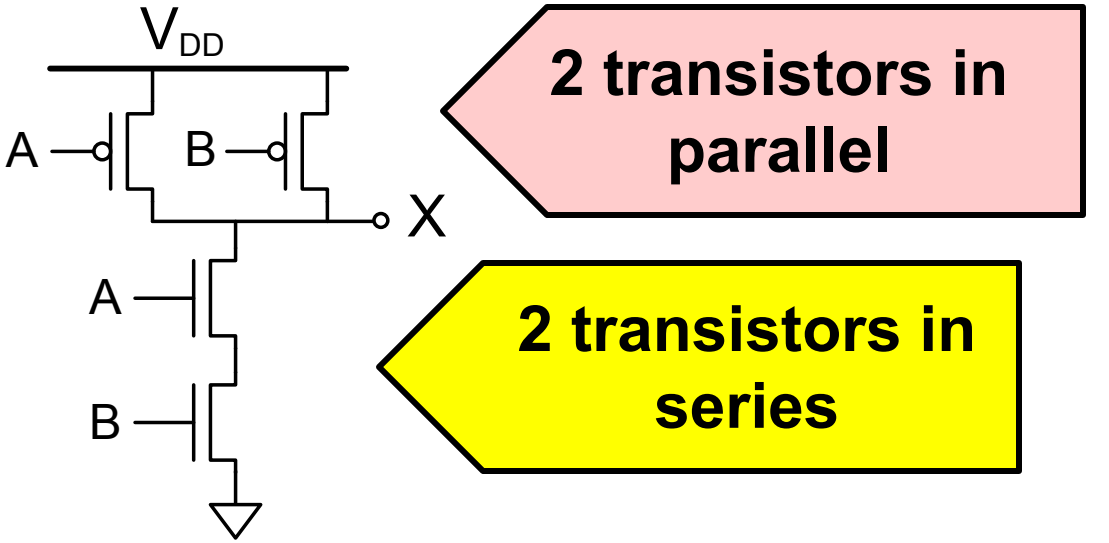


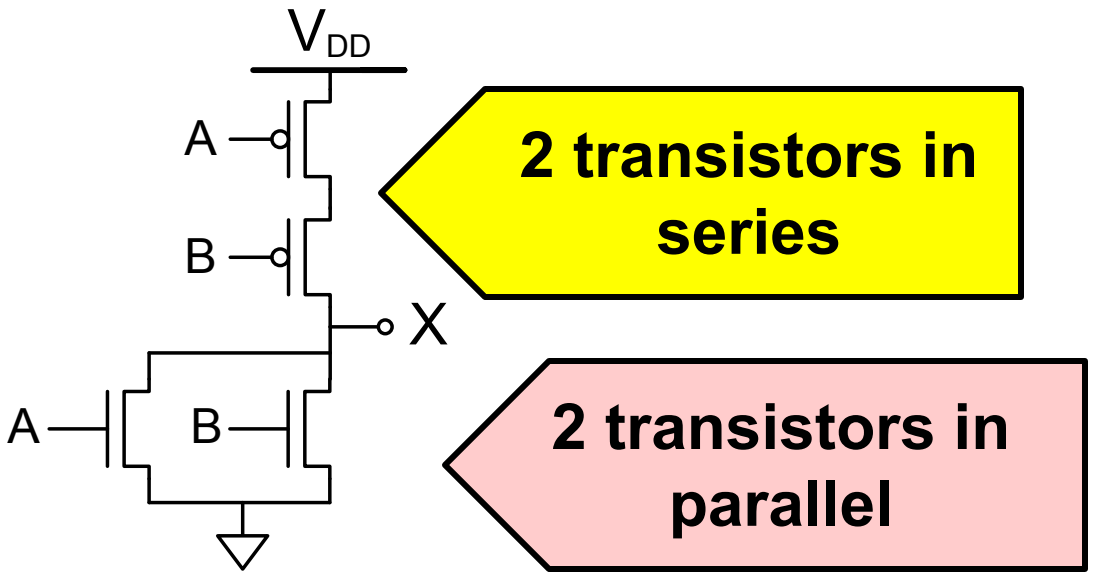
CMOS inverter

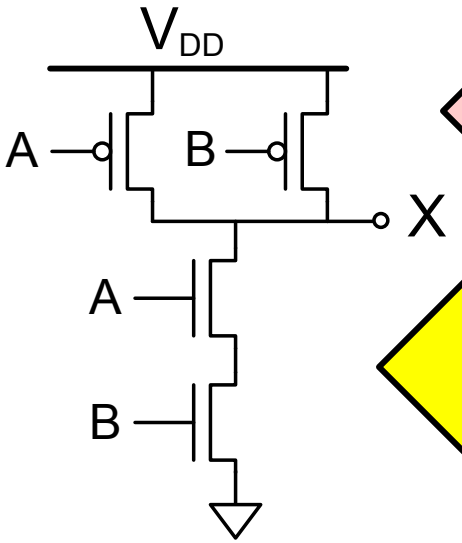




Examples of CMOS logic circuits

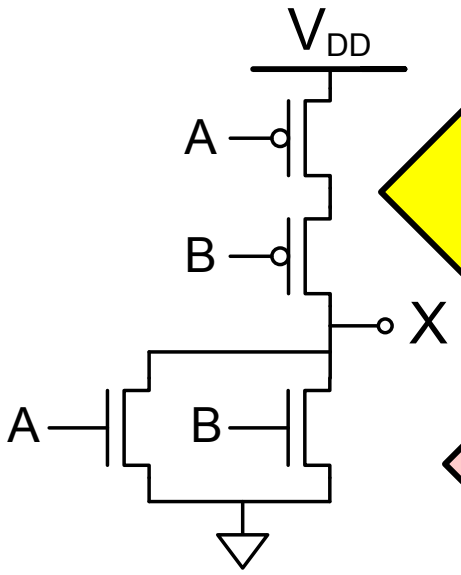

V_{DD}
A
B
X
A
B
2 transistors in parallel
2 transistors in series


V_{DD}
A
B
X
A
B
2 transistors in series
2 transistors in parallel



Either A or B=0
will cause X=1

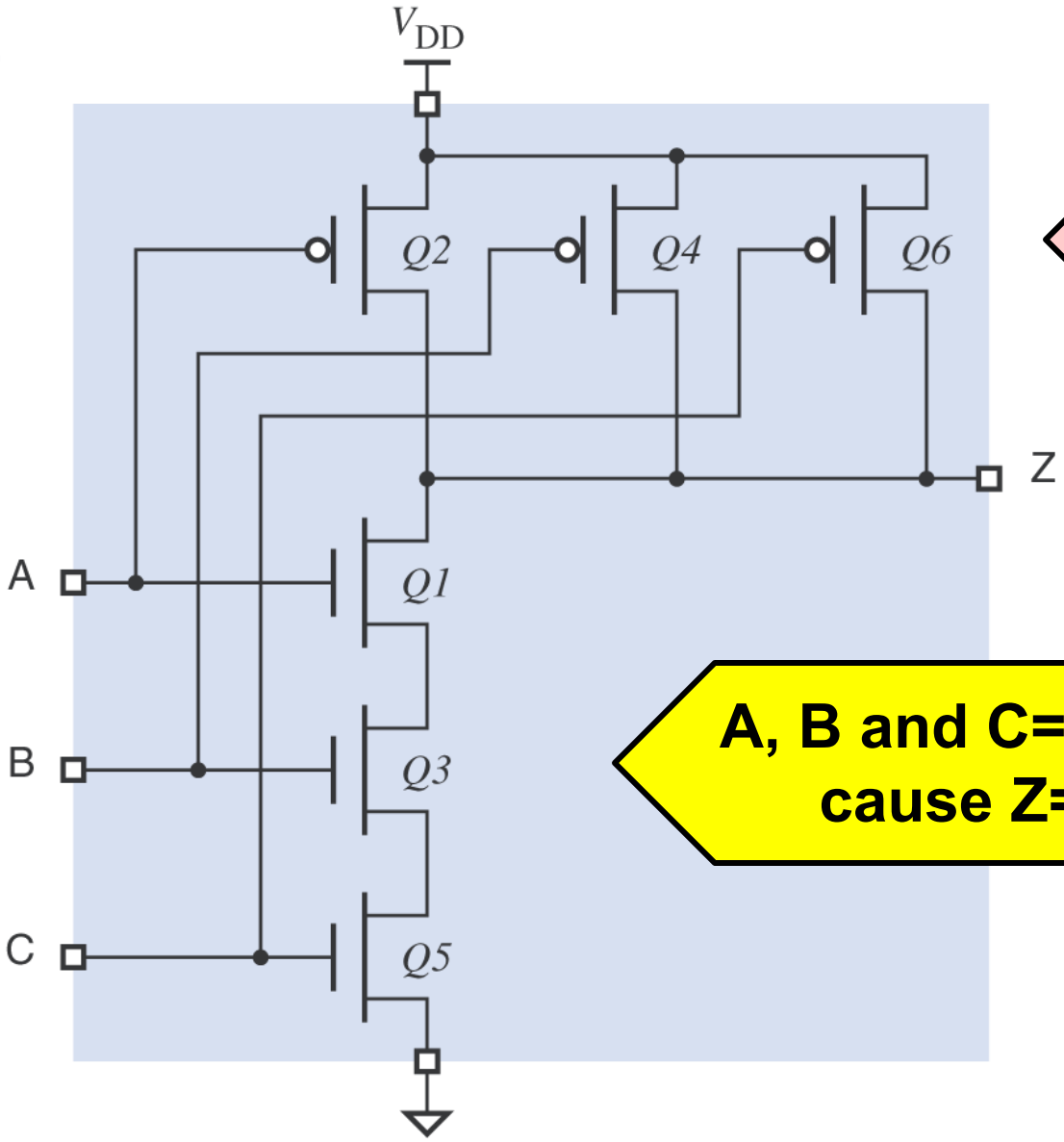
Both A and B=1
will cause X=0



Both A and B=0
will cause X=1

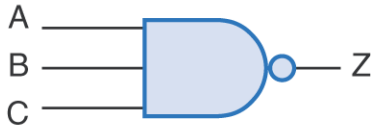
Either A or B=1
will cause X=0

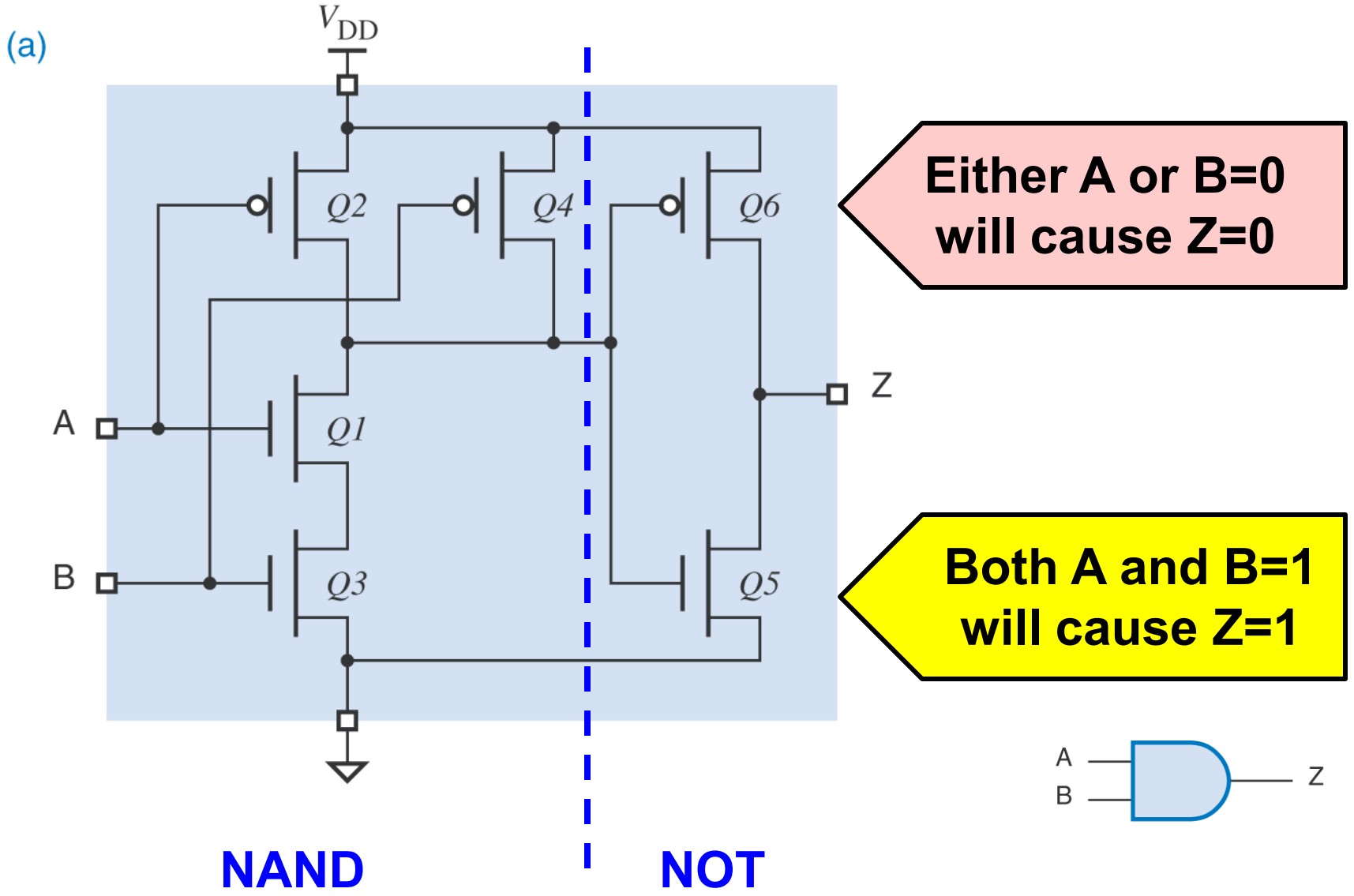
(a)



Either A, B or C=0 will cause Z=1

A, B and C=1 will cause Z=0

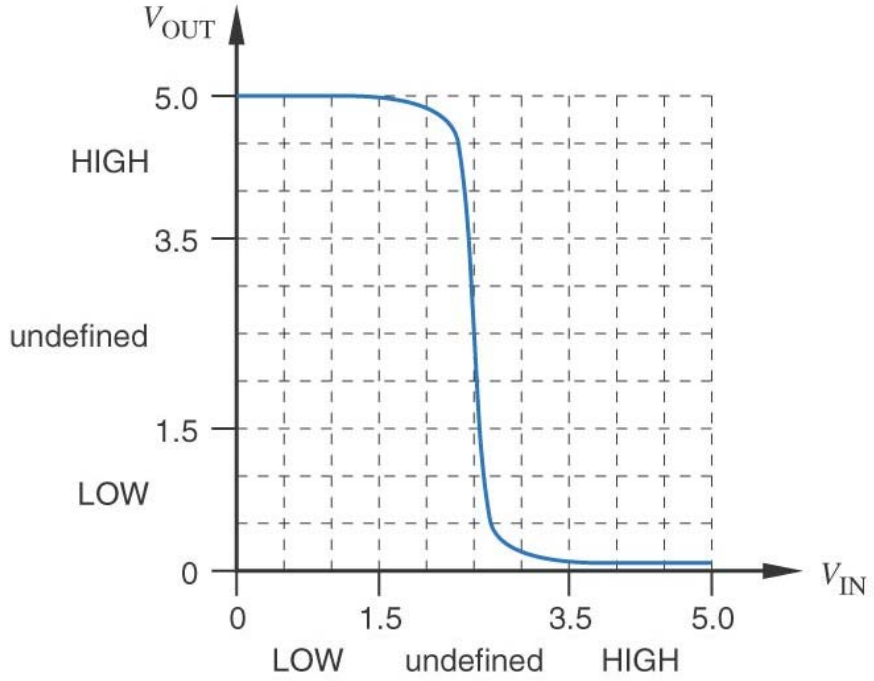



(a)
V_{DD}
Q2
Q4
Q6
Z
A
Q1
B
Q3
Q5
NAND
NOT
Either A or B=0
will cause Z=0
Both A and B=1
will cause Z=1
A
B
Z

L11 (6.40 – 6.63)

- Voltage parameters and noise margin
- Current parameters and fan-out
- Power dissipation, switching speed, propagation delay
- Rise time and fall time
- Tri-state output and open drain output

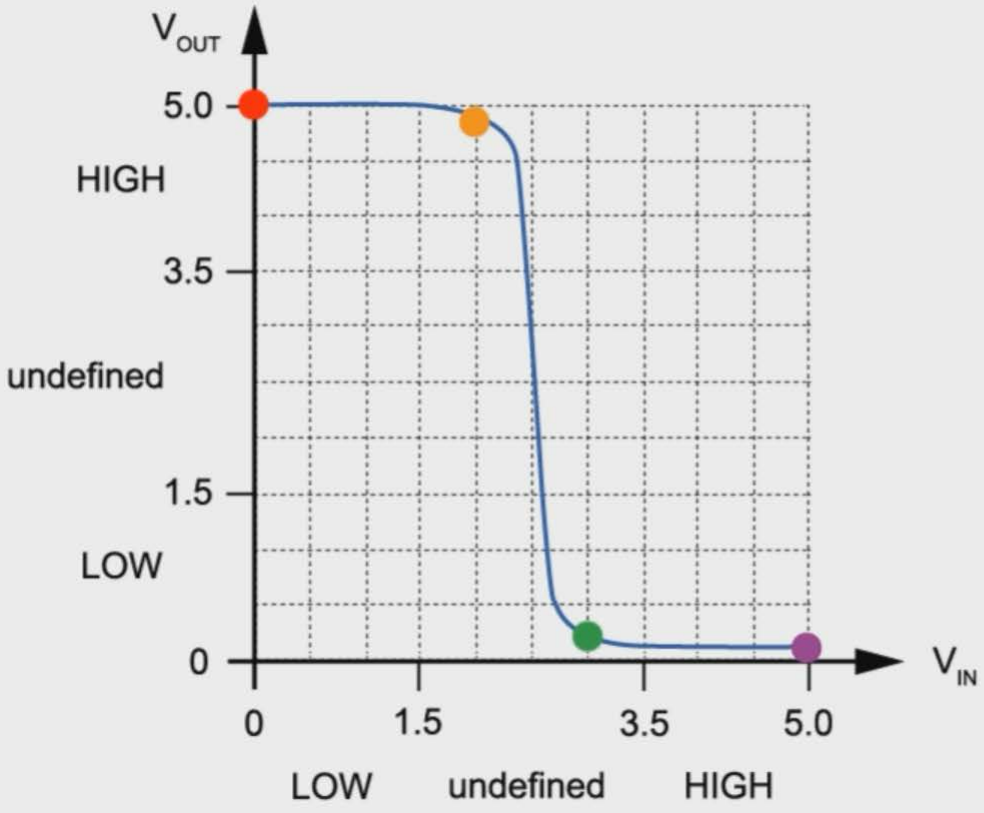
Digital Circuits Characteristics



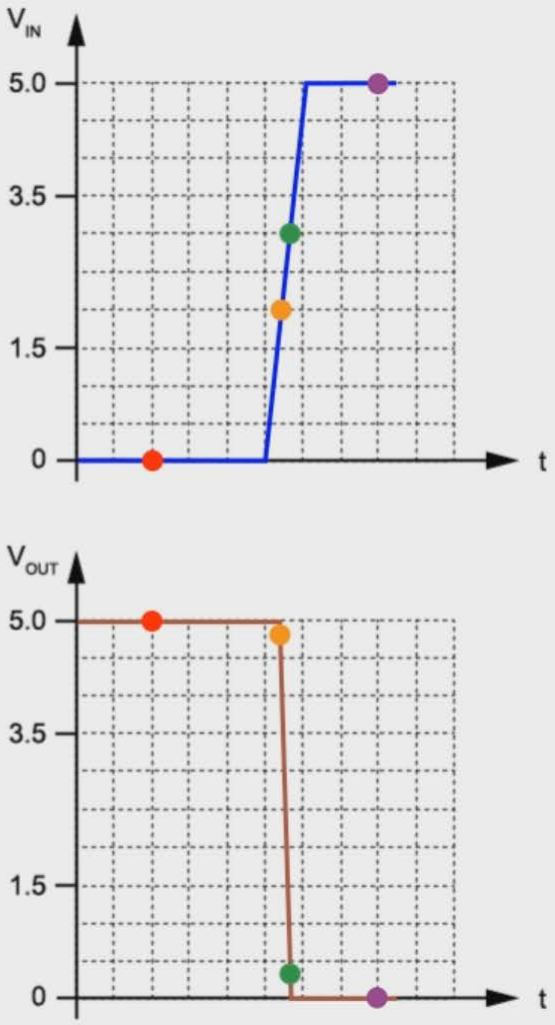
Not guaranteed

Fig. 3-25 Typical input-output transfer characteristic of a CMOS inverter

transfer characteristic:



time-based signals:



Voltage parameters:

V_{OH} (min) - Minimum output voltage produced for logic 1

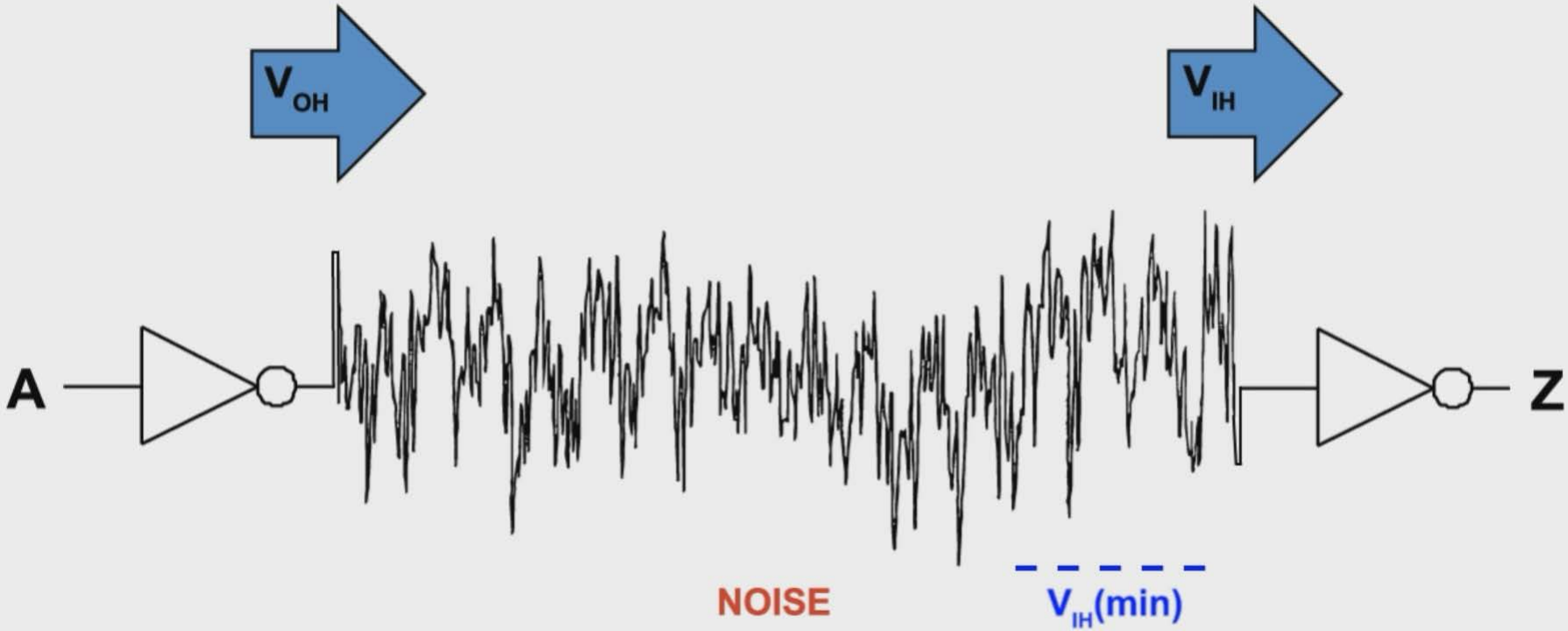
V_{IH} (min) - Minimum input voltage to be recognised as logic 1

V_{IL} (max) - Maximum input voltage to be recognised as logic 0

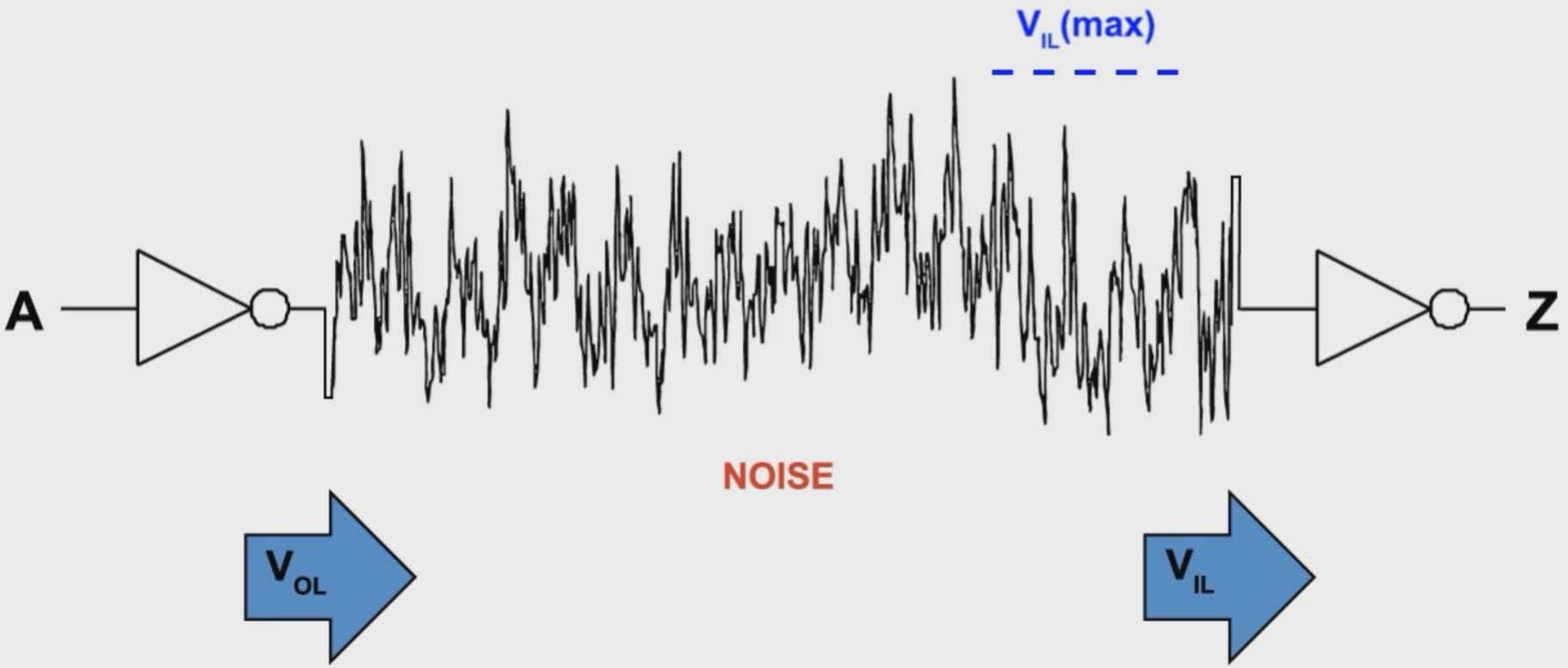
V_{OL} (max) – Maximum output voltage produced for logic 0

Voltage parameters and noise margin:

High-state DC noise margin = $V_{OH}(min) - V_{IH}(min)$

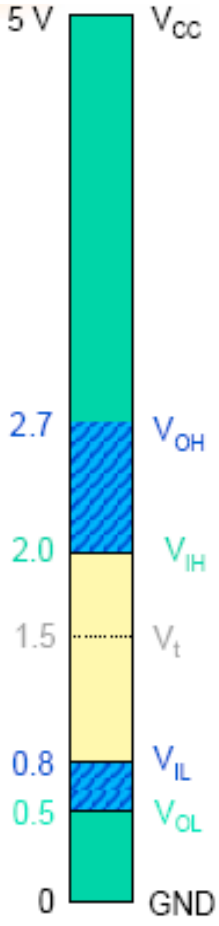


Voltage parameters and noise margin:

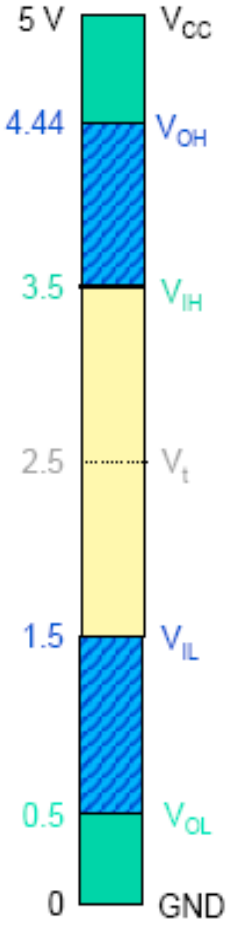


Low-state DC noise margin = V_{IL}(max) – V_{OL}(max)

Some typical values

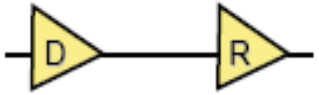


5-V LSTTL



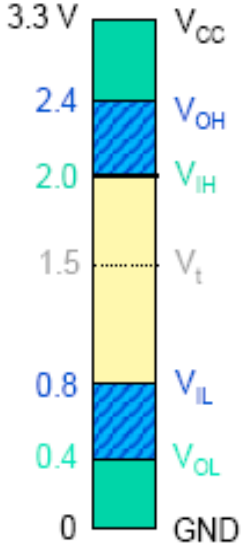
5-V CMOS

Is V_{OH} higher than V_{IH}?
Is V_{OL} less than V_{IL}?

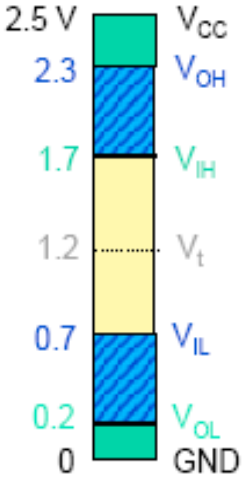


D \ R	5TTL	5CMOS	3LVTTL	2.5CMOS	1.8CMOS
5TTL	Yes	No	Yes *	Yes*	Yes*
5 CMOS	Yes	Yes	Yes*	Yes*	Yes*
3 LVTTL	Yes	No	Yes	Yes*	Yes*
2.5 CMOS	Yes	No	Yes	Yes	Yes*
1.8 CMOS	No	No	No	No	Yes

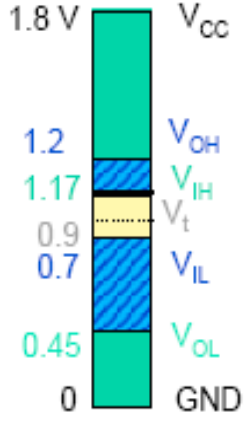
* Requires V_{IH} Tolerance



3.3-V LVTTL



2.5-V CMOS



1.8-V CMOS

Current parameters

I_{IH} – Maximum current that flows into the input at logic 1

I_{IL} - Maximum current that flows into the input at logic 0

I_{OH} – Maximum current that flows from the output at logic 1

I_{OL} – Maximum current that flows from the output at logic 0

Very small leakage current in CMOS (1μA)

0.02 - 24 mA depends on family

Other characteristics

Fan-out

- **specifies the number of standard loads that the output gate can drive**
- **More loads may reduce DC noise margins and switching speed**

Speed

- **Time taken for output to switch between 0 and 1**

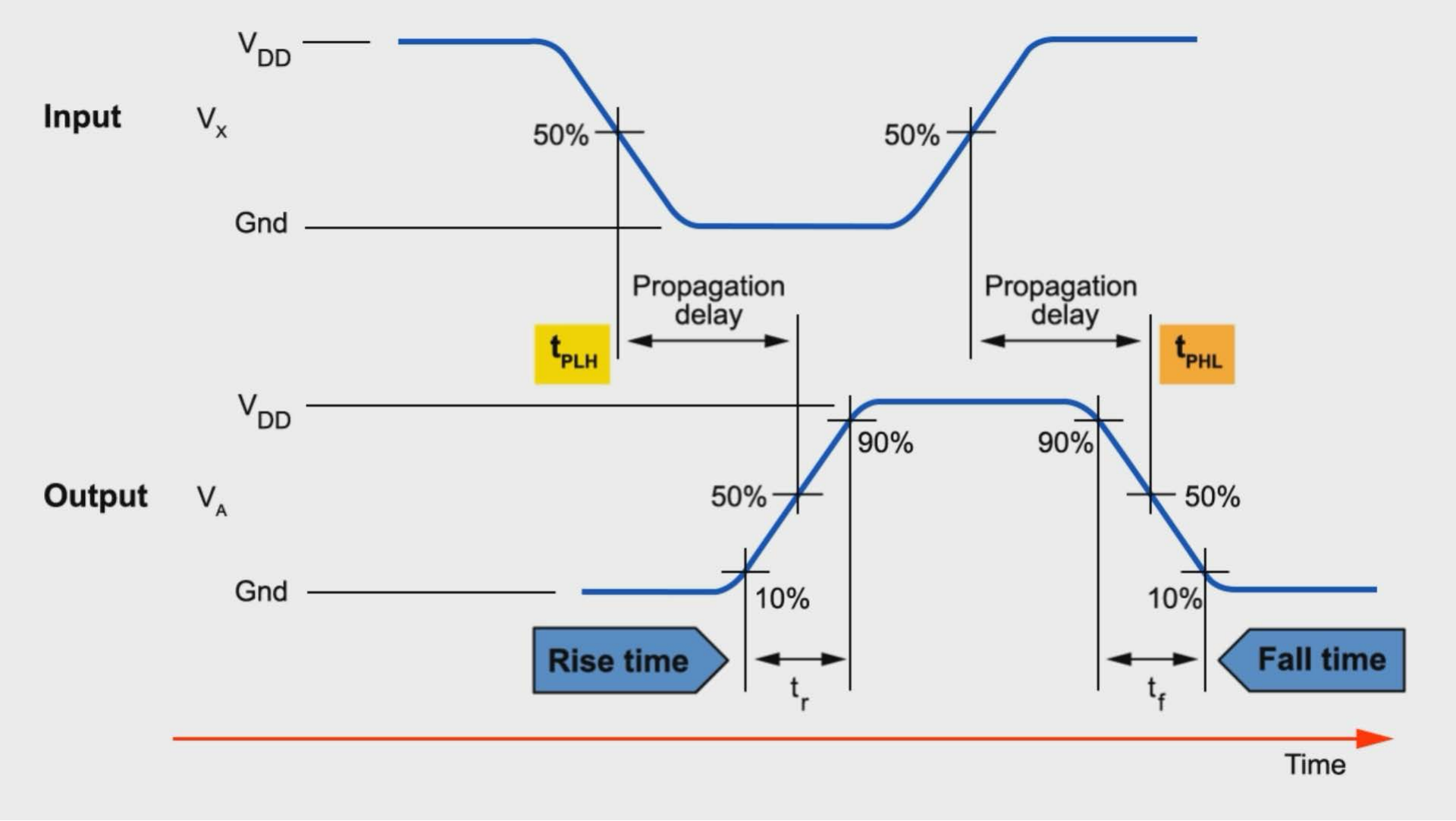
Power dissipation

- power consumed by the gate or device
- For CMOS, most of the power is consumed when output switches between 0 and 1 (dynamic power dissipation)
- $P = C V^2 f$
- Proportional to switching frequency f and square of power supply voltage V
- Static power dissipation (when there is no switching) is usually very small

Propagation delay

- average transition delay time for signal to propagate from input to output
- e.g. 9 – 19 ns for 74HC00
- t_{PD} , or t_{PHL} & t_{PLH}
- t_{PHL}= delay when output changes from High to Low
- t_{PLH}= delay when output changes from Low to High

Propagation delay (inverter):



Tristate outputs

3 output states:

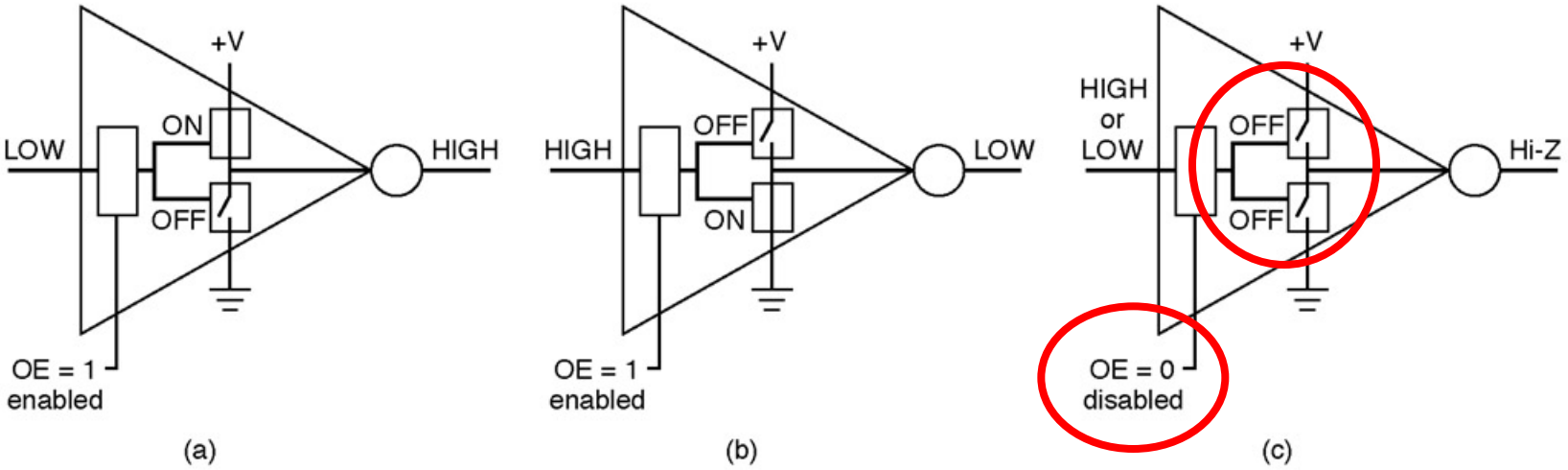
- logic 0
- logic 1
- high impedance (Hi-Z)
  - neither 0 nor 1
  - behaves like an open circuit

When a device is enabled, output is logic 0 or 1

When device is disabled, output is in Hi-Z

E.g. tristate buffer, tristate inverter

Tristate inverter

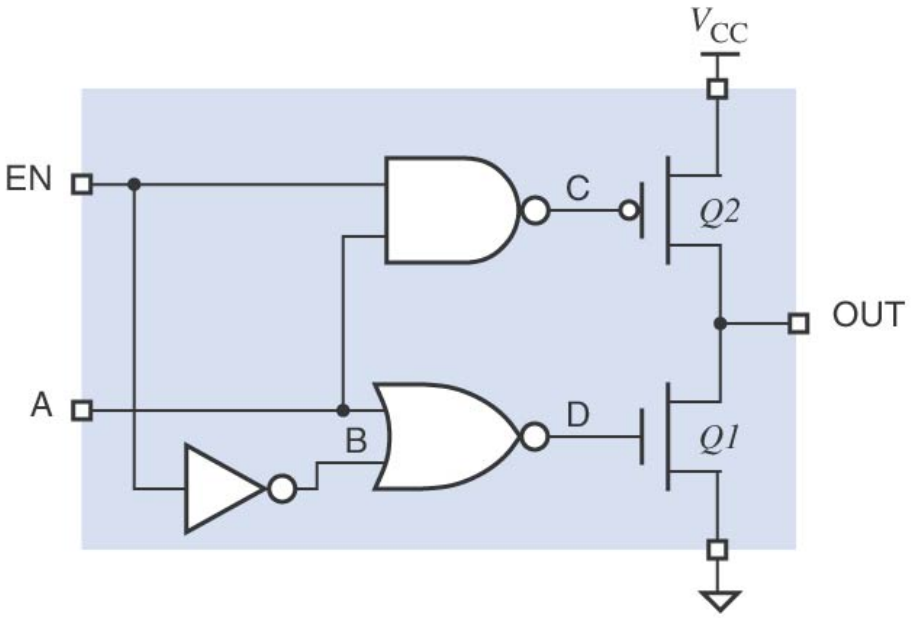


(a) & (b): Tristate device behaves as a normal inverter when enabled.

(c): Device output is in high-impedance when disabled.

CMOS Tristate buffer

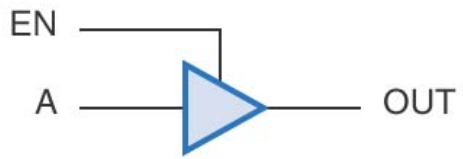
(a)



(b)

EN	A	B	C	D	*Q1*	*Q2*	OUT
L	L	H	H	L	off	off	Hi-Z
L	H	H	H	L	off	off	Hi-Z
H	L	L	H	H	on	off	L
H	H	L	L	L	off	on	H

(c)



From *Digital Design*: *Principles and Practices*, Fourth Edition, John F. Wakerly, ISBN 0-13-186389-4.
©2006, Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

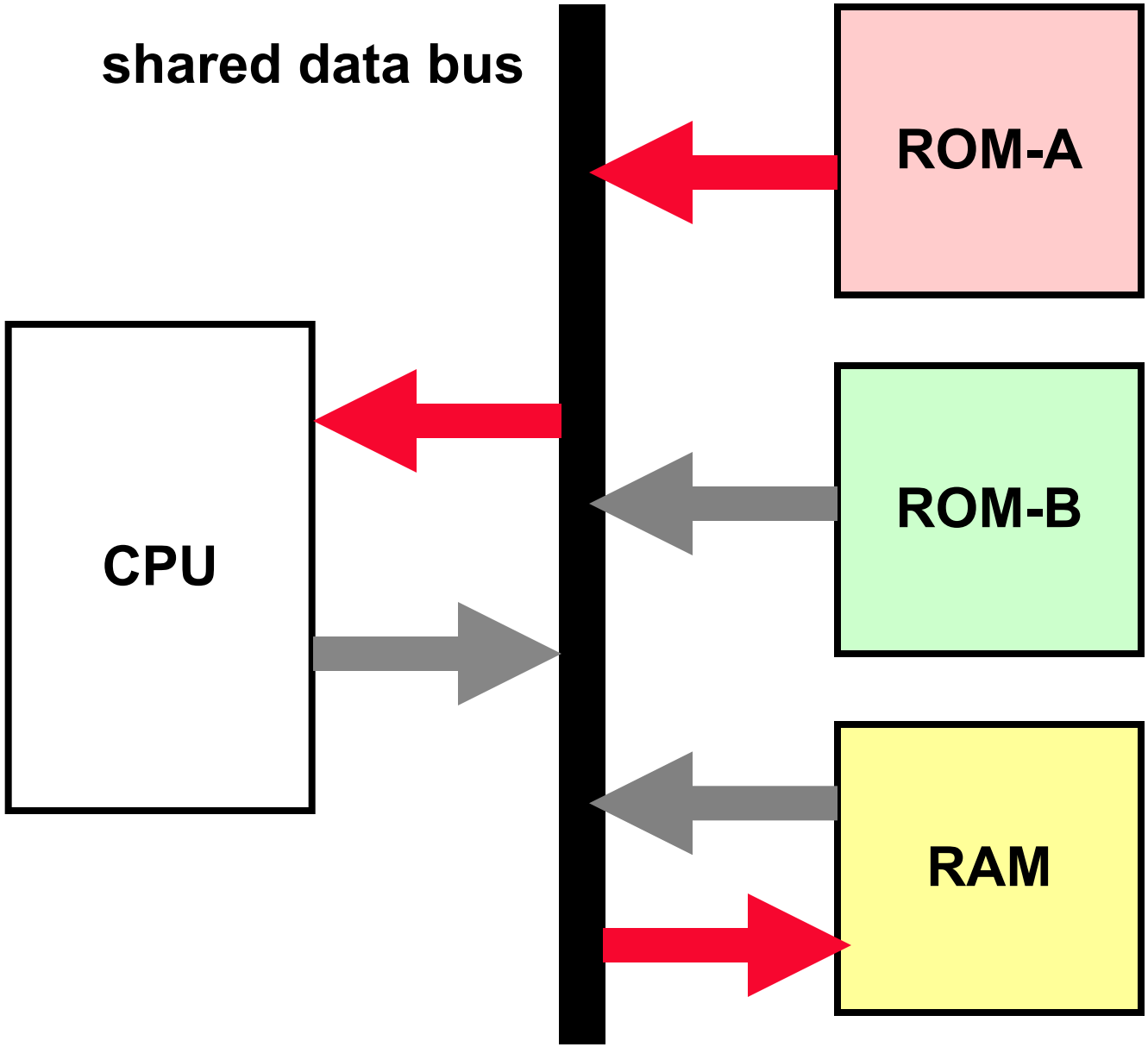
Advantage of using logic devices with tristate outputs

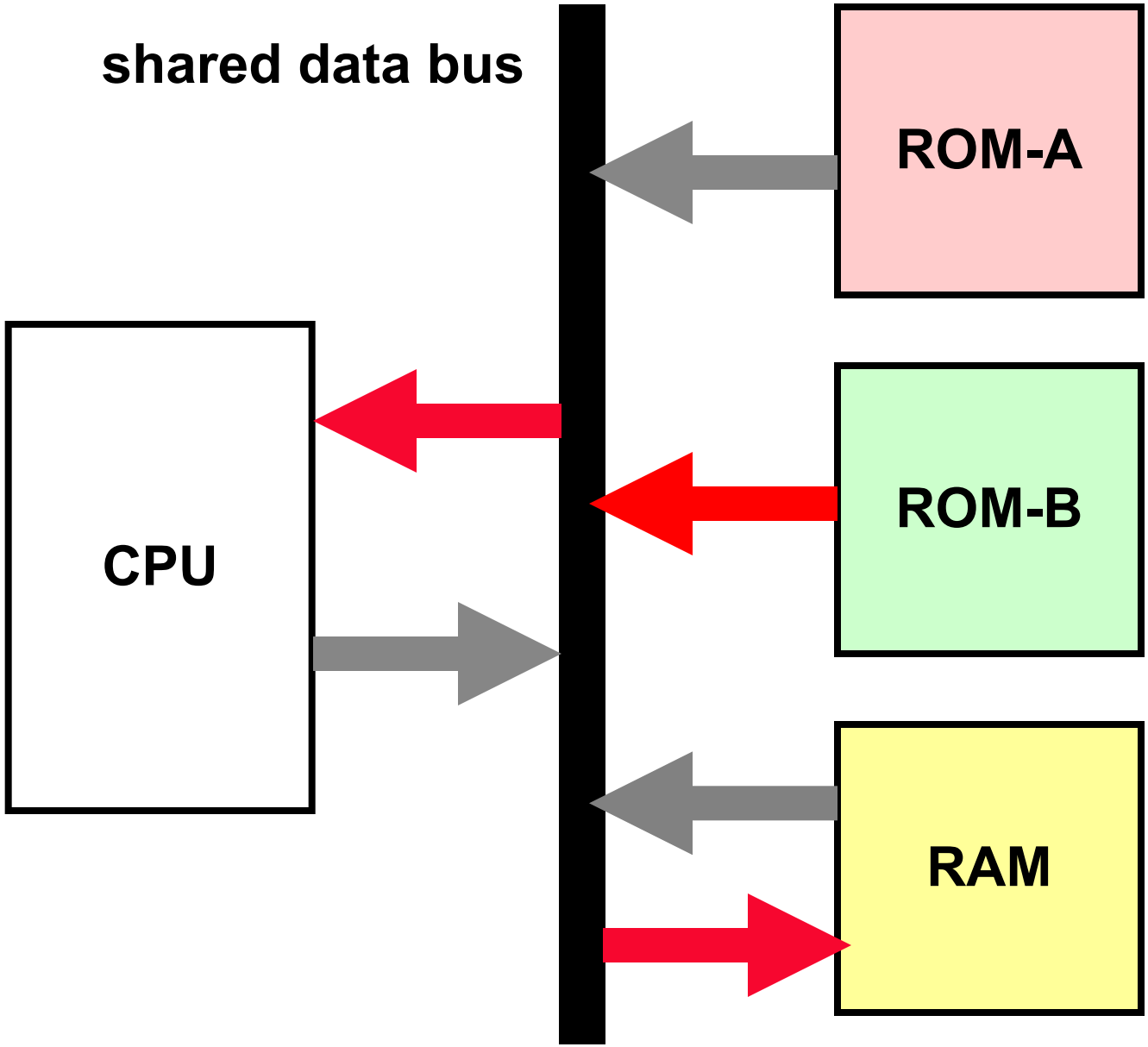
Two or more outputs can be connected together.

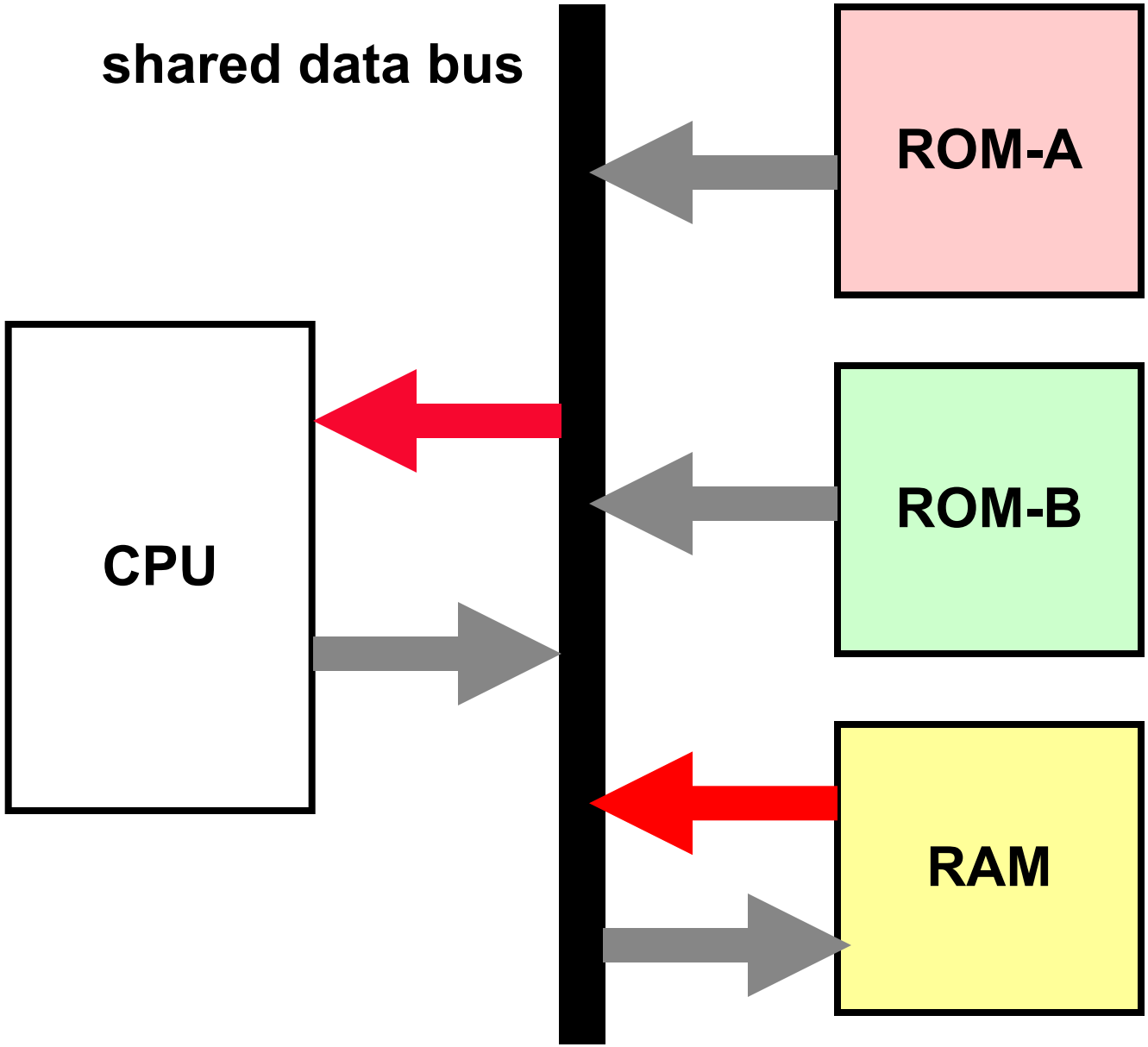
However, at any one time only one (or no) output should be enabled.

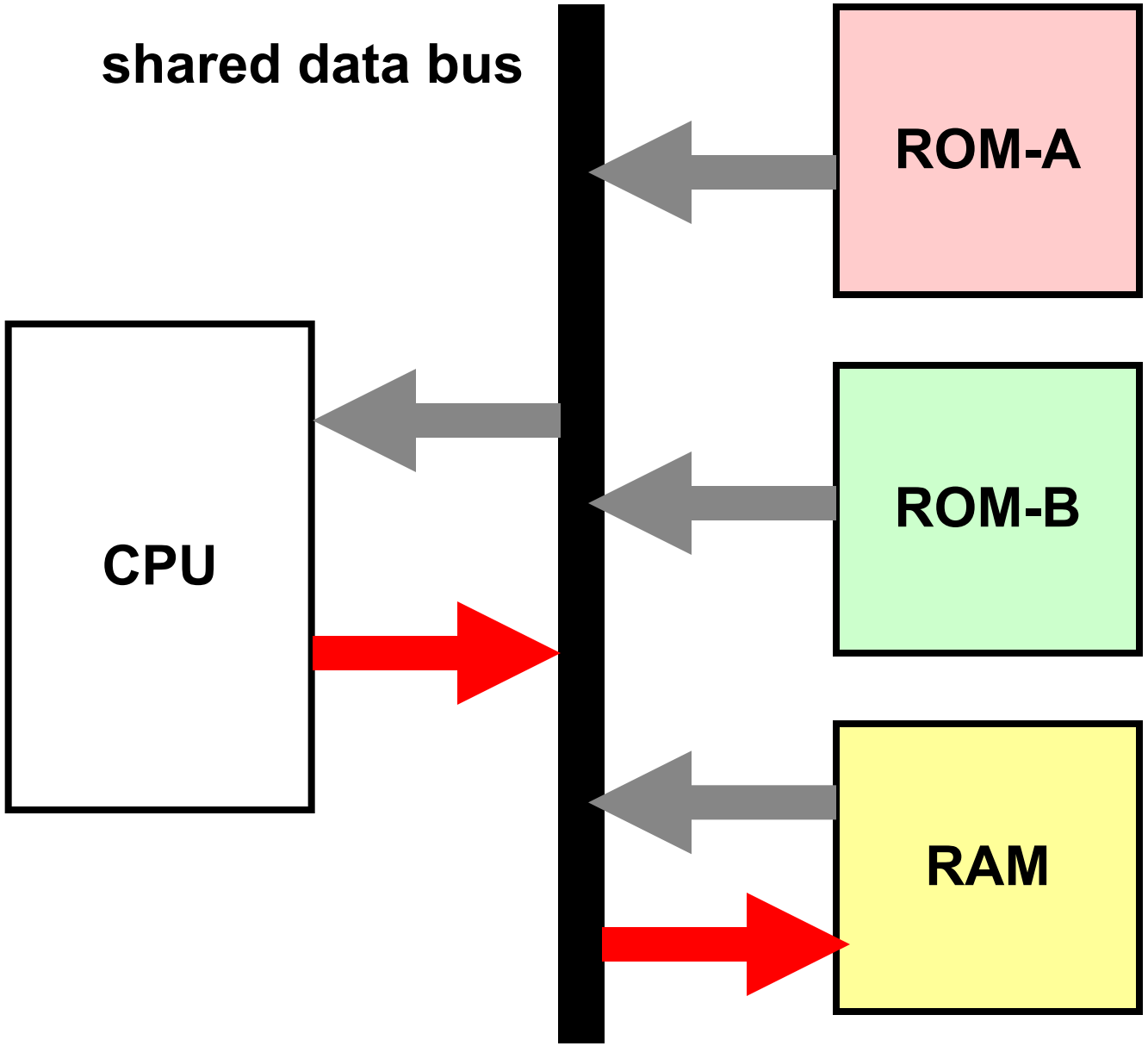
Otherwise, it can lead to bus contention and damage the devices.

Example: memory devices have tristate outputs to share the same data bus.


shared data bus
ROM-A
CPU
ROM-B
RAM

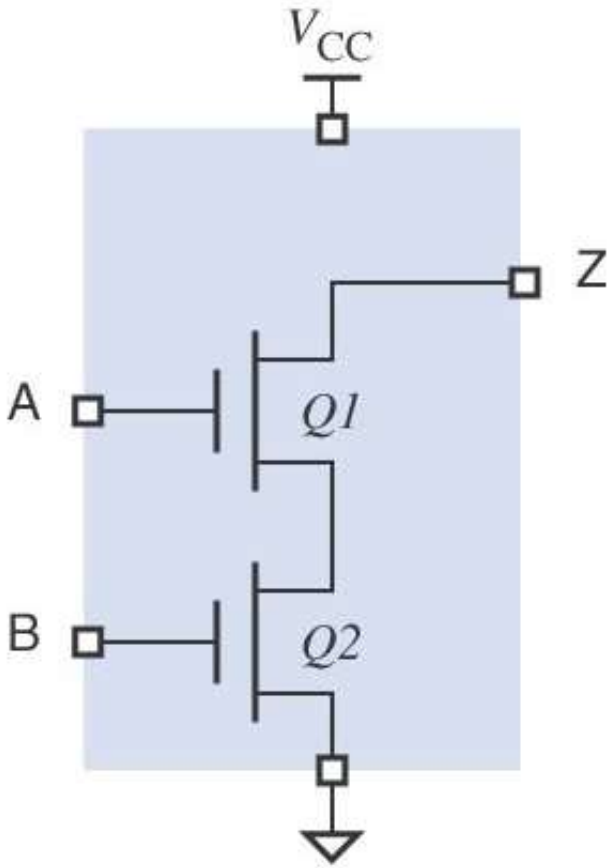

shared data bus
ROM-A
CPU
ROM-B
RAM


shared data bus
ROM-A
CPU
ROM-B
RAM


shared data bus
ROM-A
CPU
ROM-B
RAM

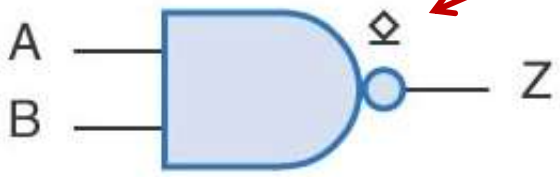
Open-collector/drain output

Need external pull-up resistor to make output=1



A	B	Q1	Q2	Z
L	L	off	off	open
L	H	off	on	open
H	L	on	off	open
H	H	on	on	L

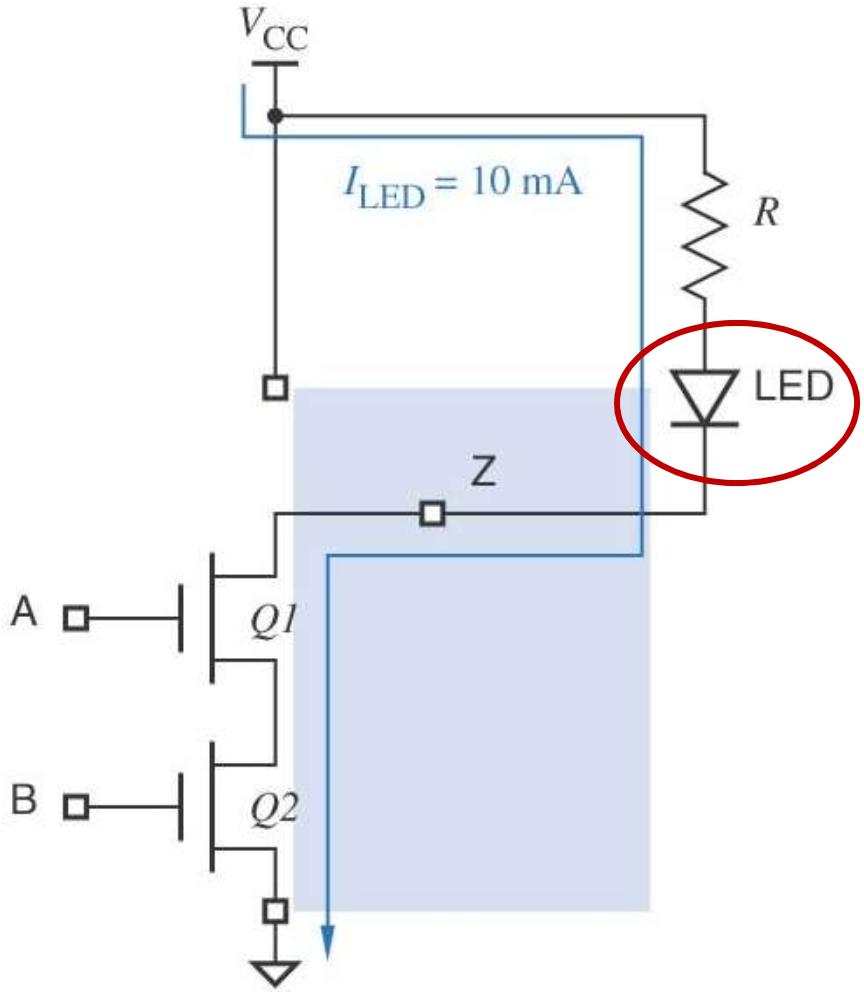
Symbol for open-drain



Application: open-drain output can drive LED

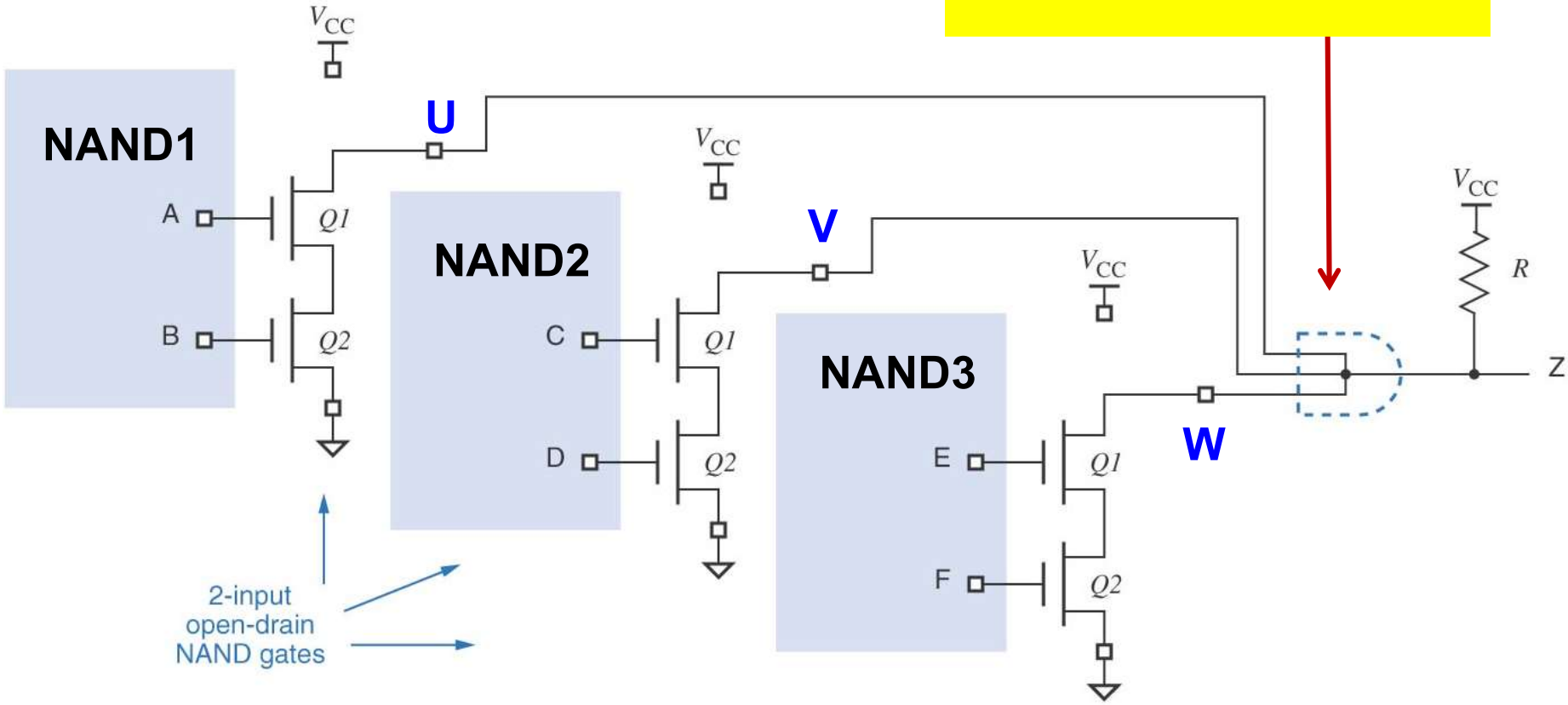
- If A=B=1
- Q1 and Q2 ON
- Z=0
- LED ON

- If A=0 or B=0
- Q1 or Q2 OFF
- Z=1
- LED OFF



Open-drain outputs can be tied together

wired-AND logic
Z = U•V•W



Summary of wired-AND circuit behaviour

Inputs of NAND gates			Outputs	
A=B=1	C=D=1	E=F=1	U, V, W	Z
No	No	No	U=1; V=1; W=1	1
No	No	Yes	W=(EF)'=0; => U=V=0	0
No	Yes	No	V=(CD)'=0; => U=W=0	0
No	Yes	Yes	V=(CD)'=0; W=(EF)'=0; => U=0	0
Yes	No	No	U=(AB)'=0; => V=W=0	0
Yes	No	Yes	U=(AB)'=0; W=(EF)'=0; => V=0	0
Yes	Yes	No	U=(AB)'=0; V=(CD)'=0; => W=0	0
Yes	Yes	Yes	U=(AB)'=0; V=(CD)'=0; W=(EF)'=0	0

Imagine an elastic band:

NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Combinational Circuits

So far, you have covered…

- Number Systems and Digital Arithmetic
- Basic logic gates and Boolean Algebra
- Truth Tables and K-Map
- Gate level combinational circuits

Adding two signed binary numbers

$$\begin{array}{rcl} +6 & & 0110 \\ +\ -3 & \Rightarrow & +\ 1101 \\ \hline +3 & & 0011 \end{array}$$

X	Y	C_i	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Truth table for Full Adder

Minterm (or Maxterm) expression

$$C_o = \sum (3, 5, 6, 7)$$

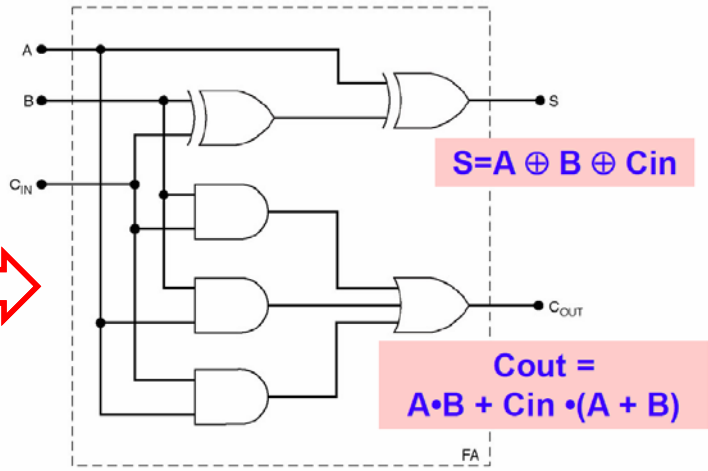
$$S = \sum (1, 2, 4, 7)$$

$$S = X \oplus Y \oplus C_i$$

$$C_o = X.Y + X.C_i + Y.C_i$$

Minimize using Boolean Algebra or K-Maps

Implementation of Full Adder using basic logic gates



Where is this going?

- Combinational circuits are used extensively within digital systems that are found in many electronic devices

Have you tried to count the number of computers in your house?

Things like microwave ovens, dishwashers, refrigerators, televisions may be obvious. But what about remote controllers for the air conditioner, the stereo, the Blu-Ray player and so on ....



***Source**: http://www.mydentistrocks.org/wp-content/uploads/2014/06/20140324200975887588.jpg*

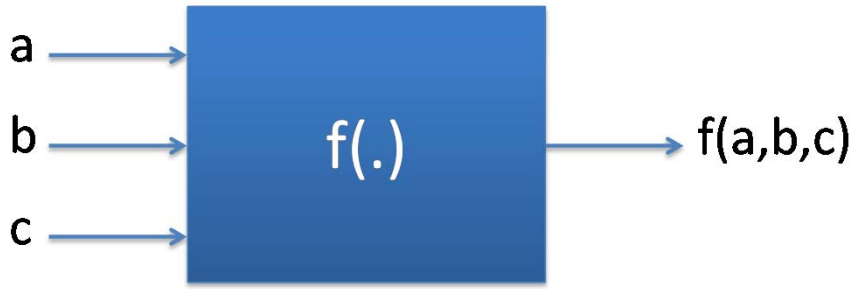
In Today's Lecture...

NANYANG TECHNOLOGICAL UNIVERSITY

- Outline
  - Combinational design process
  - Commonly used combinational circuits
    - 7-Segment Decoder
    - Decoder (One-Hot)
    - Multiplexer
- Outcomes
  - Learn how to design simple combinational circuits
  - Understand the working principles of commonly-used combinational circuits and appreciate how they can be used to build digital systems

Combinational Circuits

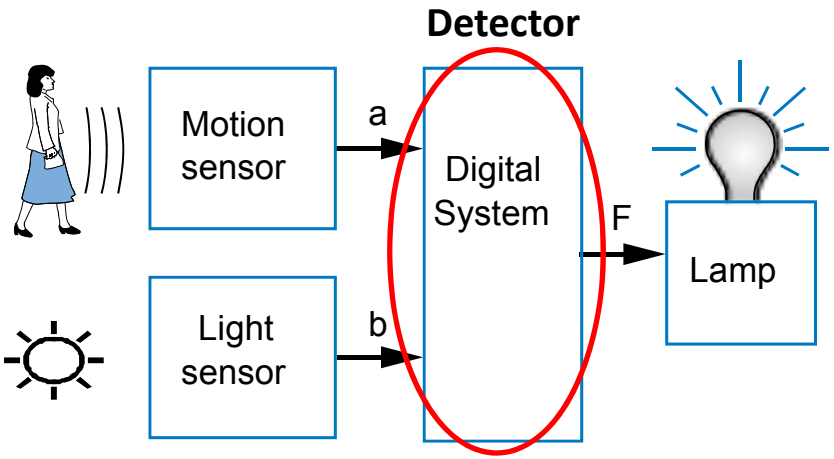
- Combinational circuits are functions:
  - *a function is a relation between a set of inputs and a set of permissible outputs with the property that each input is related to exactly one output*



- Remember, a combinational circuit takes a set of inputs, and for each input combination, always produces a corresponding output
- If I apply the same input values, I always get the same output values:
  - In any order
  - At any time

Example of Combinational Circuits

- Combinational circuits:
  - Outputs depend solely on the **present combination** of the input values



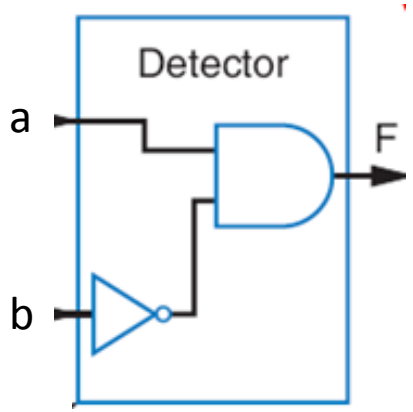
Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 35

if a=0 and b=0, then F=0

if a=0 and b=1, then F=0

if a=1 and b=0, then F=1

if a=1 and b=1, then F=0



Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 49

- Motion-in-dark example
  - Turn on lamp (F=1) when
    - Motion sensed (a=1) and no light (b=0)
  - F = a **AND NOT**(b)
  - Build using logic gates, **AND** and **NOT**, as shown

Combinational Design Process

NANYANG TECHNOLOGICAL UNIVERSITY

- Hence, we can define a two-step process for implementing combinational designs:
  - **Step 1: Capture the function**
    - Use truth-table or equations, depending on what makes sense for the application
  - **Step 2: Convert to circuit**
    - 2A: First, if you used a truth table in Step 1, create equations
    - 2B: For each output, create a circuit corresponding to that output's equation

Example: Three 1s Pattern Detector

- **Problem 1**: Detect three adjacent 1's in an 8-bit input

  abcdefgh

  000**111**01 → 1

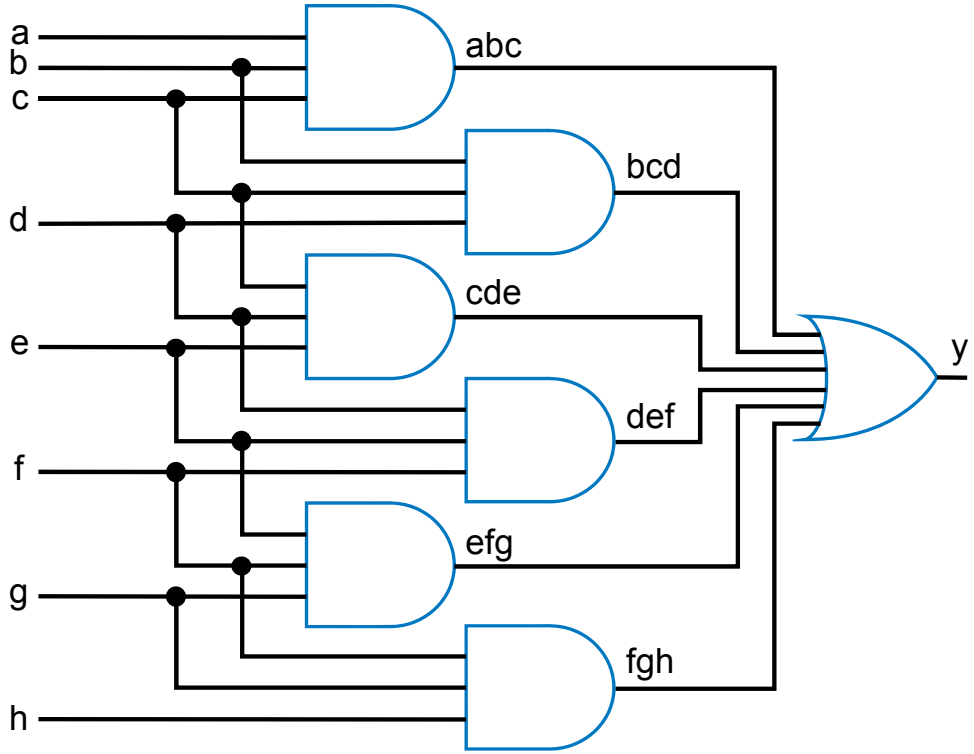
  10110011 → 0

  0**111**0001 → 1

- **Step 1**: Capture the function
  - Truth table would be too big (2^8 = 256 rows!)
  - Use an equation, with a term for each possible case:
  - y = abc + bcd + cde + def + efg + fgh
- **Step 2A**: Create equation (Already Done)
- **Step 2B**: Implement using gates

Example: Three 1s Pattern Detector

- y = abc + bcd + cde + def + efg + fgh



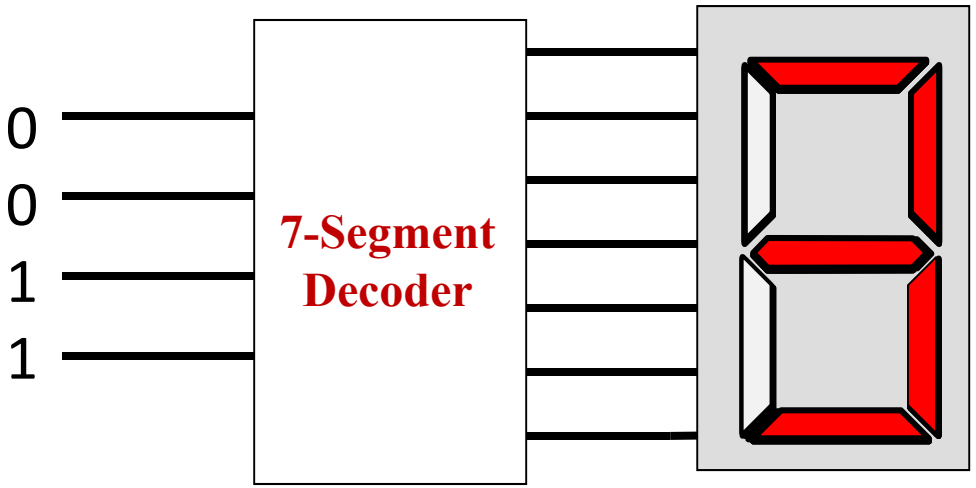
***Source**: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 75*

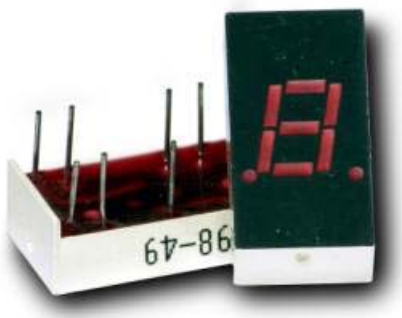
NANYANG TECHNOLOGICAL UNIVERSITY

SEVEN SEGMENT DECODER

Seven Segment Decoder

- Consider a seven segment display
- We want a circuit that maps binary numbers to the correct digit on the display

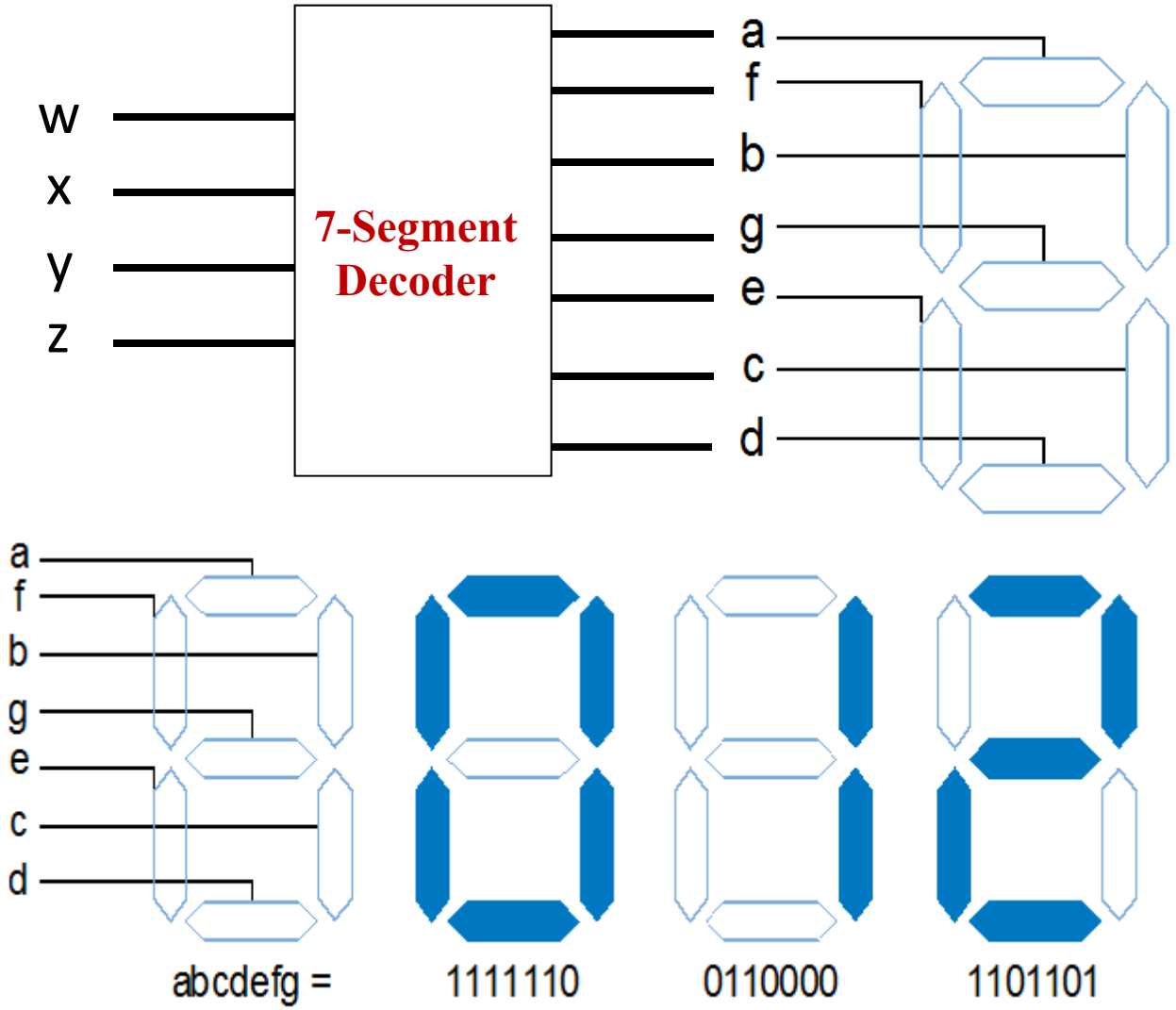




Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 72

- The 7-Segment Decoder needs to decide, for each input set, which segments should be lit to display the correct digit

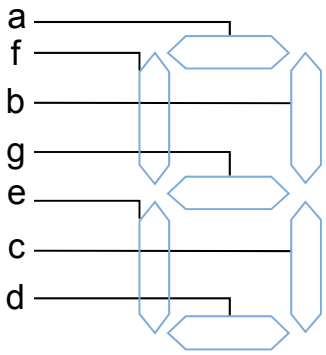
Seven Segment Decoder



***Source**: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 72*

Seven Segment Decoder

- We can determine a function for each segment, that indicates whether it should be lit for a specific input pattern





1111110

Inputs X[3:0]	Outputs Seg[0:6]						
Hexadecimal digits (binary)	Segments (1: on, 0: off)						
	a	b	c	d	e	f	g
0 (0000)	1	1	1	1	1	1	0
1 (0001)	0	1	1	0	0	0	0
2 (0010)	1						
3 (0011)	1						
4 (0100)	0						
5 (0101)	1						
6 (0110)	1						
7 (0111)	1	1	1	0	0	0	0
8 (1000)	1	1	1	1	1	1	1
9 (1001)	1	1	1	1	0	1	1
A (1010)	1						
B (1011)	0	0	1	1	1	1	1
C (1100)	1	0	0	1	1	1	0
D (1101)	0						
E (1110)	1						
F (1111)	1						

- Step 1: We can construct a truth table

Seven Segment Decoder

Inputs X[3:0] Hexadecimal digits (binary)	Outputs Seg[0:6] Segments (1: on, 0: off)						
	a	b	c	d	e	f	g
0 (0000)	1	1	1	1	1	1	0
1 (0001)	0	1	1	0	0	0	0
2 (0010)	1						
3 (0011)	1						
4 (0100)	0						
5 (0101)	1						
6 (0110)	1						
7 (0111)	1	1	1	0	0	0	0
8 (1000)	1	1	1	1	1	1	1
9 (1001)	1	1	1	1	0	1	1
A (1010)	1						
B (1011)	0	0	1	1	1	1	1
C (1100)	1	0	0	1	1	1	0
D (1101)	0						
E (1110)	1						
F (1111)	1						

- Step 2A: Create equations for each output from truth table
- Step 2B: Create a circuit corresponding to that output's equation

Kmap for a		X1,X0			
		00	01	11	10
X3,X2	00	1	0	1	1
	01	0	1	1	1
	11	1	0	1	1
	10	1	1	0	1

a = X3'X2X0 + X2X1 + X3'X1 + X3X0' + X3X2'X1' + X2'X0'

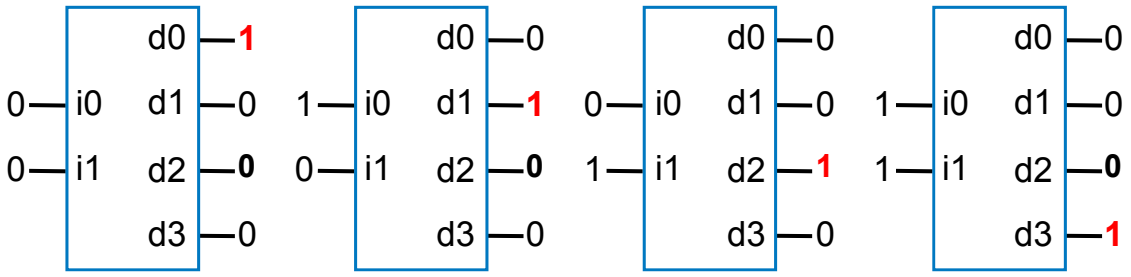
DECODER

Decoder

- Decoders are an important basic circuit, similar to the 7-Segment Decoder
- Take a binary input number, and output a corresponding one-hot output
  - Only one bit of the output is high, its position corresponds to the input value
  - Output width is always 2 to the power of input width
    - N-input Decoder: 2^N outputs

Input		Output
00	→	0001
01	→	0010
10	→	0100
11	→	1000

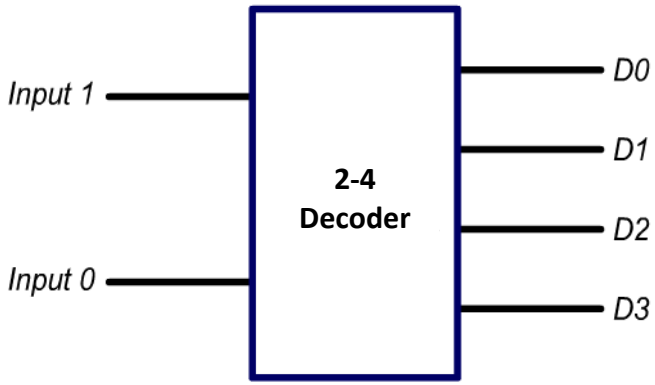
Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 84

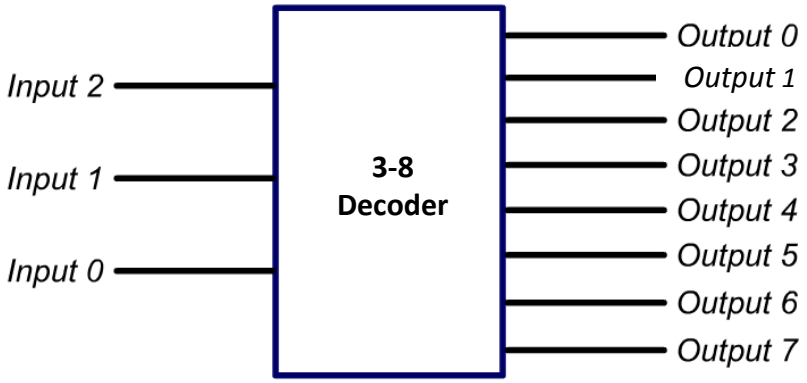


Example: A two-input decoder will have four outputs (2-4 Decoder)

Decoder

- In general, decoders can be referred to n-m decoders





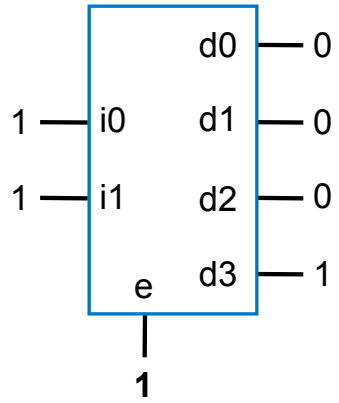
Input		Output			
1	0	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

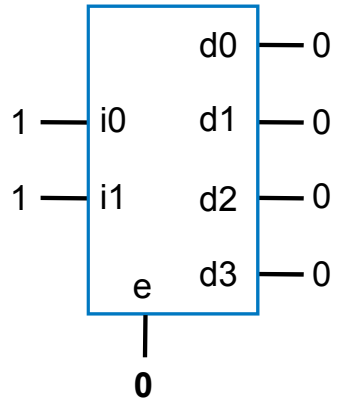
Input			Output							
2	1	0	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

***Source**: https://filebox.ece.vt.edu/~jgtront/introcomp/decoder.swf*

Decoder

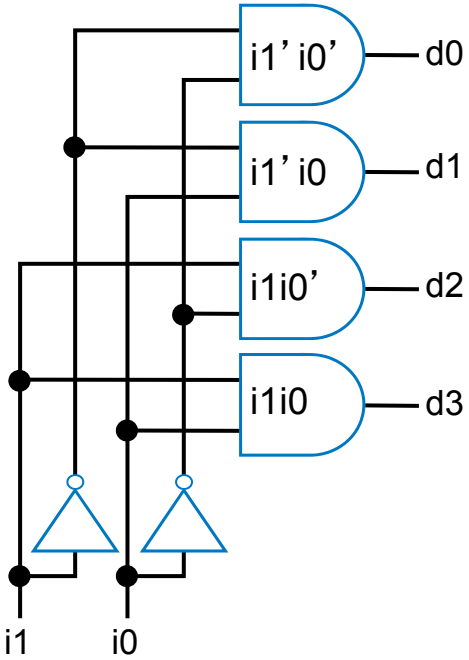
- Internal design
  - **AND** gate for each output to detect input combination
- Decoder with enable e
  - e = 0: Outputs all 0
  - e = 1: Regular behavior





Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 85

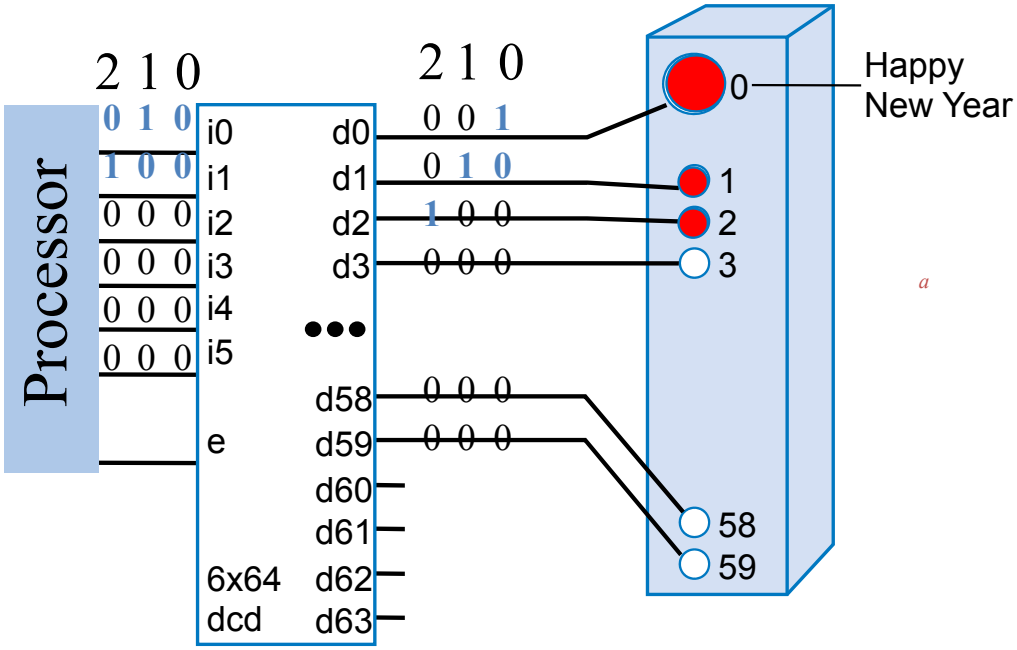
Input		Output
00	→	0001
01	→	0010
10	→	0100
11	→	1000



Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 84

Example: New Year's Eve Countdown Display

- Processor counts from 59 to 0 in binary
- Need to convert the 6-bit count to light up one bulb on the display
- 6-64 Decoder is used (4 outputs unused)



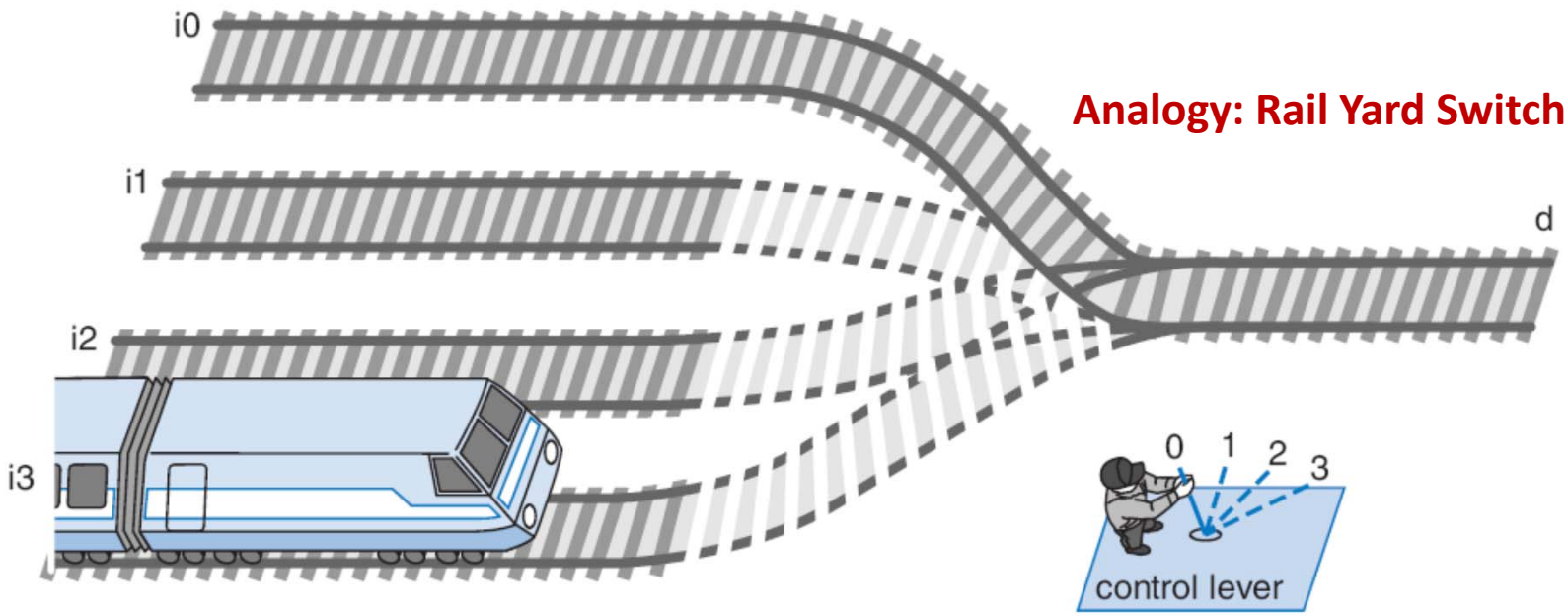
Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 86

NANYANG TECHNOLOGICAL UNIVERSITY

MULTIPLEXER

Multiplexer (Mux)

- Another important combinational circuit is the multiplexer (selects one from several inputs)
- Consists of multiple inputs, and a single output
- A select input determines which input should be connected to the output

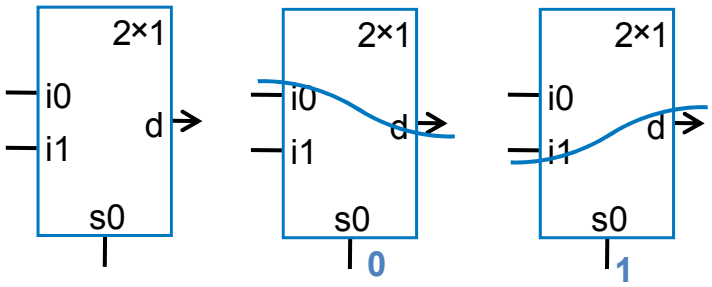


Analogy: Rail Yard Switch

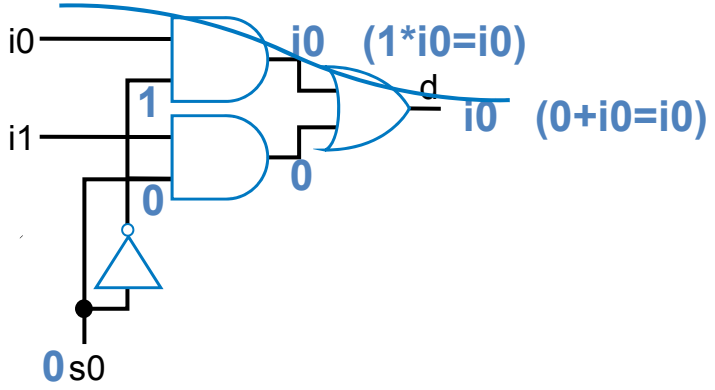
***Source**: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 87*

Multiplexer (Mux)

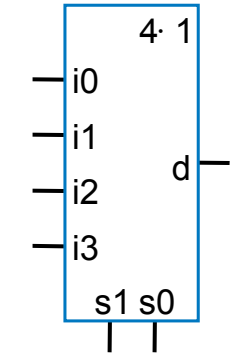
- Internal design of a 2x1 Mux



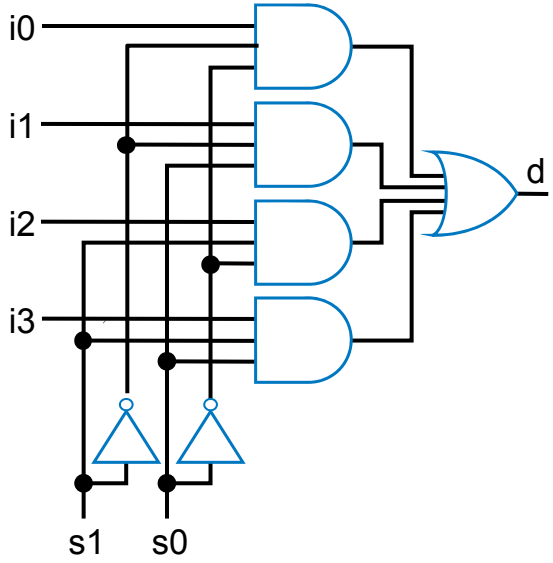
Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 87



- 2 input multiplexer (mux) needs a 1-bit select input
- 4 input mux requires a 2-bit select input
- An **n** input mux requires a **$\log_2(n)$**-bit select



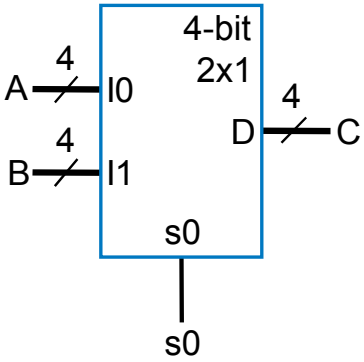
4x1 mux

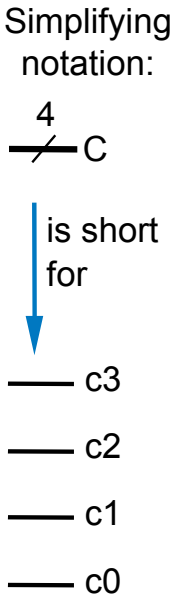


Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 88

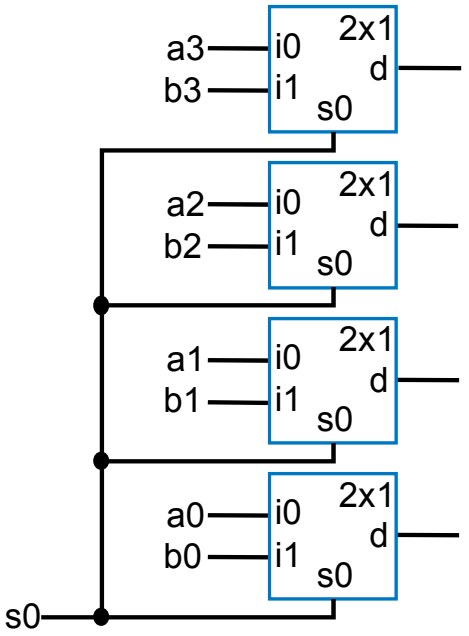
Multiplexer (Mux)

- We can combine Muxes to select multi-bit inputs, e.g. numbers
- A 4-bit 2x1 mux
  - Four 2x1 Muxes sharing the same select line to select between A and B





We will refer to this as a bus

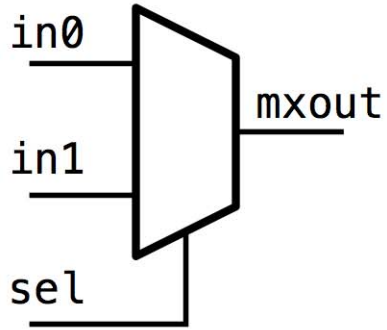


Source: Frank Vahid, "Digital Design", Wiley, 2nd Edition, pg. 89

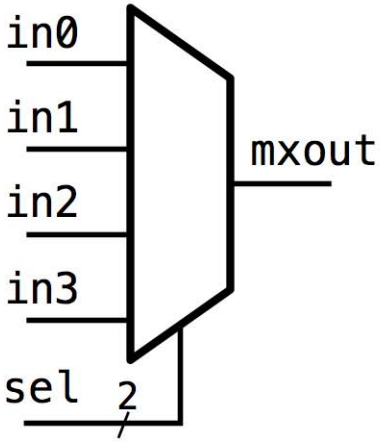
Multiplexer (Mux)

- Muxes are so common, they are often drawn using their own symbol:

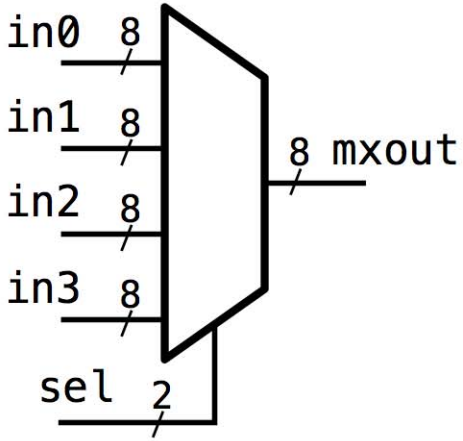
1-bit 2x1 mux



1-bit 4x1 mux



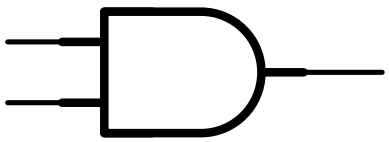
8-bit 4x1 mux



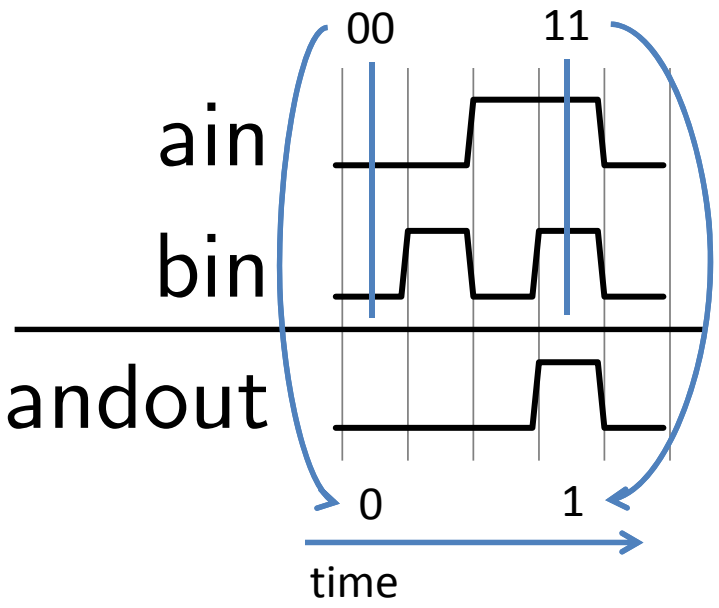
TIMING DIAGRAMS

Timing Diagrams

- Timing diagrams show the behavior of a circuit with progression of time
- Input values are changed and the resultant outputs shown
- Consider an **AND** gate:



- A timing diagram can show any combination or order of input values
- The output at any point is calculated by looking at the input values at that instance



Summary

- Combinational circuit always produces a corresponding output for each input combination

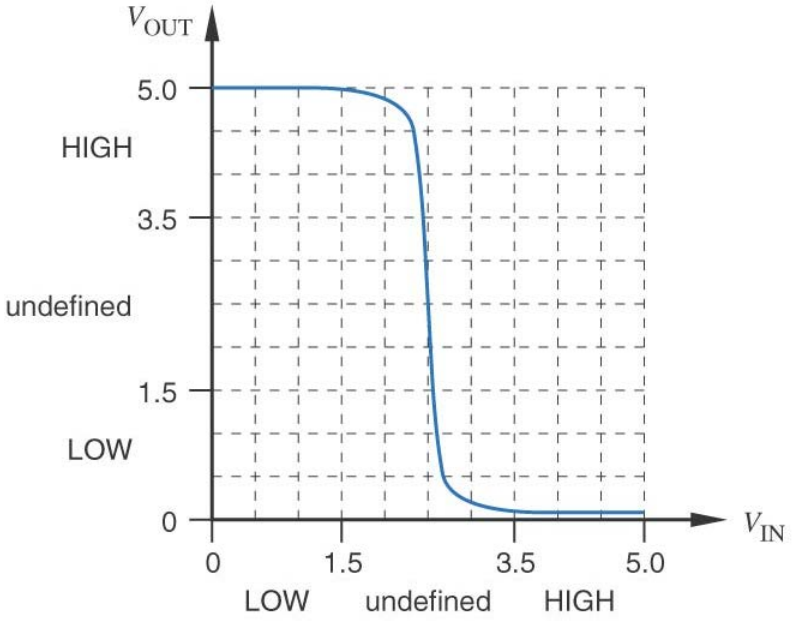
- Two-step process for implementing simple combinational circuits
  - Capture the function
  - Convert to circuit

- Commonly-used combinational circuits e.g. 7-segment decoder, decoders and multiplexers are used to build digital systems

L12 (7.1 – 7.15)

- Schmitt-trigger input
- Programmable logic array
- Fixed-point and floating-point numbers

Digital Circuits Characteristics



Not guaranteed

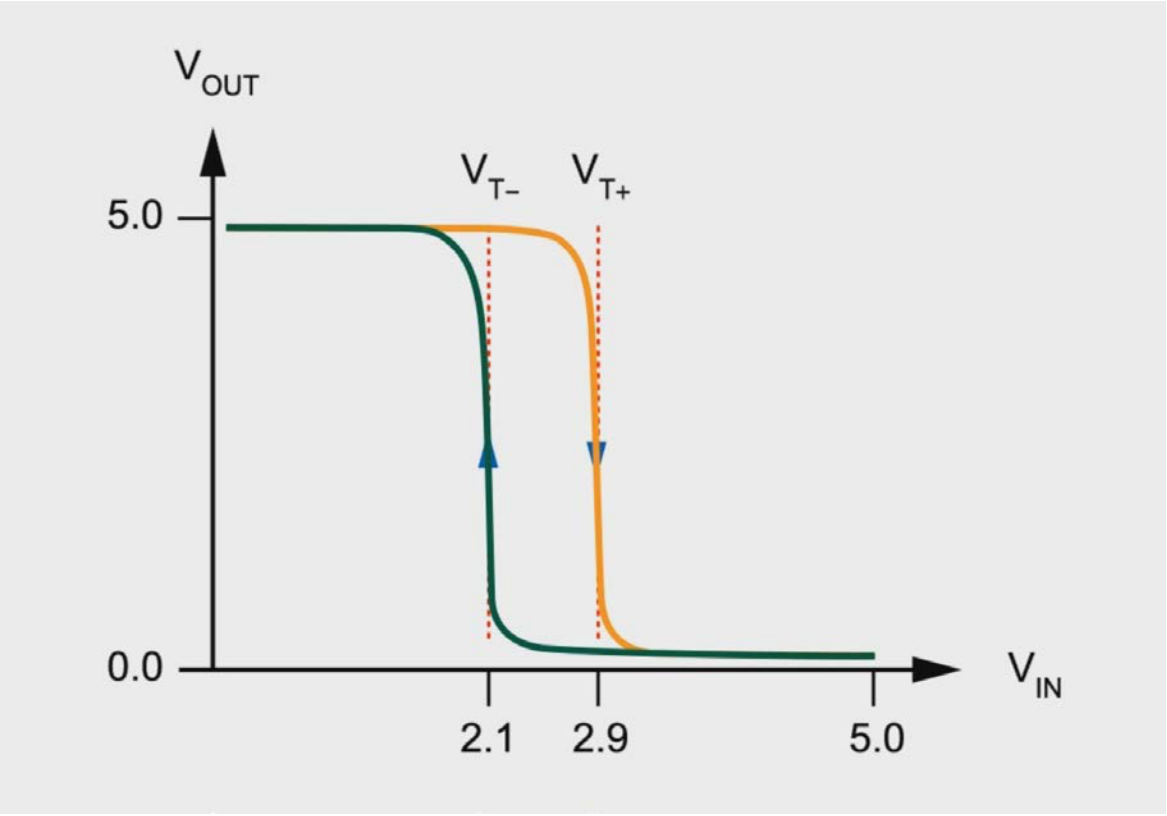
Fig. 3-25 Typical input-output transfer characteristic of a CMOS inverter

Schmitt-Trigger Inputs

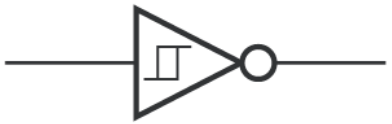
Fig. 3-47

- A Schmitt-Trigger inverter is characterised by 2 threshold voltages: VT- and VT+
- When input voltage V_{IN} rises above VT+, output V_{OUT} will switch to low.
- When input voltage V_{IN} drops below VT-, output V_{OUT} will switch to high.
- This non-symmetric behaviour is called hysteresis.
- The symbol  indicates Schmitt-Trigger input.

Schmitt-Trigger Inverter



(b)



(a) input-output transfer characteristic (b) logic symbol

Fig. 3-47 Schmitt-trigger inverter

Schmitt-Trigger Inverter

- Fig. 3-48
- When input voltage (V_{IN}) rises slowly, or has minor fluctuations around VT, a standard inverter will give unstable, fluctuating output.
- The output of a Schmitt-Trigger inverter in such situations has clean transitions and does not fluctuate.

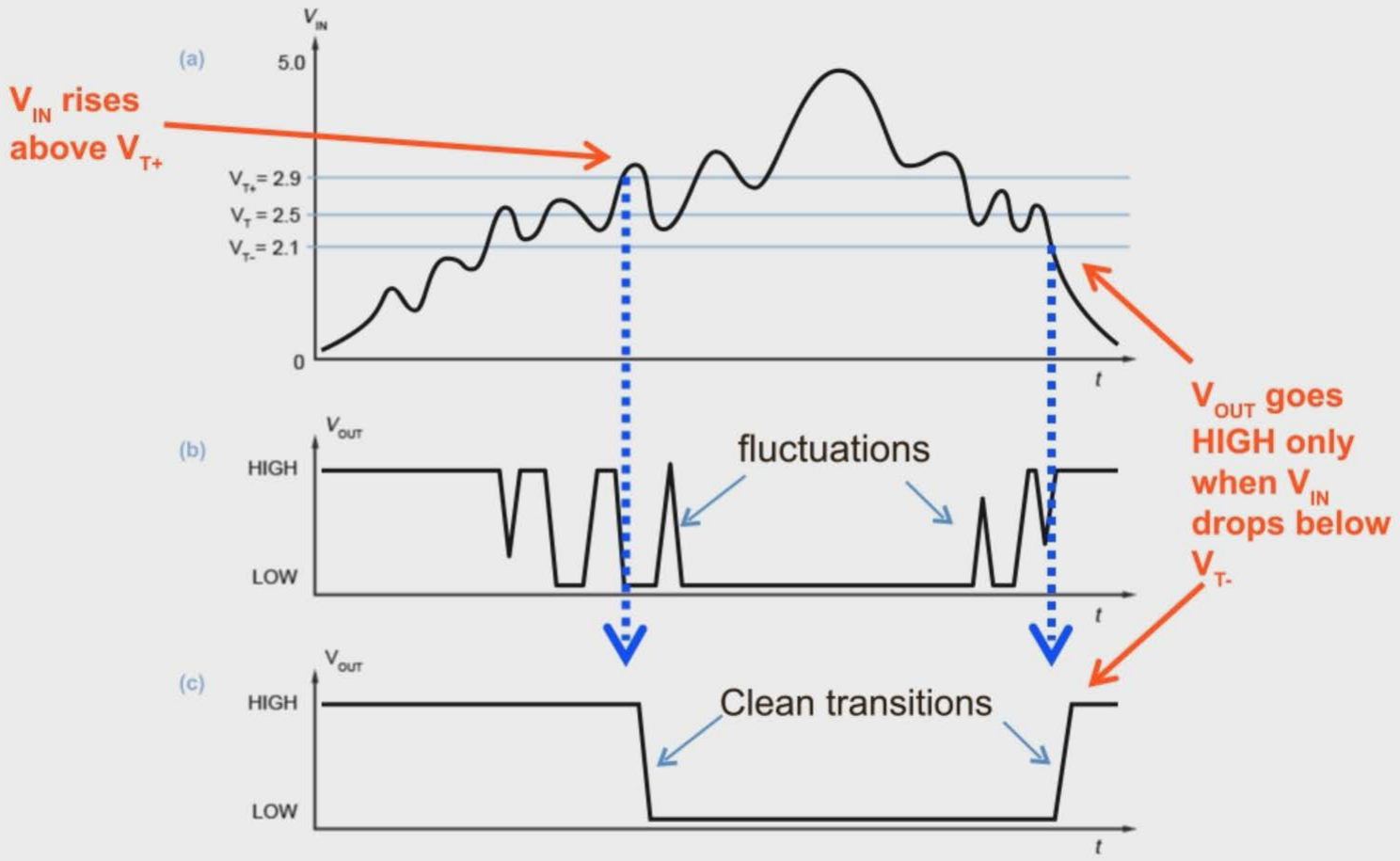


Fig. 3-48 Device operation with slowly changing inputs: (a) a noisy, slowly changing input; (b) output produced by an ordinary inverter; (c) output produced by an inverter with 0.8V of hysteresis.

Combinational PLA

- PLD = Programmable Logic Devices
- Examples are PLA, PAL, CPLD, FPGA
- PLA: Programmable Logic Array
- PAL: Programmable Array Logic
- CPLD: Complex PLD
- FPGA: Field Programmable Gate Array
- SOM/SOP Boolean expressions can be easily implemented on a PLA

PLA Example

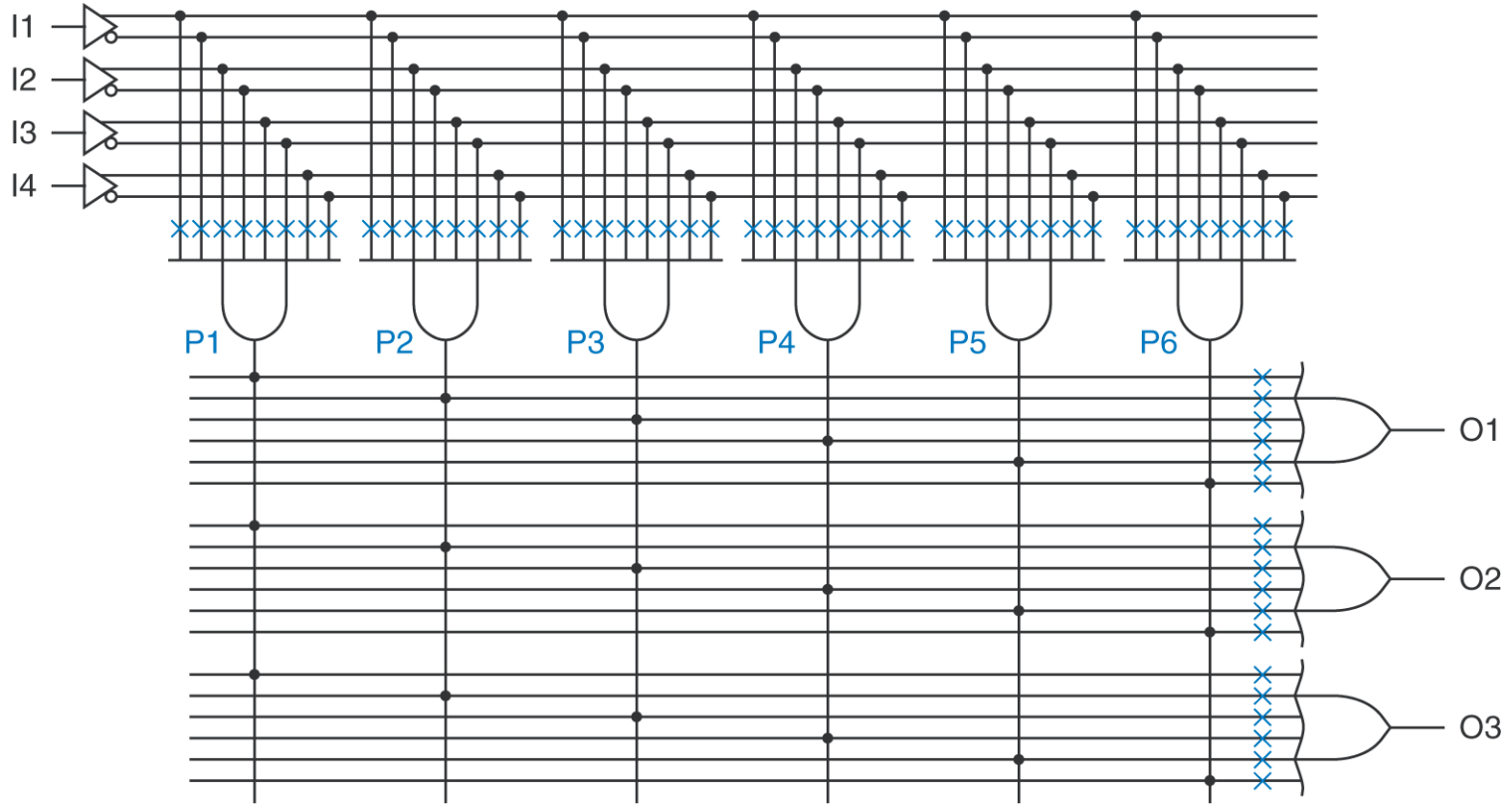


Figure 6-21

A 4 × 3 PLA with six product terms.

From *Digital Design*: *Principles and Practices*, Fourth Edition, John F. Wakerly, ISBN 0-13-186389-4.
©2006, Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

X : indicate programmable connections

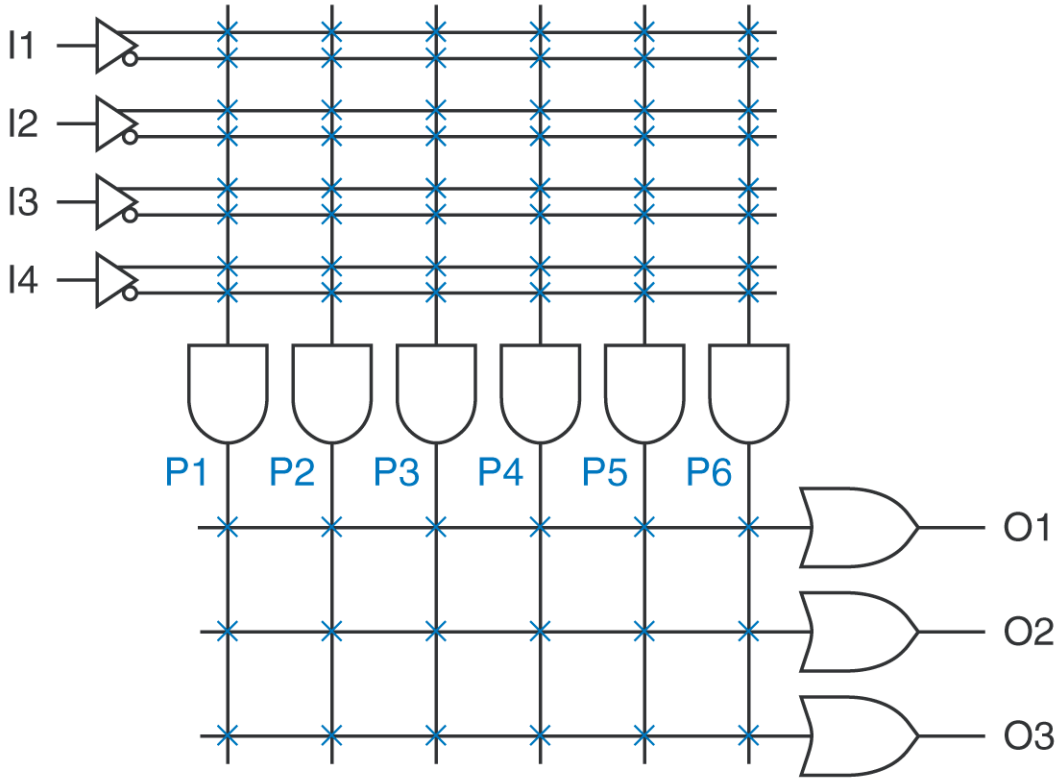
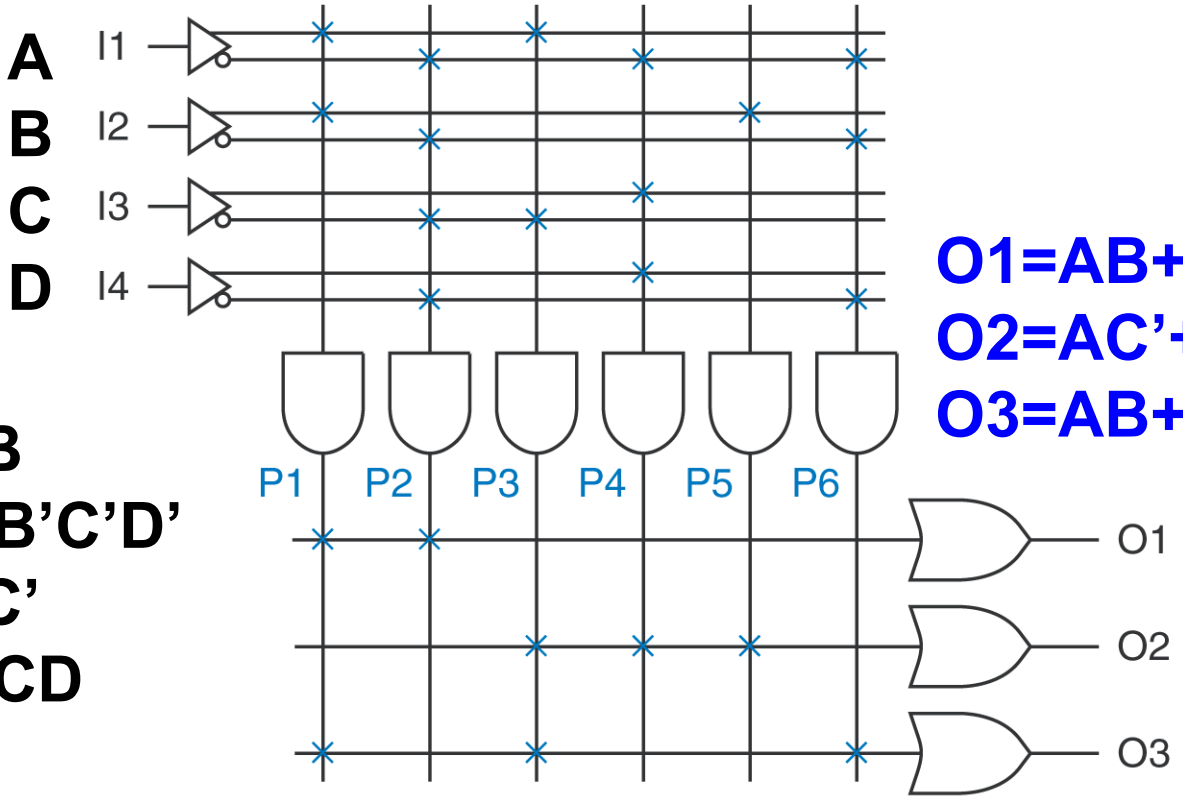


Figure 6-22

Compact representation of a 4 × 3 PLA with six product terms.

From *Digital Design*: *Principles and Practices*, Fourth Edition, John F. Wakerly, ISBN 0-13-186389-4.
©2006, Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

PLA Example: SOP implementation



O1=AB+A'B'C'D'
O2=AC'+A'CD+B
O3=AB+AC'+A'B'D'

P1=AB
P2=A'B'C'D'
P3=AC'
P4=A'CD
P5=B
P6=A'B'D'

Figure 6-23

A 4 × 3 PLA programmed with a set of three logic equations.

From *Digital Design*: *Principles and Practices*, Fourth Edition, John F. Wakerly, ISBN 0-13-186389-4.
©2006, Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

PLA Example (truth table)

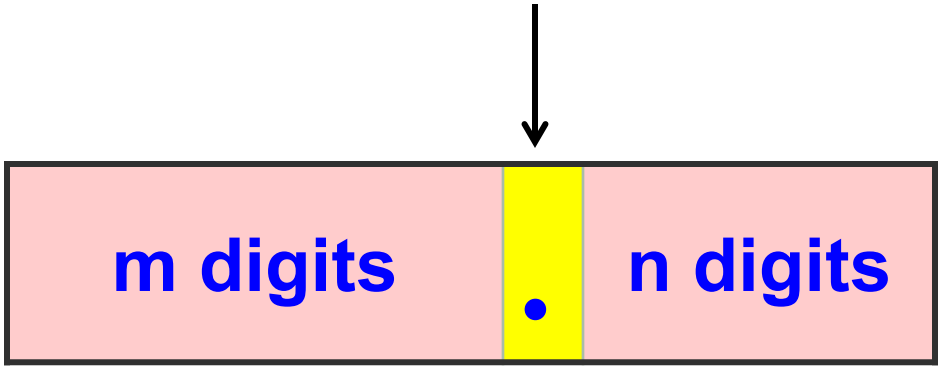
A	B	C	D	O1	O2	O3
0	0	0	0	1	0	1
0	0	0	1	0	0	0
0	0	1	0	0	0	1
0	0	1	1	0	1	0
0	1	0	0	0	1	0
0	1	0	1	0	1	0
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	0	1	1
1	0	0	1	0	1	1
1	0	1	0	0	1	0
1	0	1	1	0	1	0
1	1	0	0	1	1	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

Fixed-point numbers

Examples:

- 102_{10}
- 123.456_{10}
- $2000.AD_{16}$
- 1001.0101_{2}

The radix point has a fixed position among the integer and fraction digits



- **Fixed-point numbers are usually used for representing integers only**

- **Disadvantages of using fixed-point arithmetic for non-integers:**
  - **relatively small range of numbers that can be represented**
  - **possible loss of significant digits during computation**

Floating-point numbers

Machine equivalent of scientific notation

A floating-point number Z is represented as:

$$S \times R^E$$

e.g. 2.99792458×10^5

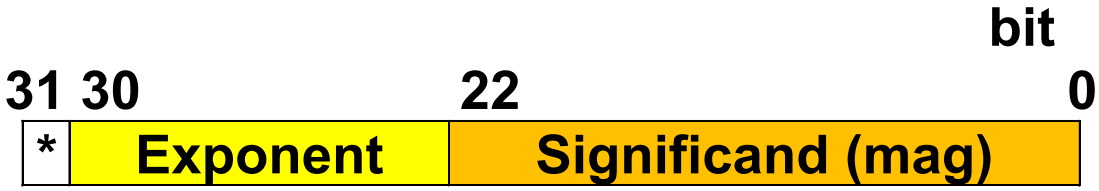
$$S \times R^E$$

Where

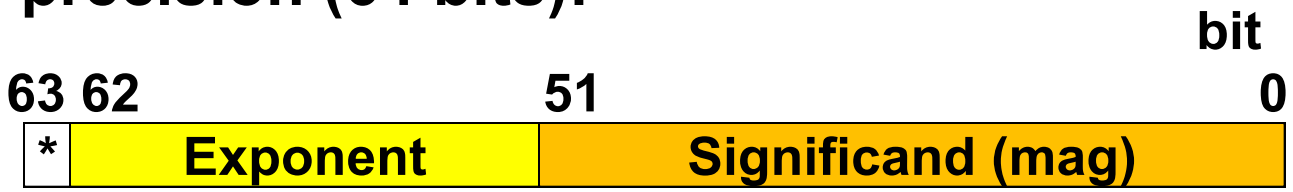
- S is the *significand*, usually a signed fractional fixed-point number
- R is the *radix* (e.g. 2 for binary, 8 for octal, 10 for decimal, etc.)
- E is the *exponent*, usually a signed fixed-point integer

Example: the IEEE Standard 754 on floating-point arithmetic

Single precision (32 bits):



Double precision (64 bits):



- * is the sign bit of significand
- radix is 2
- significand is in sign-magnitude notation
- exponent is in 2's complement notation

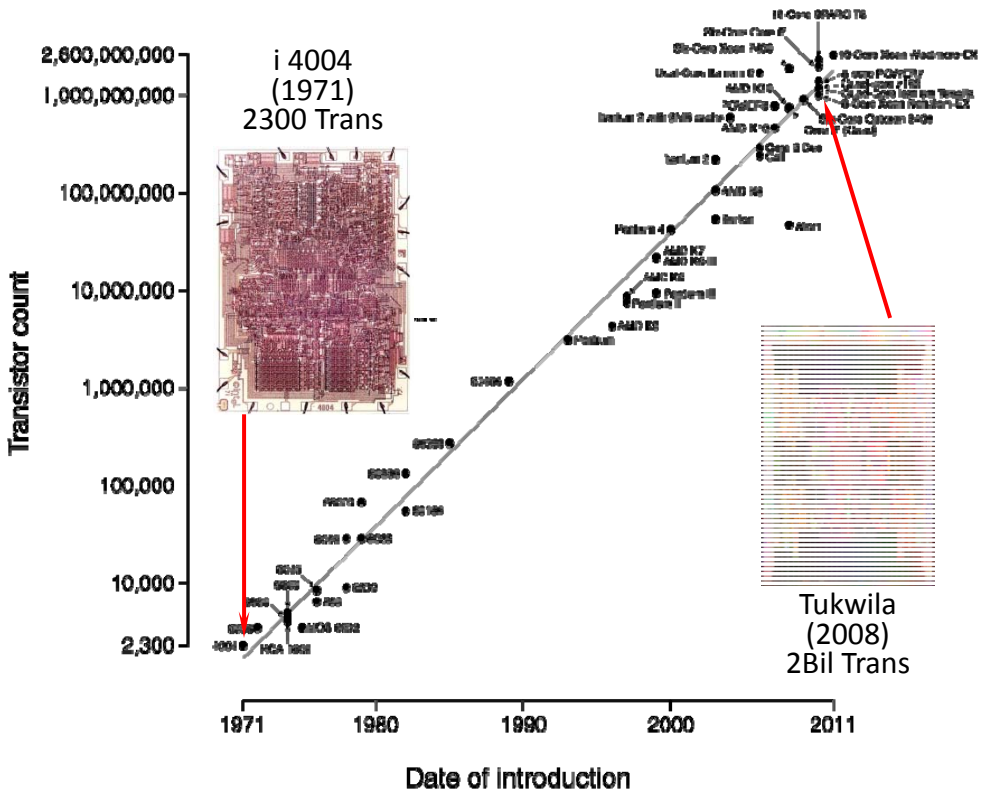
NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Introduction to Verilog

Where is this going?

- Designs are getting bigger and more complex
  - Designer productivity is an issue



So is working with individual gates going to allow you to design a 2 billion transistor CPU?

What about using Schematic Capture? Will that allow you to design a 2 billion transistor CPU?

So we need better Design Methods and Tools (EDA and CAD tools) to design large complex systems.

Verilog and the **FPGA tools** are starting you along this path…

Next, we will cover:

NANYANG TECHNOLOGICAL UNIVERSITY

- Introduction to Verilog HDL
  - Verilog is a hardware description language (HDL) and has some similarities to programming languages
- Combinational circuits in Verilog
  - Combinational circuits are circuits where the current outputs depend only on the current inputs
  - So far you have only looked at gate level combinational circuits
- Sequential circuits
  - Sequential circuits are circuits where the outputs depend on both the present input signals and on the past history of the inputs. That is it contains memory.
  - We will only consider synchronous (clocked) circuits.
- State machines
  - A state machine is a sequential circuit with a finite number of states which can be used to control the operation of physical devices

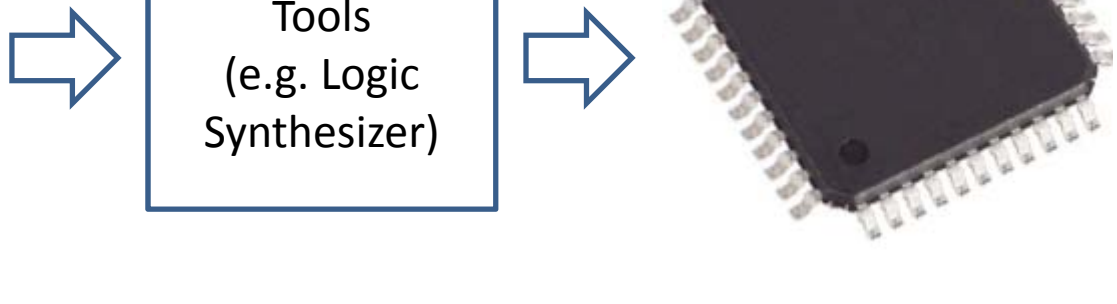
Hardware Description Languages (HDLs)

- Special languages with special constructs for describing hardware
- They can be used to describe hardware
  - At various levels of abstraction
  - Enables hierarchical design
- Sophisticated tools can then ensure the hardware generated from the description is efficient

```
module counter (clk,
reset, enable, count);
  input clk, reset,
enable;
  output [3:0] count;
  reg [3:0] count;

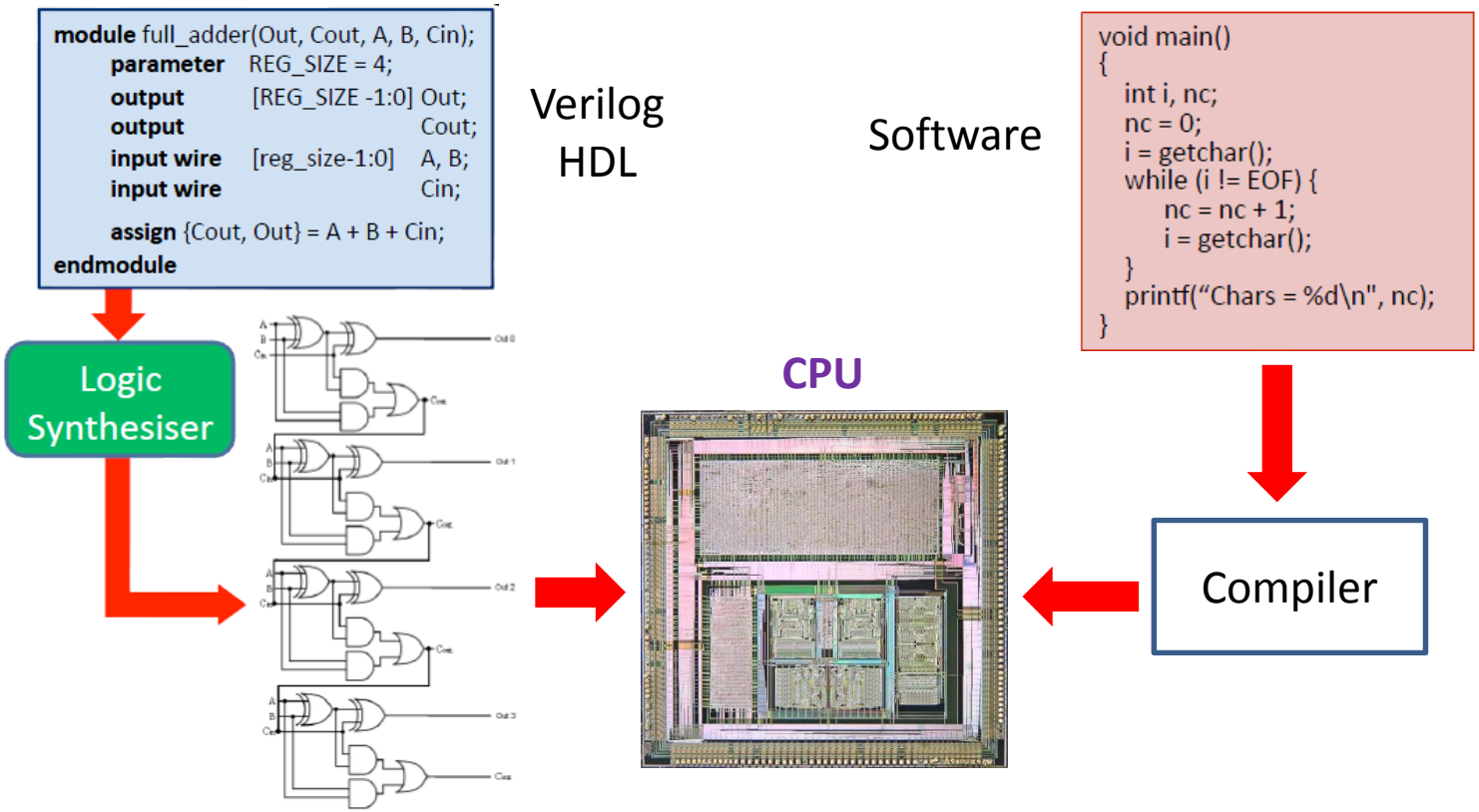
  always @ (posedge clk)
  if (reset == 1'b1)
     count <= 0;
else if ( enable == 1'b1)
     count <= count + 1;

endmodule
```



Hardware Description Languages (HDLs)

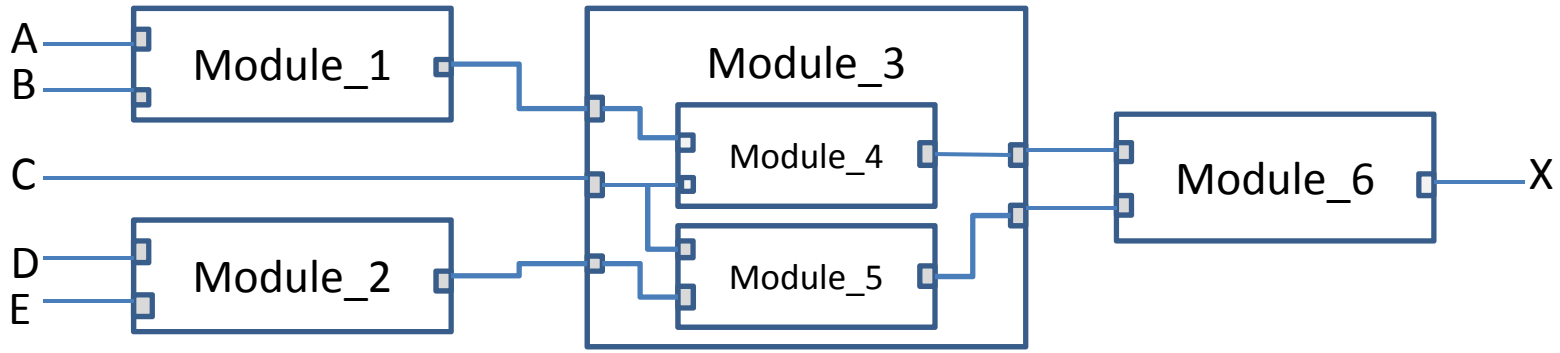
- HDLs are **programming-like** languages that are used to describe hardware
- HDLs are synthesized (and optimized) to hardware primitives
  - Note: Software is compiled to primitive instructions



MODULES

Module

- In Verilog, designs are broken down into ***modules***
- A ***module*** is a ***container*** the designer can use to ***encapsulate*** a unit of functionality
- Modules can contain code to describe hardware and also instances of other modules
- Good designs consist of sufficient (but not excessive) ***levels of hierarchy***, with modules containing instances of modules, that contain instances of other modules
- At each level in the hierarchy, a module instance is treated as a "black-box" – the internals are unknown



Verilog Module Declaration

NANYANG TECHNOLOGICAL UNIVERSITY

- We declare a module in Verilog using the **module** keyword and a list of ports:

```
module somename (port1, port2, port3);
```

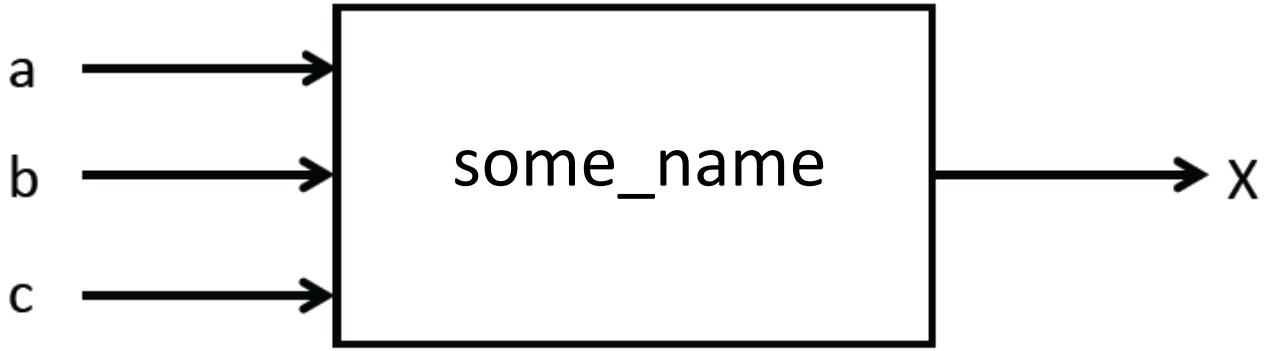
  - The above declaration describes a module with three **ports**, each a single wire

- We can indicate the direction of the ports using the keywords **input** and **output**:

```
module somename (input port1, port2,
                 output port3);
```

  - The above describes a module with two inputs and one output

Verilog Module Declaration




```
module some_name (
     input a, b, c,
     output X);

  // Describe your circuit here

endmodule
```

- The **endmodule** keyword indicates the end of statements that comprise the module description

Structural Design using Verilog

- Our first experience with Verilog will be for ***structural design***
- This allows us to ***structurally*** describe any circuit we might otherwise use a circuit diagram for
- The principles learnt in the first half of the course allow us to describe any circuit in terms of these gates

X	Y	C_i	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Truth table for Full Adder

$$S = X \oplus Y \oplus C_i$$

$$C_o = X.Y + X.C_i + Y.C_i$$

Minimize using Boolean Algebra or K-Maps

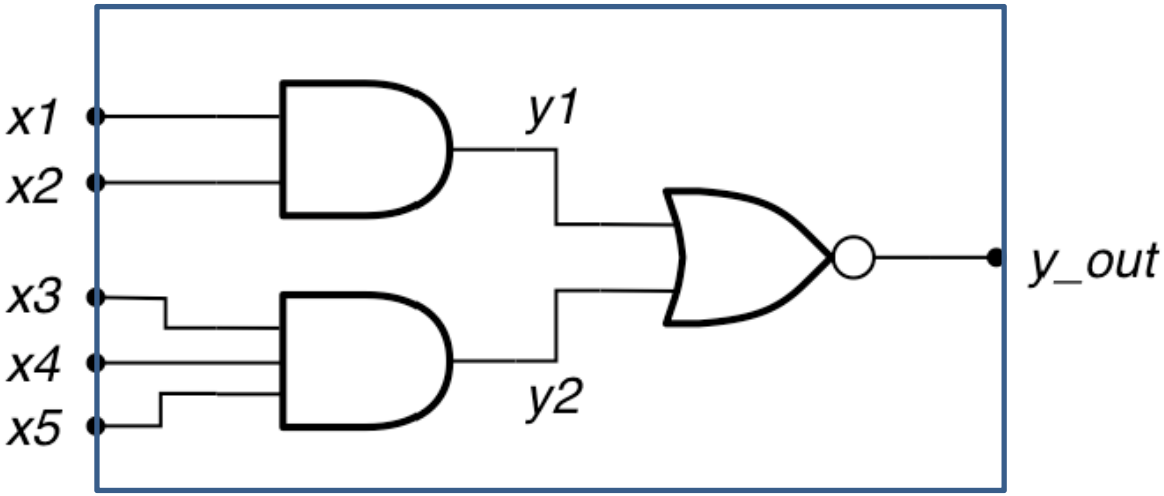
```
module full_adder (
      input X, Y, Ci,
      output S, Co);

  // Describe your structural
  // design for full adder here

endmodule
```

Structural Design using Verilog

- Describing a circuit by its internal structure
- To do this, we need:
  - A module definition – *done*
  - Some gates
  - Wires to connect those gates



We are trying to 'draw' the circuit using codes

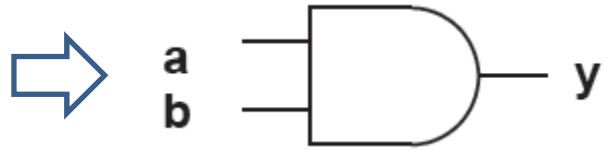
NANYANG TECHNOLOGICAL UNIVERSITY

INSTANTIATING GATE-LEVEL PRIMITIVES

Gate-Level Primitives

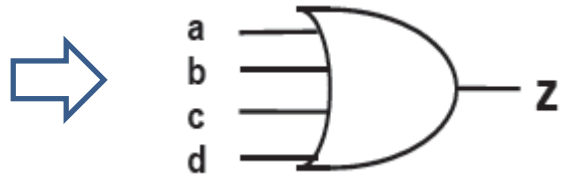
- Verilog provides us with basic ***primitives*** to model Boolean gates: and, nand, or, nor, not, xor, xnor

```
and (y, a, b);
```



- Represents an ***and*** gate with inputs connected to wires a and b, and output connected to wire y
- Gate primitives allow more than two inputs:

```
or (z, a, b, c, d);
```



- This represents an ***or*** gate with inputs a, b, and c, d and output z

The (single) output is always the first argument

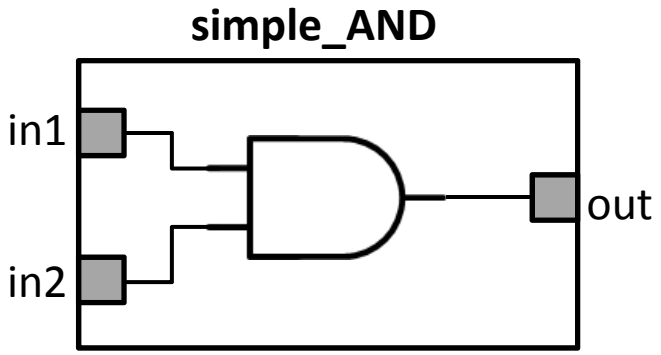
Verilog Module with Gate Level Primitives

- A simple module that contains just a two-input ***and*** gate would be written like this:

```
module simple_AND (
       input in1, in2,
       output out);

       and (out, in1, in2);

endmodule
```



- The module's ports act the same as wires (but you **cannot** connect the output of a gate to an input port!)

  Of course, we wouldn't create a module for a single *and* gate since the primitive already exists

Wires

- We can declare ***internal wires*** in a module, using the **wire** keyword:

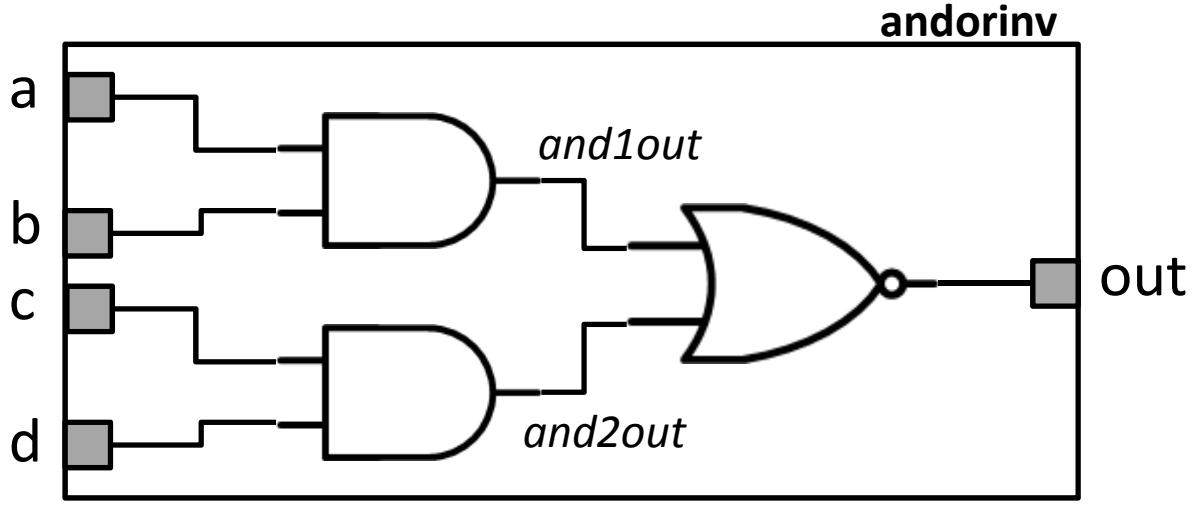
```
wire int_signal;
```



- This creates a named (1-bit) wire that can be used to connect gates together
  - Note: we will see later that it is not always necessary to declare 1-bit wires

Verilog Module with Gate Level Primitives

- Let us consider an ***and-or-inverter***
- Boolean equation: out = ((a•b)+(c•d))'




```
module andorinv (input a, b, c, d,
                 output out);

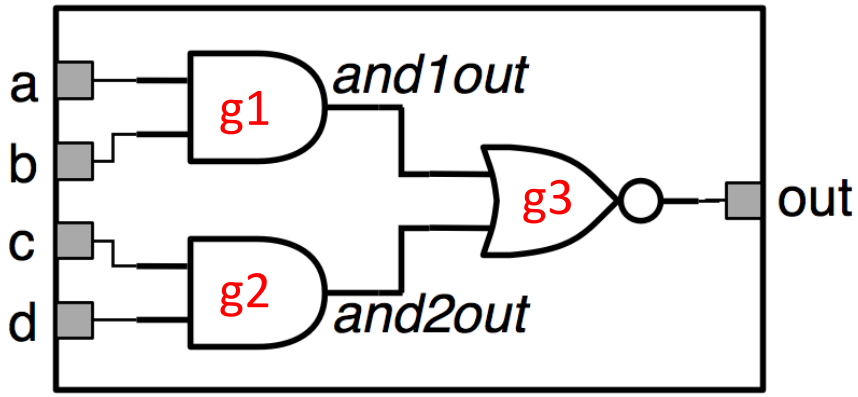
      wire and1out, and2out;

      and (and1out, a, b);
      and (and2out, c, d);
      nor (out, and1out, and2out);

endmodule
```

Verilog Module with Gate Level Primitives

- Note you can also use of a unique ***identifier*** for each gate (this can help in testing):



Gate Identifiers

```
module andorinv (input a, b, c, d,
                 output out);

        wire and1out, and2out;

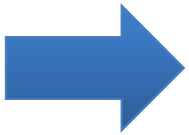
        and g1 (and1out, a, b);
        and g2 (and2out, c, d);
        nor g3 (out, and1out, and2out);

endmodule
```

Verilog Module with Gate Level Primitives

- When declaring module ports of the same type and direction, no need for separate declarations, can just use a comma between names:

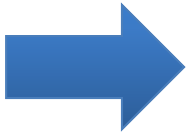
```
module hmm (input in1,
            input in2,
            input in3,
            output out);
```



```
module hmm (input in1,
                 in2, in3,
                 output out);
```

- Also, when declaring multiple wires of the same type, there is no need for separate declarations, use a comma between names:

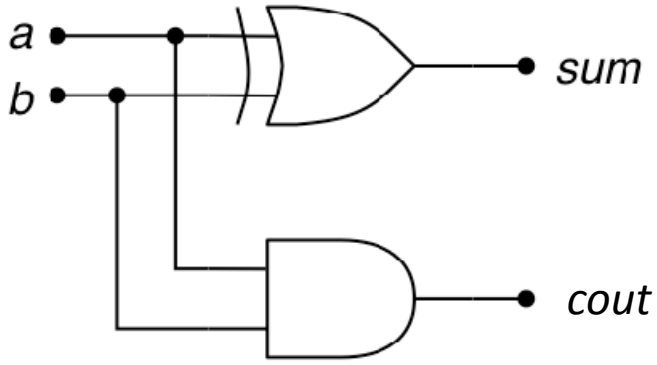
```
wire red;
wire blue;
wire green;
```



```
wire red, blue, green;
```

Example: Half Adder in Verilog

- Lets re-look at the binary adder
- A ***half-adder*** takes two 1-bit inputs and produces a 1-bit **sum** output and a 1-bit carry out (**cout**)
- We can see that **sum** is just an ***xor*** function and **cout** is just an ***and*** function



a	b	sum	cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

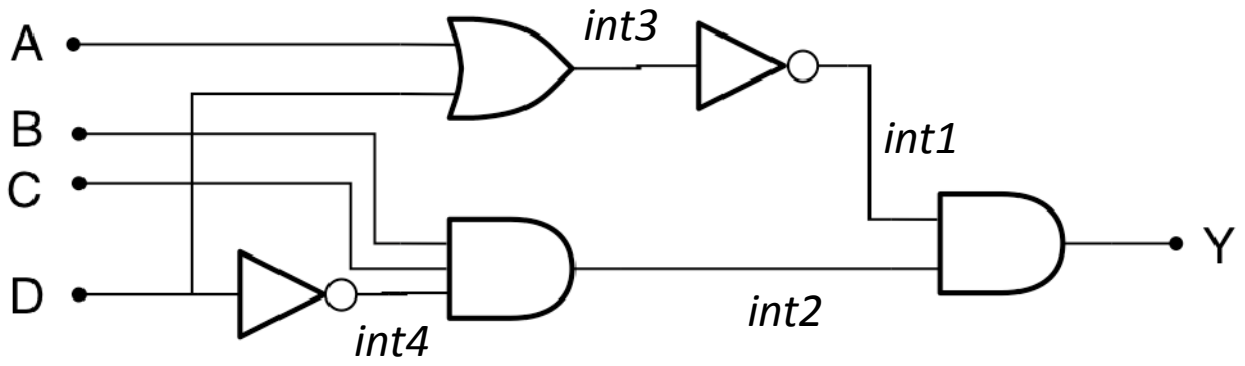
```
module add_half (
  input a, b,
  output sum, cout);

  xor g1 (sum, a, b);
  and g2 (cout, a, b);
endmodule
```

Note: g1 and g2 are gate identifiers

Exercise

- Implement a Verilog module using structural gate-level primitives for the following circuit:

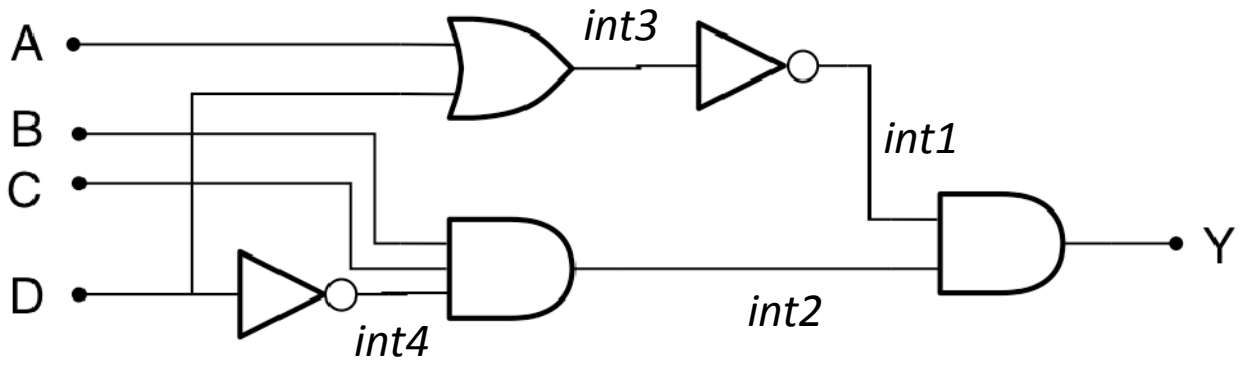



```
module simple_circ (input A, B, C, D, output Y);
   wire int1, int2, int3, int4;

   and (Y, int1, int2);
   not (int1, int3);
   or  (int3, A, D);
   and (int2, B, C, int4);
   not (int4, D);
endmodule
```

Exercise

- Implement a Verilog module using structural gate-level primitives for the following circuit:




```
module simple_circ (input A, B, C, D,
                    output Y);

   wire int1, int2, int3, int4;

   and (Y, int1, int2);
   not (int1, int3);
   or  (int3, A, D);
   and (int2, B, C, int4);
   not (int4, D);
endmodule
```

```
module simple_circ (input A, B, C, D,
                    output Y);

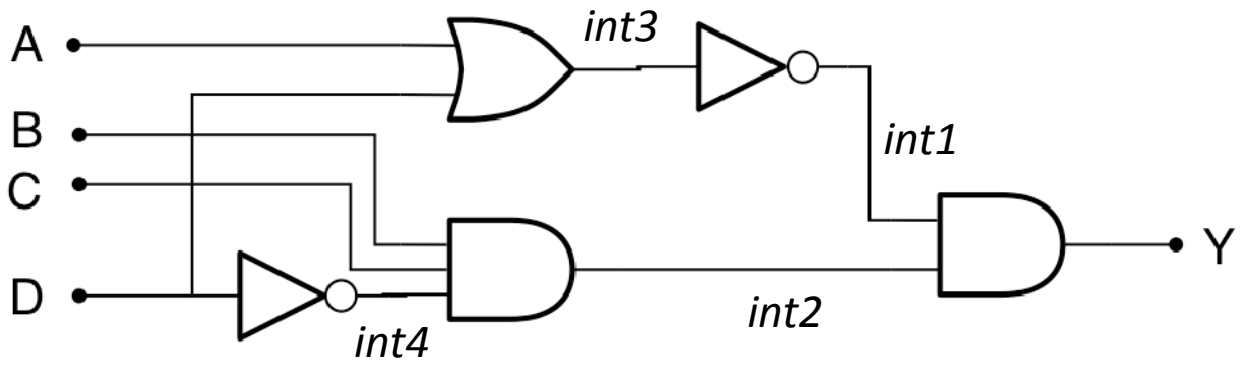
   wire int1, int2, int3, int4;

   not (int1, int3);
   not (int4, D);
   and (int2, B, C, int4);
   and (Y, int1, int2);
   or  (int3, A, D);
endmodule
```

These two implementations will produce the same circuit

Verilog Module with Gate Level Primitives

- Note that the **order** of statements is **unimportant**
- Each statement describes a piece of hardware
- There is no sequence of steps when doing structural Verilog
- Usually we declare internal signals first, and then instantiate gates



A Note on Wires

- There is no need to declare 1-bit wires for connecting modules
- Tools will infer these wires automatically by matching names
- But **only** for 1-bit wires in instantiation:

```
module simple_circ (input A, B, C, D,
                    output Y);

   wire int1, int2, int3, int4;

   and (Y, int1, int2);
   not (int1, int3);
   or  (int3, A, D);
   and (int2, B, C, int4);
   not (int4, D);
endmodule
```

So, instead of this:

```
module simple_circ (input A, B, C, D,
                    output Y);

   and (Y, int1, int2);
   not (int1, int3);
   or  (int3, A, D);
   and (int2, B, C, int4);
   not (int4, D);

endmodule
```

This is OK!

Additional Notes

- Verilog is **case-sensitive!**
  - FIFO_out, fifo_out and Fifo_Out refer to different things
- Statements are terminated with a semi-colon (;)
- Comments are ***C-style***:
  - // starts a single line comment
  - /* is used for block comments */ – cannot be nested!
- Whitespace is ignored – But you **should** use it to make your code readable
- ***Identifiers*** in Verilog:
  - Must start with **a letter** or **underscore** (_)
  - Can contain **letters**, **numbers**, **underscore**, and **dollar** ($)
  - Must not clash with keywords (e.g**. module** , **input**, etc.)

```
module and_or_inv (input a, b, c, d,
                   output out);

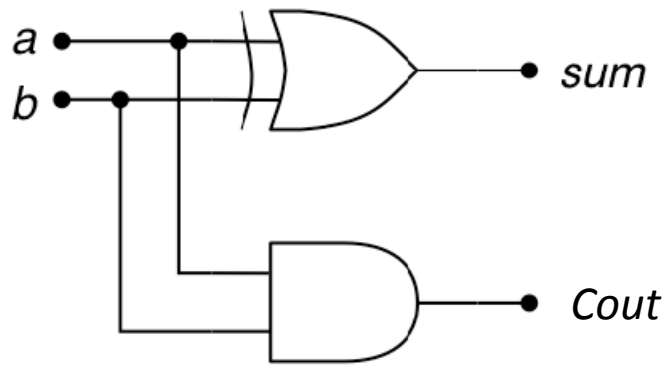
  wire and1out, and2out;

  and g1 (and1out, a, b); // AND gate
  and     g2 (and2out, c, d);
  nor g3 (out, and1out,   // NOR gate
          and2out);

endmodule
```

MODULE INSTANTIATION

Half Adder and Full Adder




```
module add_half (
   input a, b,
   output sum, Cout);

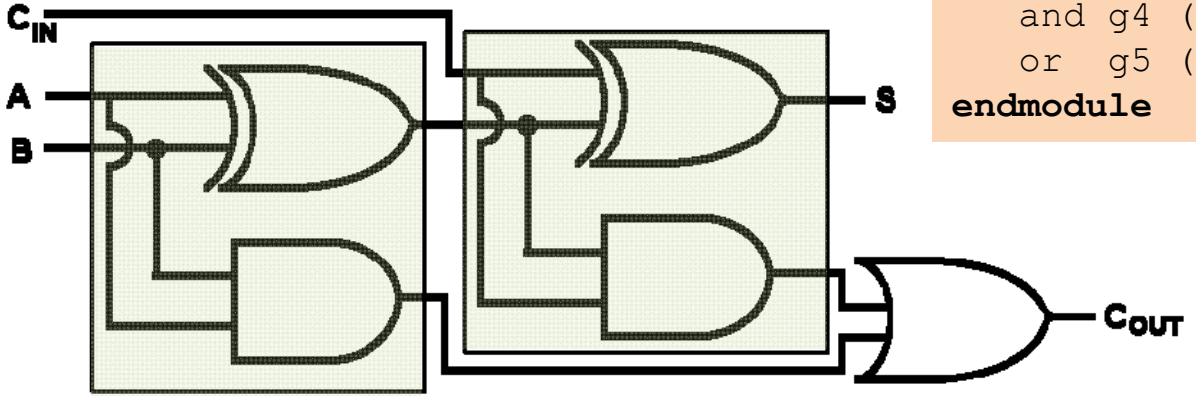
   xor g1 (sum, a, b);
   and g2 (Cout, a, b);
endmodule
```

A ***full adder*** can be implemented using 2 Half-Adders and an OR gate (see slides on Digital Arithmetic)

```
module full_add (input A, B, Cin
                 output sum, Cout);
   wire w1, w2, w3;

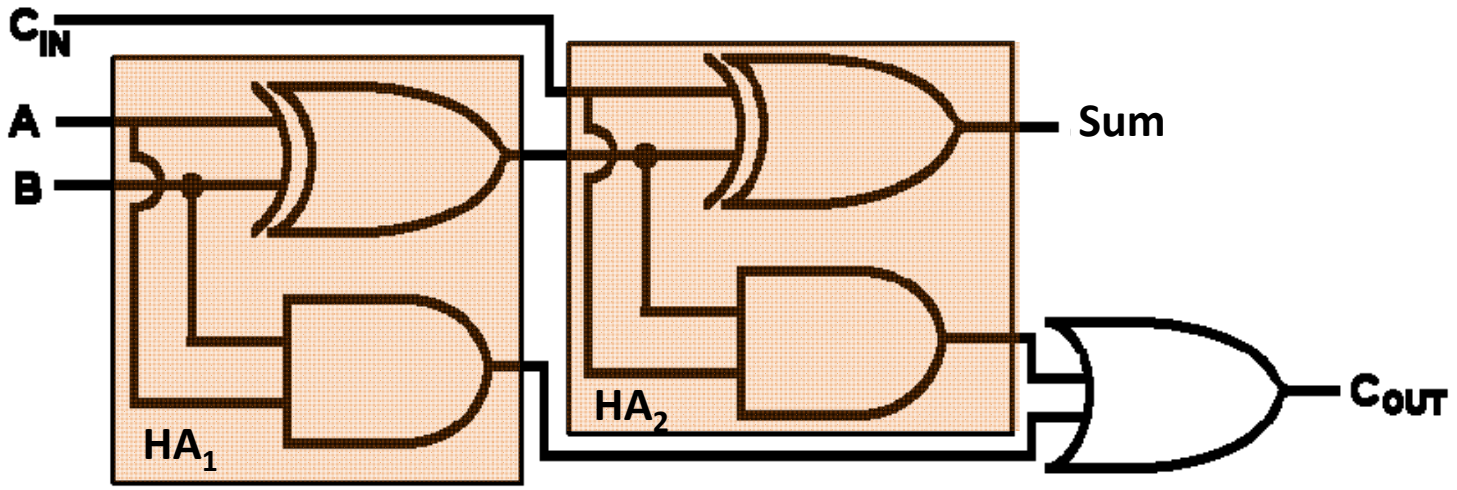
   xor g1 (w1, A, B);
   and g2 (w2, A, B);
   xor g3 (sum, Cin, w1);
   and g4 (w3, Cin, w1);
   or  g5 (Cout, w2, w3);
endmodule
```

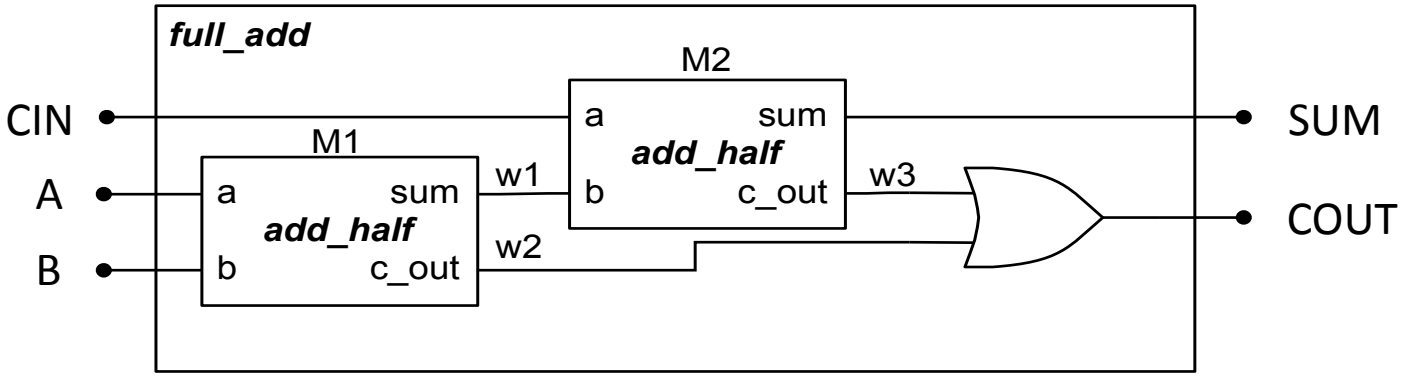
This is tedious!



Instantiation in Verilog

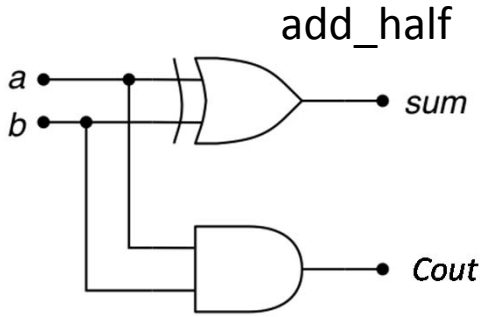
- In Verilog, we can use existing modules within new modules through ***instantiation***
- For example, we can ***instantiate*** the half adder module we previously designed **within** a new module

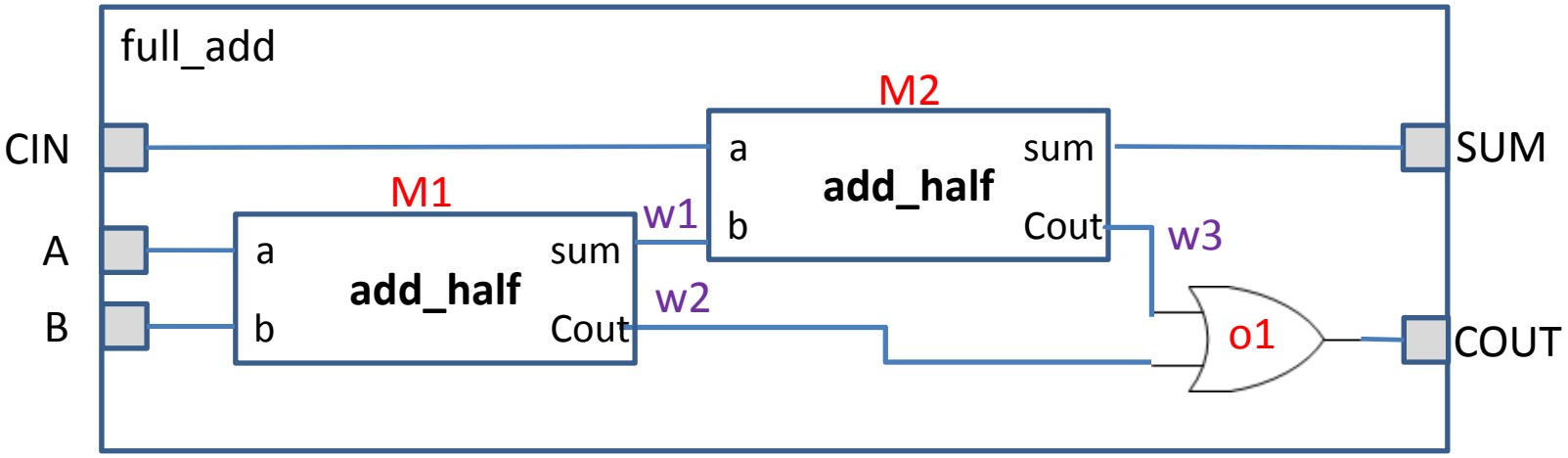




Instantiation in Verilog (Full Adder)

```
module add_half (input a, b,
                 output sum, Cout);
  xor g1 (sum, a, b);
  and g2 (Cout, a, b);
endmodule
```






```
module full_add (input A, B, CIN,
                 output SUM, COUT);
    wire  w1, w2, w3;

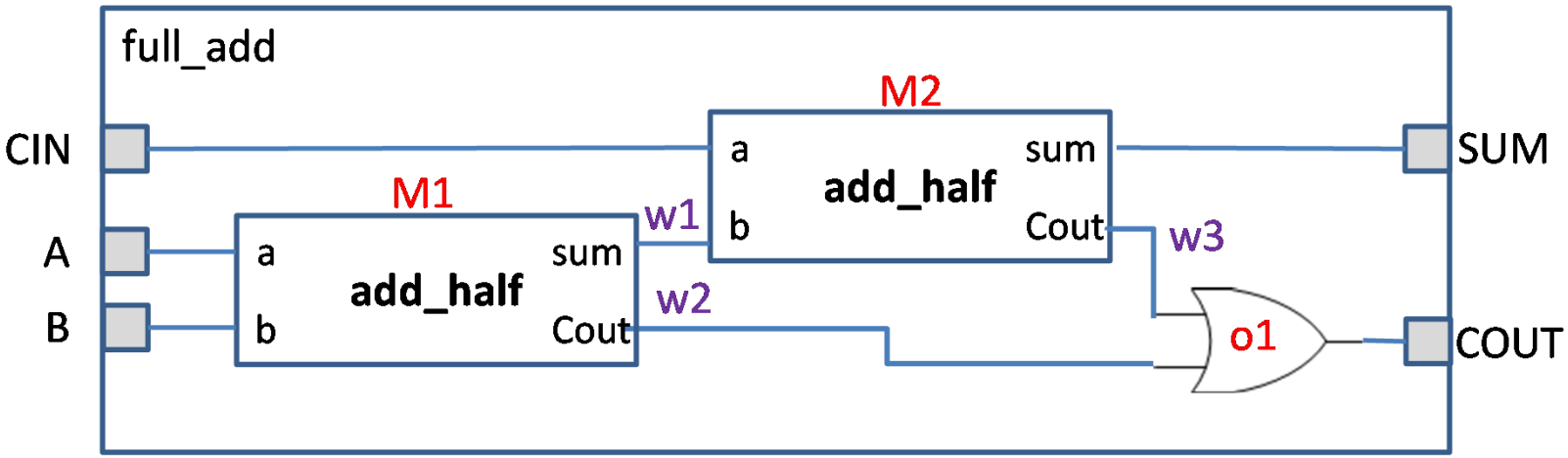
    add_half M1 (A, B, w1, w2);
    add_half M2 (CIN, w1, SUM, w3);
    or o1 (COUT, w3, w2);
endmodule
```

Instantiation in Verilog

- We ***instantiate*** a module by invoking its name and giving an ***instance*** a name:

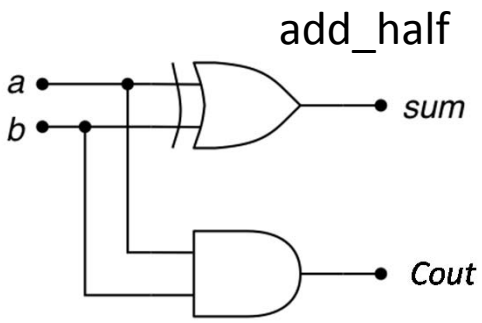
```
add_half M1 (A, B, w1, w2);
```

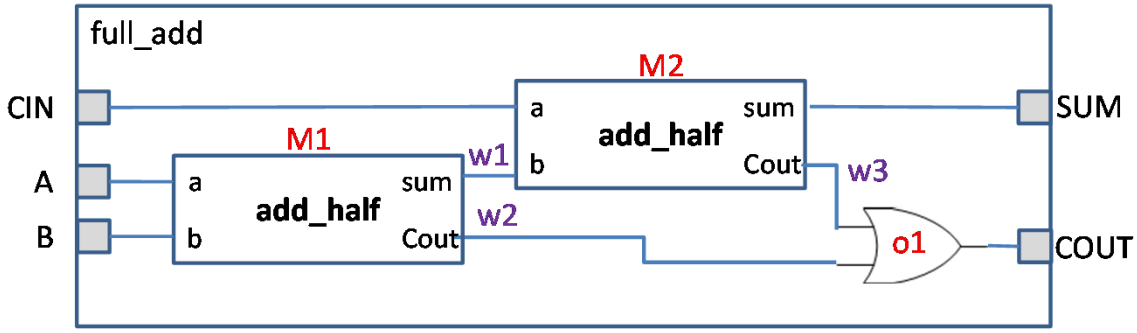
- Creates an ***instance*** called **M1** of module **add_half**
- It then **connects the signals and ports** referenced in the parentheses with the corresponding ports of the instantiated module
  - Just like we did previously with gate-level primitives



Ordered Instantiation

- The **order** determines connections
- Looking at the original **add_half** module declaration:
- Its first port is its ***a*** input, the second its ***b*** input, the third its ***sum*** output and the fourth its ***Cout*** output
- Connects a wire called ***A*** in the outer module with the ***a*** port, ***B*** with the ***b*** port, the ***w1*** wire to its ***sum*** output and the ***w2*** wire to its ***Cout*** port






```
module add_half (input a, b,
                 output sum, Cout);
  xor g1 (sum, a, b);
  and g2 (Cout, a, b);
endmodule
```

```
add_half M1 (A, B, w1, w2);
```

```
add_half M2 (CIN, w1, SUM, w3);
```

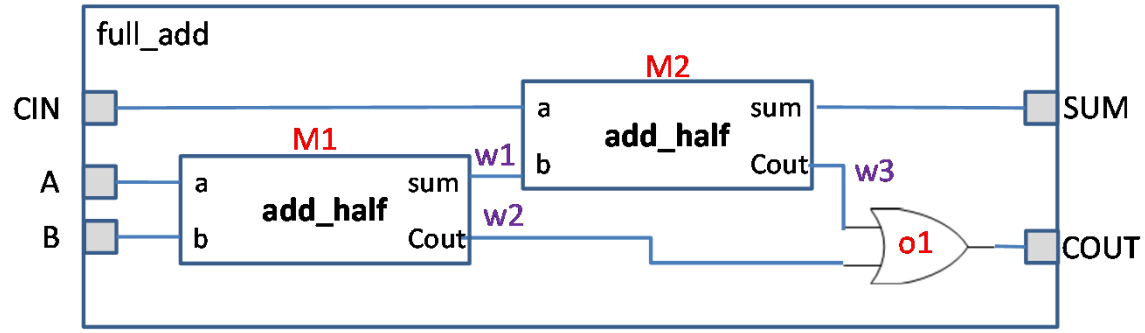
Named Instantiation

- Connecting instantiated modules to wires and ports in the manner just described can be **error-prone**:
  - Must remember the order of the ports
  - If we change the port order or add/remove ports, we have to change the instantiation
- Hence, we generally use a ***named connection***:

```
add_half M1 (.a(A),
             .b(B),
             .sum(w1),
             .Cout(w2));
```

```
add_half M2 (.a(CIN),
             .b(w1),
             .sum(SUM),
             .Cout(w3));
```

The port name is preceded by a dot, and the connected signal is placed in the brackets



Instantiation in Verilog

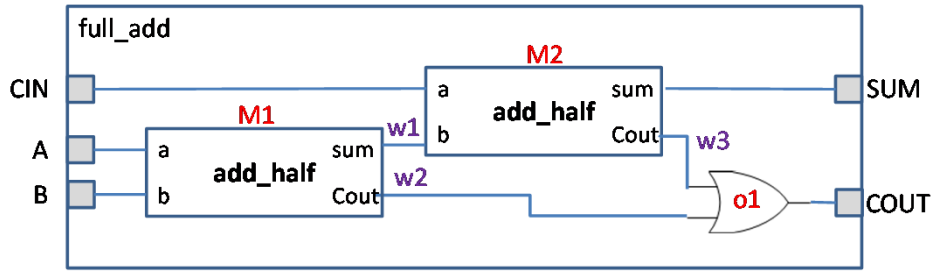
Ordered Instantiation

```
module full_add  (input A, B, CIN,
                  output SUM, COUT);
    wire  w1, w2, w3;

    add_half M1 (A, B, w1, w2);
    add_half M2 (CIN, w1, SUM, w3);
    or O1 (COUT, w3, w2);

endmodule


module add_half  (input a, b,
                  output sum, Cout);
    xor G1 (sum, a, b);
    and G2 (Cout, a, b);
endmodule
```



Named Instantiation

```
module full_add  (input A, B, CIN,
                  output SUM, COUT);
    wire  w1, w2, w3;

    add_half M1 (.a(A), .b(B),
                 .sum(w1),
                 .Cout(w2));

    add_half M2 (.a(CIN), .b(w1),
                 .sum(SUM),
                 .Cout(w3));

    or O1 (COUT, w3, w2);
endmodule

module add_half  (input a, b,
                  output sum, Cout);
    xor G1 (sum, a, b);
    and G2 (Cout, a, b);
endmodule
```

NANYANG TECHNOLOGICAL UNIVERSITY

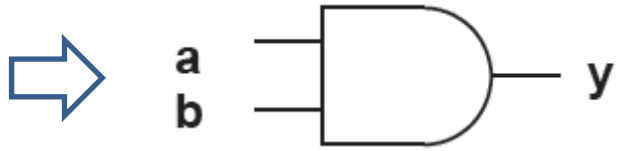
VERILOG ASSIGNMENTS

Verilog Assignments

- Implementing larger circuits using individual gates can be tedious.

Instantiating gate primitive

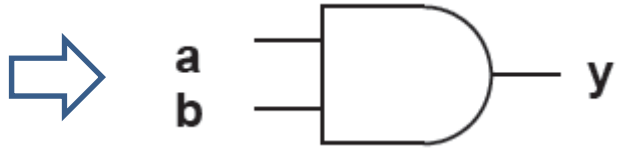
```
and (y, a, b);
```



- Verilog allows us to use combinational logic ***expressions*** through the **assign** keyword:

Continuous assignment

```
assign y = a & b;
```



- This is called a ***continuous assignment***
  - As it is always permanently assigned
- It allows us to assign the result of a Boolean expression to a signal

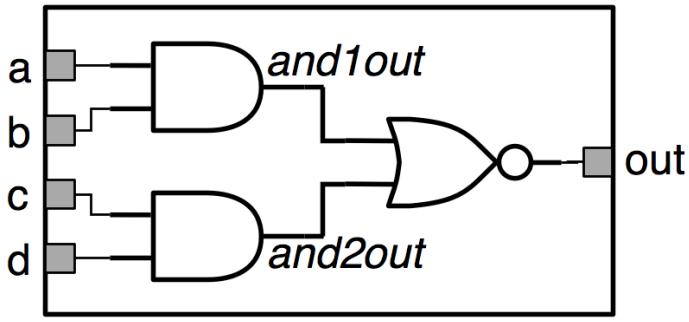
This is much simpler than using gate primitives

Verilog Assignments

- We can use a range of operators:
  - **&&** : and (bitwise is: **&**)
  - **||** : or (bitwise is: **|**)
  - **!** : not (bitwise is: **~**)
  - **^** : xor (bitwise is: **^**)

Operator	Name
~	Bitwise NOT
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~&	Bitwise NAND
~|	Bitwise NOR

- The and-or-inverter example previously shown:
  - The ***assign statement*** allows the use of more complex operators and operands – more on this later



Instantiating gate primitives

```
wire and1out, and2out;

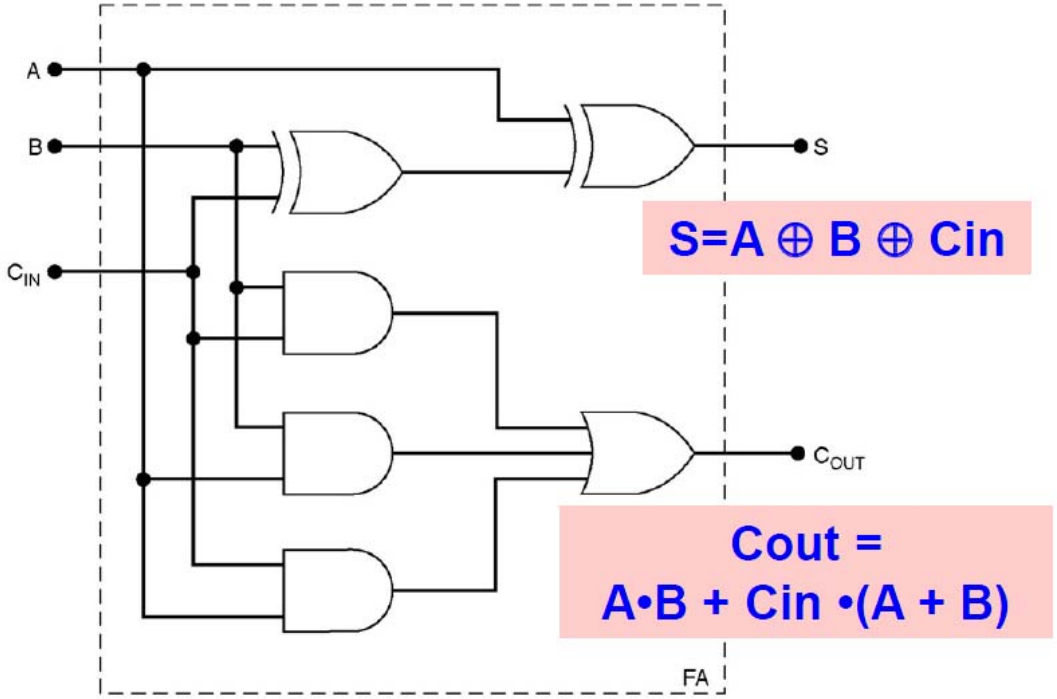
nor g3 (out, and1out, and2out);
and g2 (and2out, c, d);
and g1 (and1out, a, b);
```

```
assign out = ~((a&b) | (c&d));
```

Continuous assignment

Continuous Assignment Example

- Full Adder (using assign statements)



Remember that all assign statements happen concurrently (at the same time)

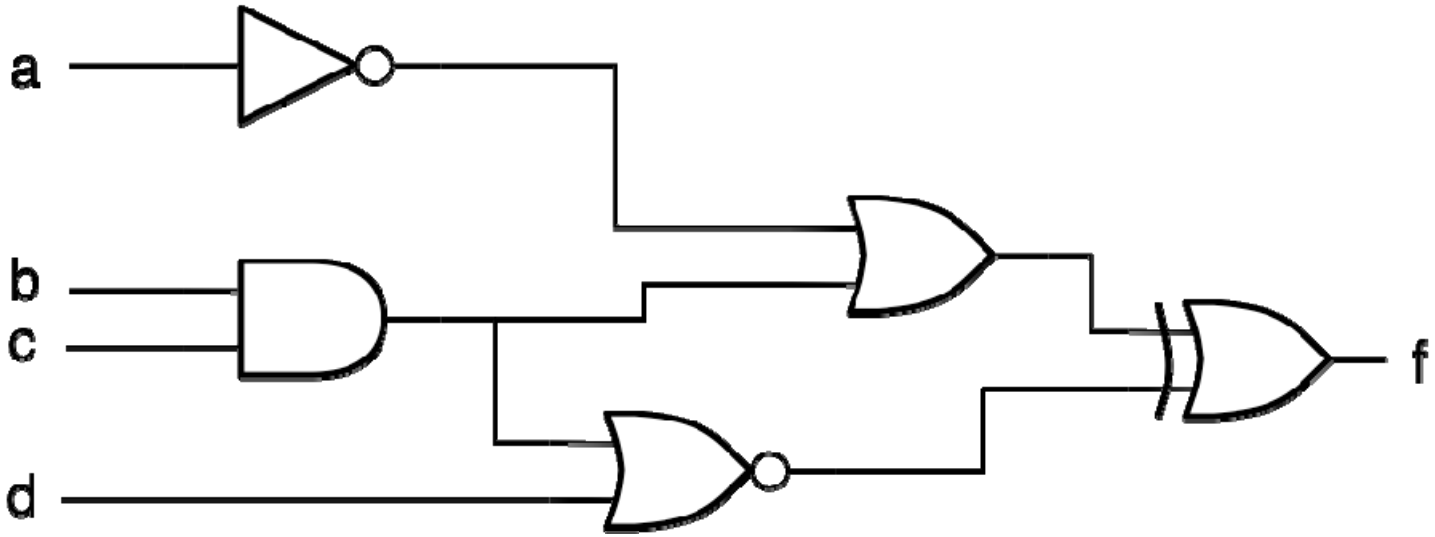
```
module full_adder (input A, B, Cin,
                   output S, Cout);

   assign S    = A ^ B ^ Cin;
   assign Cout = (A & B) | (B & Cin) | (A & Cin);

endmodule
```

Exercise

- You should now be able to go from circuit to structural Verilog description, or assign statement, and vice versa.
- Try this (use a single assign statement):




```
assign f = (~a | (b&c)) ^ ((b&c) ~| d);
```

Conditional Assignment

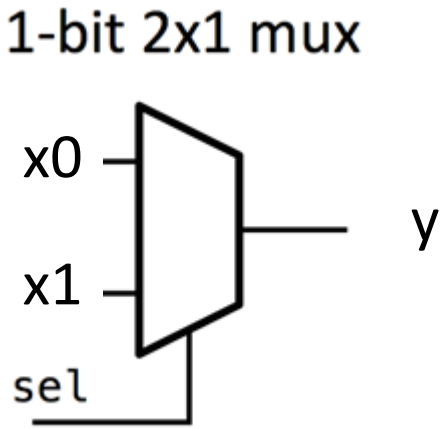
- We have already seen the continuous assignment statement:

```
assign y = ~((a & b) | (c & d));
```

- It is also possible to have a conditional assignment (as in C):

```
assign y = sel ? x1 : x0; // a multiplexer
```

The signal **y** will be connected to **x1** **if sel is 1, else** it is connected to **x0**



Basic Combinational Arithmetic

- We will not cover the details of arithmetic in Verilog in the course, but a few things are worth knowing
- Verilog supports many basic *arithmetic operations*:
  - Arithmetic: +, –, *, / (division is not generally synthesizable)
  - Comparisons: <, <=, >, >=, ==, !=
    - The comparisons are evaluated as true (1) or false (0)
- These can be useful in statements like these:

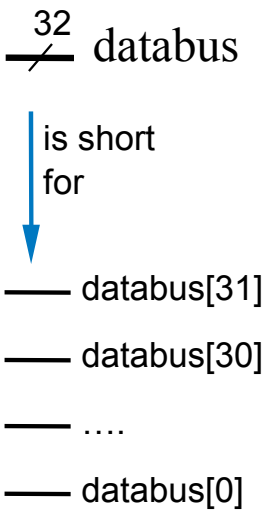
```
assign sum = a + b;
assign diffval = curr – prev;
assign max = (a>b) ? a : b;
```

VECTORS IN VERILOG

So far we have used 1-bit signals

- What happens if I want to use multi-bit signals
  - After all it is very difficult do anything with 1 binary bit
- Verilog has a special construct for handling multi-bit signals (wires). **Formed by specifying a range:**

```
wire [31:0] databus;
```



- Also for specifying multi-bit module ports

```
module add16 (input [15:0] a, b,
              output [15:0]sum,
              output cout);

    // add module internals here

endmodule
```

Vectors in Verilog

- We can declare *multi-bit* signals (or busses as we have seen them called) in Verilog:

```
…, output [3:0] y, …
wire [7:0] something;
```

- By convention, we label the *most significant bit* (MSB) using the higher number, and the *least significant bit* (LSB) using zero
- So a *16-bit* signal will be [15:0], an *8-bit* signal will be [7:0], and a *64-bit* signal will be [63:0]

Example: Adder

- Consider the 32-bit adder below (with Carry in & out)

```
module adder (
      input Cin,
      input [31:0] A, B,
      output Cout,
      output [31:0] Sum);

      wire [32:0] w1;

      assign w1   = A + B + Cin;
      assign Sum  = w1[31:0];
      assign Cout = w1[32];

endmodule
```

- We must declare multi-bit signals

Vectors in Verilog

NANYANG TECHNOLOGICAL UNIVERSITY

- *Vectors* are really easy to work with
- We can select *individual bits* of the vector:

```
assign y = some[3]; //assign 4th bit of
                    //some to y
```

- We can select a *range* of the vector:

```
assign z = some[4:3]; //assign 5th/4th bit of some
                      //to two-bit signal z
```

- We can *assign* to individual bits or a range:

```
assign x[0] = y[1];
assign x[2:1] = y[4:3];
```

Vectors in Verilog

- But be careful! Widths of vectors in assignments *should match.*
- Verilog will let you do some really bad things:
  - Like in the previous example, **so be VERY careful!!!!**

```
assign x[2:0] = y[1];
assign x[2:1] = a;    // a 1-bit
```

- With arithmetic, this can be **catastrophic!**
- Always check you are assigning signals of equal width, and remember **the LSB index is zero**!

```
Always carefully check the Synthesis warnings!
```

Number Literals

NANYANG TECHNOLOGICAL UNIVERSITY

- What about assigning a fixed bit pattern to a signal?
- Verilog allows us to use number *literals*:
  - <size>'<radix><value>
    - <size> is the width **in bits**
    - <radix>: b for binary, o for octal, h for hex, d for decimal
    - <value>: the number you want, with as many optional underscores as needed (for readability)
- Examples:
  - 4'b0000 (4 binary bits “0 0 0 0”)
  - 8'h4F (= 8'b01001111)
  - 8‘b0100_1111 (Same as above. Note the use of the underscore)
  - 1'b1 (a single “1”)

Number Literals

- Underscores can help split long strings:
  - 16'b0010_1110_0110_1001 (easier than 0010111001101001)
    - In this example, it's better to use hex instead (16'h2E69)
  - 32'h0F4B_C009
- If you assign a literal to a larger signal, it is *zero-padded at the MSB*:

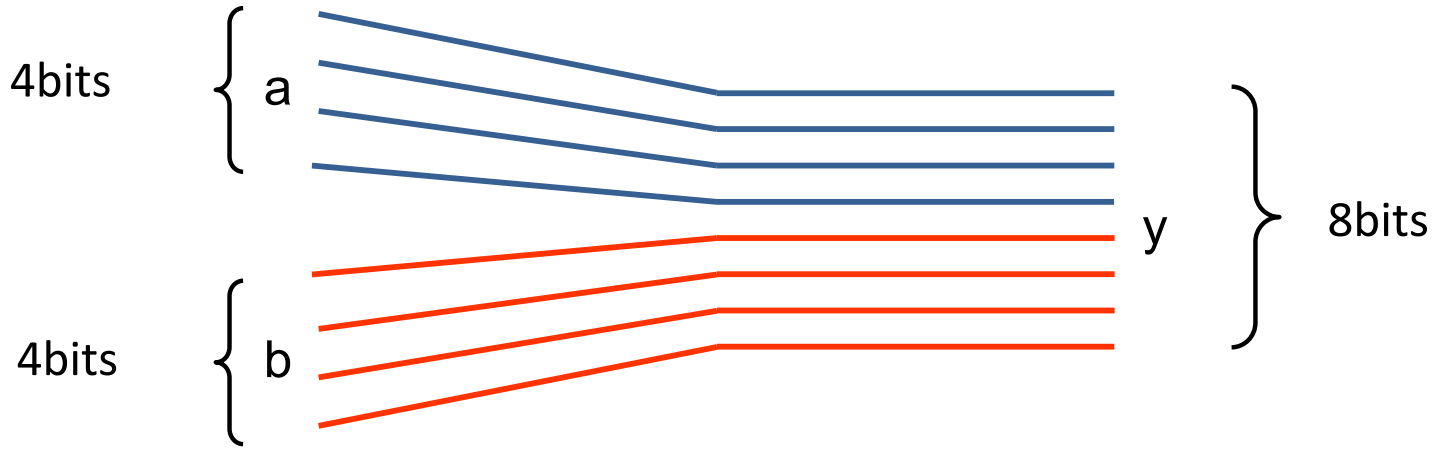
```
wire [5:0] x = 4'b1001   //x=001001
```

- If you assign a literal to a smaller signal, it is *truncated at the MSB*:

```
wire [3:0] x = 6'b110011   //x=0011
```

Concatenation

- It is sometime very useful to be able to concatenate a number of signals into a single signal.
  - Concatenation is signified by curly brackets enclosing a list




```
wire [3:0] a, b;
wire [7:0] y;
…
assign y = {a, b};
```

Concatenation and Replication

NANYANG TECHNOLOGICAL UNIVERSITY

- Concatenation

```
assign b = {a[3:0], 4'b0000};  // b is 8 bit
```

- Replication uses braces with a preceding integer or variable representing an integer.

```
assign c = {{4{a[3]}}, a[3:0]};  // c is 8-bit
```

  - The above example replicates the most significant bit 4 times

PARAMETERS

Parameters

NANYANG TECHNOLOGICAL UNIVERSITY

- A **parameter** is a constant that is local to a module
  - Can be declared in the module header
  - Can also be declared in the module body (we will not use this)

```
module some_mod #(parameter SIZE=8) (
    input  [SIZE-1:0] X, Y,
    output [SIZE-1:0] Z);
```

- Can have multiple parameters

```
module some_mod #(parameter SIZE=8, WIDTH=16)(
    input  [SIZE-1:0] X, Y,
    output [WIDTH-1:0] Z);
```

Parameters

- It is also possible to redefine a **parameter**
  - Consider the following 8-bit module

```
module submod #(parameter SIZE=8) (
    input    [SIZE-1:0] X, Y, output [SIZE-1:0] Z);
    // some statements in here
endmodule
```

- Then, note the change in the parameter in the submodule instantiation below
  - Now using two 16-bit submodules

```
module top_mod #(parameter SIZE=16) (
    input    [SIZE-1:0] a, b, c,  output [SIZE-1:0] D, E);

    submod #(.SIZE(SIZE)) U1 (.X(a), .Y(b), .Z(D));
    submod #(.SIZE(SIZE)) U2 (.X(c), .Y(b), .Z(E));

endmodule
```

In practice, it may be better to call them different names to avoid confusion

Example: Adder

NANYANG TECHNOLOGICAL UNIVERSITY

- Consider the 32-bit adder below (with Carry in & out)

```
module adder #(parameter SIZE=32)(
        input Cin,
        input [SIZE-1:0] A, B,
        output Cout,
        output [SIZE-1:0] Sum);

        assign {Cout, Sum} = A+B+Cin;

endmodule
```

Note: 1. How the concatenation operator handles the 33-bit result produced by A+B+Cin
2. The use of the arithmetic addition operator (+) to generate the sum.

NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Verilog for Combinational Circuits

So, Where are We?

NANYANG TECHNOLOGICAL UNIVERSITY

- We have covered: **modules**, **ports**, **wires**, **module instantiation**, **gates** and **assign statements**

```
module myModule (input a, b, data_i,
                 output flag, data_o, addr);
        wire or1out;

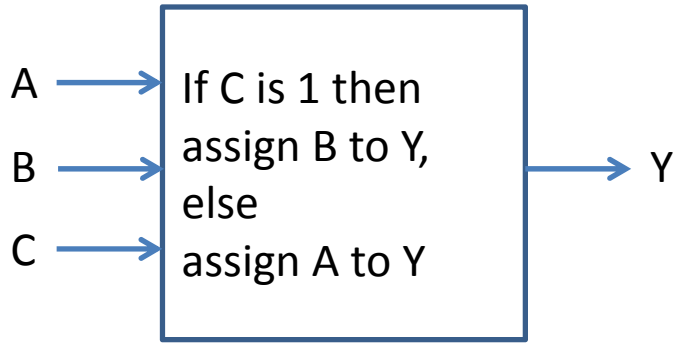
        get_Data IC1 (.din(data_i), .add(addr), .do(data_o));
        or g1 (or1out, a, b);
         assign flag = or1out & data_i;

endmodule
```

- So far we have only examined **Structural Verilog**.

Behavioral Modeling

- A more powerful level: we describe **how** the circuit **behaves**, not how it is *constructed*
- Synthesis tools work out how to make hardware that fulfills the description, taking into account the target architecture




```
module counter (clk,
reset, enable, count);
  input clk, reset,
enable;
  output [3:0] count;
  reg [3:0] count;

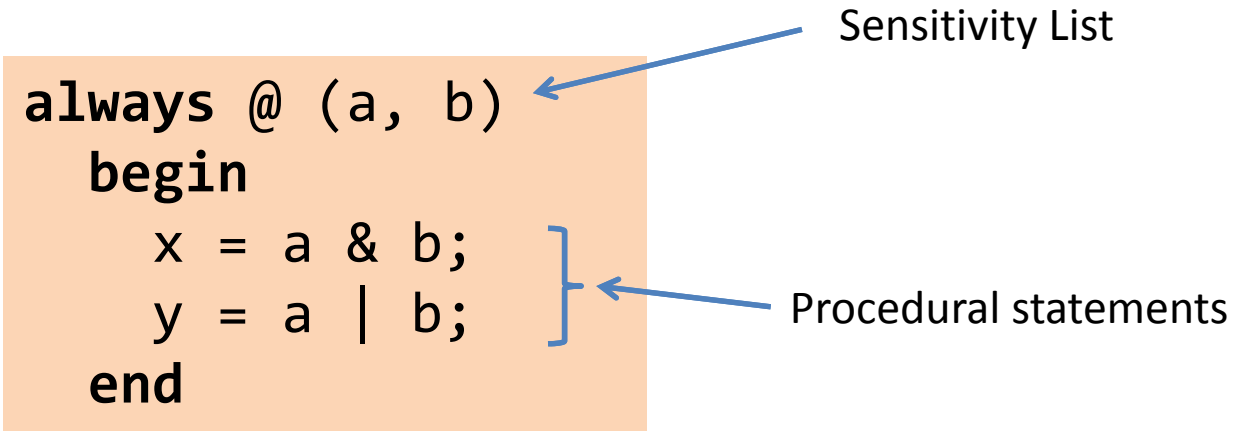
  always @ (posedge clk)
  if (reset == 1'b1)
     count <= 0;
else if ( enable == 1'b1)
     count <= count + 1;

endmodule
```



Combinational always Block

- For ***behavioral design***, we use a type of procedural block called an **always** block
- An **always** block contains procedural statements that describe the behavior of the desired hardware



- The **always** keyword starts a block
- The *sensitivity list* **must** contain the names of any signals that affect the output of the block

always @ *

- If you accidently leave a signal out you will not get the expected circuit
- Luckily, there's a shortcut: **`always @ *`** **– use it!**

```
always @ (a, b)
  begin
    x = a & b;
    y = a | b;
    z = a & c;
  end
```

This will not generate the expected output as it will not be triggered when input c changes

```
always @ *
  begin
    x = a & b;
    y = a | b;
    z = a & c;
  end
```

The "*" means all input signals inside the block.

This is OK. Whenever any signal changes (* means all input signals) a change in the output is forced.

Combinational always Block

- The statements can be assignments as in the example or more complex structures
- You do not use the **`assign`** keyword inside an always block, **ever ….**
- If there is more than one statement, use **`begin`** and **`end`** to define the start and end of the block.

```
always @ *
  begin
    x = a & b;
    y = a | b;
    z = a & c;
  end
```

Reg

- Signals you assign to from within an ***always*** *block*
  - Must be declared as being of type: **`reg`**
- **`reg`** is synonymous with wire, but these signals can be assigned to from inside an always block, **`wire`**s cannot!
- A **`wire`** is simply a *connection*, it holds no value of its own
- The **`reg`** type is more like a *variable* in programming, as we will see
- Note: you *cannot* assign to a **`reg`** signal using an ***assign*** *statement*, or connect it to module instance outputs.

```
module temp  (input a, b,
              output out);

    wire w1;

    assign w1 = a & b;
    assign out = w1;

endmodule
```

```
module temp  (input a, b,
              output out);
    reg w1;

    always @ *
        w1 = a & b;

    assign out = w1;
endmodule
```

Reg

- Declare **`reg`** signals:
  - You can set an *initial value* when you declare **reg** signals

```
reg a, b;
reg [3:0] x = 4'b0000;
```

- You can also declare your module outputs as **reg** if you plan to assign to them directly from within an **always** block:

```
module sel_one (input [5:0] a, b, c, d,
                input [1:0] sel,
                output reg [5:0] sigsel);
```

So What is Synthesized?

- Assignments in an **always** block are exactly the same as assignments using an **assign**:

```
always @ *
begin
      x = a & b;
      y = a | b;
end
```



```
assign x = a & b;
assign y = a | b;
```

Exactly the same circuit will be synthesized

If Statement

- With always blocks, we can use some powerful constructs:
- **If** statement:

```
always @ *
begin
  if (x < 6)
    alarm = 1'b0;
  else
    alarm = 1'b1;
end
```

- You can have more than one statement in each branch. If so, you use **begin** and **end**

```
always @ *
begin
  if (alarm == 1'b1)
    begin
      if (after_hours)
        siren = 1'b0;
      else
        siren = 1'b1;
      light = 1'b1;
    end
  else
    begin
      siren = 1'b0;
      light = 1'b0;
    end
end
```

A note on begin and end

- Verilog follows C standards for blocks
- The difference is that Verilog uses **begin** and **end**, rather than **{ }**
  - **Note { } is used for concatenation**
- If there is just one statement in the body of a statement (like if, for, or always block), then there is no need for **begin** and **end**
- If there is more than 1 statement, you must use **begin** and **end**

```
always @ *
begin

if (x < 6)
  alarm = 1'b0;
else
  alarm = 1'b1;

end
```

```
always @ *
if (x < 6)
begin
    alarm = 1'b0;
end
else
begin
    alarm = 1'b1;
end
```

These are the same

```
always @ *
if (x < 6)
    alarm = 1'b0;
else
    alarm = 1'b1;
```

Case Statement

- We can use **case** statements:

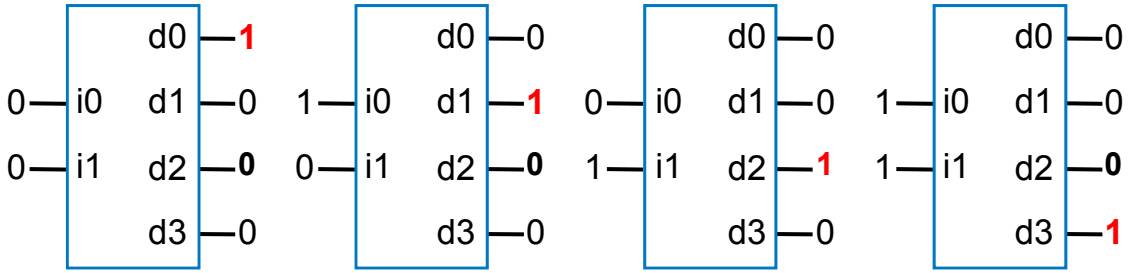
```
always @ *
case (sel)
    2'b00   : y = a;
    2'b01   : y = b;
    2'b10   : y = c;
    default : y = 4'b1010;
endcase
```

```
always @ *
case (sel)
    2'd0    : y = a;
    2'd1    : begin
                  y = b;
                  z = c;
              end
    2'd2    : y = c;
    default : y = 4'b1010;
endcase
```

- There is no need for a break in each branch (unlike some software languages)

Implementing Decoder with Verilog

Input		Output
00	→	0001
01	→	0010
10	→	0100
11	→	1000

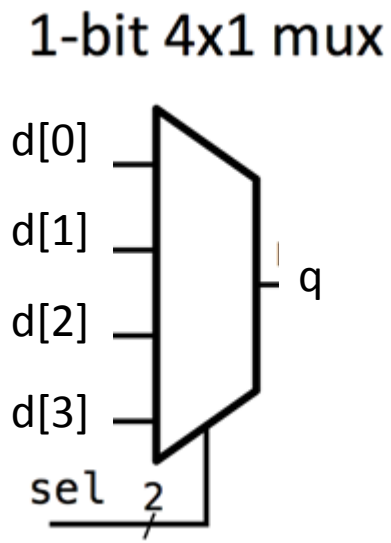


- Example: 3-to-8 decoder:

Input		Output
000	→	00000001
001	→	00000010
010	→	00000100
011	→	00001000
…		
111	→	10000000

```
module decoder3_8(output reg [7:0] d_out,
                  input  [2:0] ival);
always @ *
  case(ival)
     3'b000  :  d_out = 8'b00000001;
     3'b001  :  d_out = 8'b00000010;
     3'b010  :  d_out = 8'b00000100;
     3'b011  :  d_out = 8'b00001000;
     3'b100  :  d_out = 8'b00010000;
     3'b101  :  d_out = 8'b00100000;
     3'b110  :  d_out = 8'b01000000;
     3'b111  :  d_out = 8'b10000000;
  endcase
endmodule
```

Implementing Multiplexer with Verilog

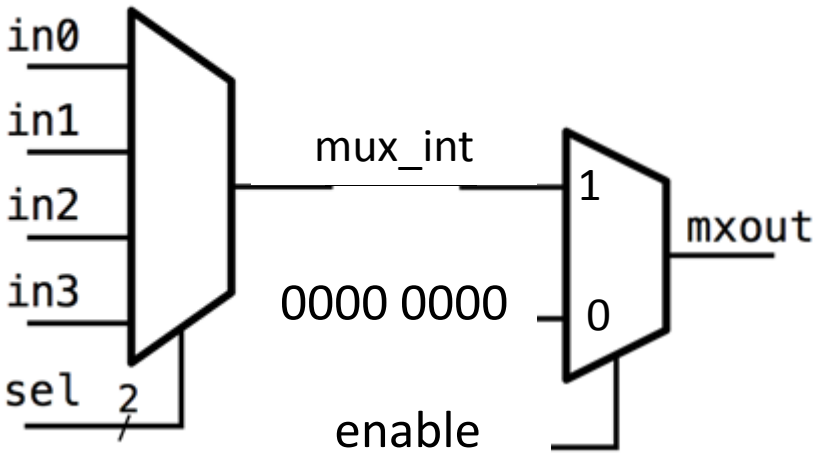



```
module mux4 (output reg  q,
             input [3:0] d,
             input [1:0] sel);

   always @ * begin
      case (sel)
         2'b00 : q = d[0];
         2'b01 : q = d[1];
         2'b10 : q = d[2];
         2'b11 : q = d[3];
      endcase
   end
endmodule
```

Note the begin and end. These are not needed as a case is a single statement. But makes it easier to read!!

Implementing Multiplexer with Verilog




```
module mux_4_32(output [7:0] mxout,
                input  [7:0] in3, in2,
                input  [7:0] in1, in0,
                input  [1:0] sel,
                input        enable);

   reg [7:0] mux_int;

   assign mxout = enable ? mux_int : 8'd0;

   always @ *
     begin
       case(sel)
         2'b00   :  mux_int = in0;
         2'b01   :  mux_int = in1;
         2'b10   :  mux_int = in2;
         2'b11   :  mux_int = in3;
       endcase
     end
endmodule
```

NANYANG TECHNOLOGICAL UNIVERSITY

IMPORTANT CONSIDERATIONS IN BEHAVIORAL VERILOG

Signal Assignments - always Block

- Notice that we can assign to ***multiple*** signals from inside one **always** block
- If you assign to a signal from inside an **always** block, you must **never** assign to it from **anywhere else!**
- That is, not from:
  - **assign** statements
  - Other **always** blocks
- Each **always** block represents the logic that generates a certain signal or set of signals
  - Hence, you should only assign to those signal(s) from inside that **always** block
- Only group signals into a single **always** block if it makes sense to do so – generally when they follow similar logic

```
assign x = . . .;

always @ *
begin
  x = a & b;
  y = a | b;
end

always @ *
begin
     y = . . .;
end
```

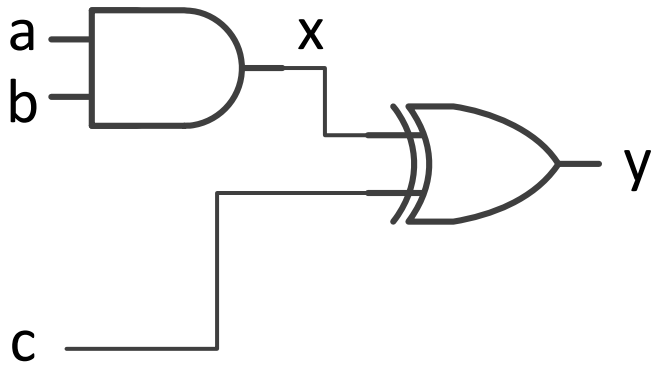




Statement Order in always block

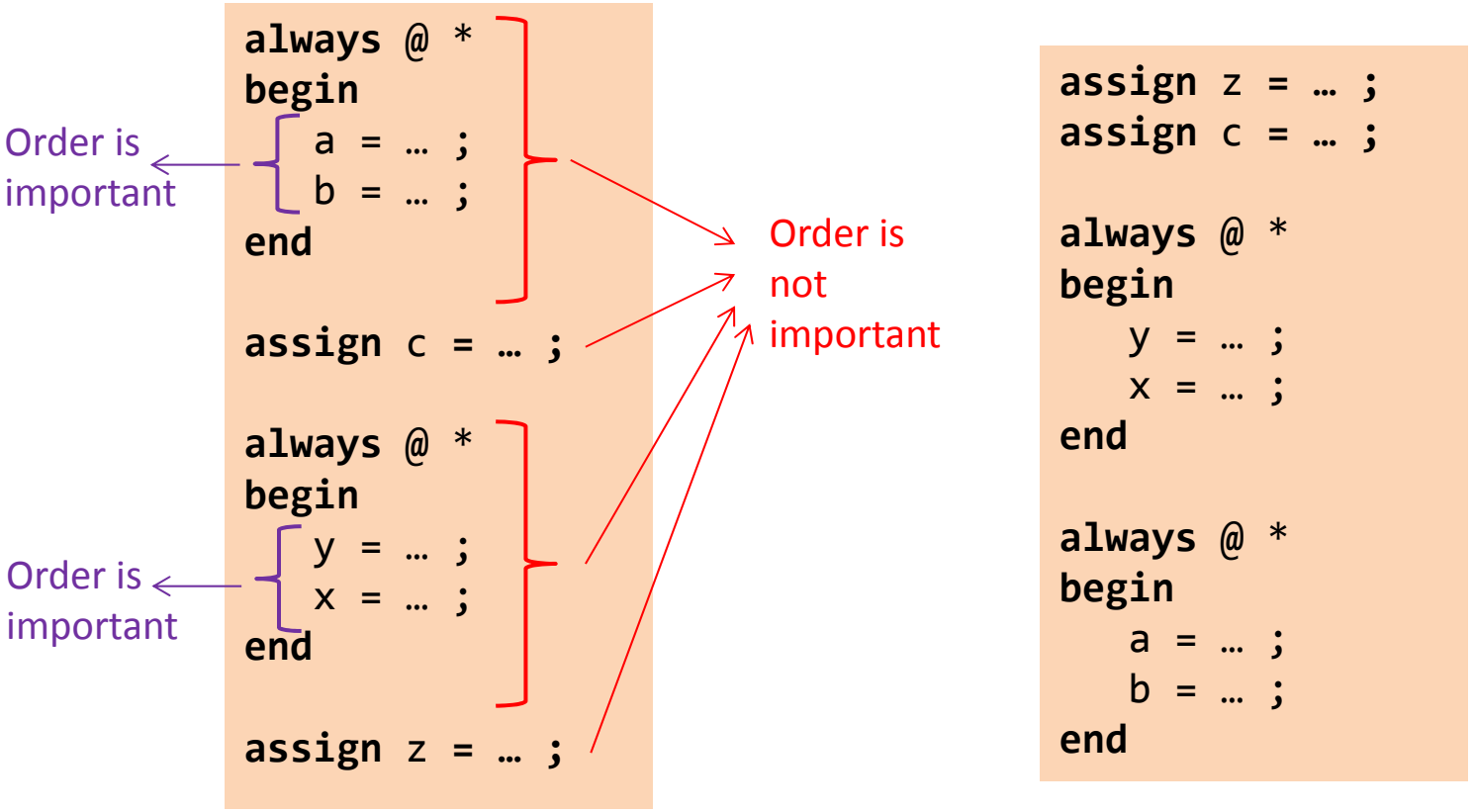
- The issue of statement order can be a little confusing
- When we read the statements in an **always** block, we read them in order, and **order matters**
- The synthesis tools turn the combined behavior described into a piece of combinational hardware
- Hence, if we assign to a signal more than once, in the end, the last assignment wins, since that's what the tools will use

```
always @ *
begin
      x = a & b;
      y = x | c;
      y = x ^ c;
end
```



Multiple always blocks

- The order of multiple **always** blocks in a module doesn't matter
  - They occur concurrently
- You can have as many **always** blocks as you want, and they each implement a piece of concurrent hardware




```
assign z = … ;
assign c = … ;

always @ *
begin
   y = … ;
   x = … ;
end

always @ *
begin
   a = … ;
   b = … ;
end
```

Avoiding Latches

- What happens to *y* in the **else** branch?

```
always @*
begin
   if(valid) begin
      x = a | b;
      y = c;
   end
   else
      x = a;          //y?
end
```



- This would imply *y* should keep its value – This is the behavior of a latch not combinational logic!

Avoiding Latches

- One way to fix this is to use a default assignment at the top of the **always** block:



```
always @*
begin
   y = x;
   if(valid) begin
      c = a | b;
      y = z;
   end
   else
      c = a;
end
```

- The default is overwritten by any subsequent assignment
- Assignments at the top of an **always** block are hence a great way of remembering to assign to all your signals

Avoiding Latches

```
always @ *
  case (sel)
    2'b00   : y = a;
    2'b01   : begin
                y = b;
                z = c;
              end
    2'b10   : y = c;
    default : y = 4'b1010;
endcase
```



```
always @ *
begin
  z = a;          // default
  y = 4'b1010; // default
  case (sel)
    2'b00 : y = a;
    2'b01 : begin
              y = b;
              z = c;
            end
    2'b10 : y = c;
  endcase
end
```

But, as we only assign to z in one instance, this implies that you are storing the data. **Must not do this as it synthesises storage.**

Summary

NANYANG TECHNOLOGICAL UNIVERSITY

- You must always include *any* signal that affects the output in the *sensitivity list*
  - else the synthesizer will produce something unexpected!!
  - *So,* easier to just use **always @ ***
- You must assign to the output signal in **all** possible cases
  - As you are designing combinational logic, it makes no sense to sometimes assign and other times not – that implies storing a value – That is not combinational logic
  - If you forget to do this, you will end up with a latch in your circuit, which is not wanted!!!
  - More on this when you start to look at **Sequential Circuits**

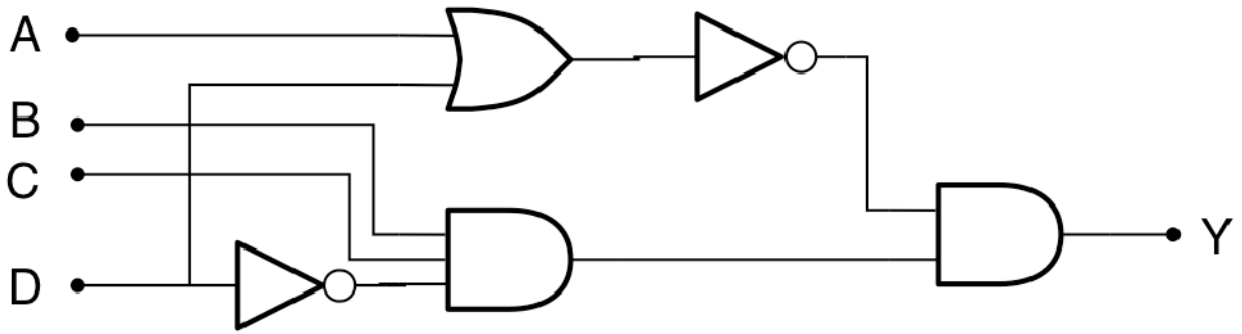
NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Sequential Circuits

Review of Combinational Circuits

- So far, we have investigated ***combinational*** circuits
- The output of a combinational circuit is purely a function of the present inputs to it
  - *output = f(input)*
- When the inputs are changed, the output will change to that determined by those inputs (after a short propagation delay)
- Knowing only the ***current*** inputs, we can always determine the output



Combinational and Sequential

NANYANG TECHNOLOGICAL UNIVERSITY

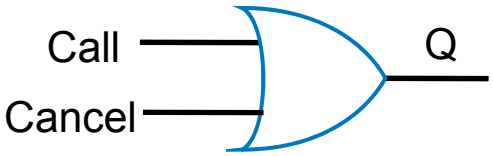
- ***Sequential*** circuits introduce the idea of ***state***
  - The state of a circuit encodes sufficient information about the past to determine how it will react with inputs to produce an output

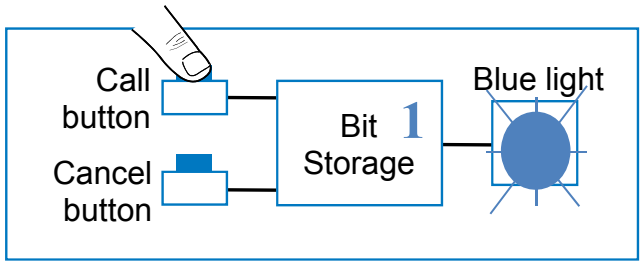
- The output of a sequential circuit depends on **both** the ***current*** inputs and ***previous*** inputs

- In this sense, sequential circuits have ***memory***, whereas combinational circuits have no memory.

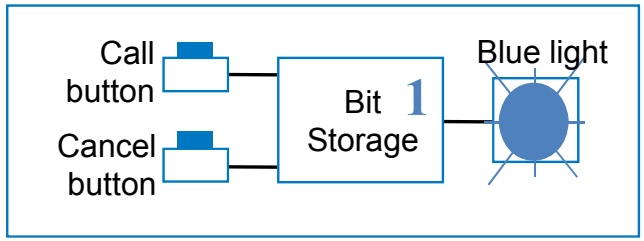
Sequential Logic

- We'd like to store a single bit
- Flight attendant call button
  - Press "Call": light on
    - Stays on after button released
  - Press "Cancel": light off
    - Stays off after button released
  - Can we use a gate?

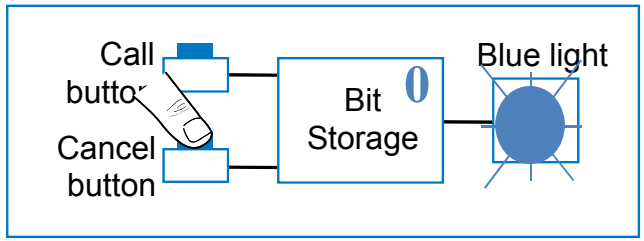




1. Call button pressed – light turns on



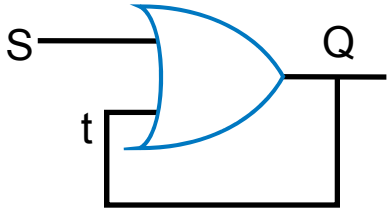
*2. Call button released – light **stays on***

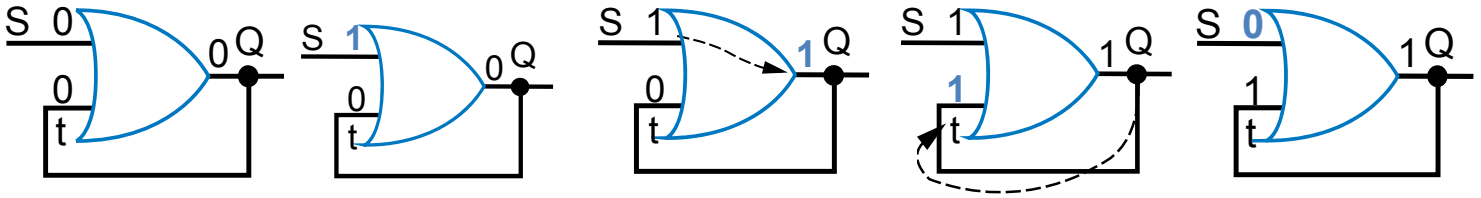


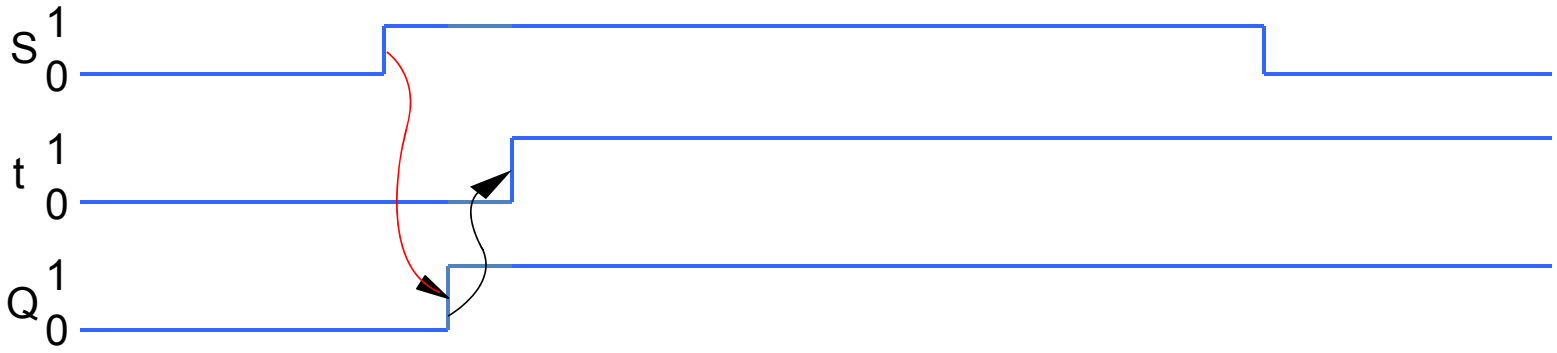
3. Cancel button pressed – light turns off

Sequential Logic

- What if we add feedback?
  - S will now cause 1 to be stored

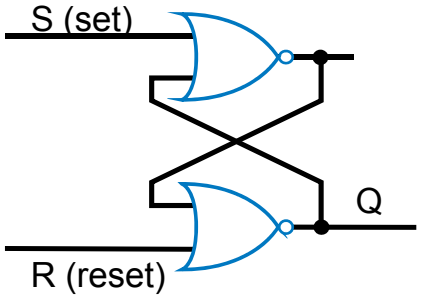


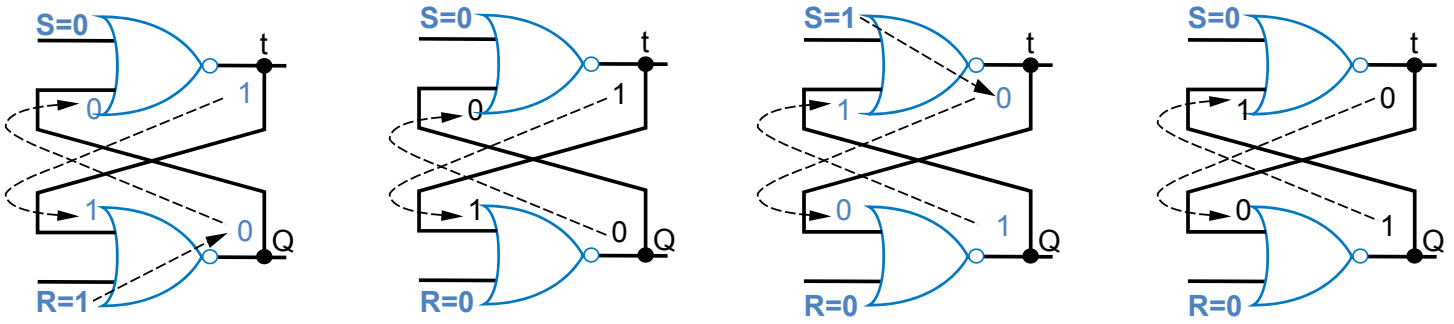


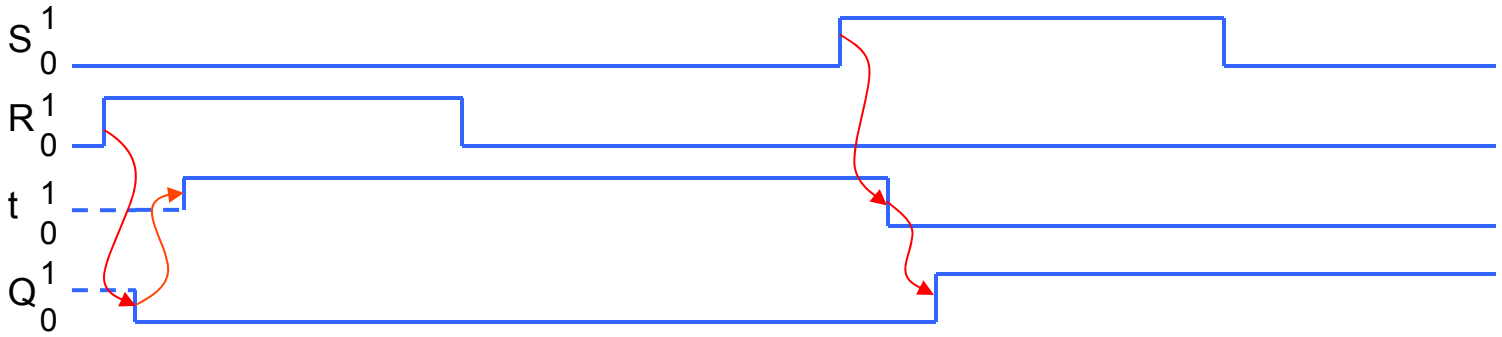


Set-Reset (SR) Latch

- The most basic circuit for storing a bit is the *SR latch*:

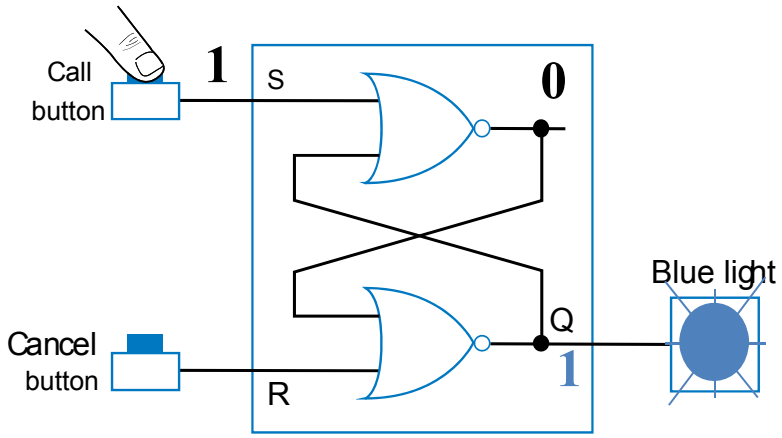






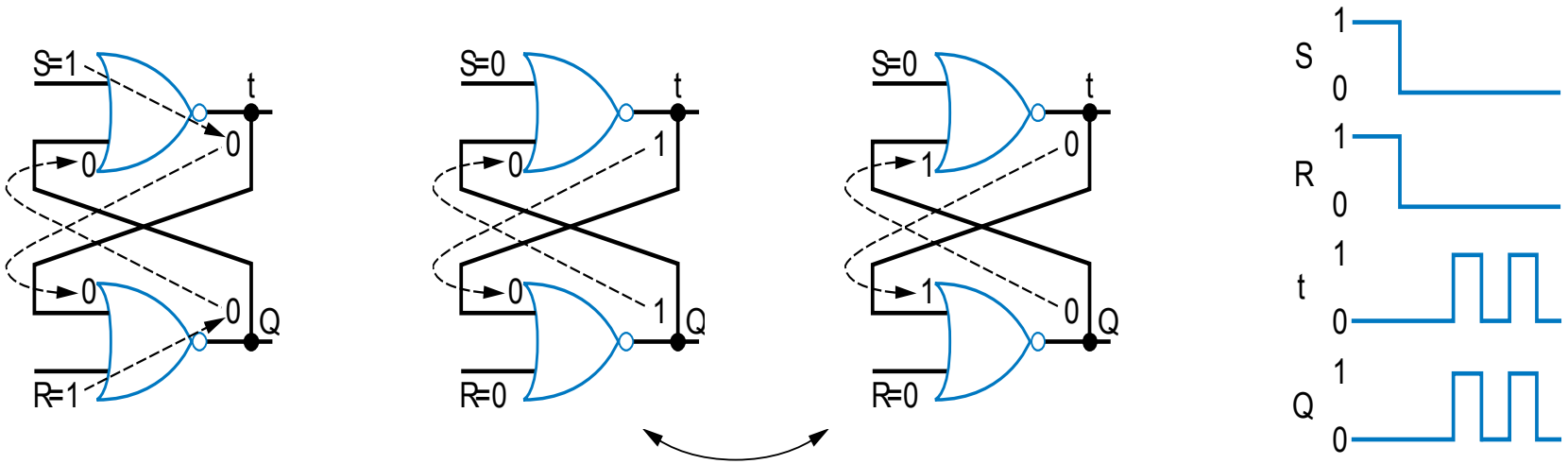
Set-Reset (SR) Latch

- The SR latch allows us to implement the flight attendant call system easily
  - Connect “call” to S, and “cancel” to R
  - Q stays at 1 even after “call is released”



Set-Reset (SR) Latch

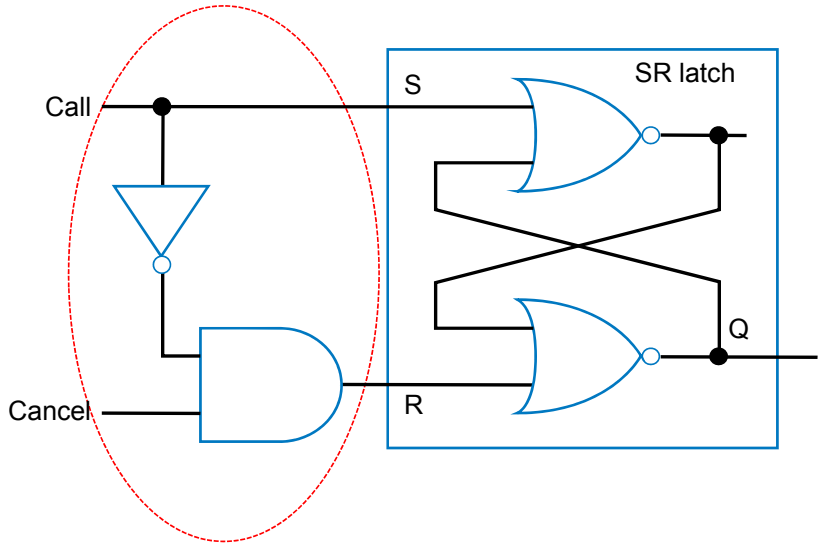
- There is, though, a problem with the SR Latch
- What happens when both S and R are asserted?

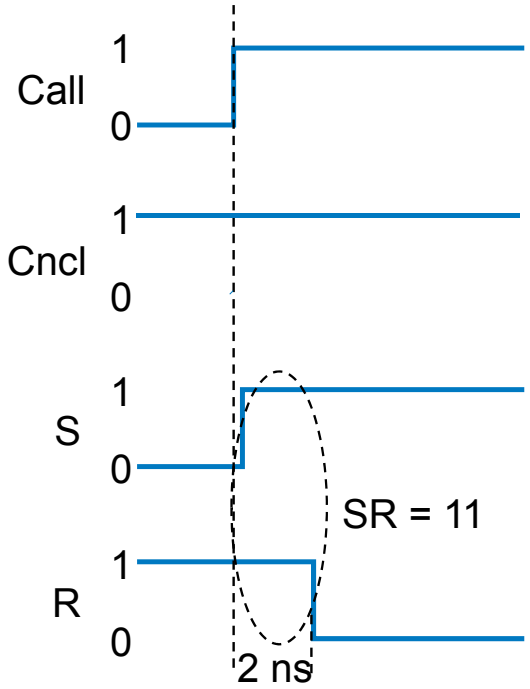


- When both S and R is then de-asserted at the same time, we may get oscillation at the output:
  - Constant swapping from 0 to 1 to 0 …
- Normally, one gate is a little faster so output will eventually stabilize, but which?
  - Impossible to know

Set-Reset (SR) Latch

- Maybe the designer can stop the problem of both S and R being 1

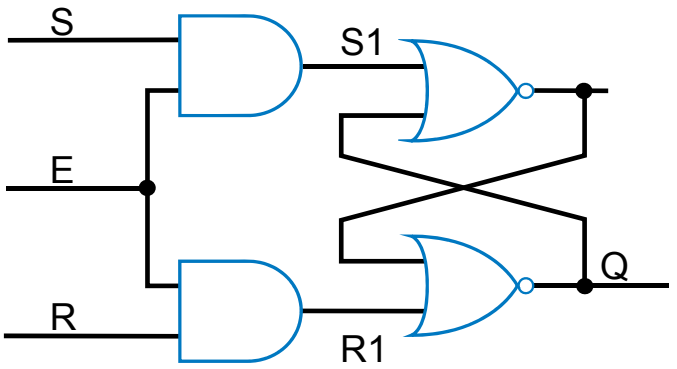


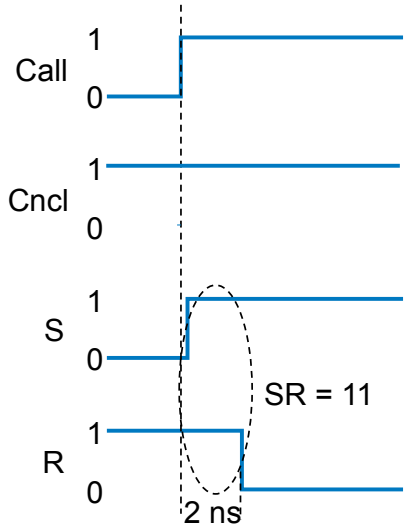


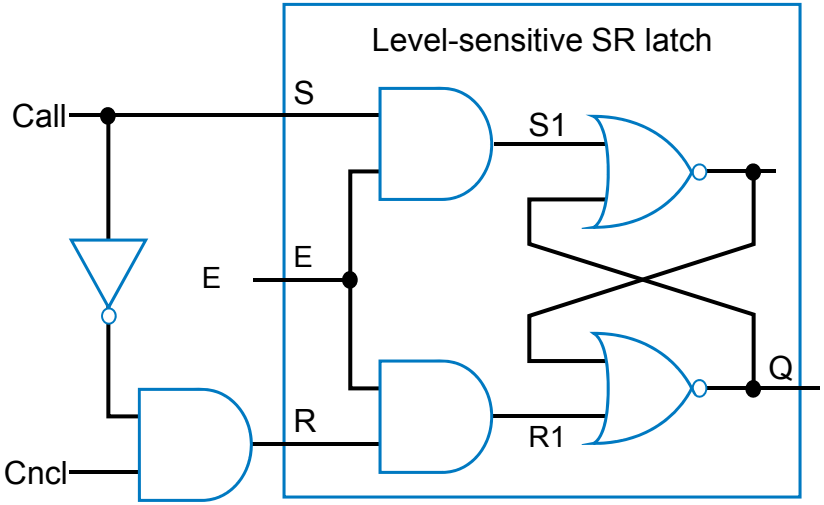
- The problem is, signals take time to travel through gates, so call to R is longer than call to S

Gated/Enabled SR Latch

- The *enabled SR latch* will only respond to inputs when E is high – *level-sensitive*
- Can now ensure S and R never both high when E high – glitches unimportant

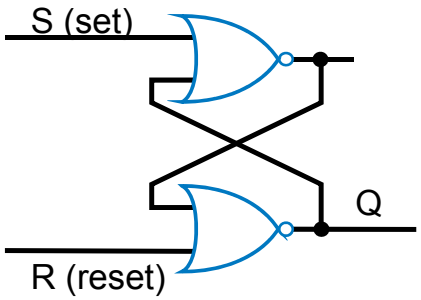






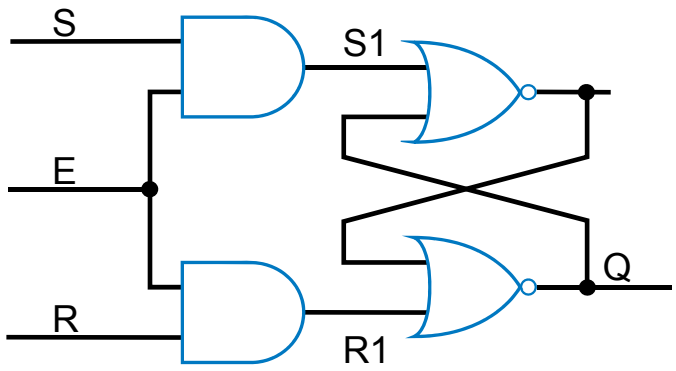
SR Latches

- We can capture the behavior of the above latches using a truth table:
  - Q means the current state
  - Q+ means the next state



SR Latch

S	R	Q+	Function
0	0	Q	Store
0	1	0	Reset
1	0	1	Set
1	1	?	Undefined



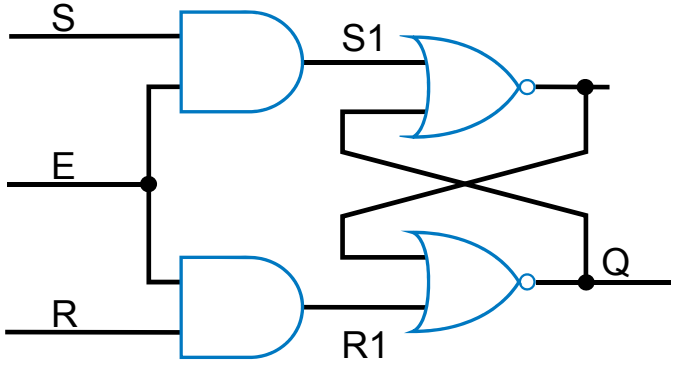
Enabled SR Latch

E	S	R	Q+	Function
0	X	X	Q	Store
1	0	0	Q	Store
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	?	Undefined

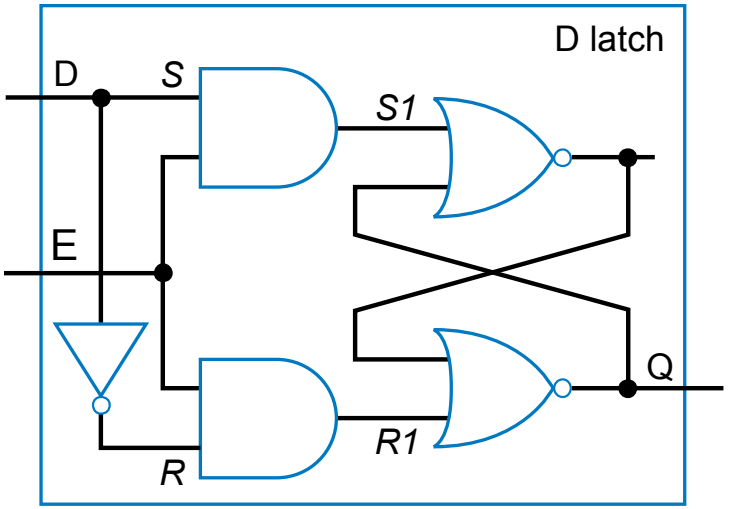
Transparent D Latch

- The D latch, also called the transparent latch
- Removes the possibility of invalid inputs
  - Inverter ensures R is always opposite of S
  - A single input controls both states of the latch

Enabled SR Latch



E	S	R	Q+	Function
0	X	X	Q	Store
1	0	0	Q	Store
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	?	Undefined

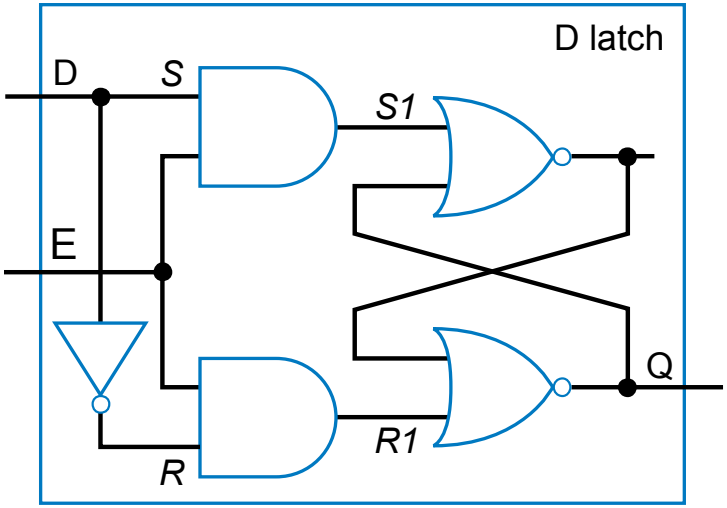


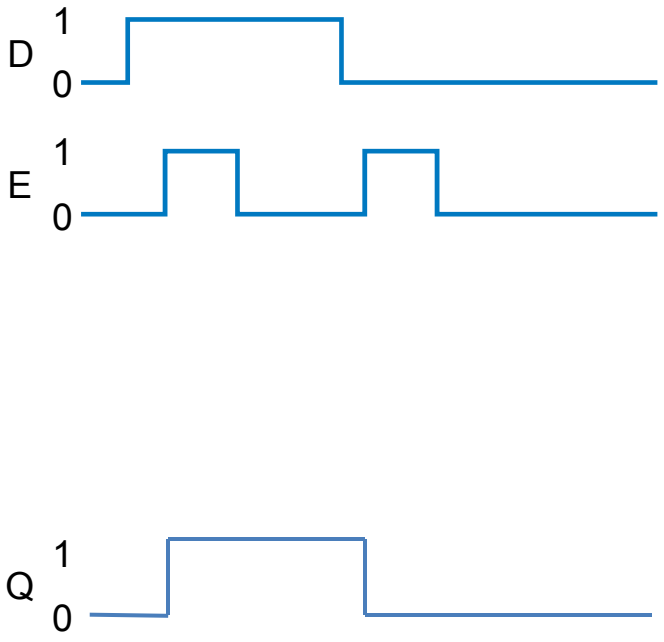
E	D	Q+
0	X	Q
1	0	0
1	1	1

Transparent D Latch

- The D latch effectively passes its input through to its output whenever the enable input is high
- Hence, “transparent”
- If enable is low, the output is held
- Truth table for a D latch:

E	D	Q+	Function
0	X	Q	Store
1	0	0	Transparent
1	1	1	Transparent

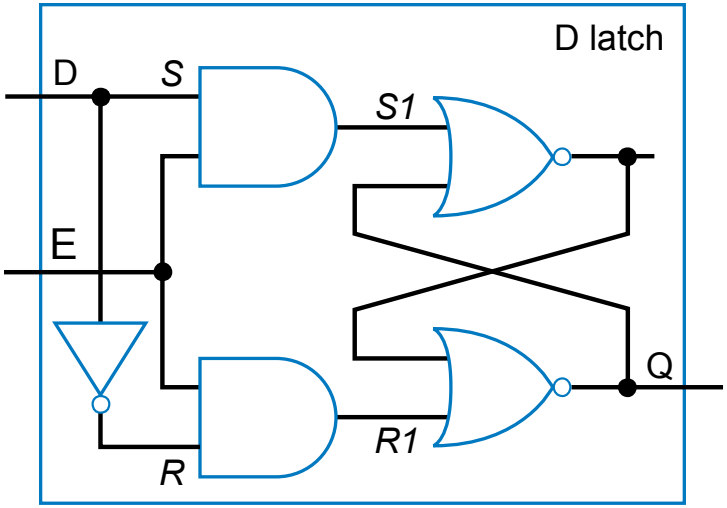


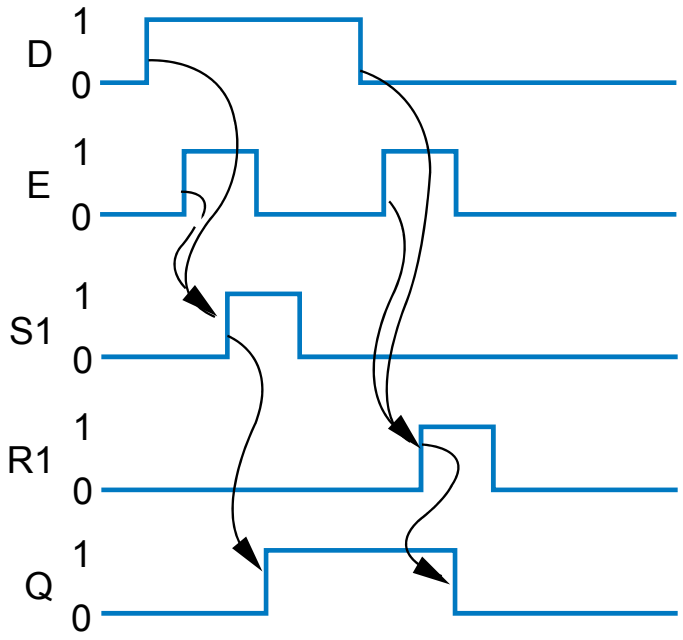


Transparent D Latch

- The D latch effectively passes its input through to its output whenever the enable input is high
- Hence, “transparent”
- If enable is low, the output is held
- Truth table for a D latch:

E	D	Q+	Function
0	X	Q	Store
1	0	0	Transparent
1	1	1	Transparent

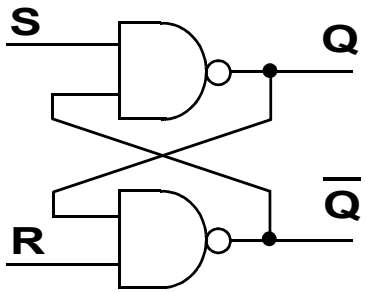




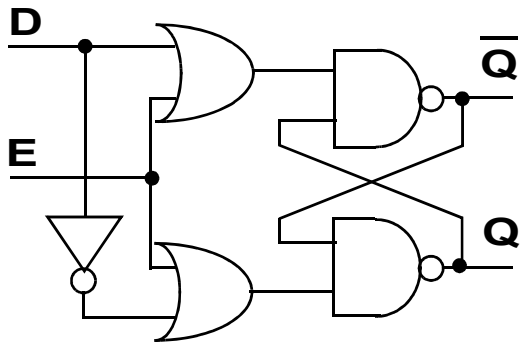
Active-Low Latches

- Note that it's possible to create active-low versions of the latches previously shown
- Swap the **nor** for **nand** and the **or** for **and**:

Active-Low SR Latch



Active-Low Transparent Latch



E	D	Q+	Function
1	X	Q	Store
0	0	0	Transparent
0	1	1	Transparent

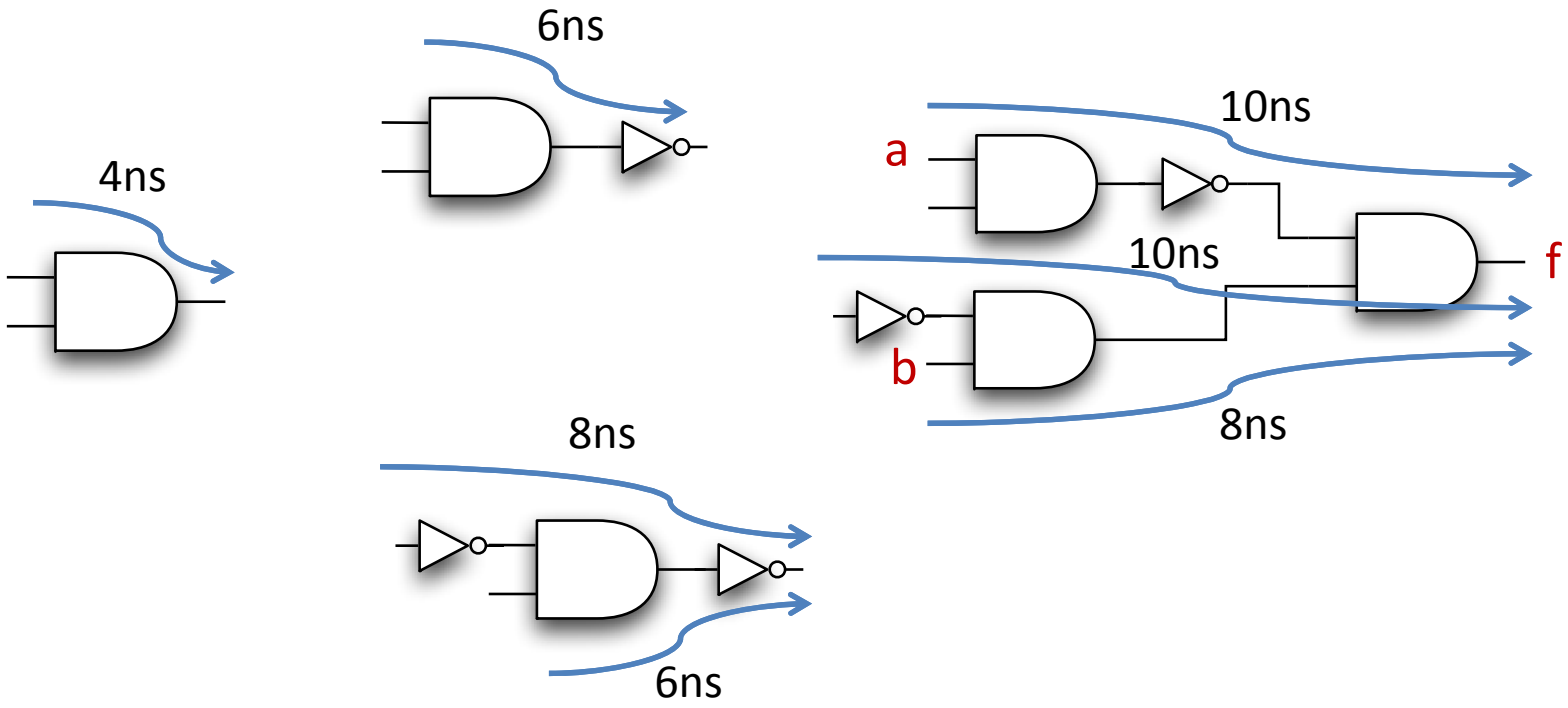
Timing in Circuits

NANYANG TECHNOLOGICAL UNIVERSITY

- So far, we have assumed that inputs cause a change in the output of a gate immediately
- The reality is it takes a short amount of time
- We call this the propagation delay
- This is the time it takes for a change at the input to create the requisite change at the output
- We can work out the propagation delay through a circuit by summing the propagation delays of the individual gates

Timing in Circuits

- Assume an **and** gate has a delay of 4ns and an **inverter** has a delay of 2ns:

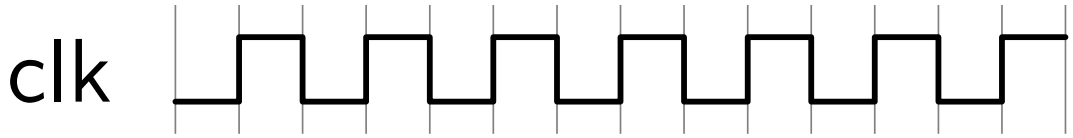


Timing in Circuits

- The propagation delay, t_p, may differ between different inputs and outputs in the same circuit
- Hence we can give the propagation delay for each input-output pair:
  - a input to f output t_p = 10ns
  - b input to f output t_p = 8ns
- If we report a single number, we always use the worst (i.e. longest path)

The Clock

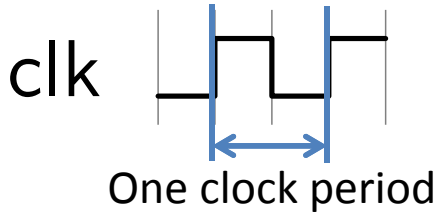
- In digital systems, sequential components are connected to what we call a "clock"
- The clock is a signal that continuously toggles between 0 and 1 at a fixed rate:



- It is called a clock because it oscillates much like the "tick-tock" of a clock
- This is where the MHz and GHz numbers for chips come from – how fast does the clock oscillate?

The Clock

- The period of the clock is the time it takes to complete one complete cycle – we call this a cycle



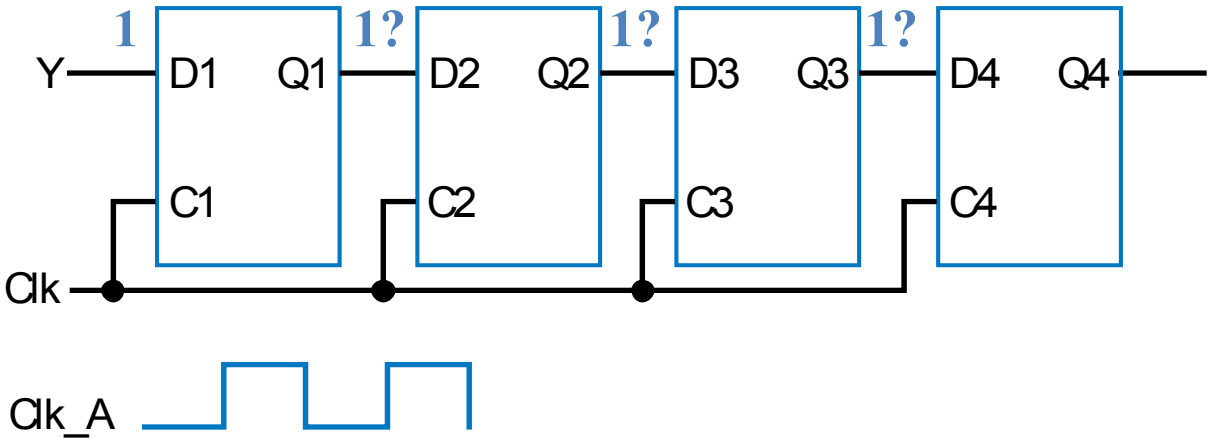
- The frequency is the inverse of the period:
  - 10ns period = 10×10^{-9}s, $1/10 \times 10^{-9} = 10^{8}$Hz = 100MHz
  - 1ns period = 1×10^{-9}s, $1/1 \times 10^{-9} = 10^{9}$Hz = 1GHz
  - 4ns period = 4×10^{-9}s, $1/4 \times 10^{-9} = 2.5 \times 10^{8}$Hz = 250MHz

Timing of Latches

Truth table of a D Latch

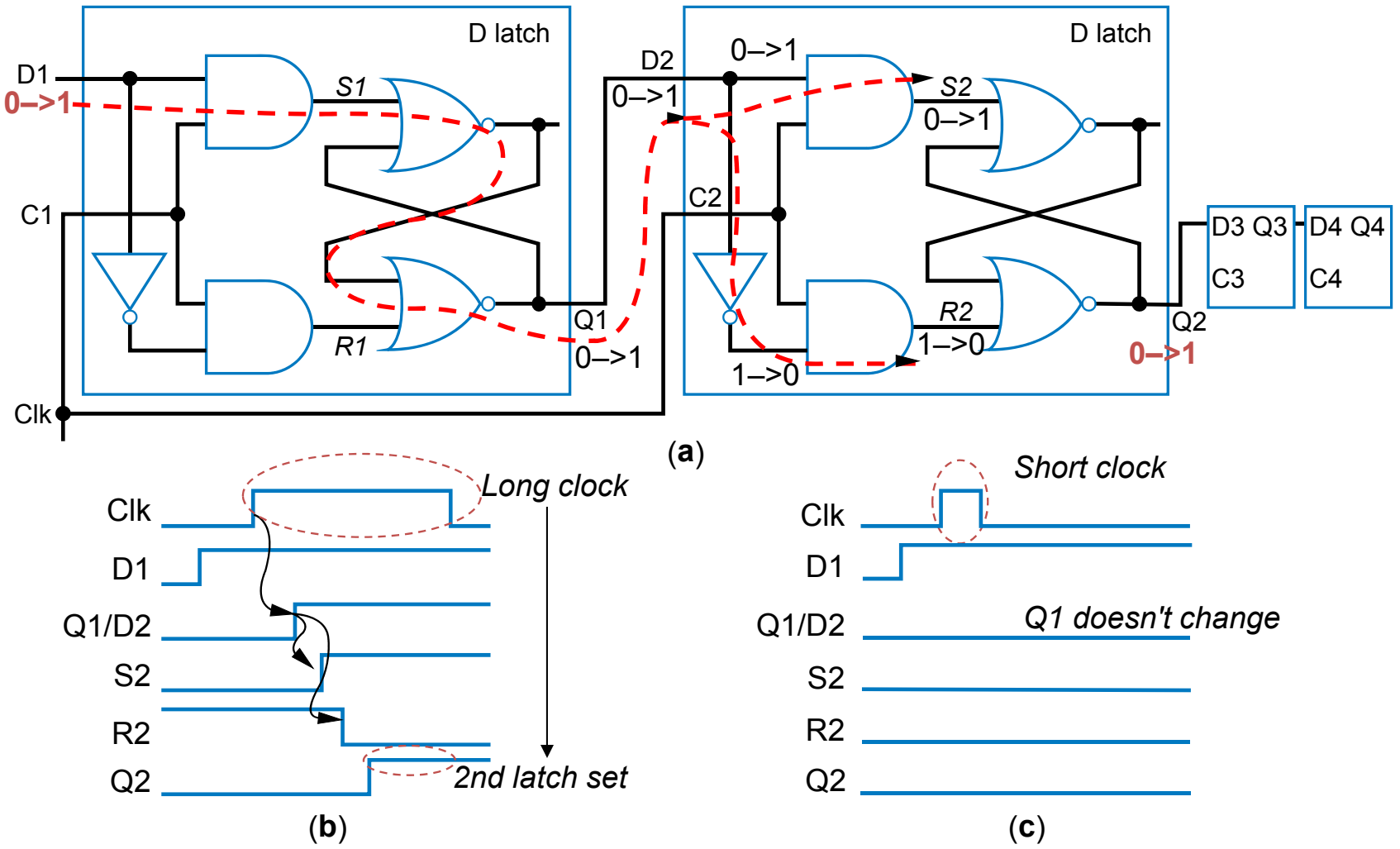
E	D	Q+	Function
0	X	Q	Store
1	0	0	Transparent
1	1	1	Transparent

- Consider a chain of latches:



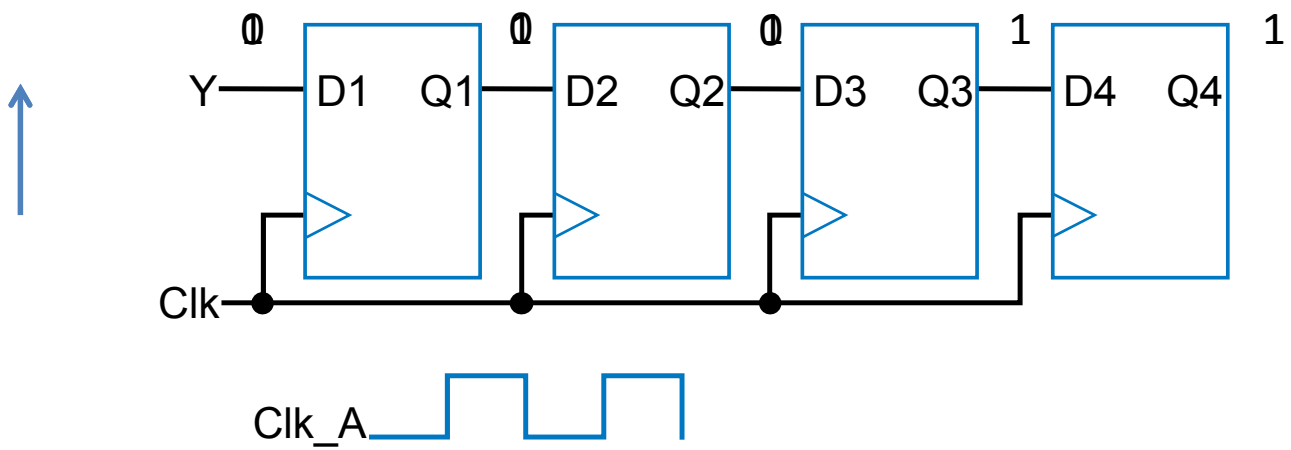
- When Clk_A is 1, how many latches will the input pass through?
- With Clk_A, it may be more, Clk_B, fewer

Timing of Latches



Edge Triggered Flip-Flops

- We can use an edge triggered flip-flop to overcome the limitation of D latches
- At each clock rising edge each flip flop will take its D input and produce that value on its Q output
- How do we implement an edge triggered flip flop?



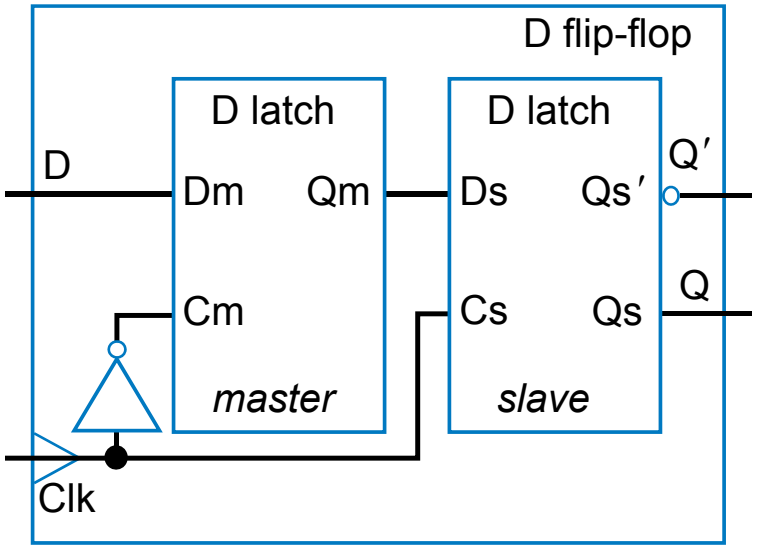
Edge-Triggered Flip-Flops

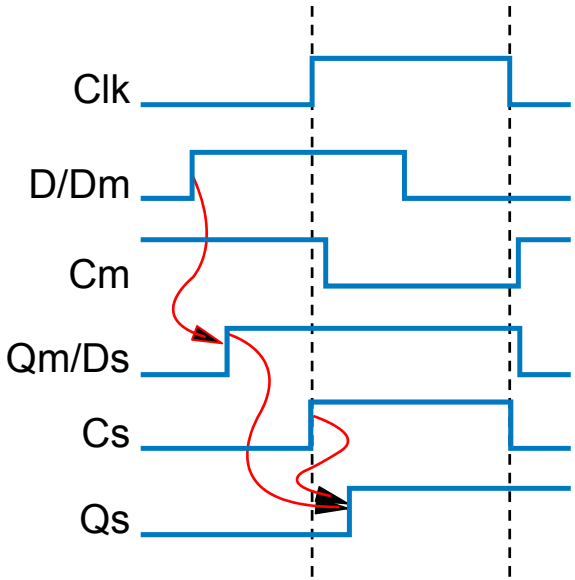
NANYANG TECHNOLOGICAL UNIVERSITY

- Master-Slave implies two latch stages:
  - Master stage is enabled when control is low, and samples the current input, while slave is disabled, locking it in the middle
  - When control goes high, the master is disabled, and the slave uses the value from the middle to determine its output
  - Hence, the only time an output will change is at the rising edge
  - For a negative-edge triggered setup, the master responds to high enable, and the slave to low enable

Edge-Triggered Flip-Flops

- The master and slave are each just a D latch:



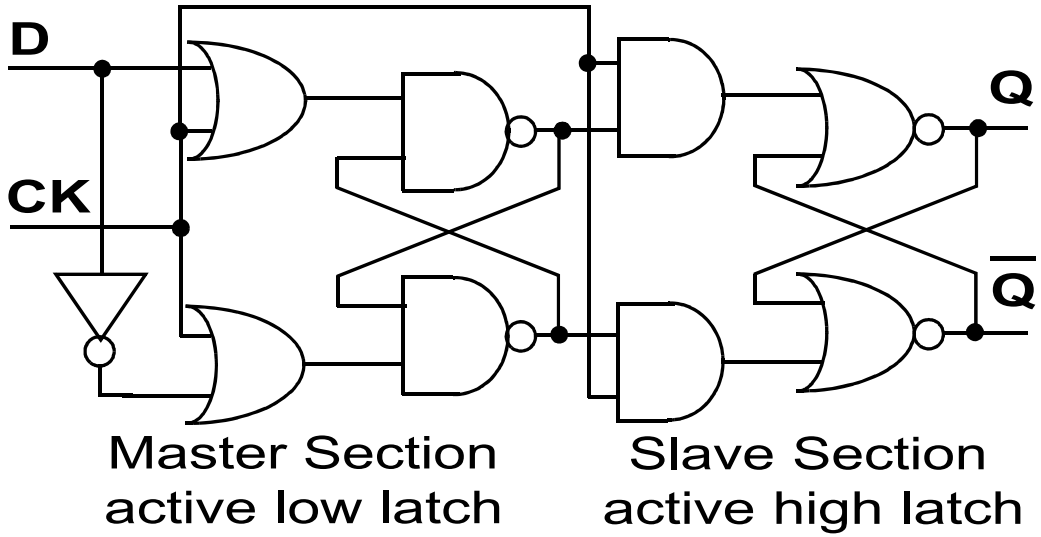


Truth table of a D Latch

E	D	Q+	Function
0	X	Q	Store
1	0	0	Transparent
1	1	1	Transparent

Edge-Triggered Flip-Flops

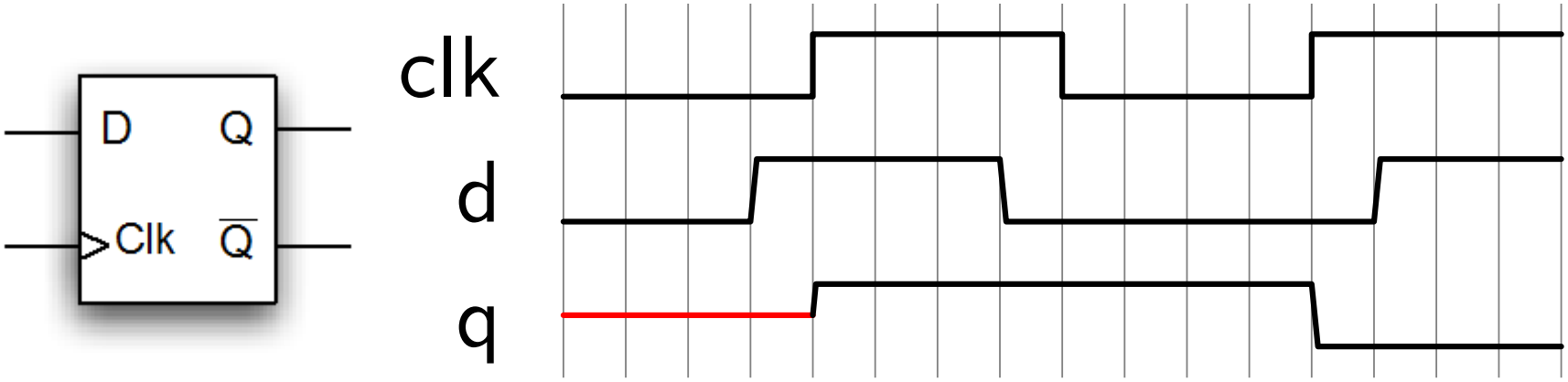
- Rather than invert the clock as in the last slide, we can use an active low version (more correct)
- Example design of D flip-flop:



- Note that there are many possible designs, and many are more compact than above

Edge-Triggered Flip-Flops

- The timing diagram for for a D-type flip-flop is as follows:



- Note transitions on the output only occur at the (rising) clock edge

Edge-Triggered Flip-Flops

NANYANG TECHNOLOGICAL UNIVERSITY

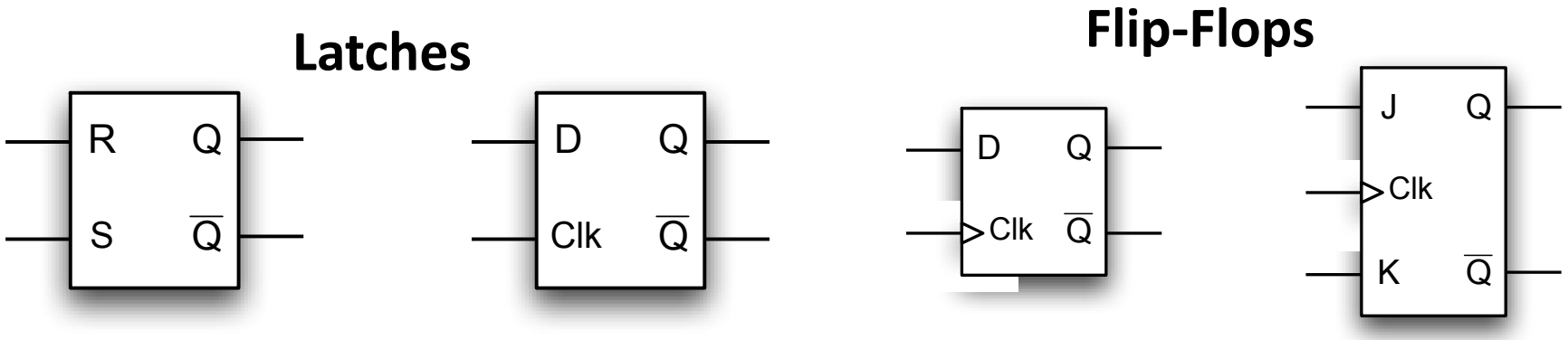
- **Latches** can be level-sensitive, i.e., their outputs change when the control (enable) input is high

- A **flip-flop** is a circuit that changes its outputs only at the **control signal's edges**
  - **Positive edge-triggered**: output changes when control goes from 0 to 1
  - **Negative edge-triggered**: output changes when control goes from 1 to 0

- We can build such circuits using a master-slave setup

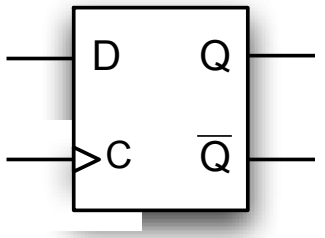
Edge-Triggered Flip-Flops

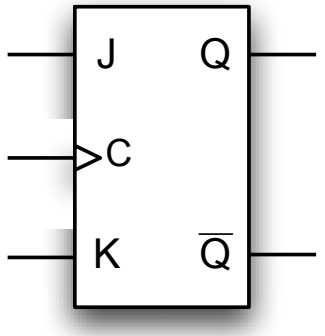
- A few notes:
  - In this course, *latch* means **level-sensitive**, *flip-flop* means **edge-triggered** – correct modern convention
    - Flip-flops can be **positive edge-triggered** or **negative edge-triggered**
    - In our lectures, we will assume that flip-flops are positive edge-triggered unless we specify otherwise
  - Some older references may not follow this convention, so **be careful** when using other resources
  - Good news: the symbol for an edge triggered component always has a small triangle on the control input:

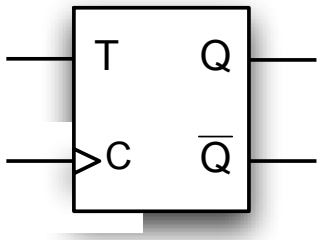


Edge-Triggered Flip-Flops

- We've seen the D flip-flop:
  - Q takes the value on D at the rising edge of C
- There's also a J-K flip-flop:
  - Q becomes 1 if J is asserted at the rising edge and 0 if K is asserted
  - If both J and K are asserted, the output toggles at the rising edge
- And a T flip-flop:
  - Q toggles at the rising edge if T=1
- (Also a lesser seen SR flip-flop)





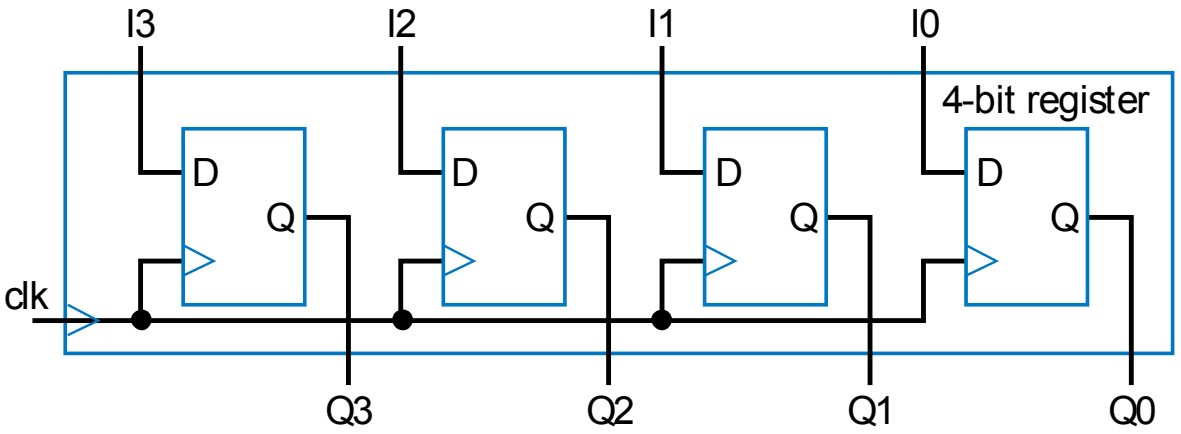


The Clock

- In synchronous design, we generally use one clock for all the sequential components in our circuit
- Hence, the outputs of all those blocks will only update at the clock (rising) edge
- That's why prefer (edge-triggered) flip-flops
- That way, we can more easily analyze our circuit, and model its behavior

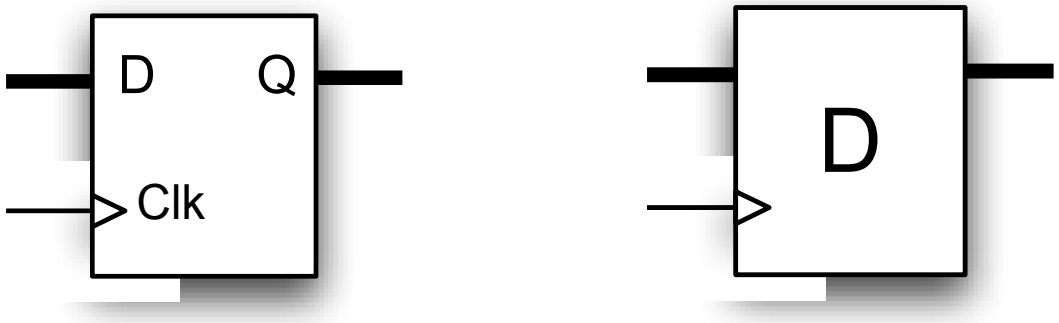
Registers

- The D-type flip-flop is the fundamental building block in synchronous design
- But we often deal with multi-bit signals
- When we combine multiple D flip-flops together to store multiple bits, we call this a **register**:



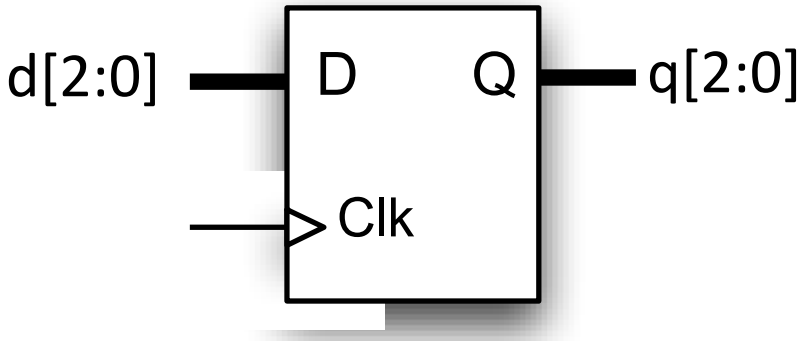
Registers

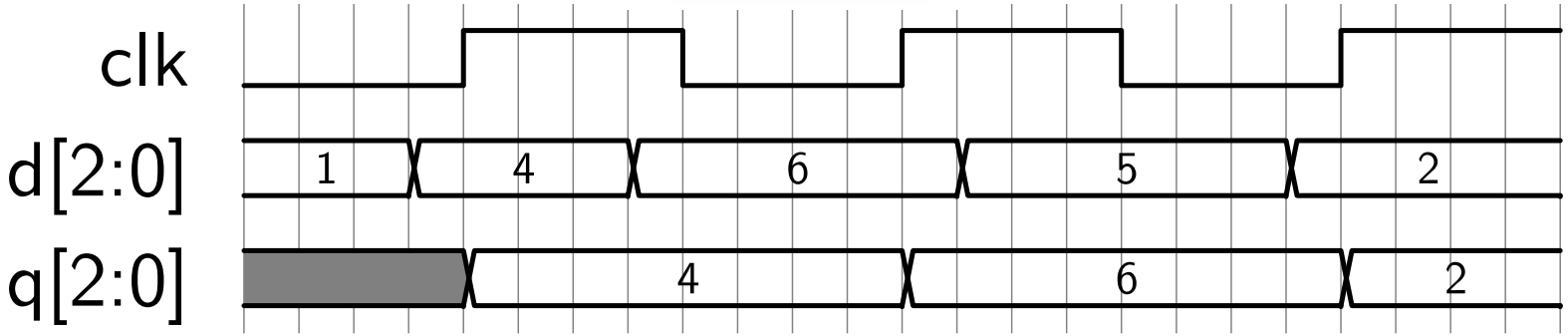
- Henceforth, we can use only registers as our fundamental sequential component
- We now know how they are constructed internally, but we no longer need to worry about latches and flip-flops
- Even a 1-bit signal is stored in a “register”
- We can use a similar symbol to that used for a D flip-flop:



Registers

- When dealing with multi-bit registers, we can show these values in a timing diagram without splitting up the bits:

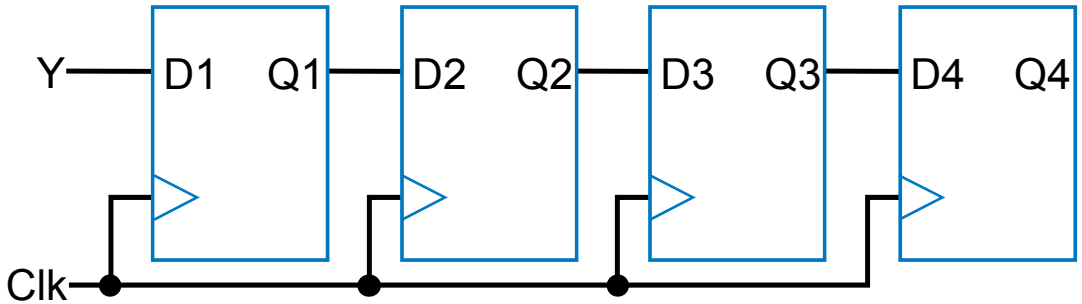


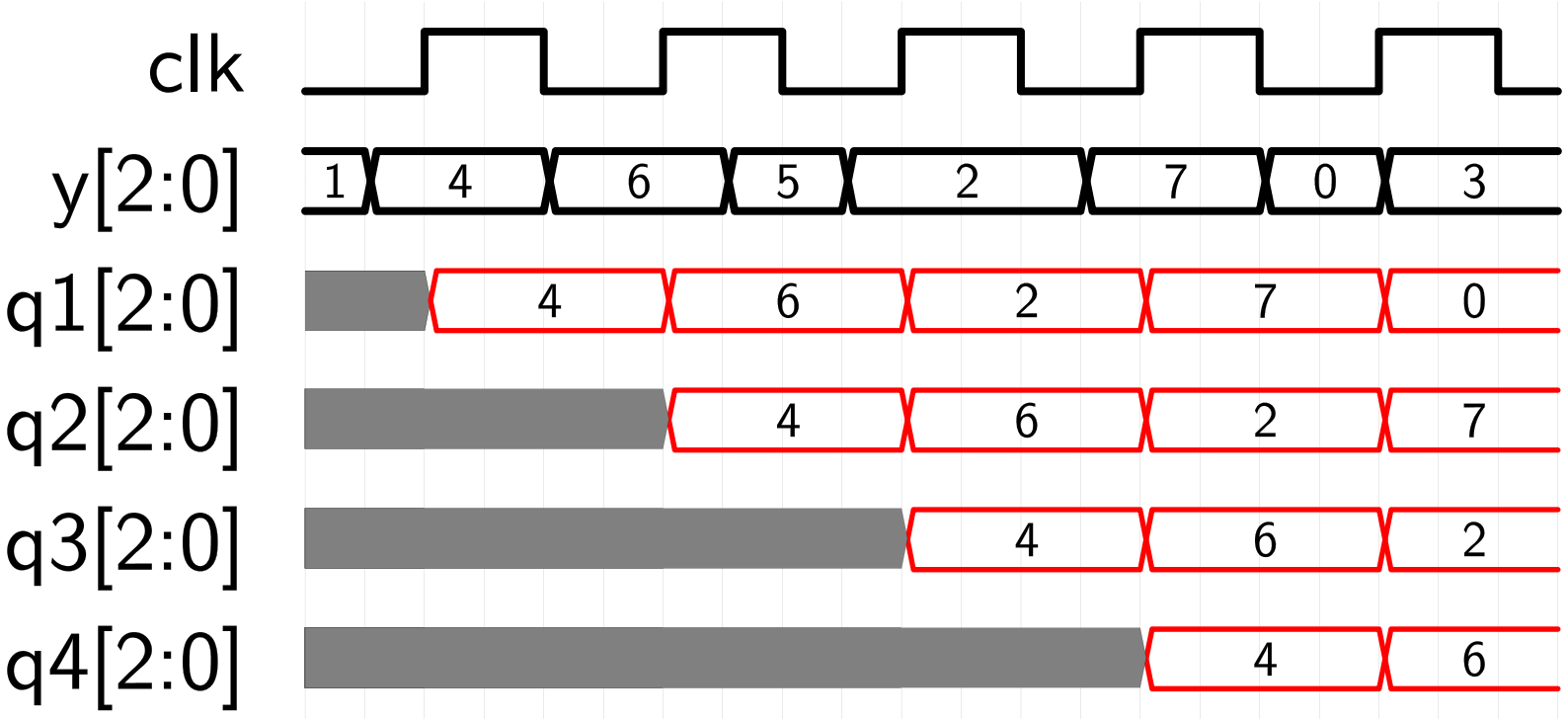


- We can use decimal or hex, but make it clear
- Again, transitions only happen at the rising edge

Registers

- What about this arrangement?





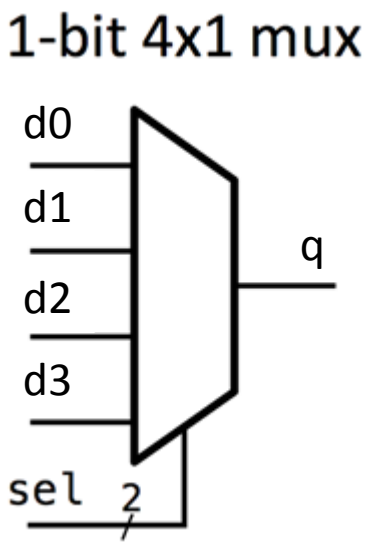
NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Sequential Circuits in Verilog

Combinational Verilog

- All the Verilog we have looked at so far allows us to implement purely combinational circuits
- It is important to be wary of the rules introduced there to avoid inadvertently writing sequential code that would not function as intended




```
module mux4 (output reg q,
             input [3:0] d,
             input [1:0] sel);
   always @* begin
      case (sel)
         2'b00 : q = d[0];
         2'b01 : q = d[1];
         2'b10 : q = d[2];
         2'b11 : q = d[3];
      endcase
   end
endmodule
```

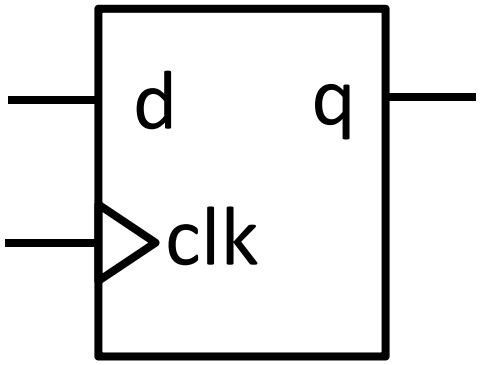
Sequential Verilog

NANYANG TECHNOLOGICAL UNIVERSITY

- We will only introduce the basics of sequential Verilog in this course
- The small blocks we show can be incorporated into larger designs using instantiation
- While it is possible to describe general sequential blocks in Verilog, it is generally used to describe ***synchronous*** circuits, i.e. **edge-triggered components**
- Synthesis tools will generally convert designs to D-type flip-flops/registers
- We can design many variations of the basic components

Registers in Verilog

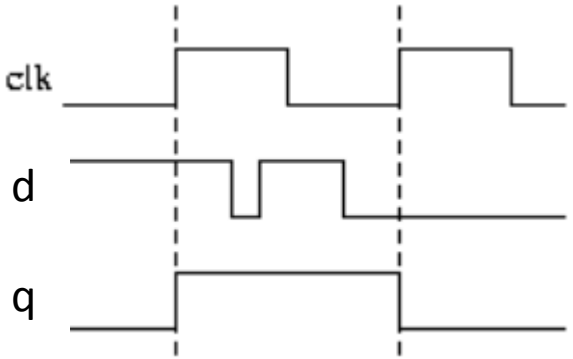
- The basic structure of an edge-sensitive block in Verilog is as follows:




```
module simplereg (input d, clk,
                  output reg q);

always@(posedge clk)
    q <= d;

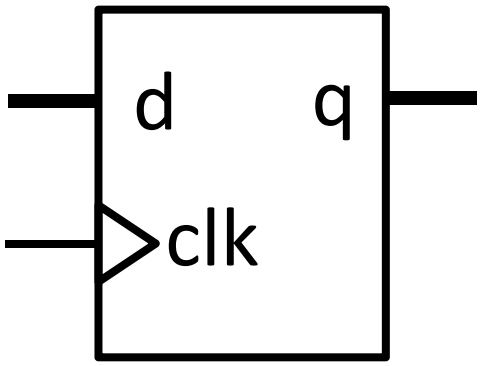
endmodule
```



- This creates a 1-bit register/D flip-flop with input *d* and output *q*

Registers in Verilog

- Note the new **always** block format:
  - For combinational, we list signals, or use always@*
  - For synchronous, we use always@(**posedge** clk)
- This tells the synthesis tools that the block's behavior should only happen at the **clock rising edge**, hence creating a flip-flop/register
- We can create a multi-bit register:



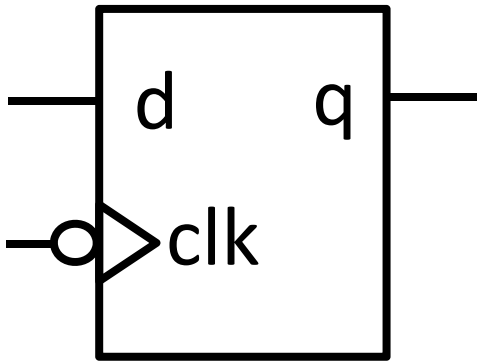

```
module simplereg (input [7:0] d, input clk,
                  output reg [7:0] q);

always@(posedge clk)
    q <= d;

endmodule
```

Clock and Reset

- In the previous examples, we used *clk* for the clock input
- The signal name should be whatever the clock signal is: *i_clk*, *gen_clk*, *clock*, *clk2400*
- If we're naming it, we often just use *clk*
- Remember to add the clock input to your module port list
- All **synchronous always blocks** should use the same clock signal
- For falling-edge triggered, use always@(**negedge** clk)



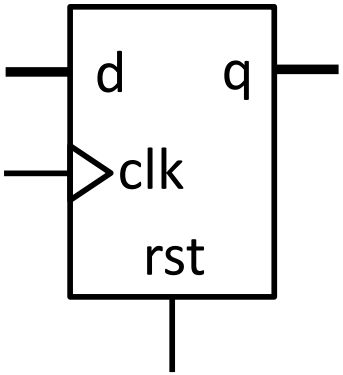
Clock and Reset

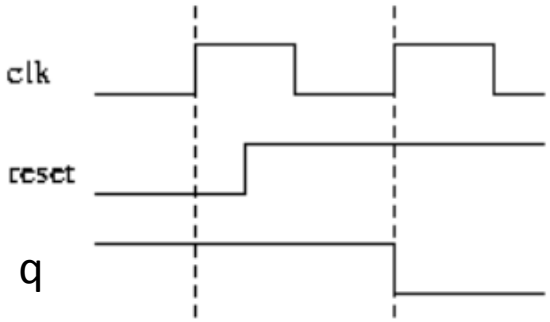
NANYANG TECHNOLOGICAL UNIVERSITY

- Registers are very useful, but often, we want to be able to *reset* the value in a register
- Two types of reset:
  - ***Asynchronous***: whenever the reset input is asserted, the contents of the register are set to the reset value
  - ***Synchronous***: at a rising edge, if reset is asserted, the contents of the register are set to the reset value
- In modern FPGA design, we use synchronous reset
- Generally, **every** synchronous component should be implemented with a reset

Clock and Reset

- A register with *synchronous* reset:





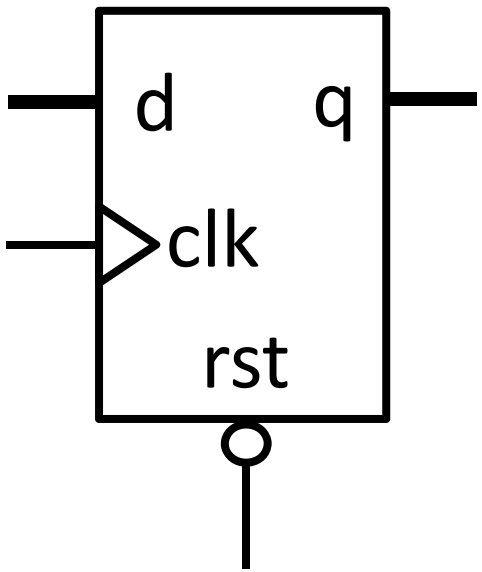

```
module simplereg (input [7:0] d,
                  input clk, rst,
                  output reg [7:0] q);

always@(posedge clk)
begin
    if(rst) // same as (rst==1'b1)
        q <= 8'b0000_0000;
    else
        q <= d;
end

endmodule
```

Clock and Reset

- Sometimes the reset is *active low*:



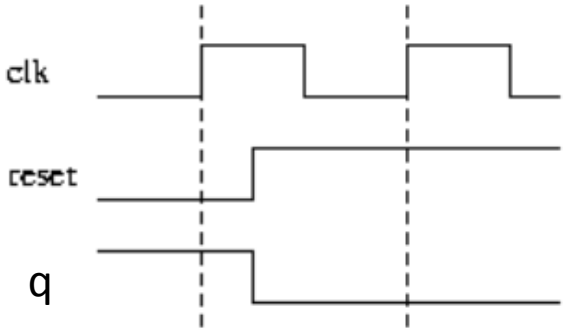

```
module simplereg (input [7:0] d,
                  input clk, rst,
                  output reg [7:0] q);

always@(posedge clk)
begin
    if(!rst)
        q <= 8'b0000_0000;
    else
        q <= d;
end

endmodule
```

Clock and Reset

- For an *asynchronous* reset, we would need to add the reset signal to the sensitivity list:



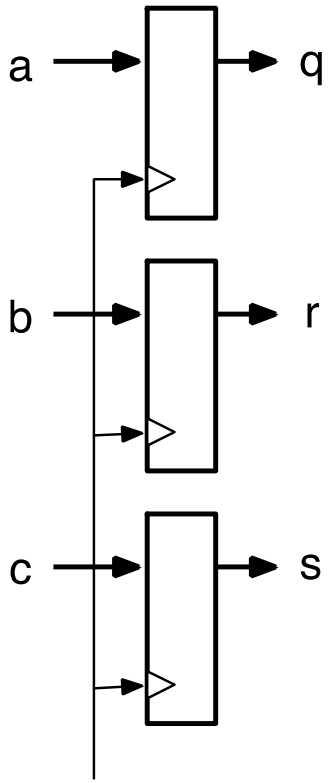

```
module simplereg (input [7:0] d,
                  input clk, rst,
                  output reg [7:0] q);

always@(posedge clk or posedge rst)
begin
    if(rst)
        q <= 8'b0000_0000;
    else
        q <= d;
end

endmodule
```

Registers in Verilog

- We can create multiple registers by including multiple assignments
- Each *assignment* in a synchronous always block results in a *register*



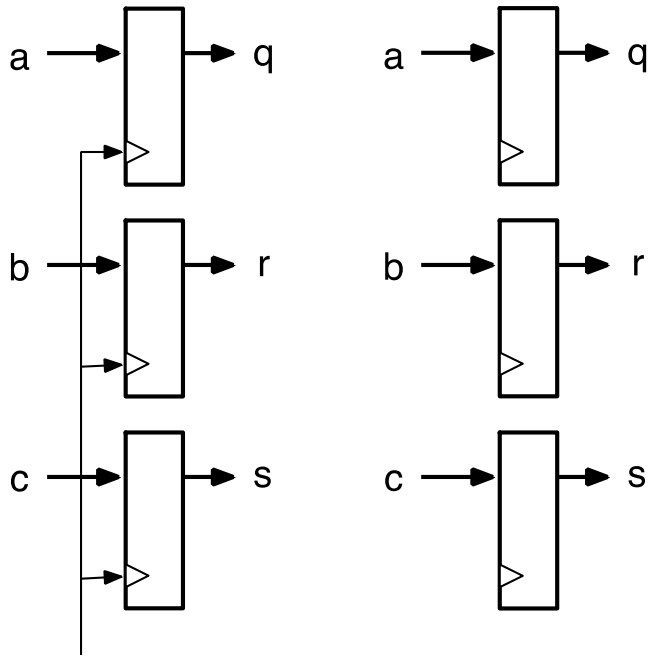

```
module multireg (input [7:0] a, b, c,
                 input clk, rst,
                 output reg [7:0] q, r, s);

always@(posedge clk)
begin
    if(!rst) begin
        q <= 8'b0000_0000;
        r <= 8'b0000_0000;
        s <= 8'b0000_0000;
    end else begin
        q <= a;
        r <= b;
        s <= c;
   end
end

endmodule
```

Registers in Verilog

- Even if registers are not connected, we can combine them in the same always block
- Each assignment in an always @ posedge clk block results in a register
- Remember, we **always** have a reset for registers
- We usually leave out the reset wires, and sometimes the clk wires in diagrams



Assignments in Always Blocks

NANYANG TECHNOLOGICAL UNIVERSITY

- You may have noticed we are using a new assignment operator: <=
- This is called a *non-blocking* assignment
- For ***combinational*** always blocks, we *always* use a blocking assignment (=), and **order matters**
- For ***synchronous*** always blocks, we *always* use non-blocking assignments (<=), and **order does not matter**
- This explanation is sufficient for this course

Synchronous Components

NANYANG TECHNOLOGICAL UNIVERSITY

- Registers are useful for storing values, and organizing the timing in a circuit
- There are a number of other basic sequential blocks we can use within our designs
  - Counters
  - Shift Registers
  - Serial-to-Parallel and Parallel-to-Serial Converters
  - Memories

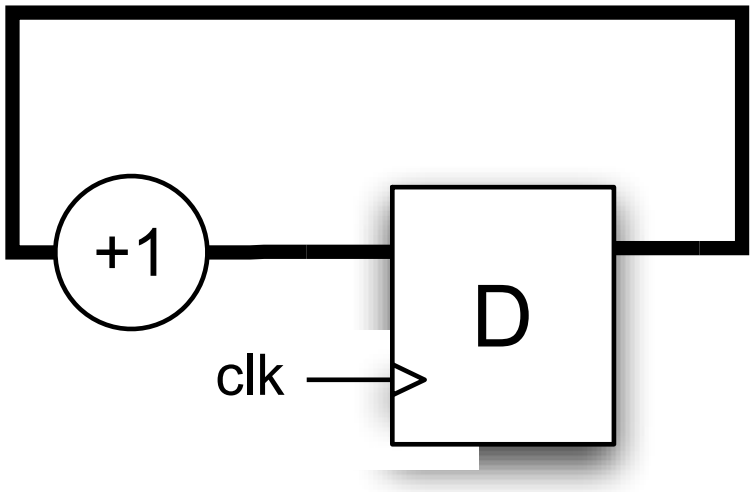
Binary Counters

- A *binary counter* is a circuit that outputs an increasing output value in each clock cycle
- Consider the count sequence of a 3-bit counter:

0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

Synchronous Counters

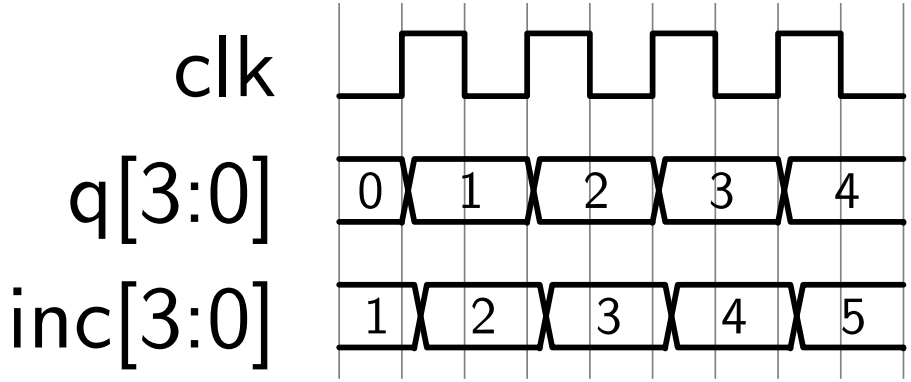
- There is another way to think about counters that may be closer to what we would do in Verilog:

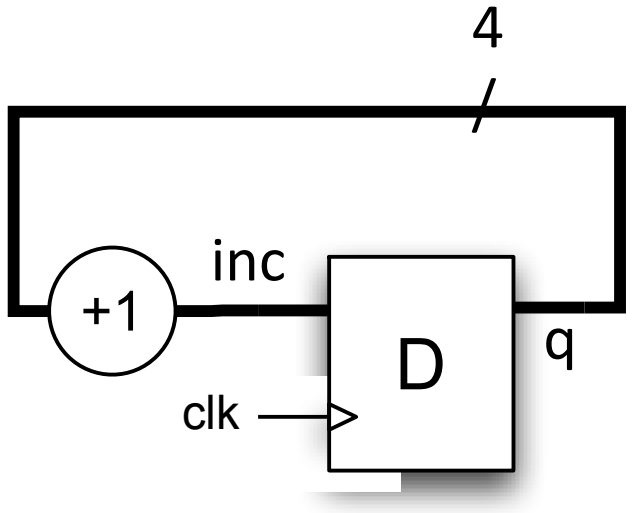


- At each rising edge, we pass through the incremented value of the current count
- The data width can be any number of bits

Synchronous Counters

- Timing diagram helps:

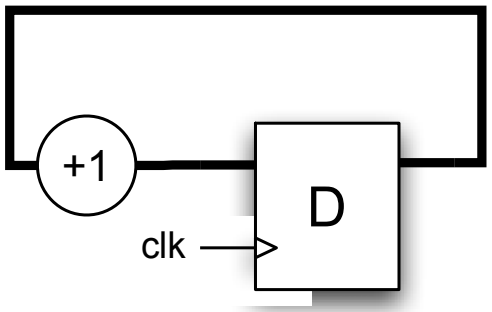




- At each clock edge, the incremented value, *inc*, derived from the current output, is passed to *q*

Synchronous Counters in Verilog

- We can describe such a counter in Verilog as follows:



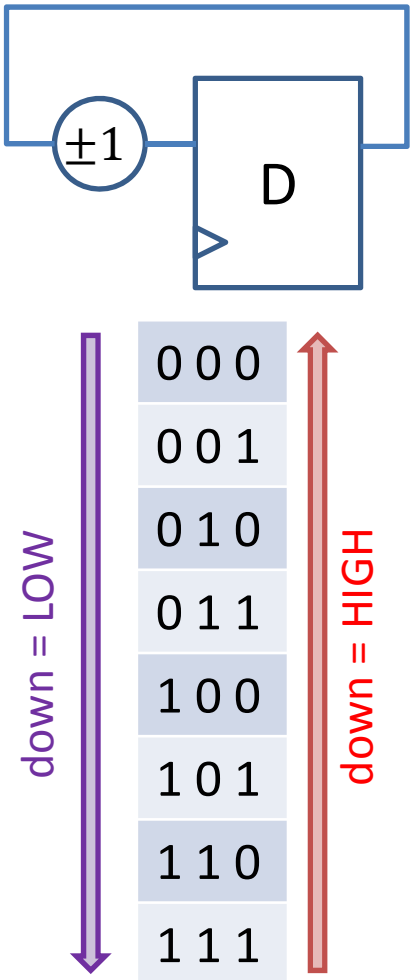

```
module simplecnt (input clk, rst,
                  output reg [3:0] q);

always@(posedge clk)
begin
    if(rst)
        q <= 4'b0000;
    else
        q <= q + 1'b1;
end

endmodule
```

Synchronous Counters in Verilog

- We can extend the capabilities of our counter quite easily; here, an up-down counter:



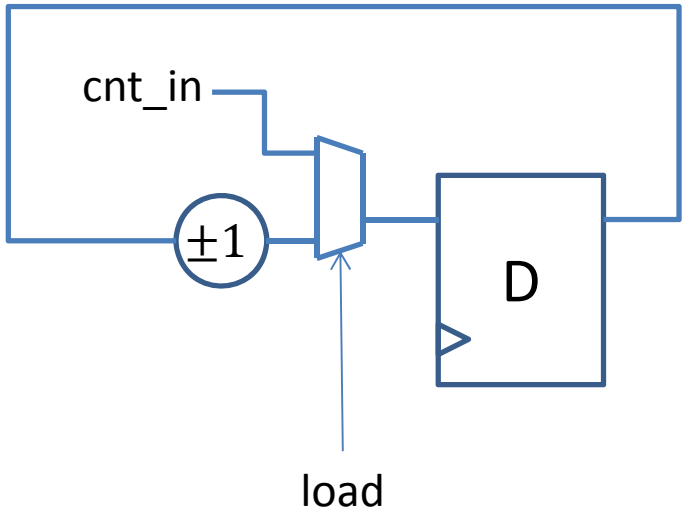

```
module simplecnt (input clk, rst, down,
                  output reg [3:0] q);

always@(posedge clk)
begin
    if(rst)
        q <= 4'b0000;
    else
        if(down)
            q <= q - 1'b1;
        else
            q <= q + 1'b1;
end

endmodule
```

Synchronous Counter in Verilog

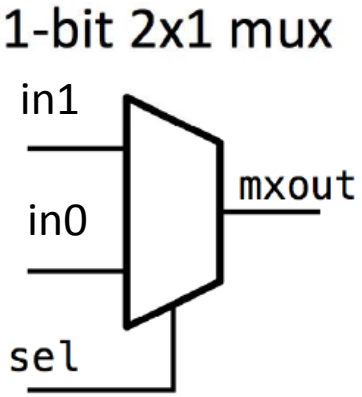
- The important thing to remember, is the *q* output is only updated at the rising edge, so when we use its value inside the block, it is the **old** value
- What if we want to be able to load a custom value into the counter?
  - load input: when high, the counter takes its value from cnt_in
  - cnt_in: custom load value
  - Hence, should load if required, otherwise count up or down depending on the down input

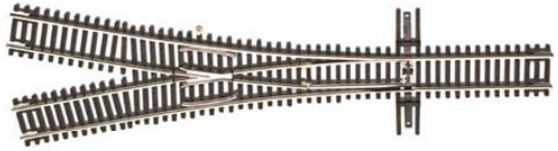


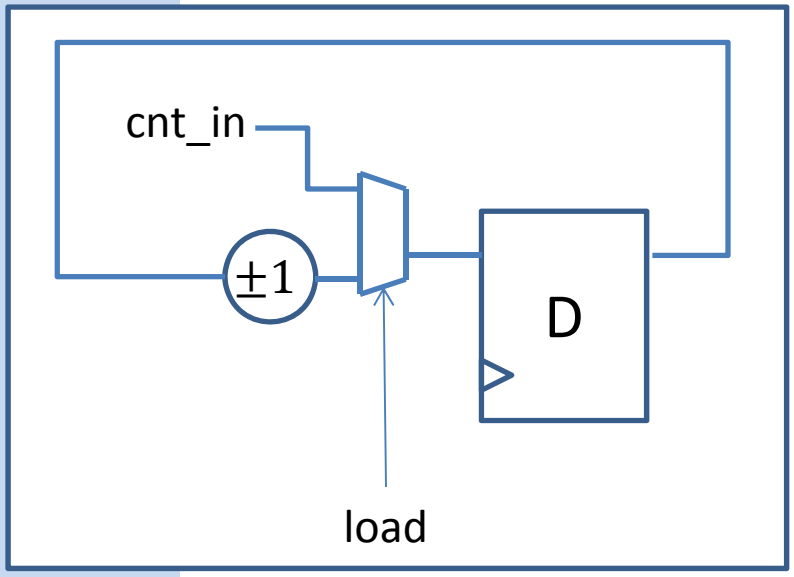
Synchronous Counters in Verilog

```
module simplecnt (input clk, rst,
                  input down, load,
                  input [3:0] cnt_in,
                  output reg [3:0] q);

always@(posedge clk)
begin
    if(rst)
        q <= 4'b0000;
    else
        if(load)
            q <= cnt_in;
        else
            if(down)
                q <= q - 1'b1;
            else
                q <= q + 1'b1;
end
endmodule
```



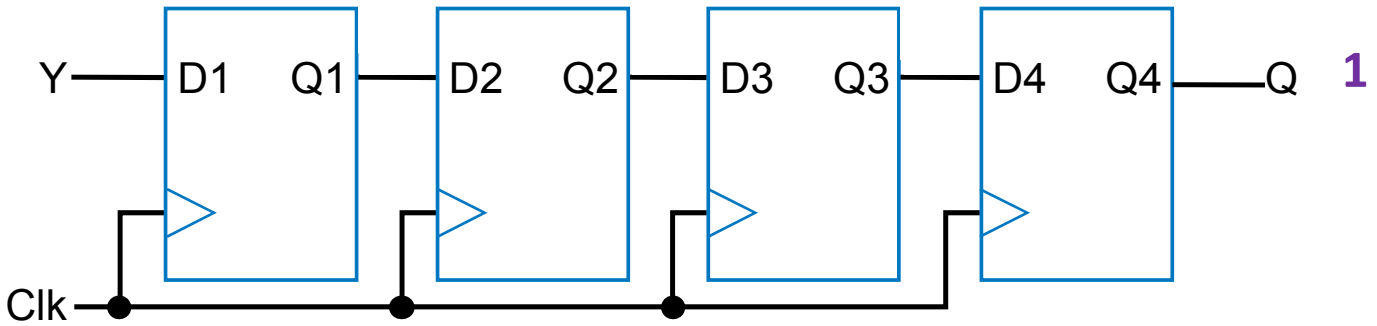




Note: Synthesis tool may not instantiate a multiplexer

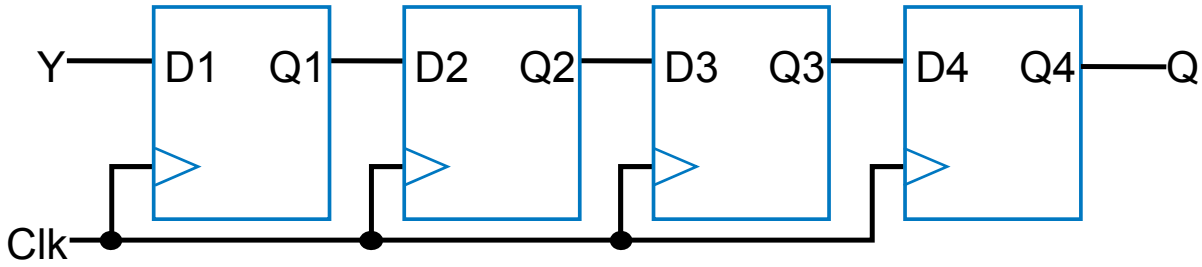
Shift Registers in Verilog

- Shift registers take a single input and pass it through a chain of flip-flops
- At each clock cycle, the input progresses one stage



- We need internal signals to connect the intermediate registers, must be declared as **reg** type
- We then write a single assignment for each register:

Shift Registers in Verilog




```
module shiftreg (input clk, y,
                 output reg q);

reg q1, q2, q3;

always@(posedge clk)
begin
    q1 <= y;
    q2 <= q1;
    q3 <= q2;
    q  <= q3;
end

endmodule
```

```
module shiftreg (input clk, y,
                 output reg q);

reg q1, q2, q3;

always@(posedge clk)
begin
    q2 <= q1;
    q1 <= y;
    q  <= q3;
    q3 <= q2;
end

endmodule
```

What if I change the order of the assignments?

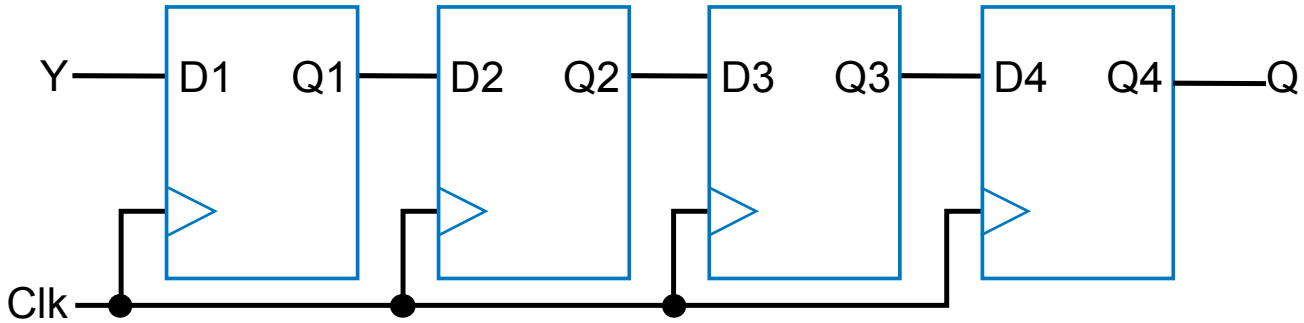
Shift Registers in Verilog

NANYANG TECHNOLOGICAL UNIVERSITY

- What if I change the order of the assignments?
  - The circuit will function identically
  - Each assignment is a register, and that assignment only occurs on the rising edge
  - Hence, they all transfer their input just **before** the rising edge to their output just **after** the rising edge
  - It would take 4 clock cycles for an input value to reach the final output in that example

Shift Registers in Verilog

- We can use vectors to make the code easier:




```
module shiftreg (input clk, y,
                 output reg q);

reg q1, q2, q3;

always@(posedge clk)
begin
    q1 <= y;
    q2 <= q1;
    q3 <= q2;
    q  <= q3;
end

endmodule
```

```
module shiftreg (input  clk, y,
                 output reg q_out);

reg [3:1] q;

always@(posedge clk)
begin
    q[1]   <= y;
    q[3:2] <= q[2:1];
    q_out  <= q[3];
end

endmodule
```

Shift Registers in Verilog

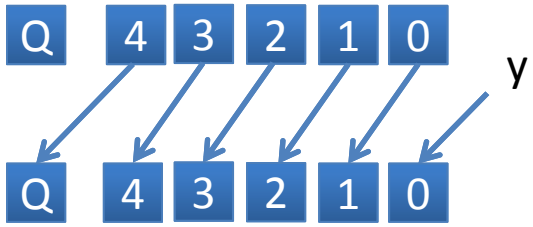
- So now it's easy to extend this to more flip-flops:

```
module shiftreg (input clk, y,
                 output reg Q);

reg [4:0] q;

always@(posedge clk)
begin
    q[0]   <= y;
    q[4:1] <= q[3:0];
    Q      <= q[4];
end

endmodule
```



More Shift Registers

- We can now think of the contents as a single word
- We now hard wire the output to the MSB

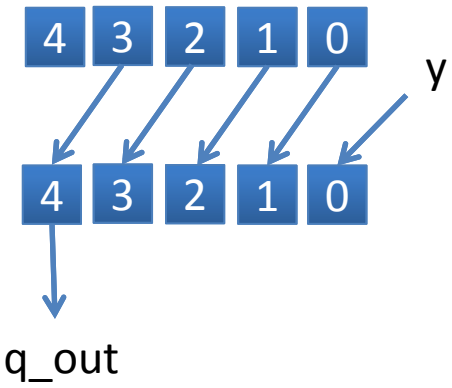
```
module shiftreg (input clk, y,
                 output q_out);

reg [4:0] q;

always@(posedge clk)
begin
    q[0]   <= y;
    q[4:1] <= q[3:0];
end

assign q_out = q[4];

endmodule
```



More Shift Registers

- What if we want to only shift sometimes?
- Now, a shift will only occur when the *sh* input is high

```
module shiftreg2 (input clk, y, sh,
                  output q_out);

reg [4:0] q;

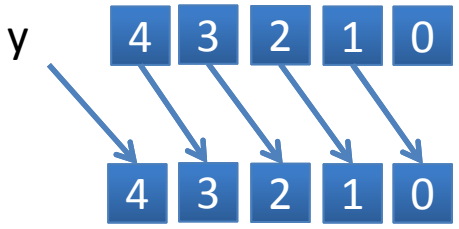
always@(posedge clk)
begin
    if (sh) begin
        q[0]   <= y;
        q[4:1] <= q[3:0];
    end
end

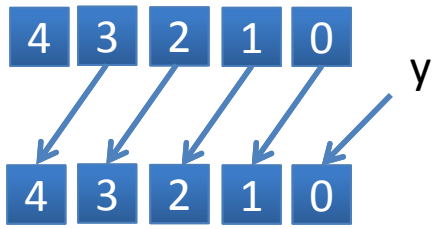
assign q_out = q[4];

endmodule
```

More Shift Registers

- We can also add the capability to shift in the other direction
- When the *rt* input is high, the new sample is registered at the MSB, and the rest shift right






```
module shiftreg3 (input clk, y, sh, rt,
                  output q_out);

reg [4:0] q;

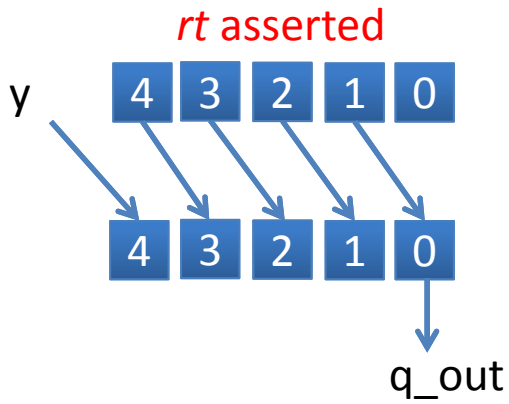
always@(posedge clk)
begin
    if (sh) begin
        if (rt) begin
            q[4] <= y; q[3:0] <= q[4:1];
        end else begin
            q[0] <= y; q[4:1] <= q[3:0];
        end
    end
end

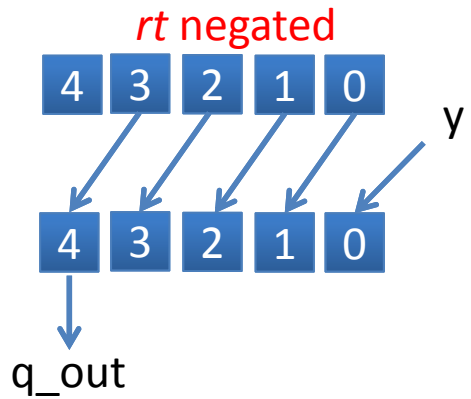
assign q_out = rt ? q[0] : q[4];

endmodule
```

More Shift Registers

- We might want to be able to access the whole word






```
module shiftreg4 (input clk, y, sh, rt,
                  output q_out,
                  output [4:0] q_word);

reg [4:0] q;

always@(posedge clk)
begin
    if (sh) begin
        if (rt) begin
            q[4] <= y; q[3:0] <= q[4:1];
        end else begin
            q[0] <= y; q[4:1] <= q[3:0];
        end
    end
end

assign q_out = rt ? q[0] : q[4];
assign q_word = q;

endmodule
```

More Shift Registers

- Shift registers can also have a load input
- Whenever *ld* is high, the shift register is loaded with the value on *ld_val*

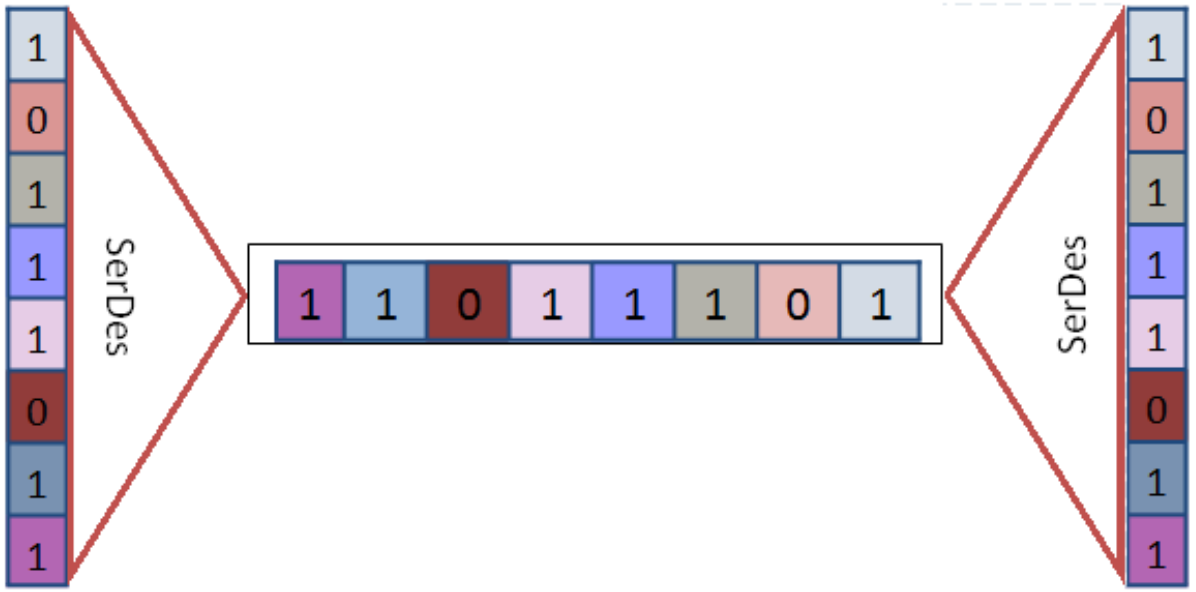
```
module shiftreg5 (input clk, y, sh, rt, ld,
                  input [4:0] ld_val,
                  output q_out,
                  output [4:0] q_word);
reg [4:0] q;

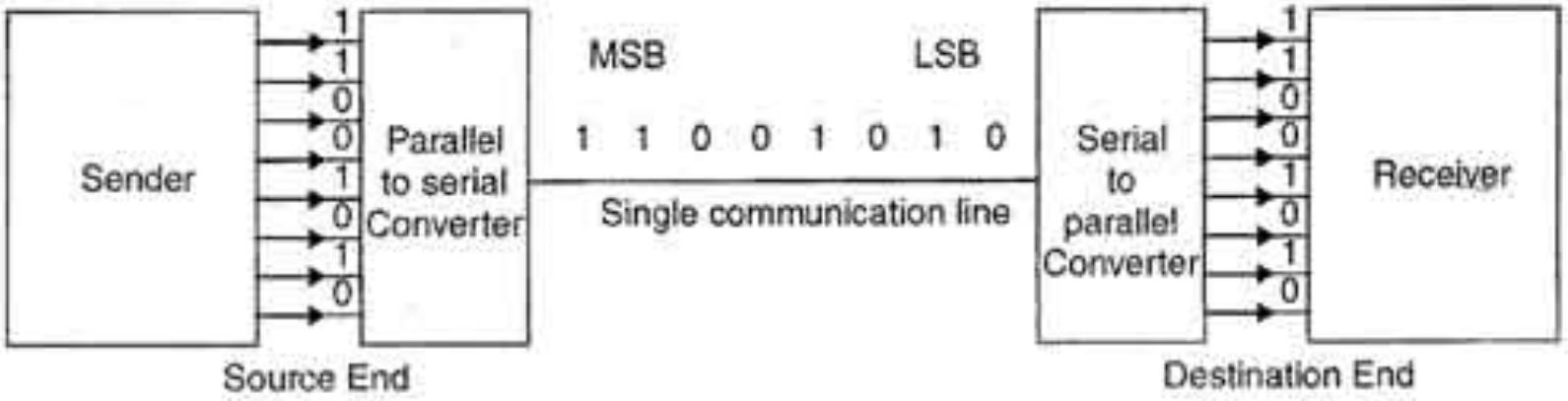
always@(posedge clk)
begin
    if (ld) q <= ld_val; else
    if (sh) begin
        if (rt) begin
            q[4] <= y; q[3:0] <= q[4:1];
        end else begin
            q[0] <= y; q[4:1] <= q[3:0];
        end
    end
end

assign q_out = rt ? q[0] : q[4];
assign q_word = q;

endmodule
```

Serial Data Transfer

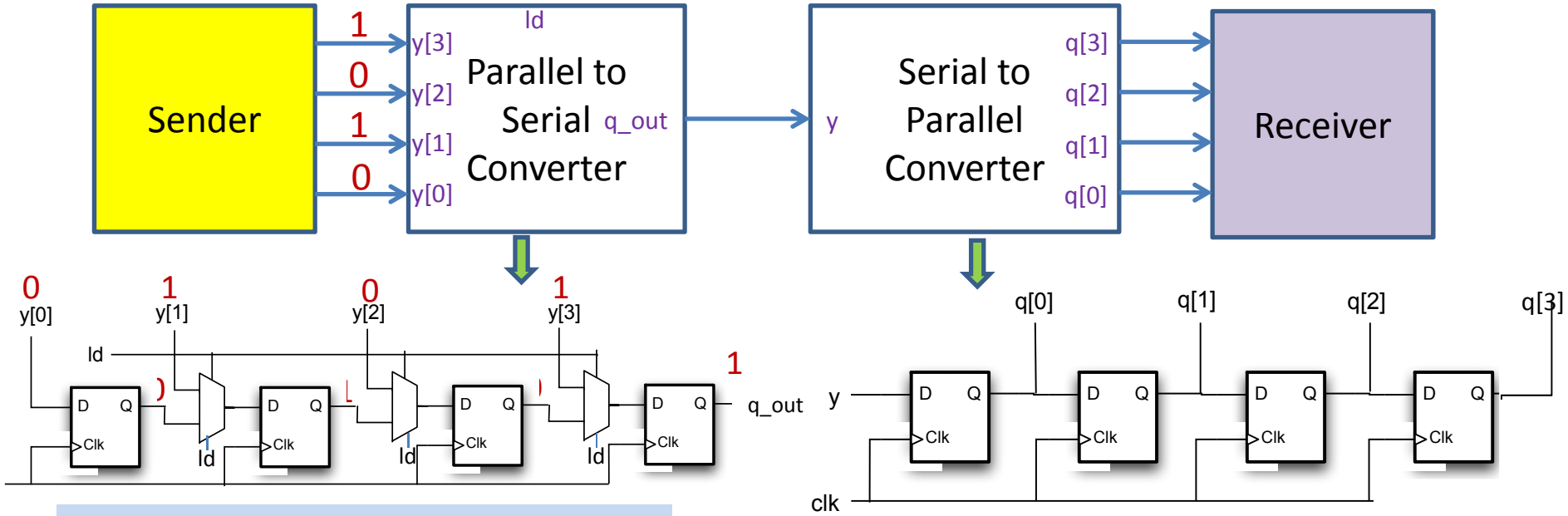




Source:
http://ecomputernotes.com/computernetworkingnotes/communication-networks/data-transmission

Serial transmission

Serial Data Transfer

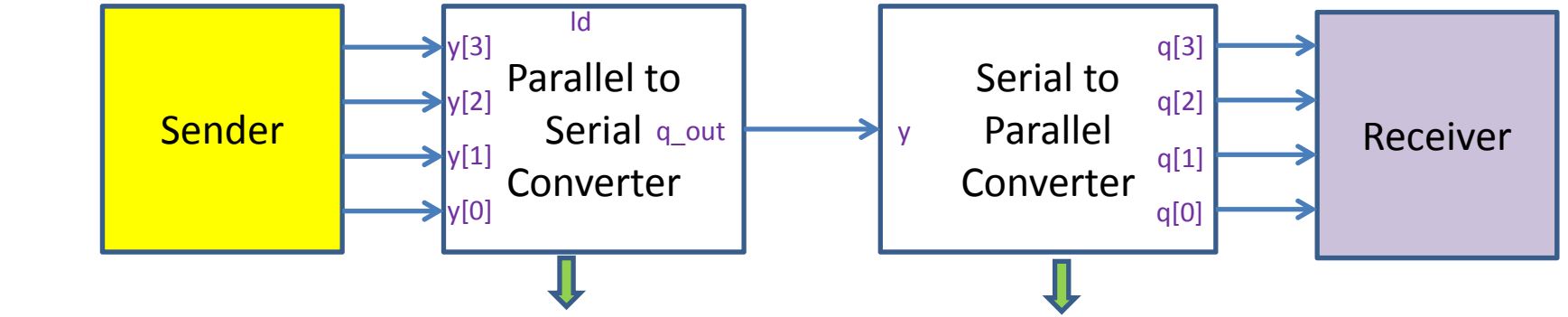


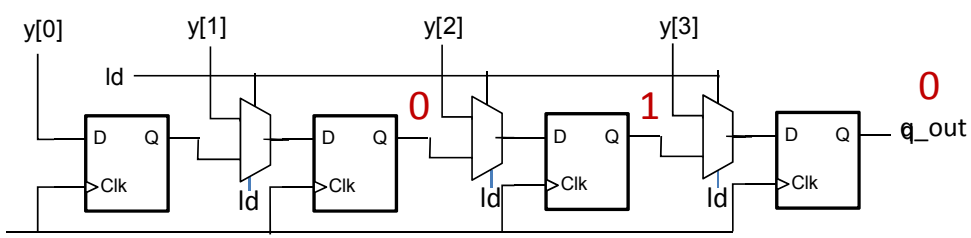

```
module piso4 (input clk, ld,
              input [3:0] y,
              output q_out);
reg [3:0] q;

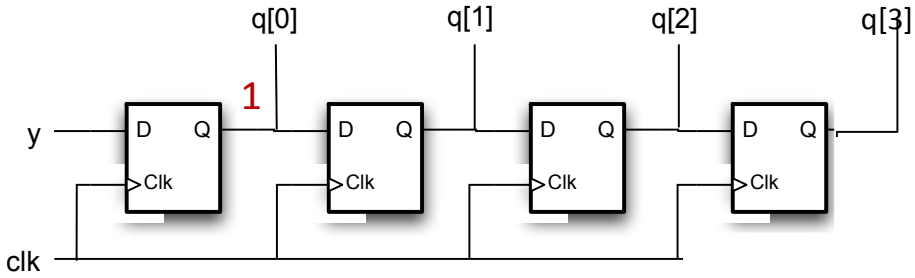
always@(posedge clk)
begin
    if (ld) q <= y;
    else begin
        q[0]   <= y[0];
        q[3:1] <= q[2:0];
    end
end
assign q_out = q[3];

endmodule
```

Serial Data Transfer





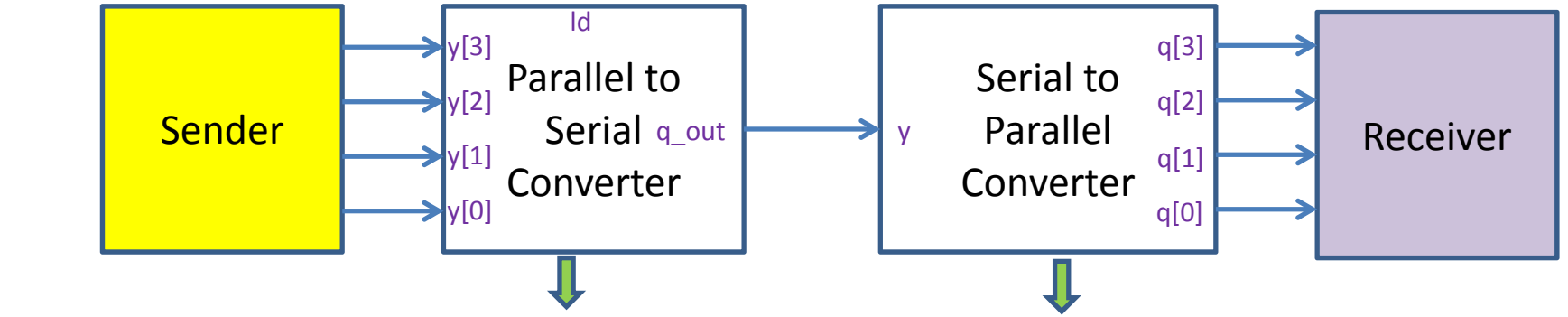


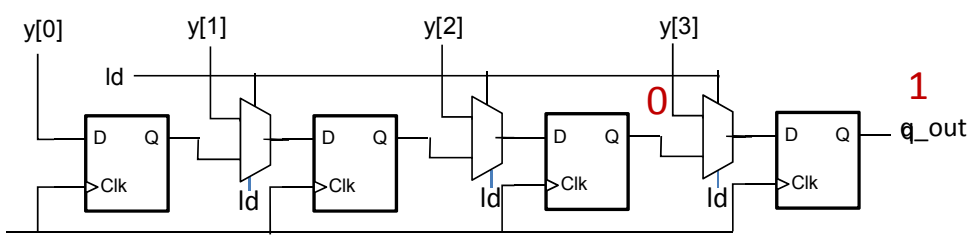

```
module piso4 (input clk, ld,
              input [3:0] y,
              output q_out);
reg [3:0] q;

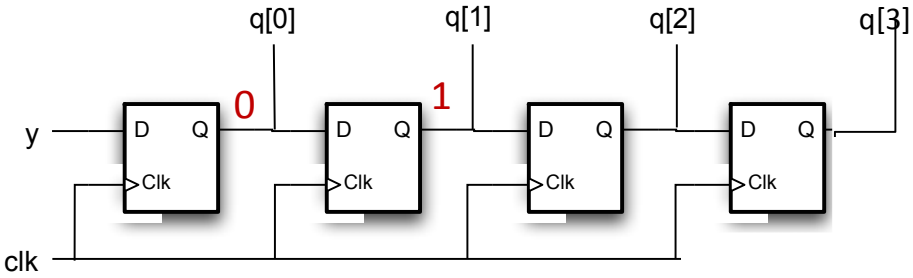
always@(posedge clk)
begin
    if (ld) q <= y;
    else begin
        q[0]   <= y[0];
        q[3:1] <= q[2:0];
    end
end
assign q_out = q[3];

endmodule
```

Serial Data Transfer





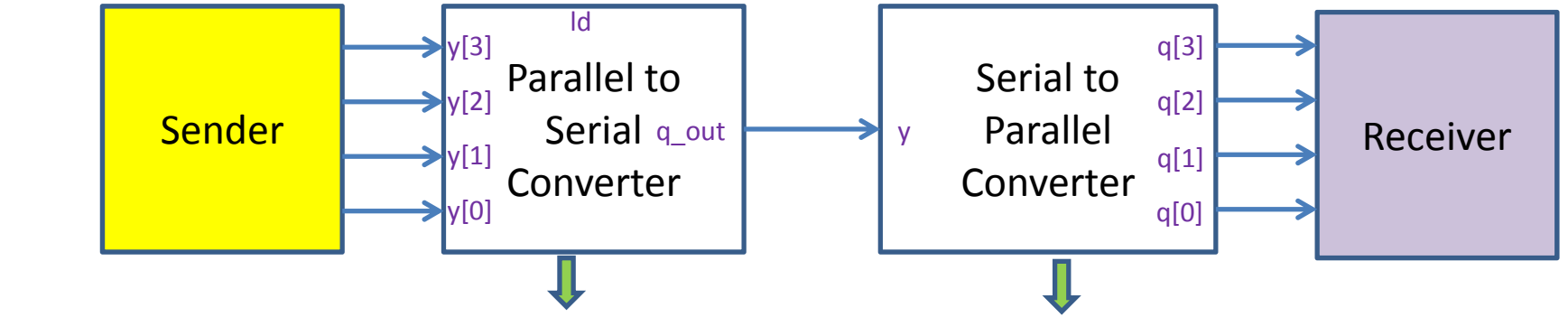


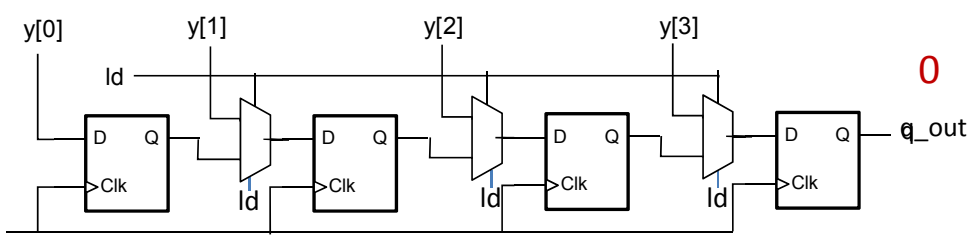

```
module piso4 (input clk, ld,
              input [3:0] y,
              output q_out);
reg [3:0] q;

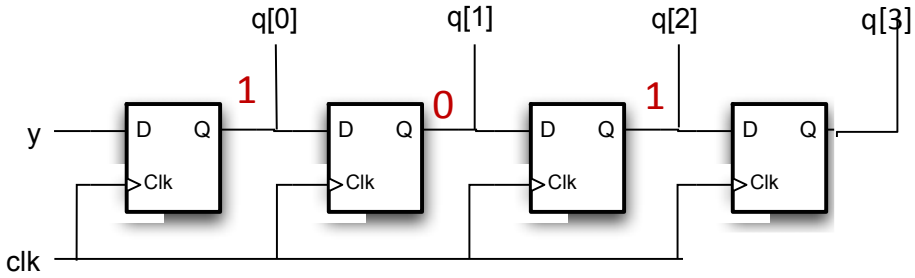
always@(posedge clk)
begin
    if (ld) q <= y;
    else begin
        q[0]   <= y[0];
        q[3:1] <= q[2:0];
    end
end
assign q_out = q[3];

endmodule
```

Serial Data Transfer





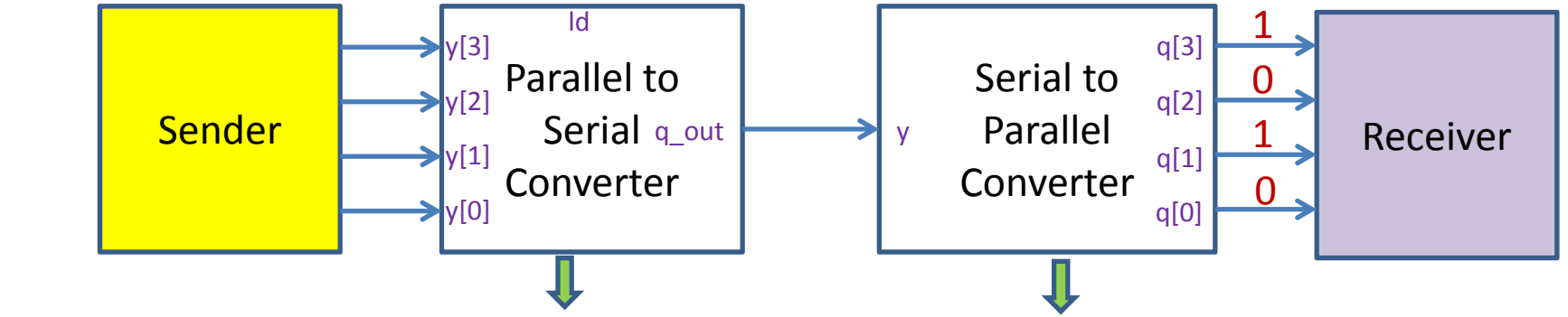


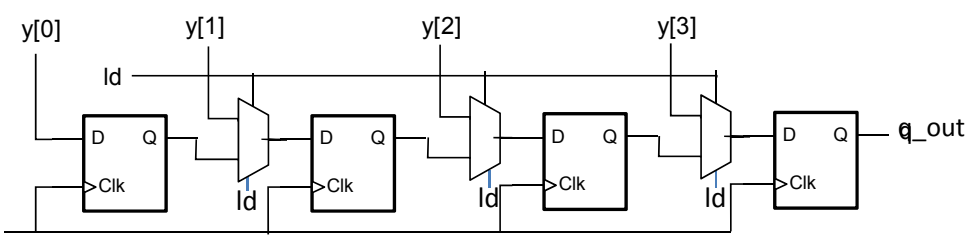

```
module piso4 (input clk, ld,
              input [3:0] y,
              output q_out);
reg [3:0] q;

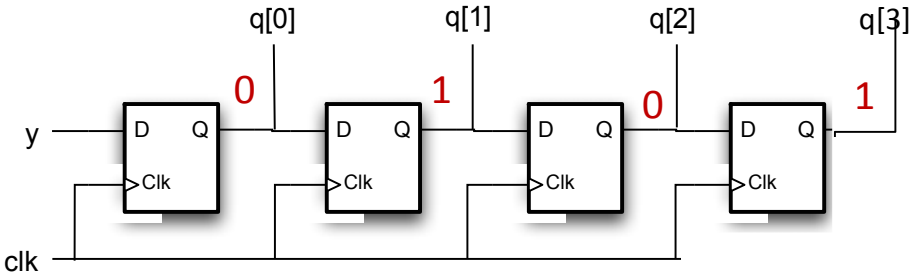
always@(posedge clk)
begin
    if (ld) q <= y;
    else begin
        q[0]   <= y[0];
        q[3:1] <= q[2:0];
    end
end
assign q_out = q[3];

endmodule
```

Serial Data Transfer








```
module piso4 (input clk, ld,
              input [3:0] y,
              output q_out);
reg [3:0] q;

always@(posedge clk)
begin
    if (ld) q <= y;
    else begin
        q[0]   <= y[0];
        q[3:1] <= q[2:0];
    end
end
assign q_out = q[3];

endmodule
```

```
module sipo4 (input clk, y,
              output reg [3:0] q);
always@(posedge clk)
begin
    q[0]   <= y;
    q[3:1] <= q[2:0];
end

endmodule
```

First-In-First-Out Buffers

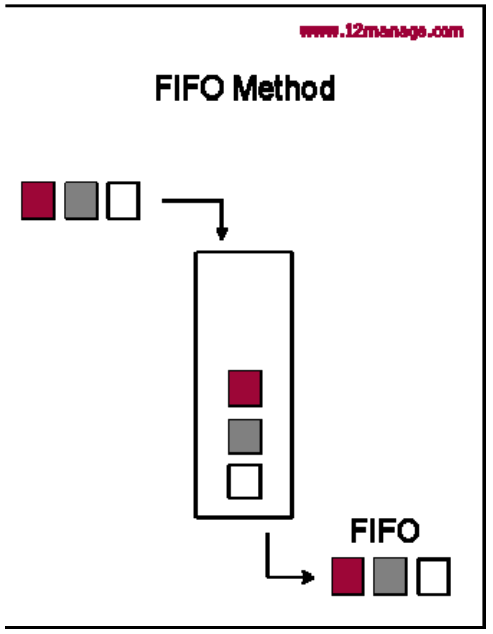
- When we build shift register structures for multi-bit signals, we typically refer to them as FIFOs (First-In-First-Out)
- They are useful for managing the flow of data when building datapaths (later)
- The structure is identical to shift registers, but now each register is multi-bit
- The input, output, and internal signal widths should match:

```
module fifo4 (input clk, input [3:0] y,
              output reg [3:0] q);

reg [3:0] q1, q2, q3;

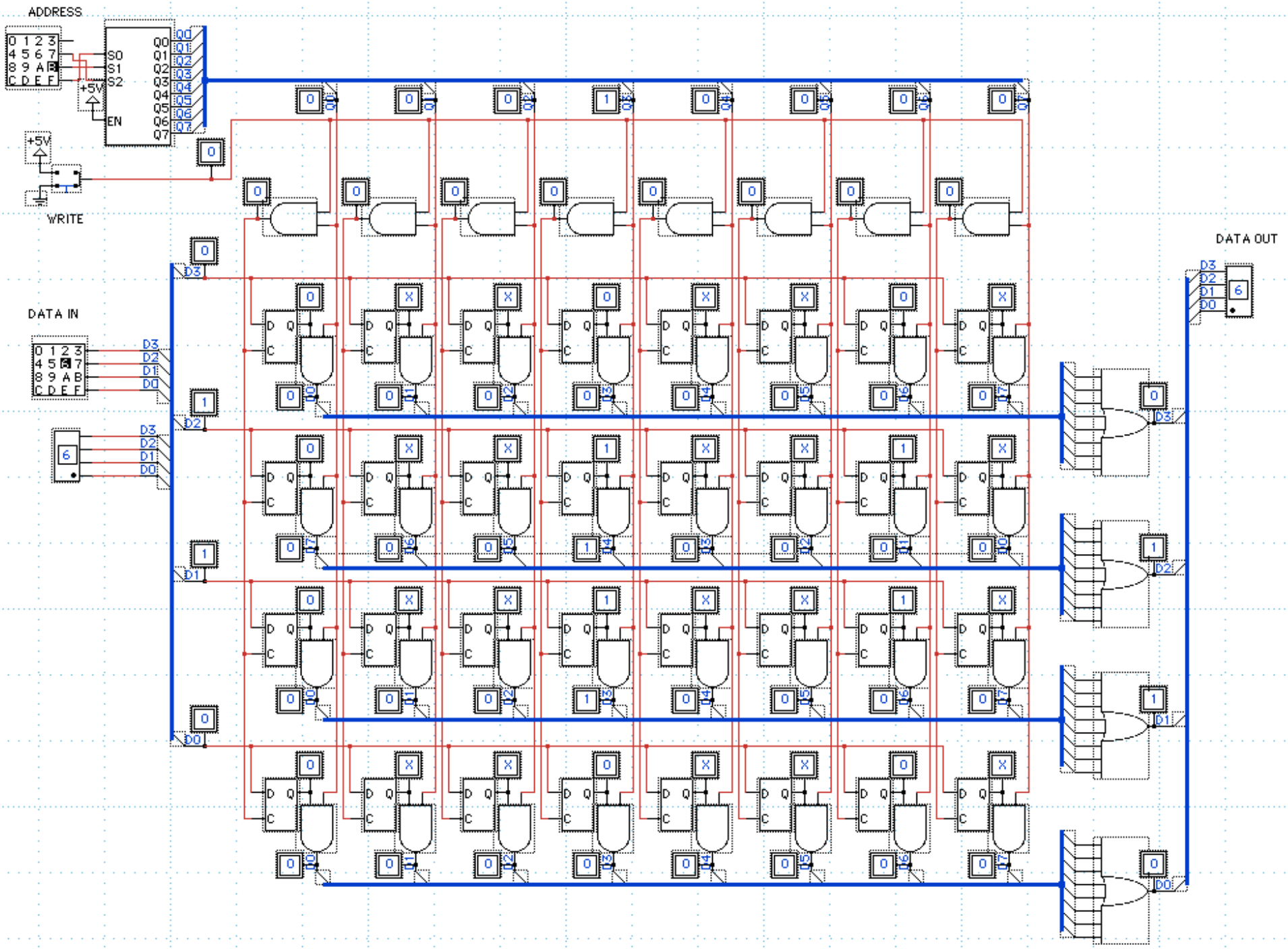
always@(posedge clk)
begin
    q1 <= y;
    q2 <= q1;
    q3 <= q2;
    q  <= q3;
end

endmodule
```



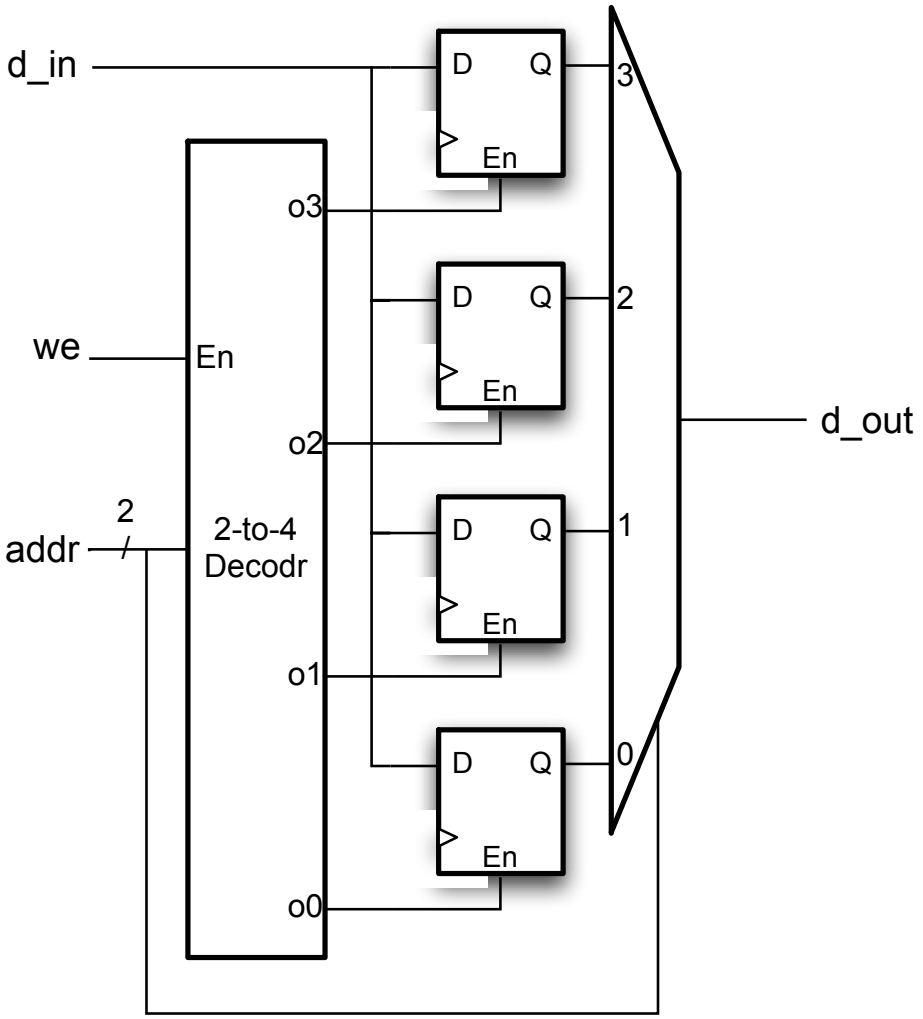
Memories

- Memories consists of an array of storage elements
- You can think of a register as one storage element
- Each storage element has a unique address
- We should be able to select which register to store to using an address
- We should be able to read the value out of any register, again using the address


ADDRESS
0 1 2 3
4 5 6 7
8 9 A B
C D E F
S0
S1
S2
EN
+5V
Q0
Q1
Q2
Q3
Q4
Q5
Q6
Q7
WRITE
DATA IN
D3
D2
D1
D0
DATA OUT
D Q
C

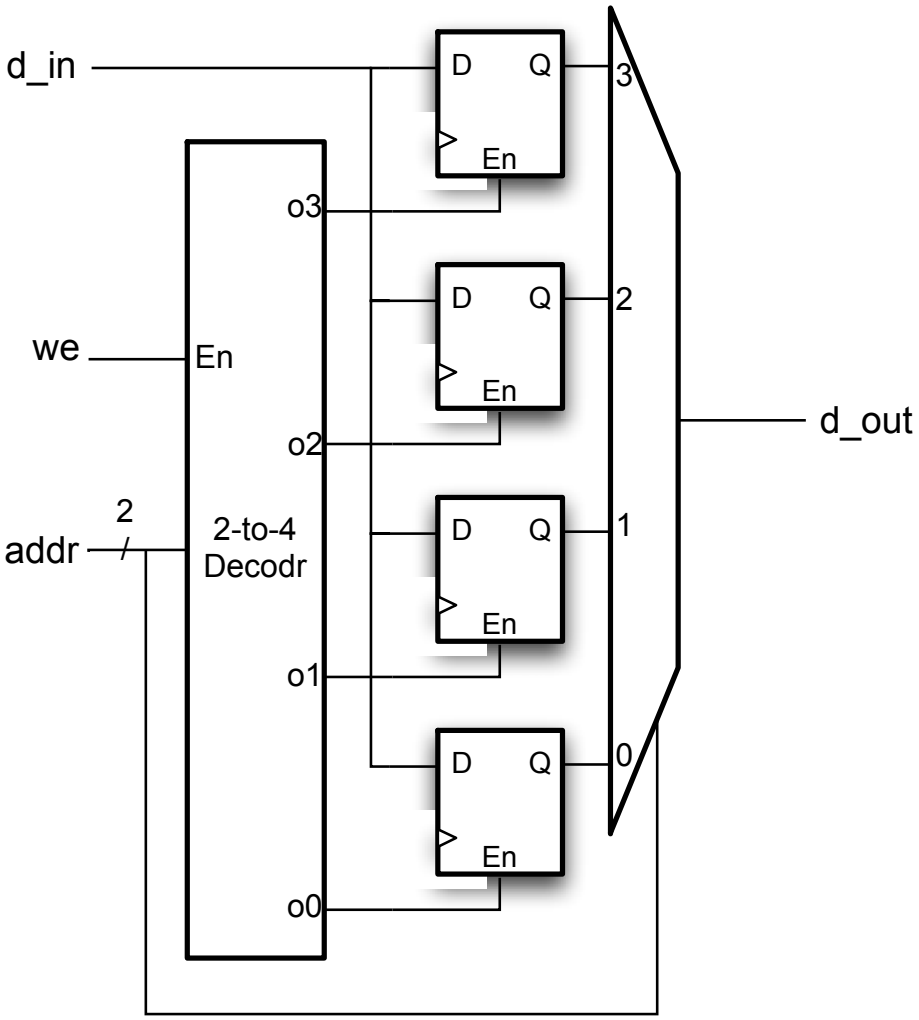
Memories

- In order to store a value, we must assert *we* (write enable) and provide an address
- The decoder outputs a 1 to enable the corresponding register to store whatever is on *d_in* at the clock edge



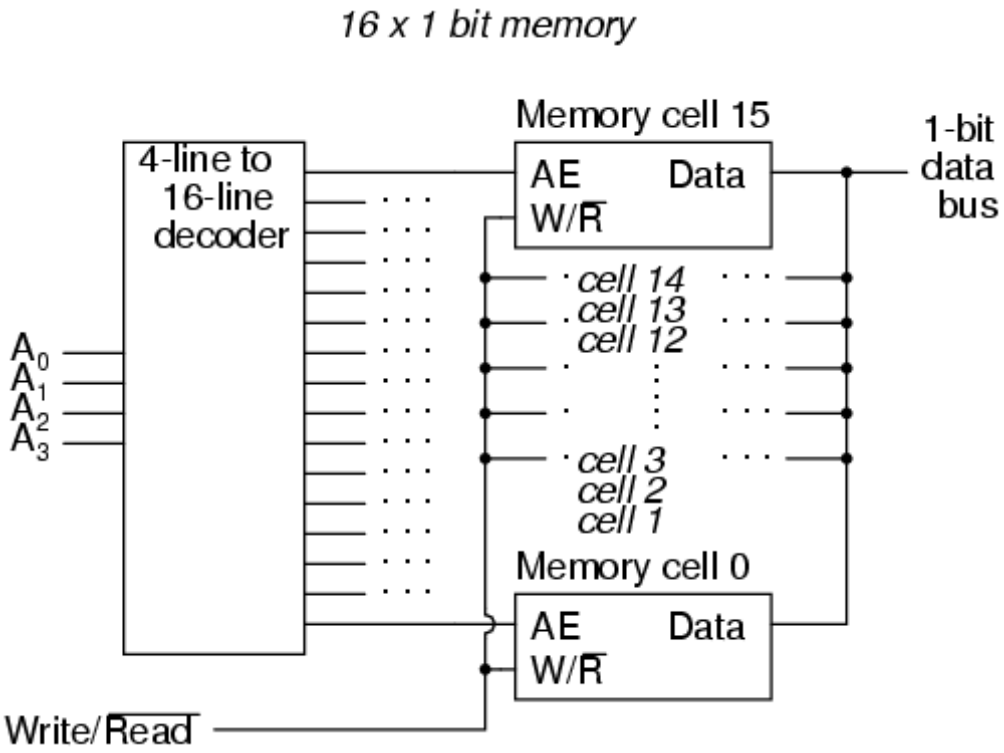
Memories

- This basic structure shows one possible arrangement
- The *addr* signal selects which location we want to read from or write to
- For reading, a mux connects the corresponding register to the *d_out* output



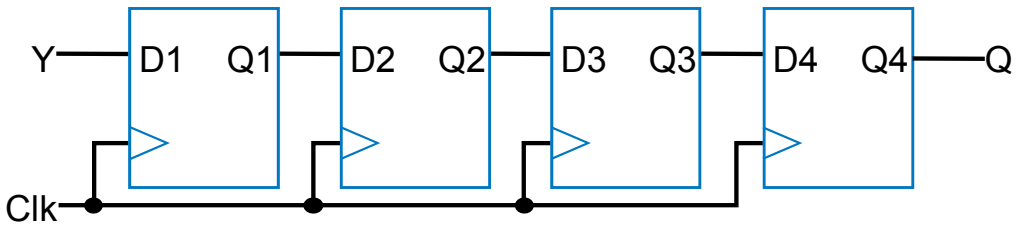
Memories

- Many structures exist for addressable memories
- Standard signal names
  - *we* : write enable
  - *re*: read enable
  - *addr*: address
  - *ce* : chip enable
  - *ae*: address enable



What Gets Synthesized?

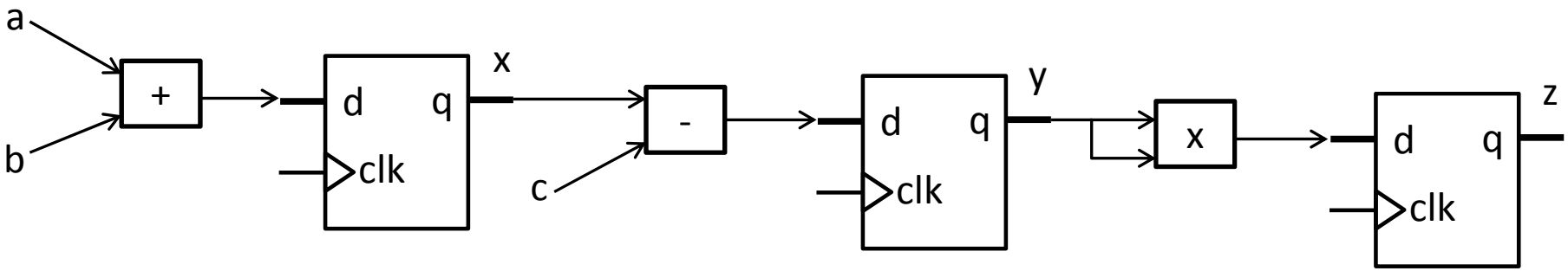
- Important to understand what is synthesized from a synchronous always block
- Each (non-blocking) assignment results in a register, with its input connected to a circuit based on the right hand side, and the output connected to the signal the left hand side
- **Every assignment is a register (i.e. synchronous)**




```
module shiftreg (input clk, y,
                 output reg q);
reg q1, q2, q3;
always@(posedge clk)
begin
    q1 <= y;
    q2 <= q1;
    q3 <= q2;
    q  <= q3;
end
endmodule
```

What Gets Synthesized?

```
always@(posedge clk)
begin
    x <= a + b;
    y <= x - c;
    z <= y * y;
end
```



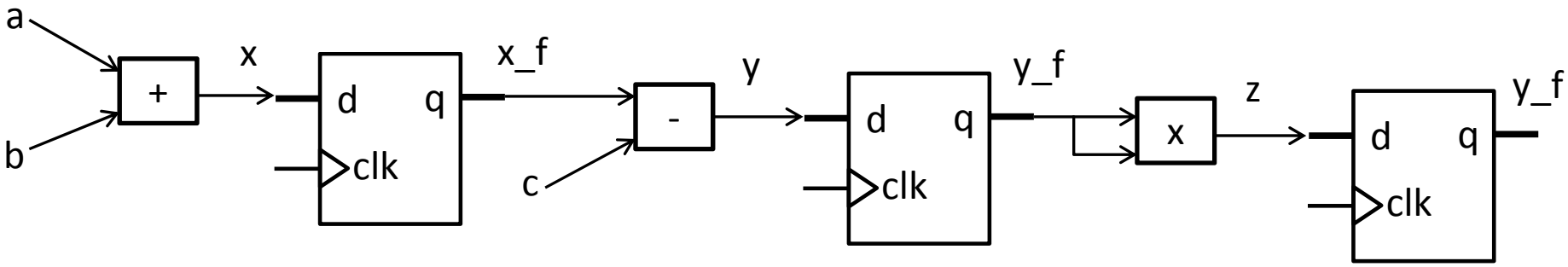
What Gets Synthesized?

- Some prefer to code with the combinational and synchronous aspects separated

```
always@(posedge clk)
begin
    x <= a + b;
    y <= x - c;
    z <= y * y;
end
```

```
always@(posedge clk)
begin
    x_f <= x;
    y_f <= y;
    z_f <= z;
end

always@*
begin
    x = a + b;
    y = x_f - c;
    z = y_f * y_f;
end
```



What Gets Synthesized?

NANYANG TECHNOLOGICAL UNIVERSITY

- The important thing is to make your code readable
- Breaking it down into smaller statements makes it easier to debug
- Remember when reading code:
  - The values "read" – i.e. on the right hand side – are considered **just before** the rising edge
  - The values "written" – i.e. on the left hand side – are only updated **just after** the rising edge
- Remember you cannot assign to a signal from more than one place!

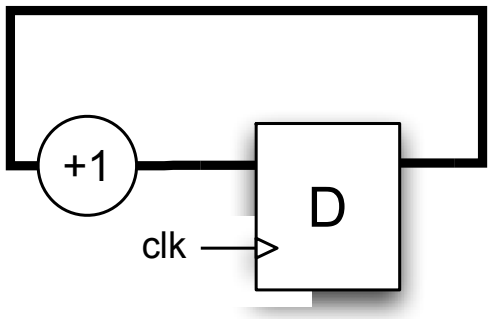
NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Finite State Machines

Sequences and States

- We have previously seen how we can design a binary counter
- Binary counters output a sequence of numbers, each being one more than the previous value
- Can I provide an abstract description of the behavior of the counter?




```
module simplecnt (input clk, rst,
                  output reg [3:0] q);

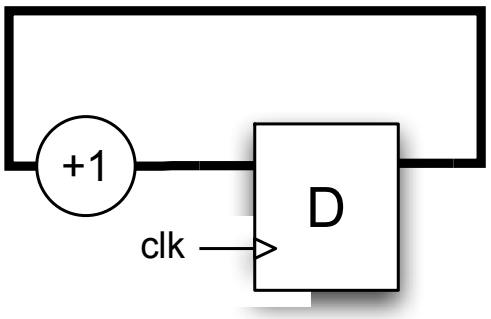
always@(posedge clk)
begin
    if(!rst)
        q <= 4'b0000;
    else
        q <= q + 1'b1;
end

endmodule
```

Sequences and States

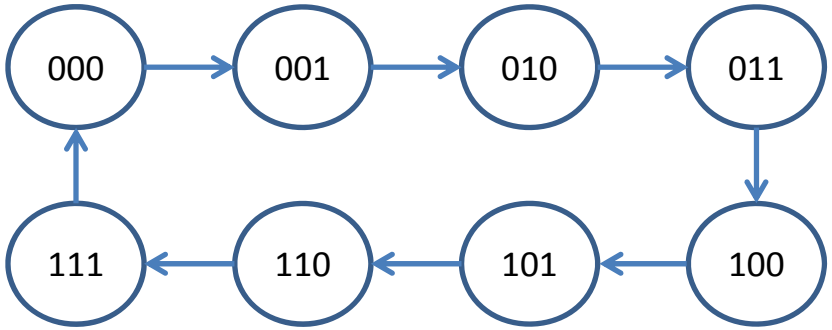
- For the counter, the sequence is easy to reason about
- At each point in time, the system is in a *state*, that determines what the output should be
- At each transition point, the circuit moves from its current state to the next state, corresponding to the one with the next largest output
- We can think of circuits as being *state machines*

0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1



Sequences and States

- We can use a figure to show these transitions:



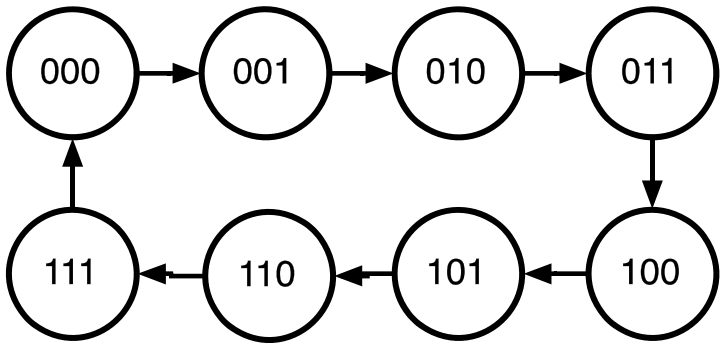
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

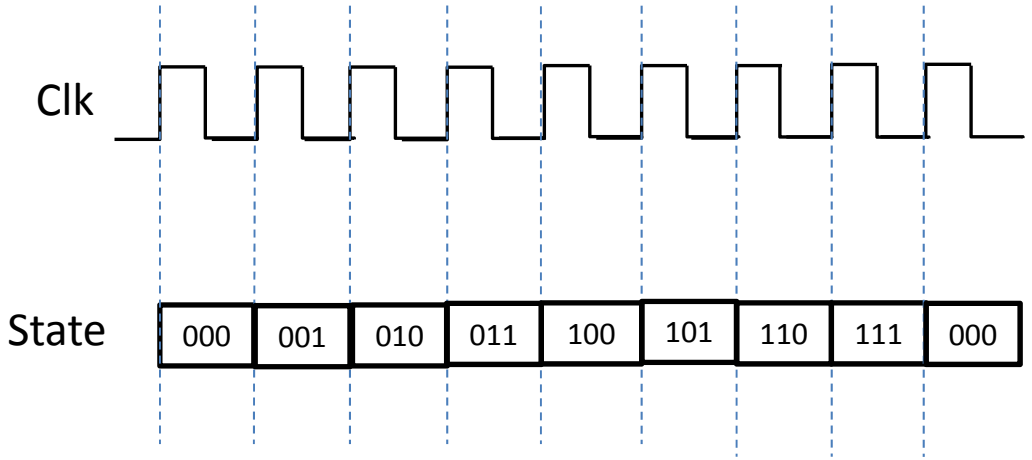
This is a called *state transition diagram*

- Each node is a possible *state* of the system
- It shows the movement from one state to the next
- In this case, the states are simply labeled with their output values

Sequences and States

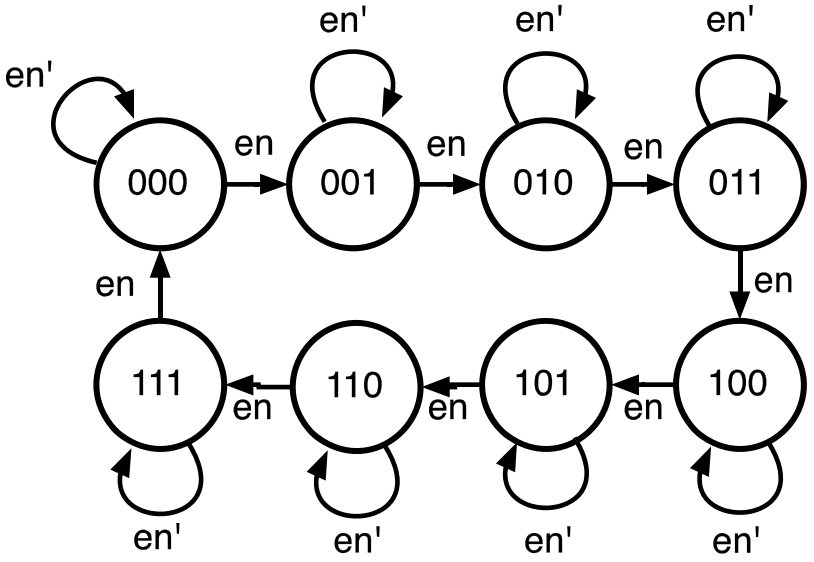
- In the previous diagram, the system always moved from one state to the next

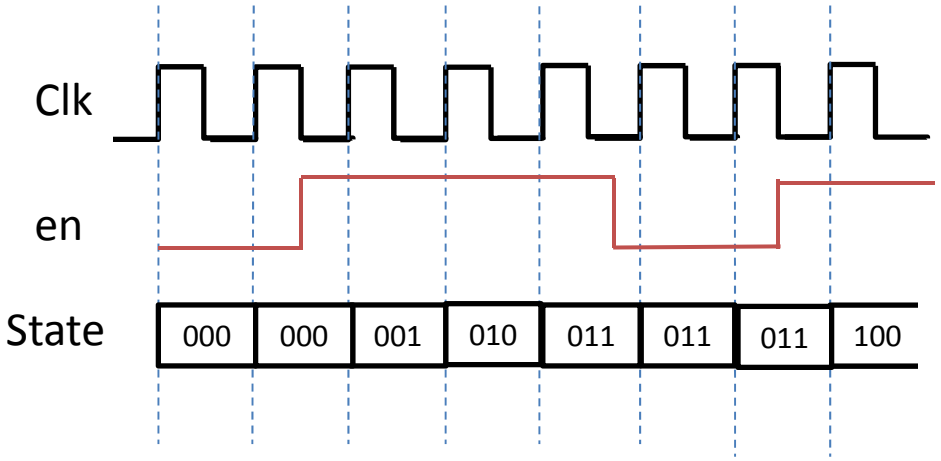




Sequences and States

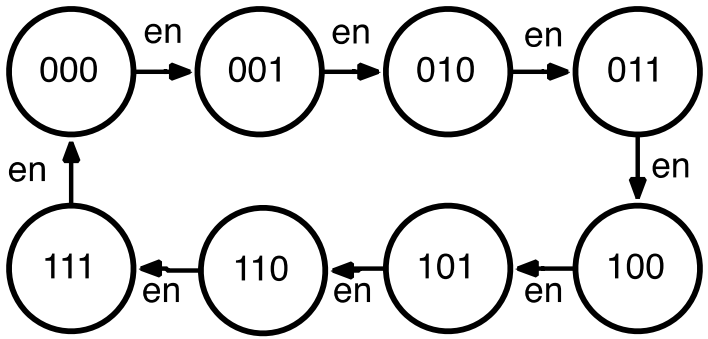
- In the previous diagram, the system always moved from one state to the next
- What about an enabled counter that only counts when an *en* input is high?
- We can indicate conditions for transition on the transition arrows:





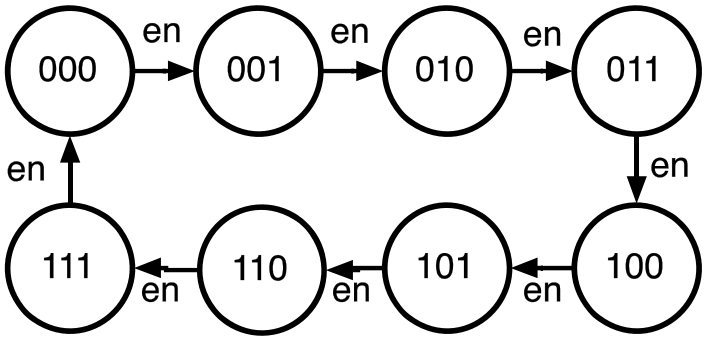
Sequences and States

- Here, we follow the standard count sequence when *en* is high
- If *en* is low, we stay in the current state
- We can simplify the diagram by assuming that conditions not shown result in self-transitions
- The state only changes when *en* is 1, otherwise, stay



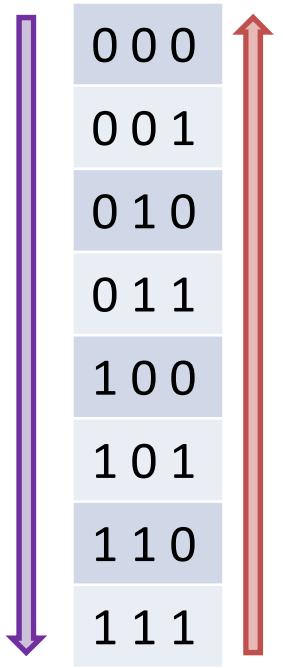
Sequences and States

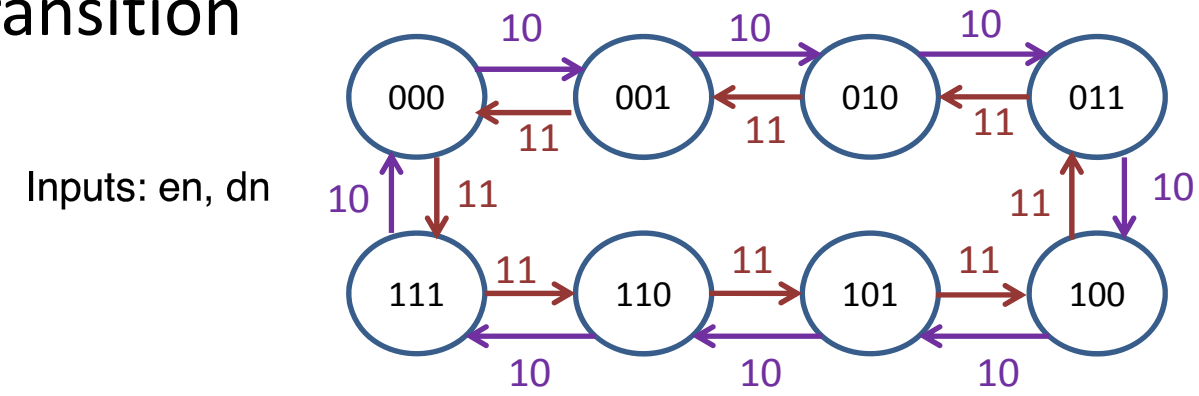
- So the state transition diagram consists of:
  - Circle nodes that represent the states
  - Arrows between states to represent possible transitions
  - Conditions place on the arrows to indicate when the transition occurs



Sequences and States

- What about an up/down counter?
  - Counts up whenever *dn* input is 0
  - Counts down whenever *dn* input is 1
- Labeling all inputs by name on arrows can be cumbersome
- Hence, use a legend: in this case, 10 means *en*=1, *dn*=0
- Other input combinations result in no transition





Sequences and States

- The preceding dealt with counters, with a relatively fixed sequence
- We used the output as the state label
- The sequence was quite restricted
- How can we generalize this for more complex systems?
- *Finite State Machines (FSMs)* are a key modelling method for digital systems

Finite State Machines

NANYANG TECHNOLOGICAL UNIVERSITY

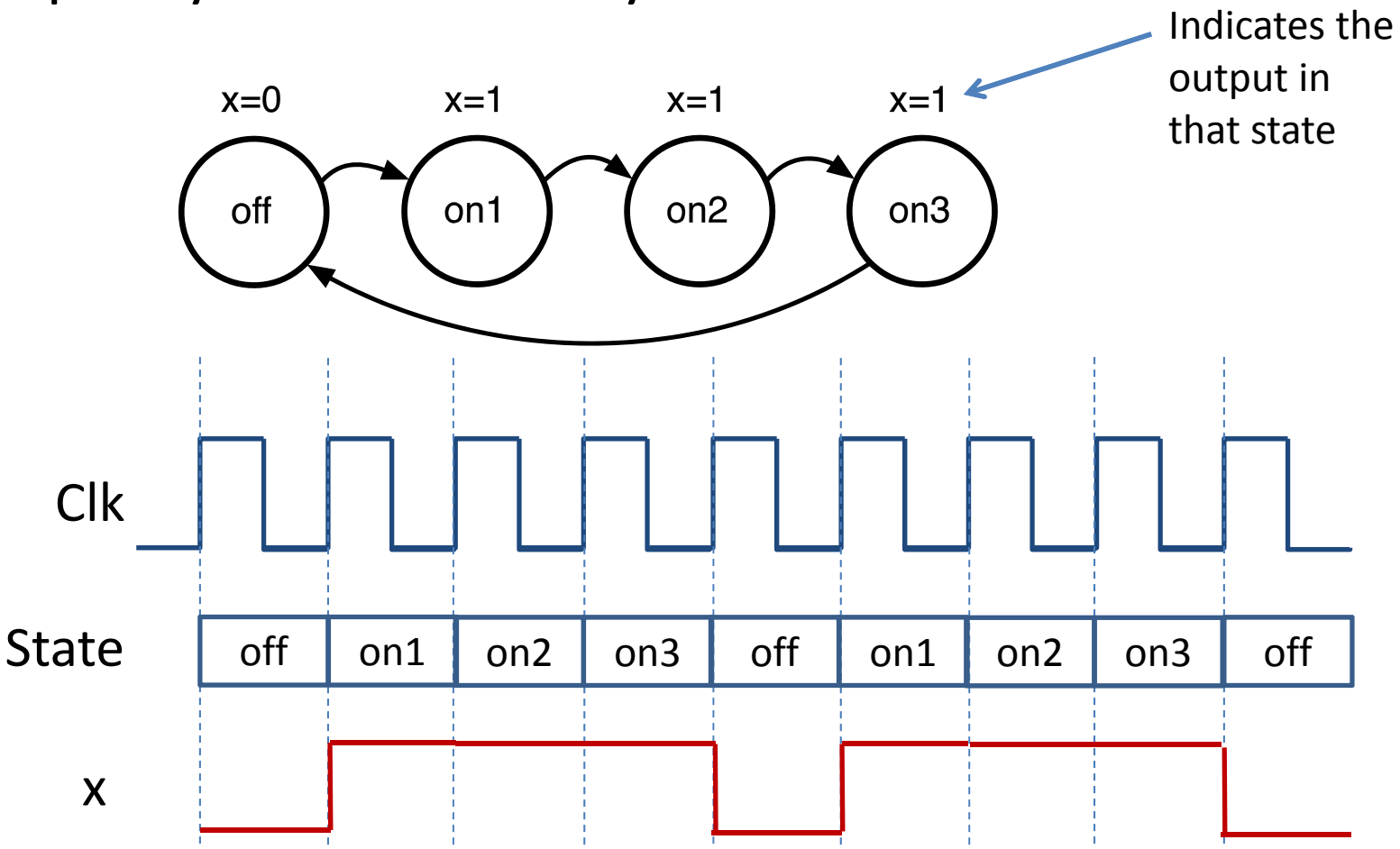
- A *finite state machine (FSM)* describes a system using a finite number of states and the associated transitions
- In synchronous design, an FSM is in one state for the duration of each clock cycle
- At every rising clock edge, the FSM may transition to another state
- Whether it transitions or not depends on the input values at that rising edge
- Once it makes the transition, it remains in that state for the next cycle

Finite State Machines

- We can label the states in a finite state machine with any meaningful name
  - In the counter sequences we saw, we just used the counter value, but often we want more meaningful information
- They are called “finite” because FSMs have a finite number of states (unlike some similar representations in other domains)
- “Machine” refers to a general object that can execute an abstract language

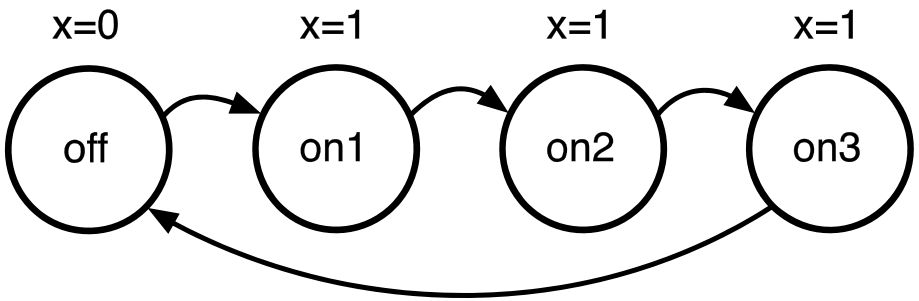
A Simple Example

- An always-on state machine that produces three high output cycles followed by one low:



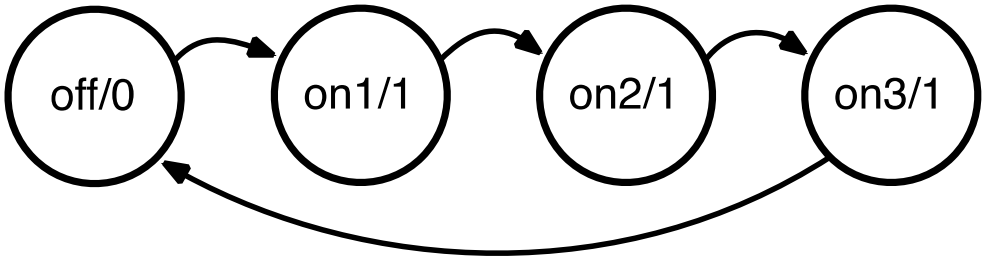
A Simple Example

- That FSM will always output high for three cycles, then low for one cycle
- The names of the states can be anything: using meaningful names helps us understand the behavior
- In this case, the output depends on what state the FSM is in: in *off*, it is low, in *on1*, *on2*, and *on3*, it is high



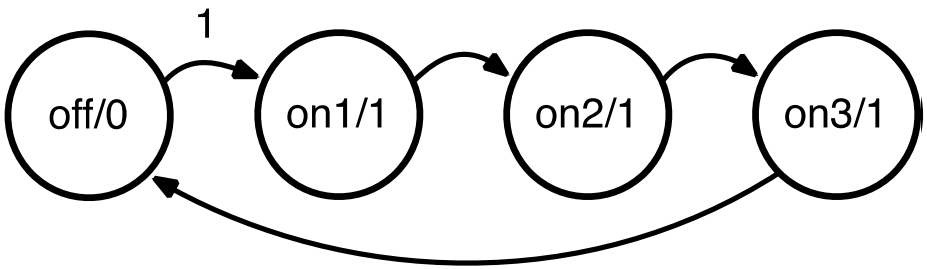
A Simple Example

- We can indicate the outputs above, below, or beside the corresponding states
- It is also possible to put the outputs inside the state bubbles (can be on a separate line to state name)
- Include a legend so it is clear what the output signal is called



Adding Inputs

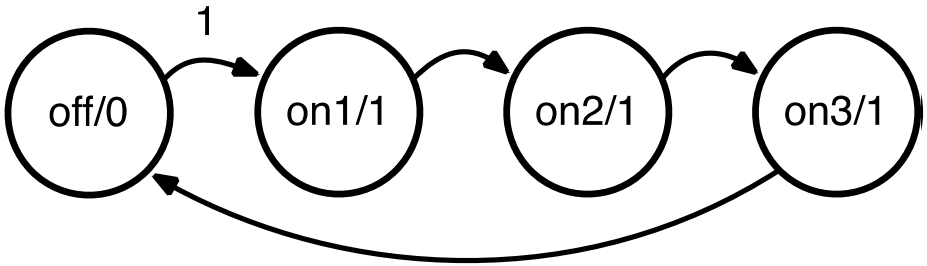
- What if we want the previous FSM to only output the three cycle high pulse when an input signal is high
- We showed that we can indicate input transition conditions on the arrows:



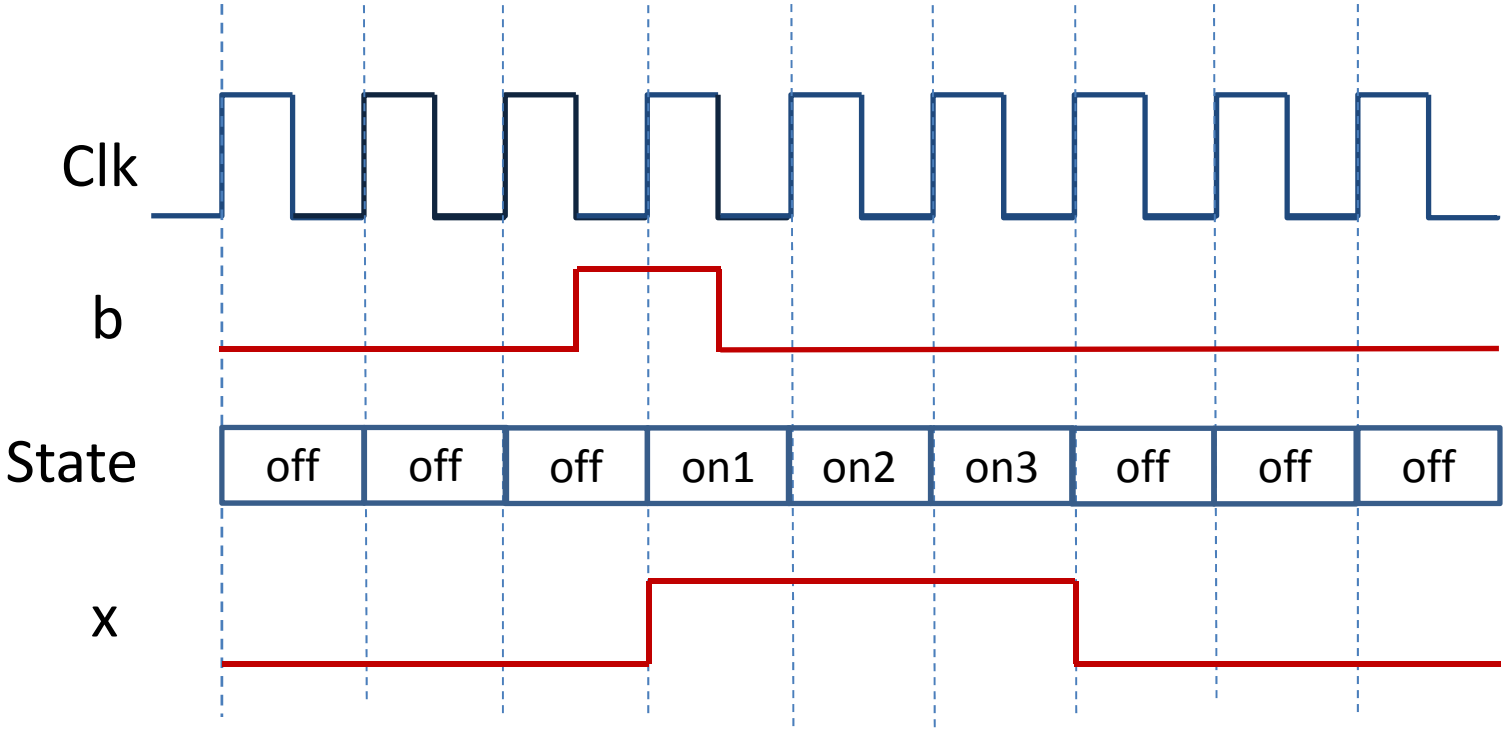
Input: b
Output: x

- Now, the FSM remains in state *off* until the *b* input goes high
- At that point it outputs a 1 as it passes through *on1*, *on2*, and *on3*, before returning to *off*

Adding Inputs

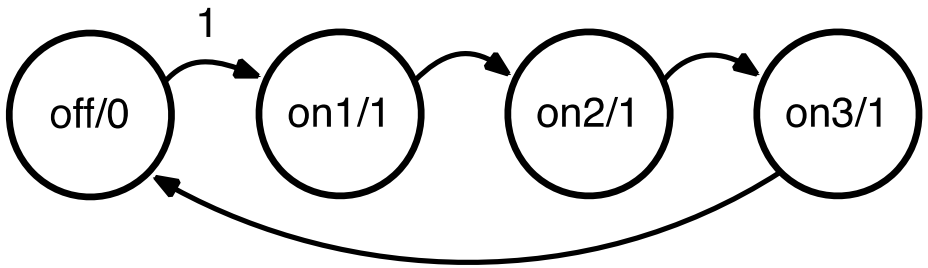


Input: b
Output: x



State Tables

- While diagrams can be useful, sometimes it helps to write the FSM information in a state transition table



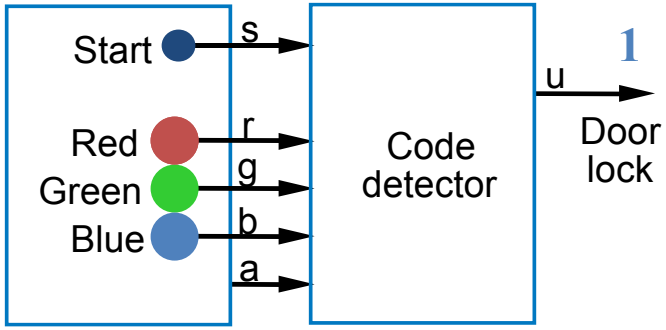
Input: b
Output: x

	Next State		
Current State	b=0	b=1	Output
off	off	on1	0
on1	on2	on2	1
on2	on3	on3	1
on3	off	off	1

Current State	b	Next State	Output x
off	0	off	0
off	1	on1	0
on1	0	on2	1
on1	1	on2	1
on2	0	on3	1
on2	1	on3	1
on3	0	off	1
on3	1	off	1

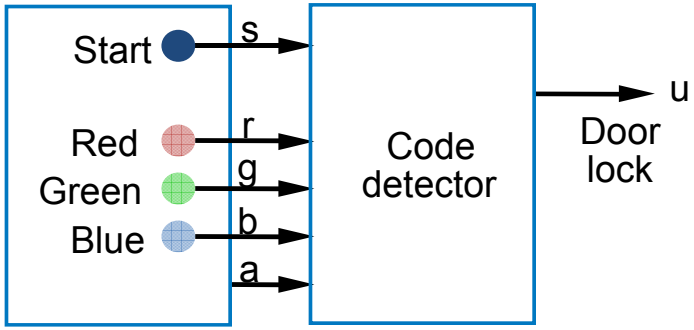
FSM Example

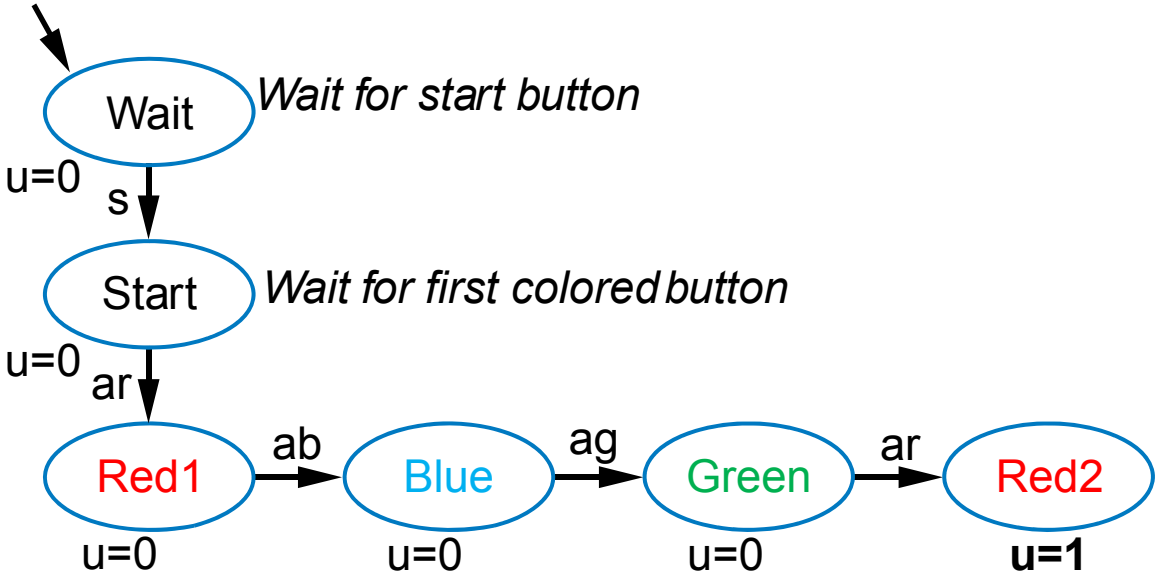
- Design a door lock that only unlocks (outputs u=1) when input buttons are pressed in a fixed sequence:
  - start, red, blue, green, red
- Inputs are s, r, g, b
- Input *a* indicates that a button has been pressed



FSM Example

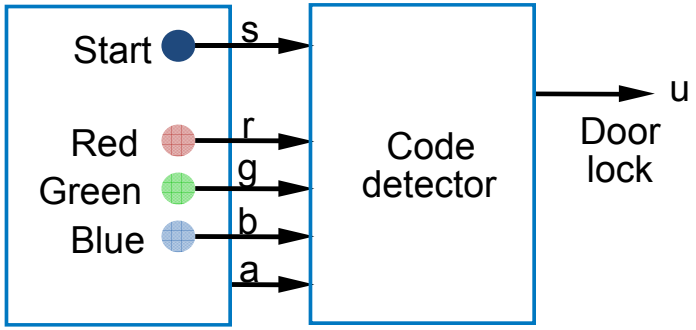
- We can capture the FSM behavior in a state diagram

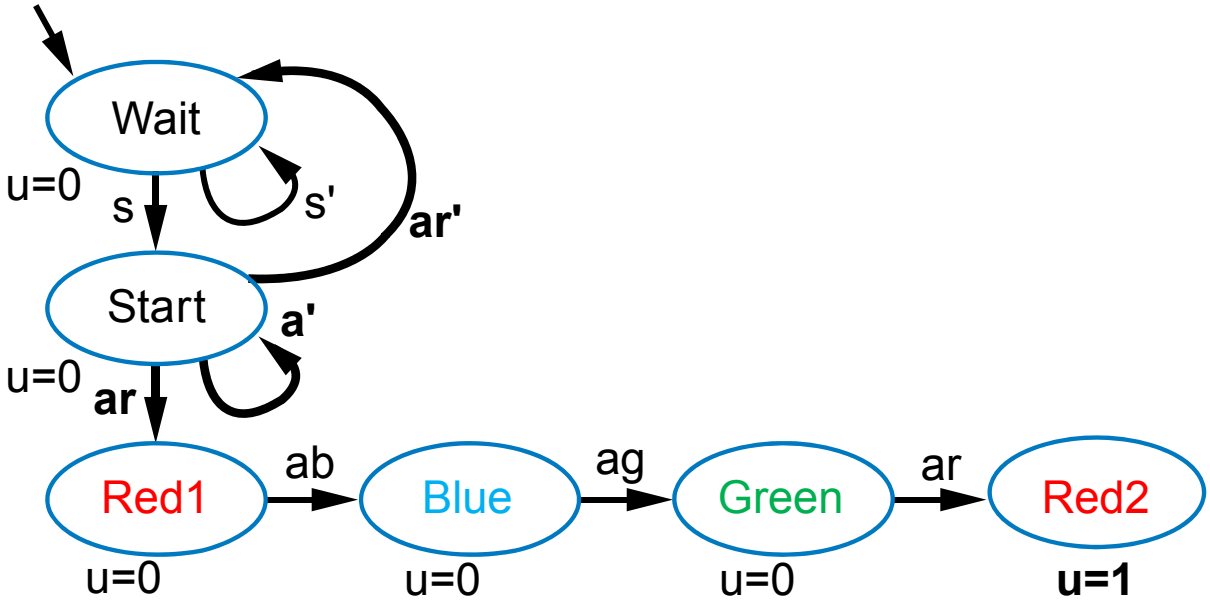




FSM Example

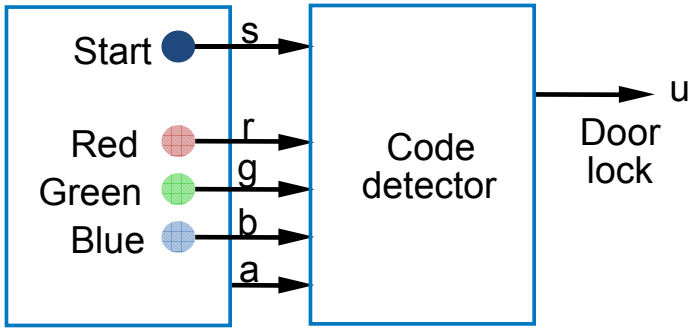
- We then need to add other transitions:

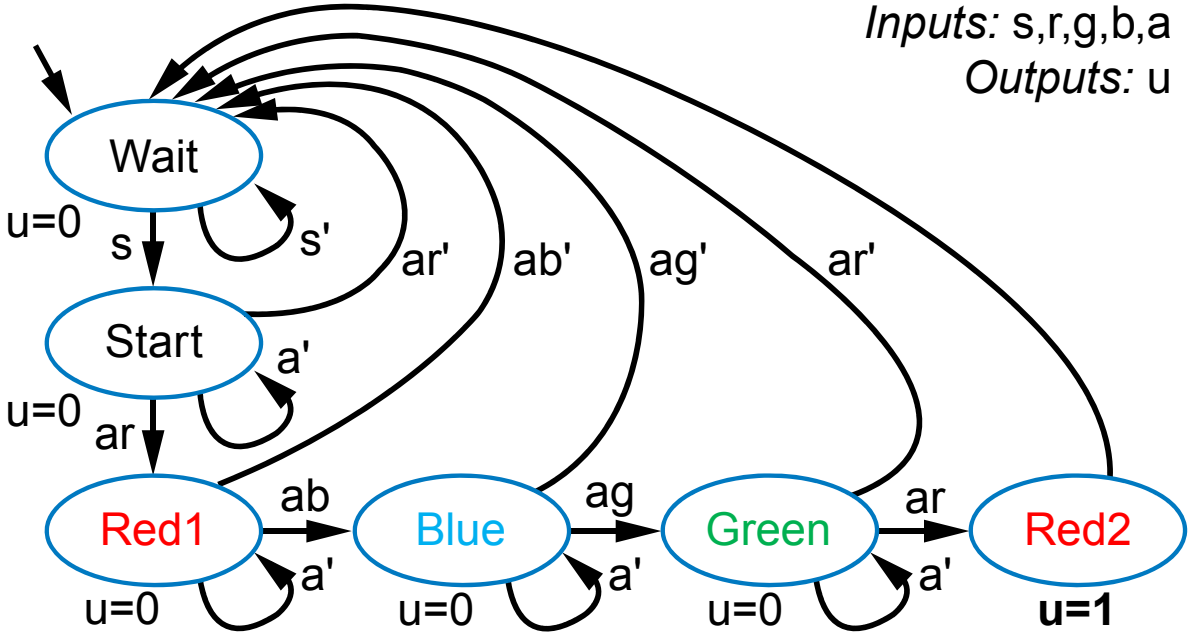




FSM Example

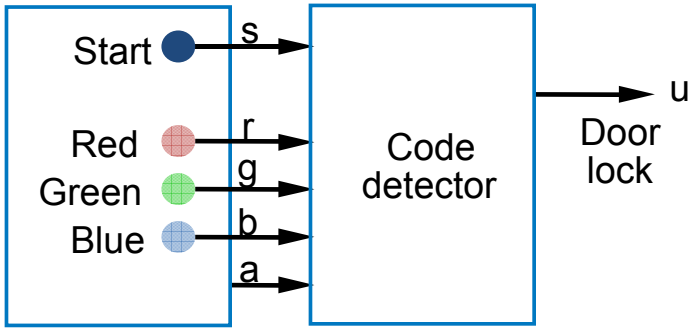
- We do the same for all states:

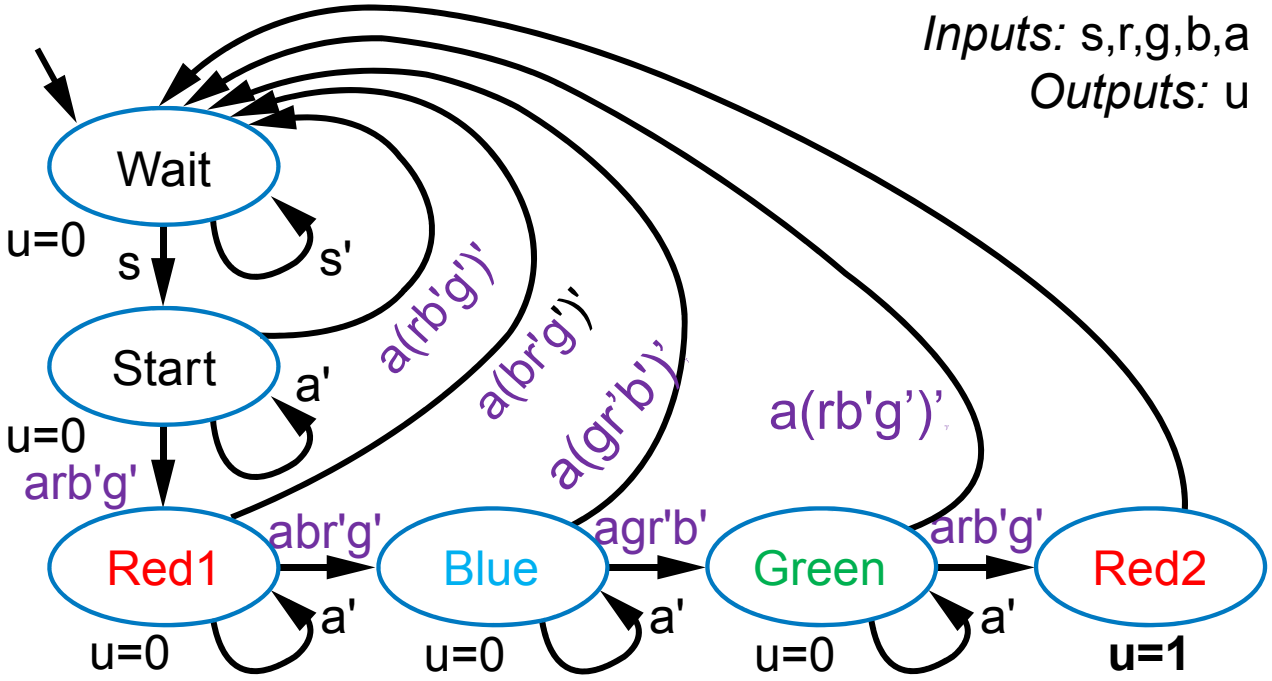




FSM Example

- However, we can see that if you press all buttons at the same time, the FSM will reach the end state, so we need to add further conditions:





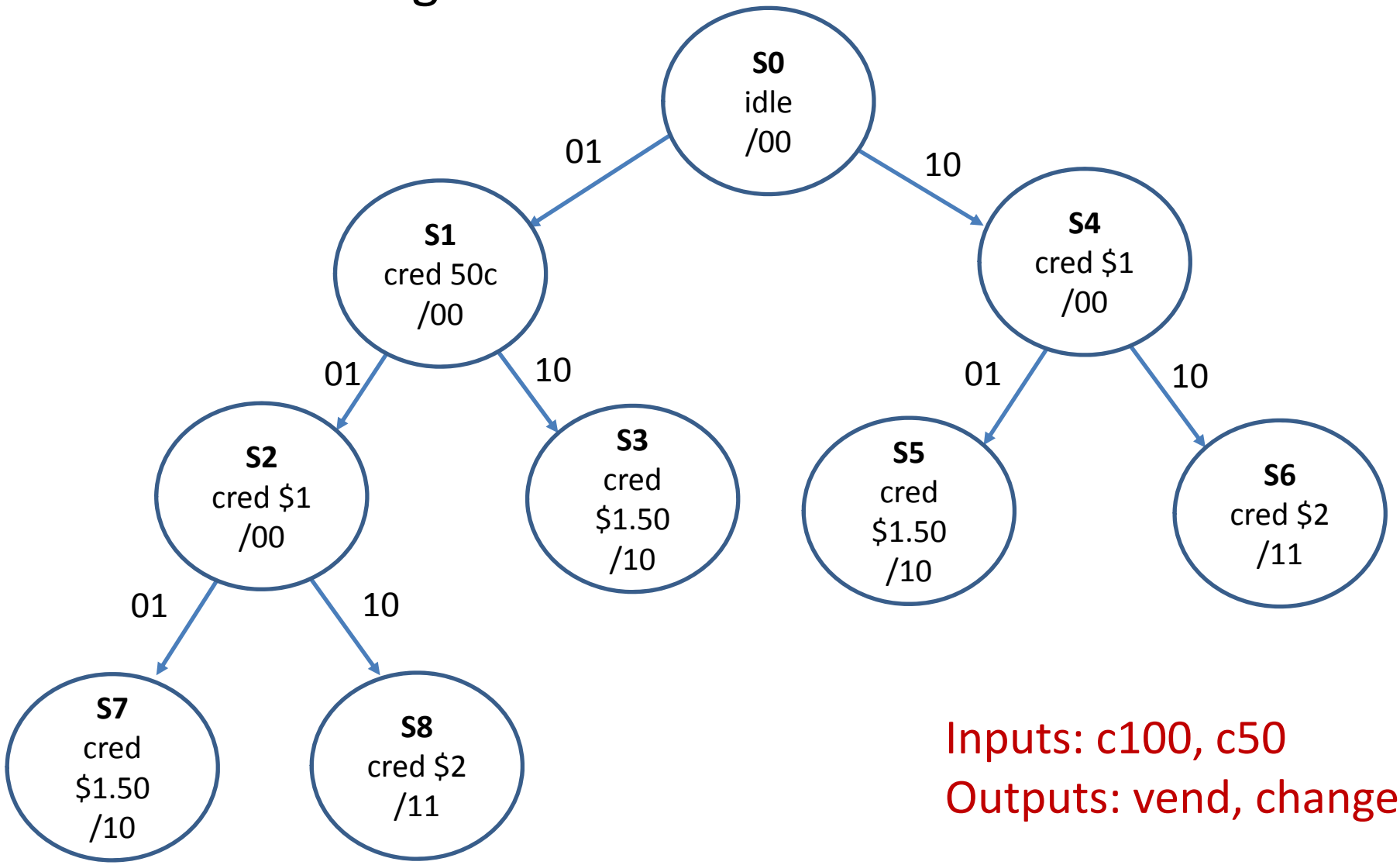
Another Example

- We want to design an FSM for a vending machine, that accepts only $1 and 50c coins, dispensing a drink that costs $1.50, and change if necessary
- This can be a circuit with two inputs, c100, for a $1 coin inserted and c50 for a 50c coin
- We assume only 1 input is ever high, and it tells us what coin was inserted
- The FSM has two outputs, vend, to release a drink, and change, to give 50c of change



Another Example

- The state diagram would look like this:

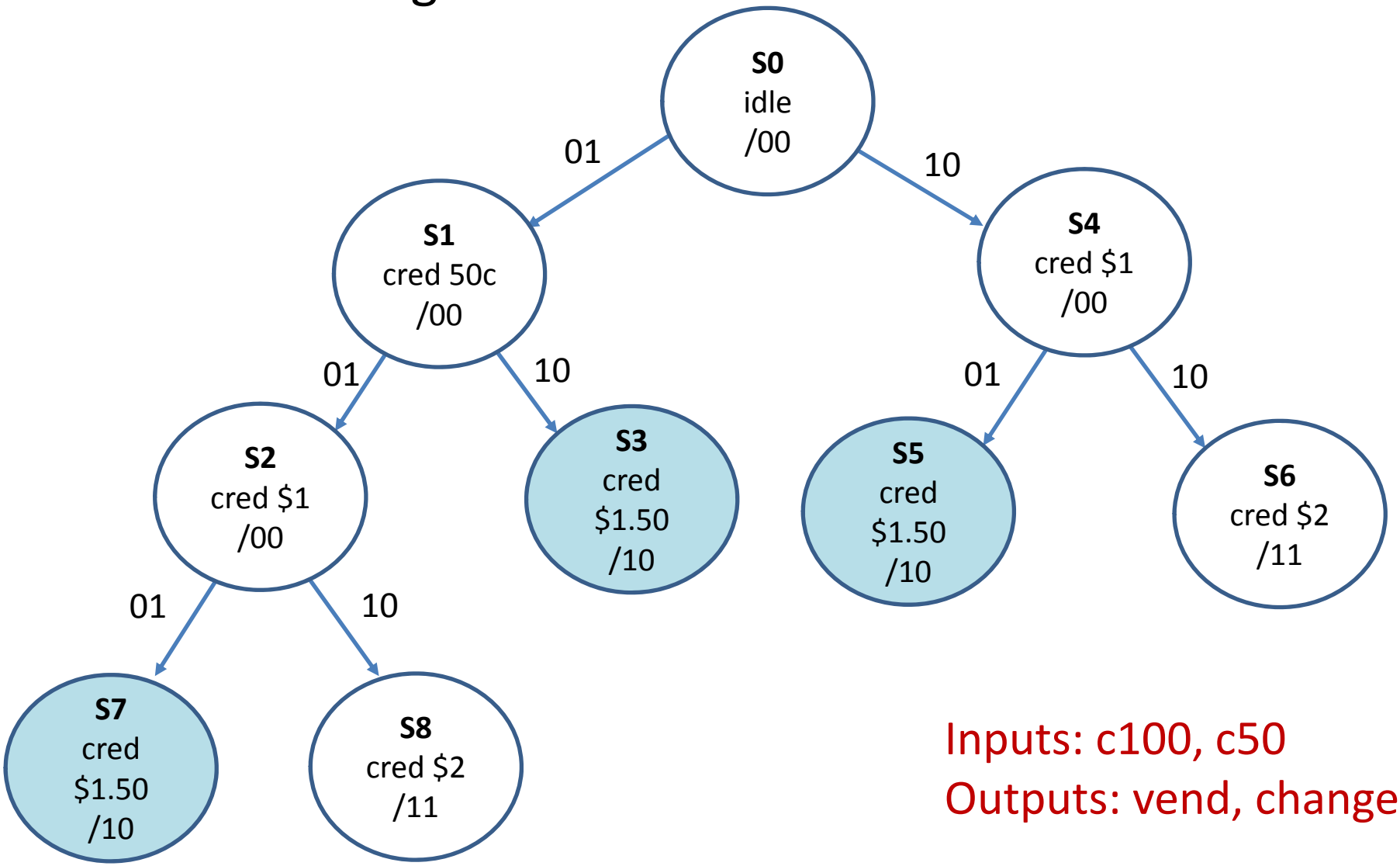


Inputs: c100, c50

Outputs: vend, change

Another Example

- The state diagram would look like this:

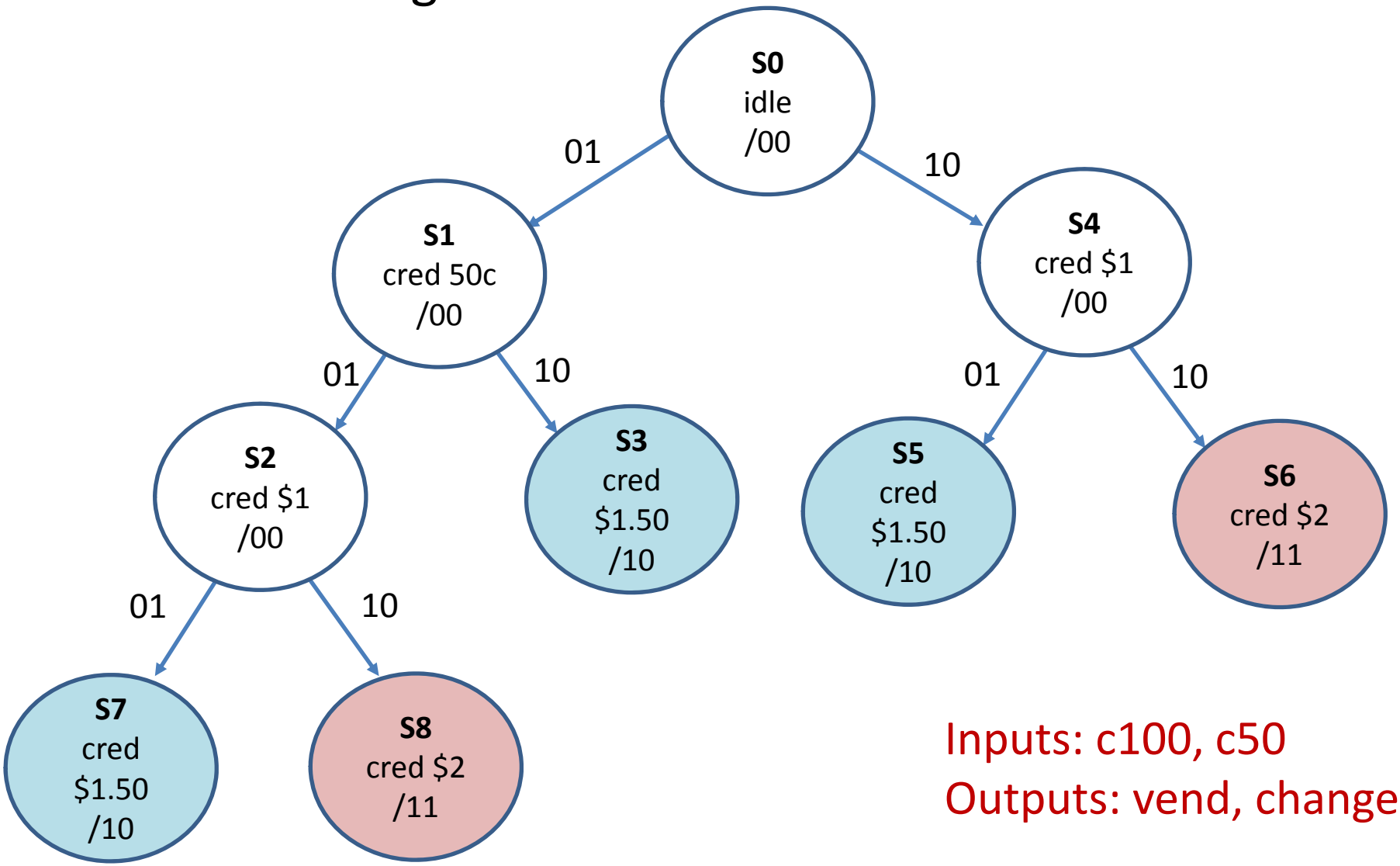


Inputs: c100, c50

Outputs: vend, change

Another Example

- The state diagram would look like this:

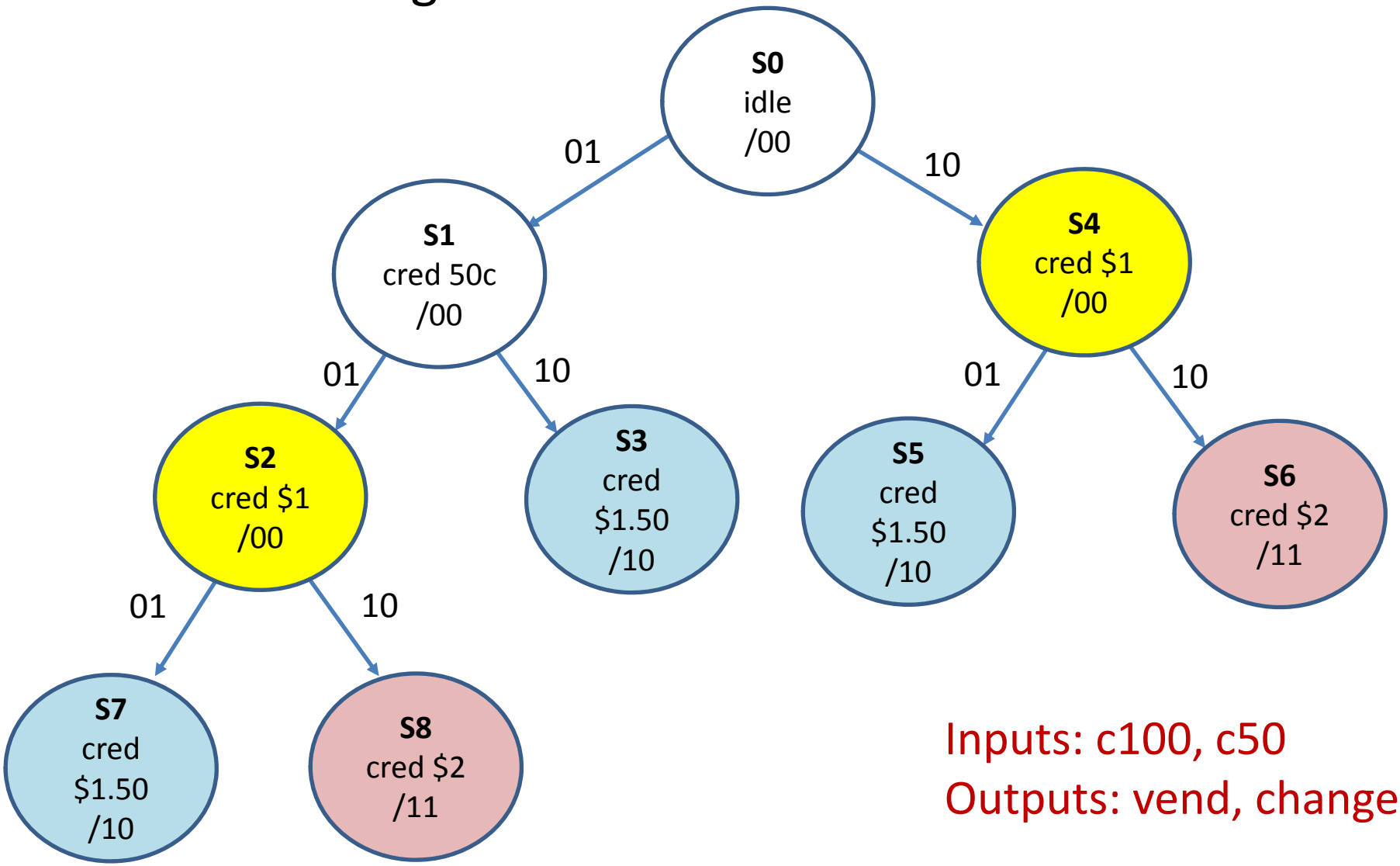


Inputs: c100, c50

Outputs: vend, change

Another Example

- The state diagram would look like this:



Inputs: c100, c50

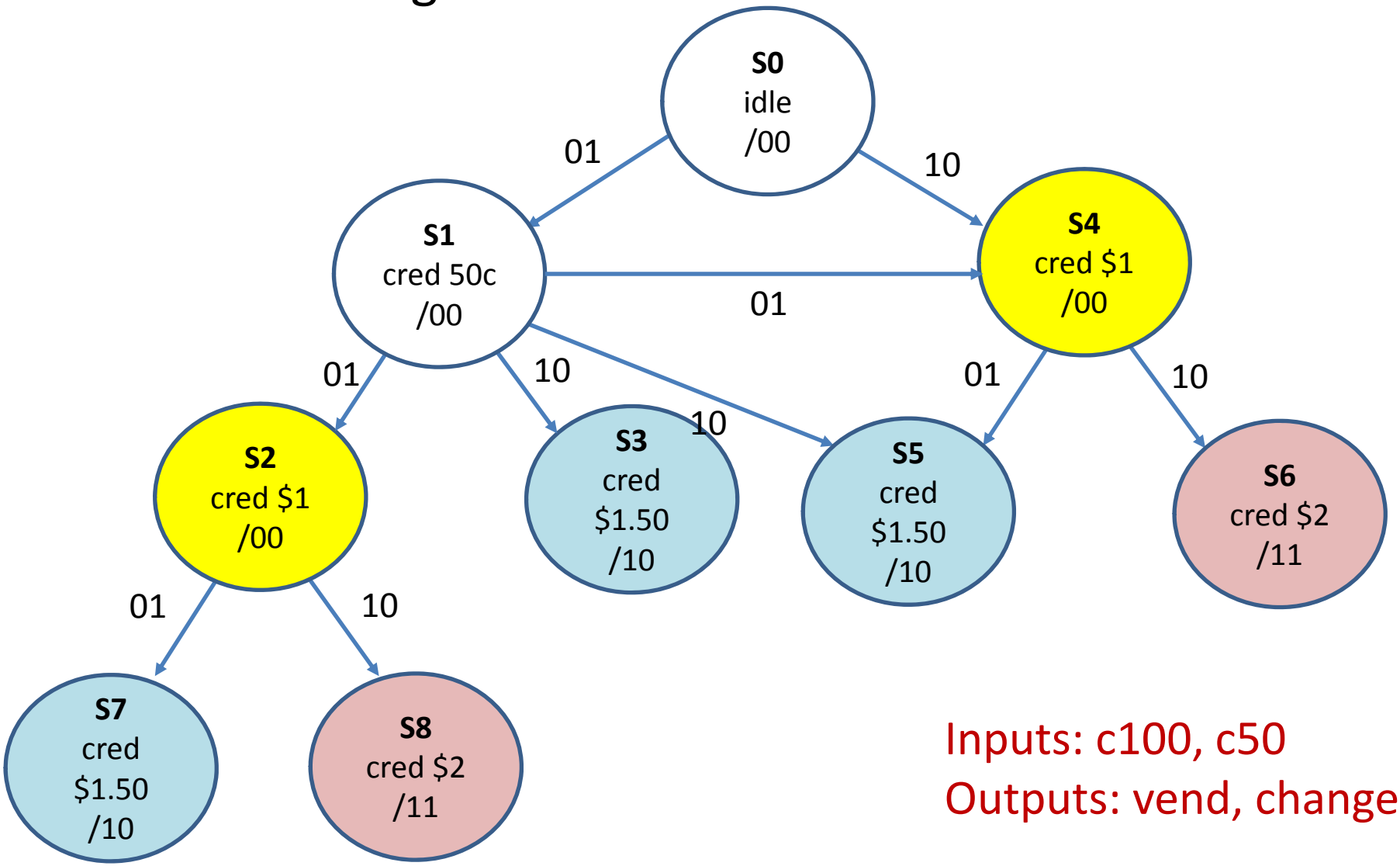
Outputs: vend, change

Another Example

- Another strength of using FSMs, is that we can see where simplifications can be made in the behavior representation
- In the previous diagram, in states S2 and S4, the vending machine has $1 credit and will react identically to future inputs, hence we can merge them
- Similarly, states S3, S5, and S7 are the same. So are states S6 and S8

Another Example

- The state diagram would look like this:



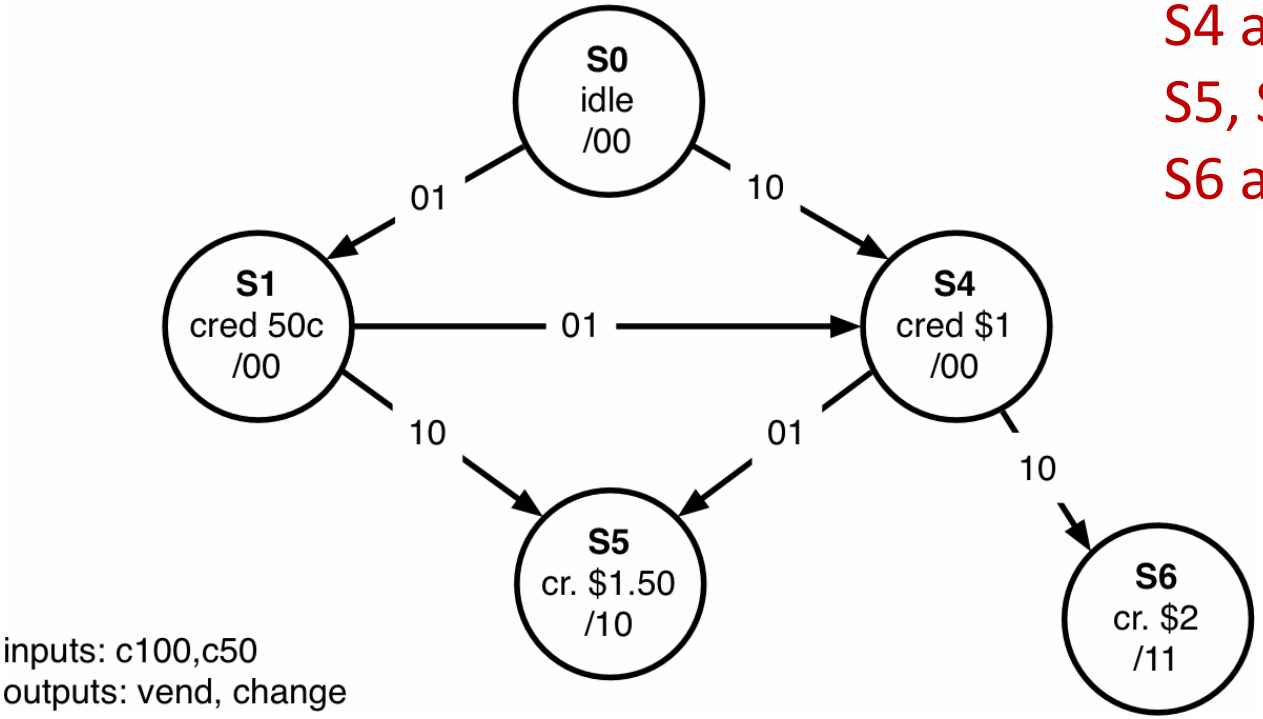
Inputs: c100, c50

Outputs: vend, change

Another Example

- Merging states is done simply by redrawing the arrows to the merged states:

Merge:
S4 and S2;
S5, S3 and S7;
S6 and S8



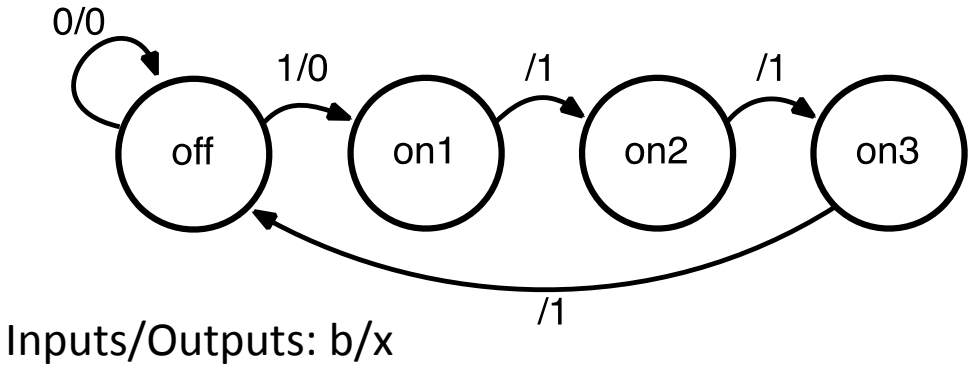
Moore and Mealy Machines

NANYANG TECHNOLOGICAL UNIVERSITY

- In all previous examples, the outputs depend only on the state, i.e. in state *x*, the output is always *f*
- These are called Moore Machines
- Another type of state machine is called a Mealy Machine
- In Mealy Machines, the output depends on the state and the current value of the inputs
  - Outputs can change mid-state, if the inputs change
- Mealy machines are a little harder to design and analyze, but can be more compact

Mealy Machines

- For Mealy machines, we can't draw the outputs inside the state circles, so we add them to the arrows instead, after a slash
- The output written on an arrow tells us what the output should be during the emitting state, when the input values match those on the same arrow
- A previous example with outputs on arrows:



NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Finite State Machines in Verilog

Mapping to Implementation

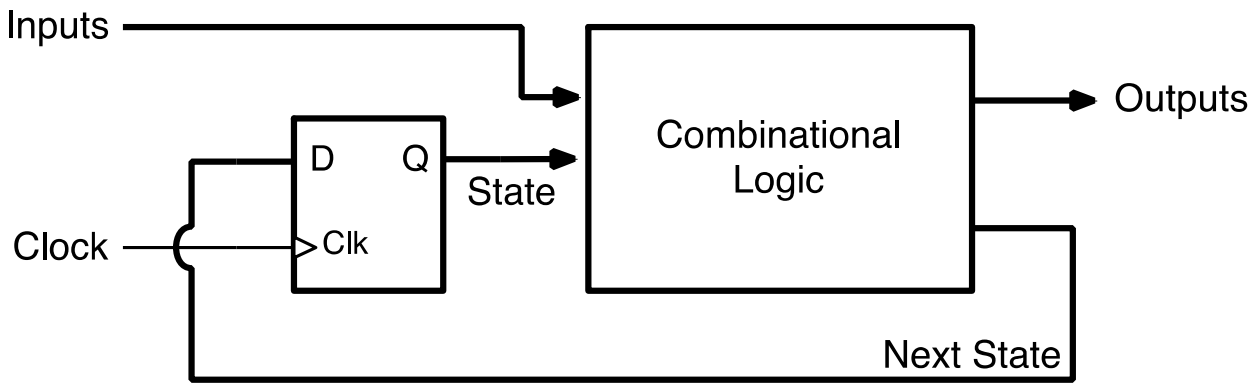
NANYANG TECHNOLOGICAL UNIVERSITY

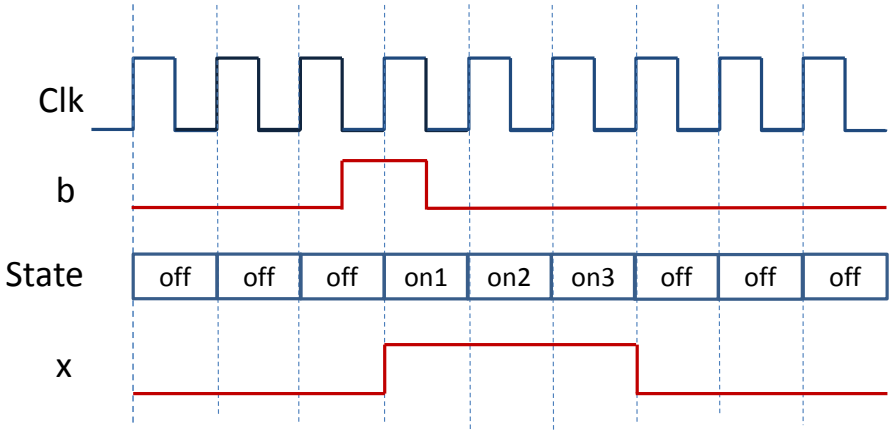
- This FSM is easy to understand, but how can we then turn this into a circuit?
- The first thing to consider is the structure we are trying to build:
  - A state register that holds the current state
  - Some logic to determine what the next state should be
  - Some logic to drive the output correctly
- At each clock cycle, the state will be updated to whatever was determined to be the next state

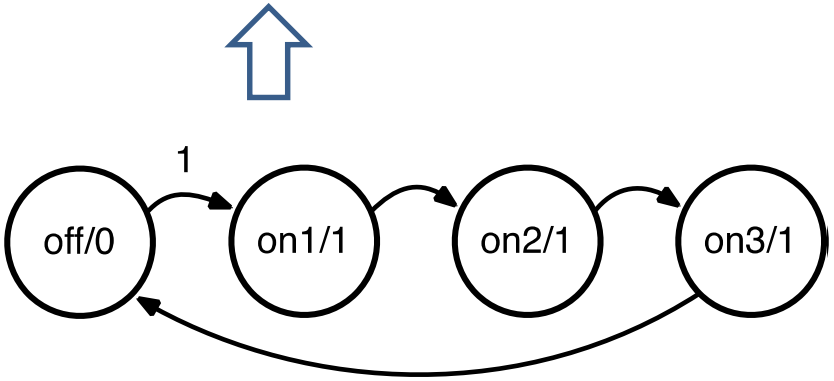
Mapping to Implementation

- Hence, the basic structure looks like this:

- A state register that holds the current state
- Some logic to determine what the next state should be
- Some logic to drive the output correctly



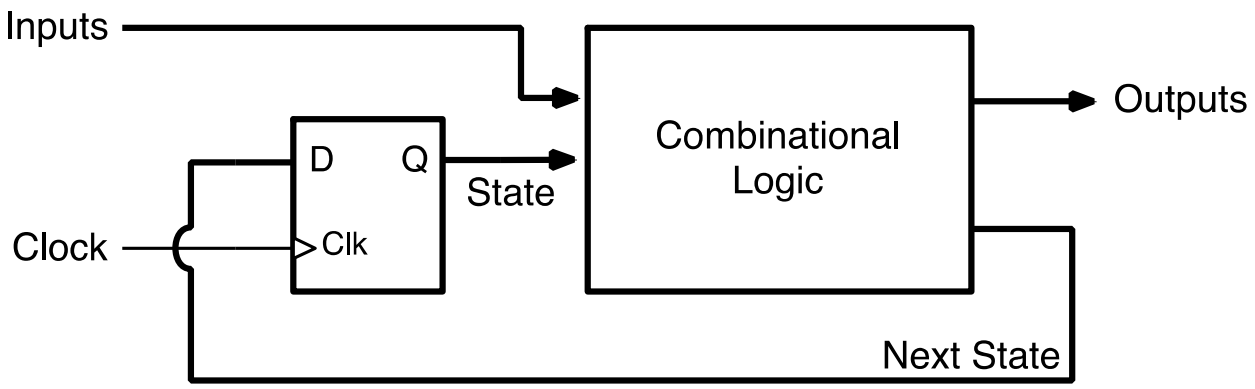




Input: b
Output: x

Mapping to Implementation

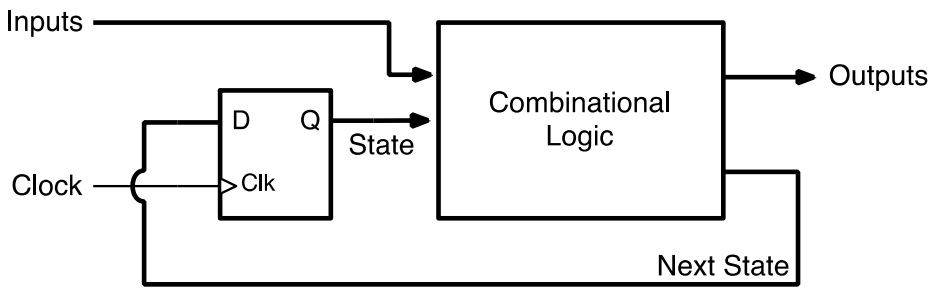
- Hence, the basic structure looks like this:

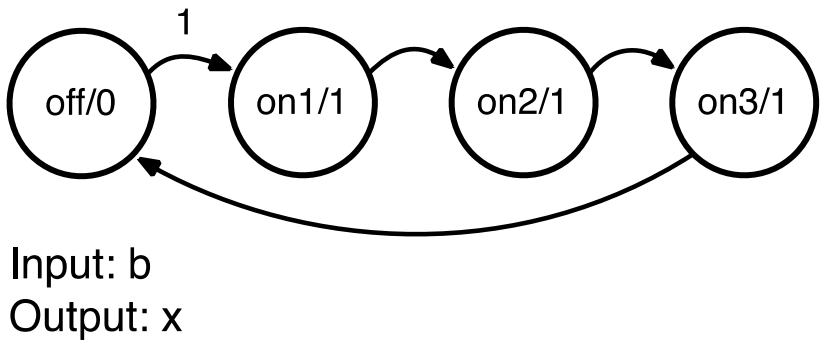


- The combinational logic the following based on the current state and inputs
  - Next State
  - Outputs
- We must turn the states into some binary representation, then we can use those values and the inputs to write the logic to produce the next state and outputs

Mapping Finite State Machines

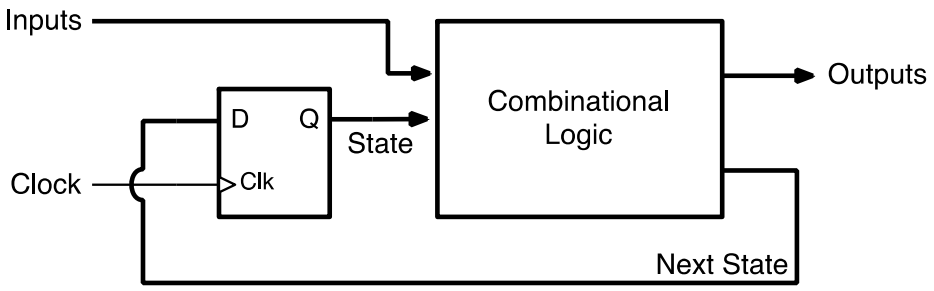
- The state register stores the current state
- To encode the state, we must assign a binary value to each state
- Hence for *m* states, we need ($\log_2 m$) bits to encode the state
- For four states, we would need two bits, so we could name them s[1:0]
- This represents the current state

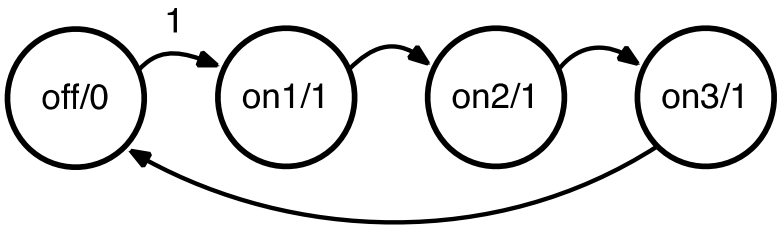




Mapping Finite State Machines

- We then need corresponding values to represent the next state, ns[1:0]
- We must then determine the combinational logic that gives the corresponding next state for the current state, as per our transition diagram
- Separately, we can then work out the logic for the output

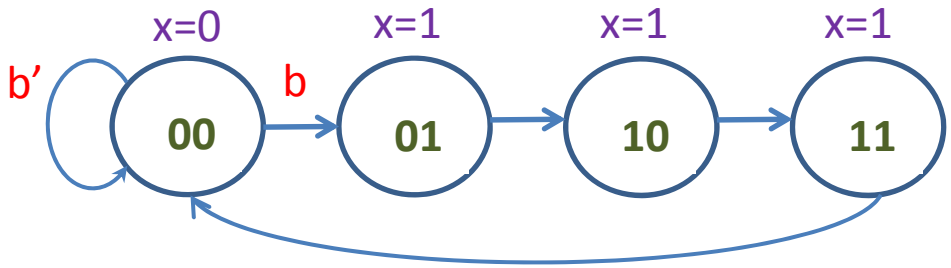




Input: b
Output: x

Mapping Example

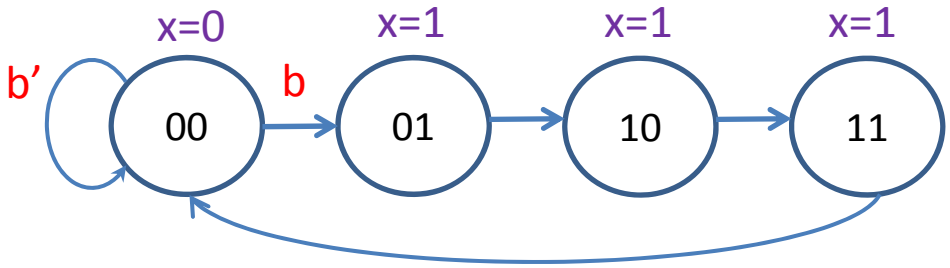
- First, we take our transition diagram

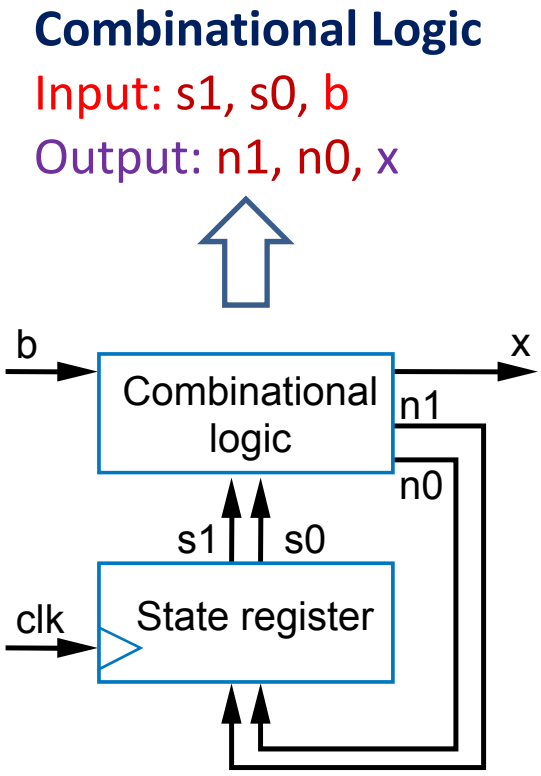


- We then give each state a unique binary value
- We call this encoding the states
- Any encoding will work as long as each state is unique

Mapping Example

- The circuit we are designing looks like this:
- We now have a value of s1, s0 for each state
- We can use this to build a table that captures the transitions to ns1, ns0, and the output

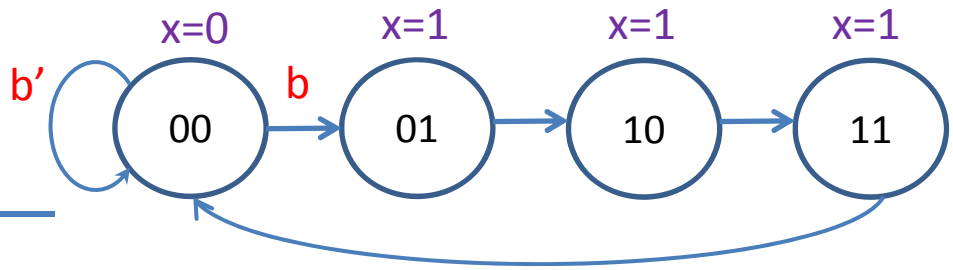




Current State: **s[1:0]**
Next State: **n[1:0]**

Mapping Example

- The transition table:



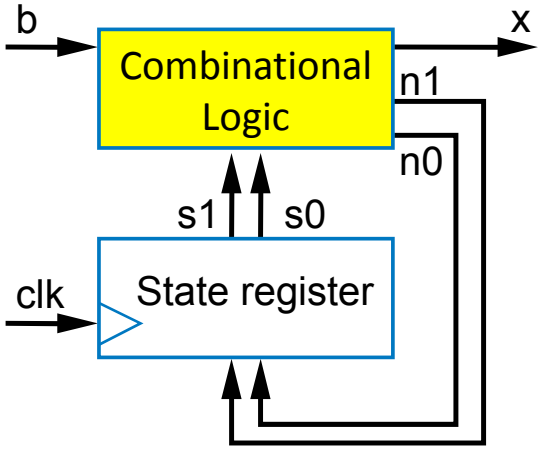
	Inputs			Outputs		
	s1	s0	b	x	n1	n0
off	0	0	0	0	0	0
	0	0	1	0	0	1
on1	0	1	0	1	1	0
	0	1	1	1	1	0
on2	1	0	0	1	1	1
	1	0	1	1	1	1
on3	1	1	0	1	0	0
	1	1	1	1	0	0

Current State **Input** **Output** **Next State**

Combinational Logic

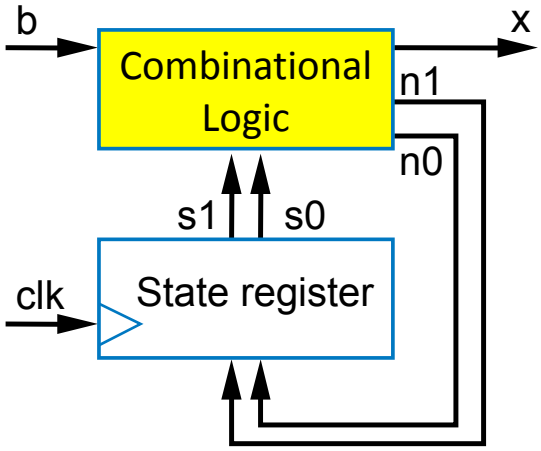
Input: s1, s0, b

Output: n1, n0, x



Mapping Example

- Now determine the combinational logic:



	Inputs			Outputs		
	s1	s0	b	x	n1	n0
off	0	0	0	0	0	0
	0	0	1	0	0	1
on1	0	1	0	1	1	0
	0	1	1	1	1	0
on2	1	0	0	1	1	1
	1	0	1	1	1	1
on3	1	1	0	1	0	0
	1	1	1	1	0	0

x = s1 + s0
(note that x=1 if s1=1 or s0=1)

n1 = s1' s0b' + s1' s0b + s1s0' b' + s1s0' b
n1 = s1' s0 + s1s0'

n0 = s1' s0' b + s1s0' b' + s1s0' b
n0 = s1' s0' b + s1s0'

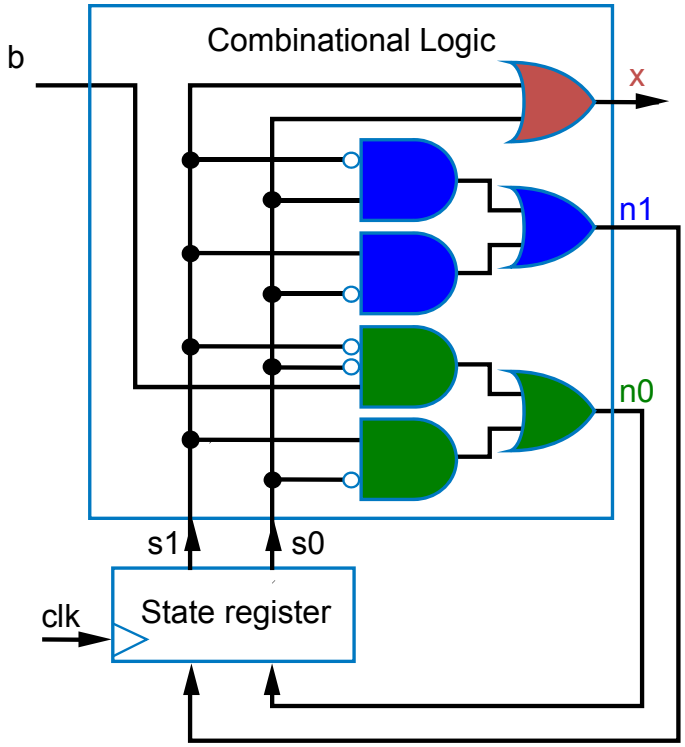
Mapping Example

- The circuit can then be implemented:

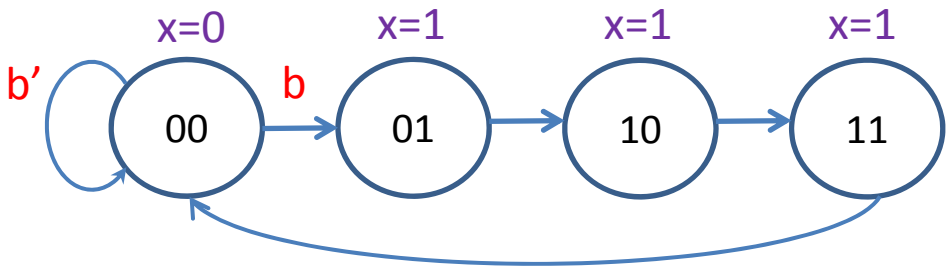
$x = s1 + s0$

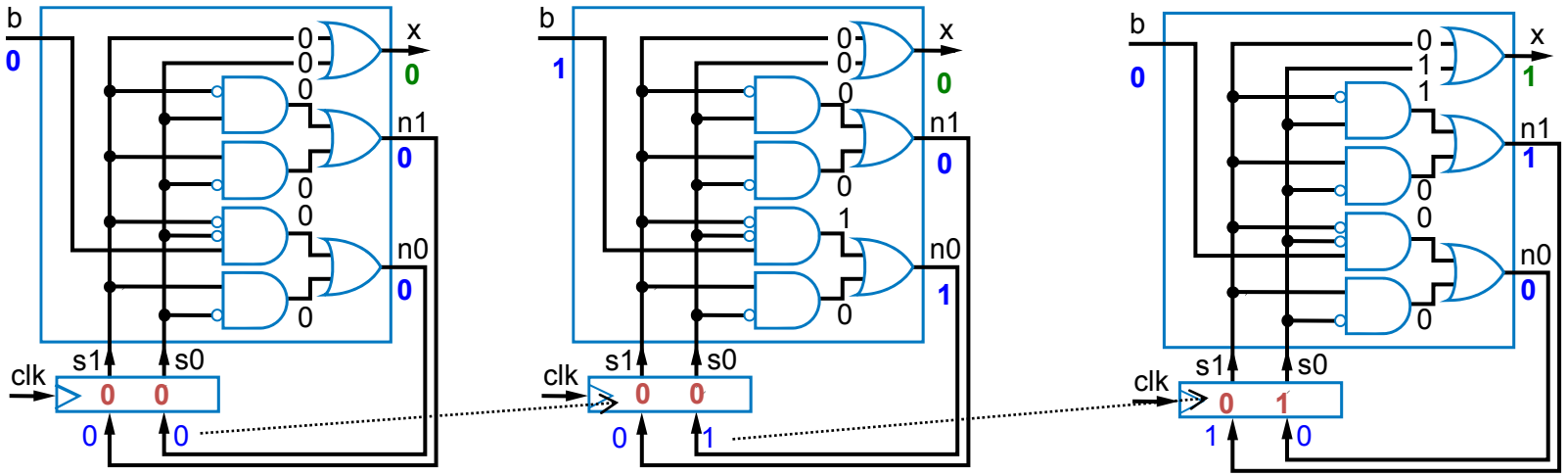
$n1 = s1' s0 + s1 s0'$

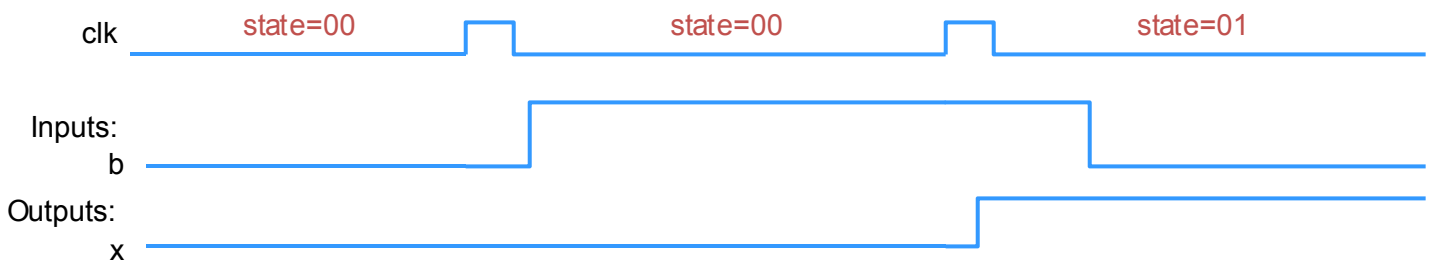
$n0 = s1' s0' b + s1 s0'$



Mapping Example







Mapping Finite State Machines

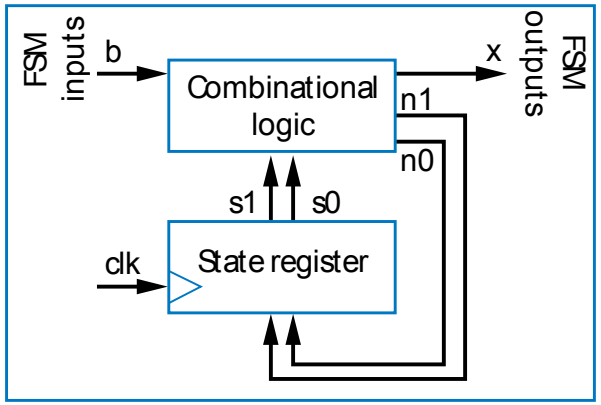
NANYANG TECHNOLOGICAL UNIVERSITY

- Hence, the stages of implementing FSMs are:
  - Draw state transition diagram using names for states
  - Construct state transition table using names – *optional*
  - Encode the states
  - Write a state transition table using encoded states
  - Determine the logic equations for the next state bits and the outputs
  - Implement the circuit

- Note that different encodings will give different circuits, but the resulting functionality will be identical

Example in Verilog

- So to implement the previous example, we simply implement the logic in Verilog
- The three equations we obtained earlier plus a state register will give the full implementation
- Three assign statements implement the combinational logic
- A single register block creates the state register

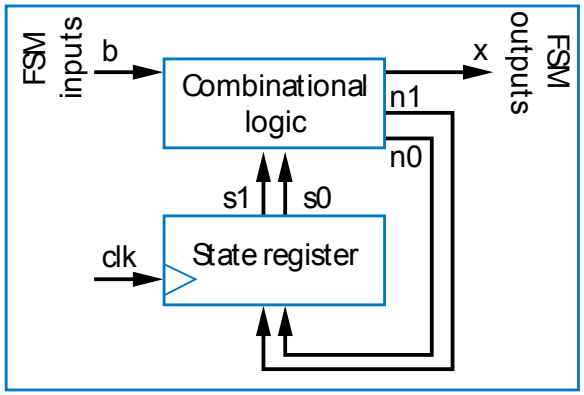


x = s1 + s0

n1 = s1's0 + s1s0' = s1 xor s0

n0 = s1's0'b + s1s0'

Example in Verilog



x = s1 + s0

n1 = s1’s0 + s1s0’ = s1 xor s0

n0 = s1’s0’b + s1s0’

```
module pulse3 (input b,
               input clk, rst,
               output x);

wire n1, n0;
reg  s1, s0;

assign n1 = s1 ^ s0;
assign n0 = (~s1&~s0&b) | (s1&~s0);
assign x = s1 | s0;

always @ (posedge clk)
begin
    if(rst) begin
       s1 <= 1'b0;
       s0 <= 1'b0;
    end else begin
       s1 <= n1;
       s0 <= n0;
    end
end
endmodule
```

Reset in State Machines

NANYANG TECHNOLOGICAL UNIVERSITY

- Whenever implementing a finite state machine, **always** ensure the state register has a reset and that you have defined a valid state
- Otherwise, we can't always predict which state the FSM will start in
- If the FSM starts in an invalid state, we have no idea where it will transition, since we may not have fully specified the transition logic
- This is especially important when the number of states is not a power of two, and hence some state values may be unused

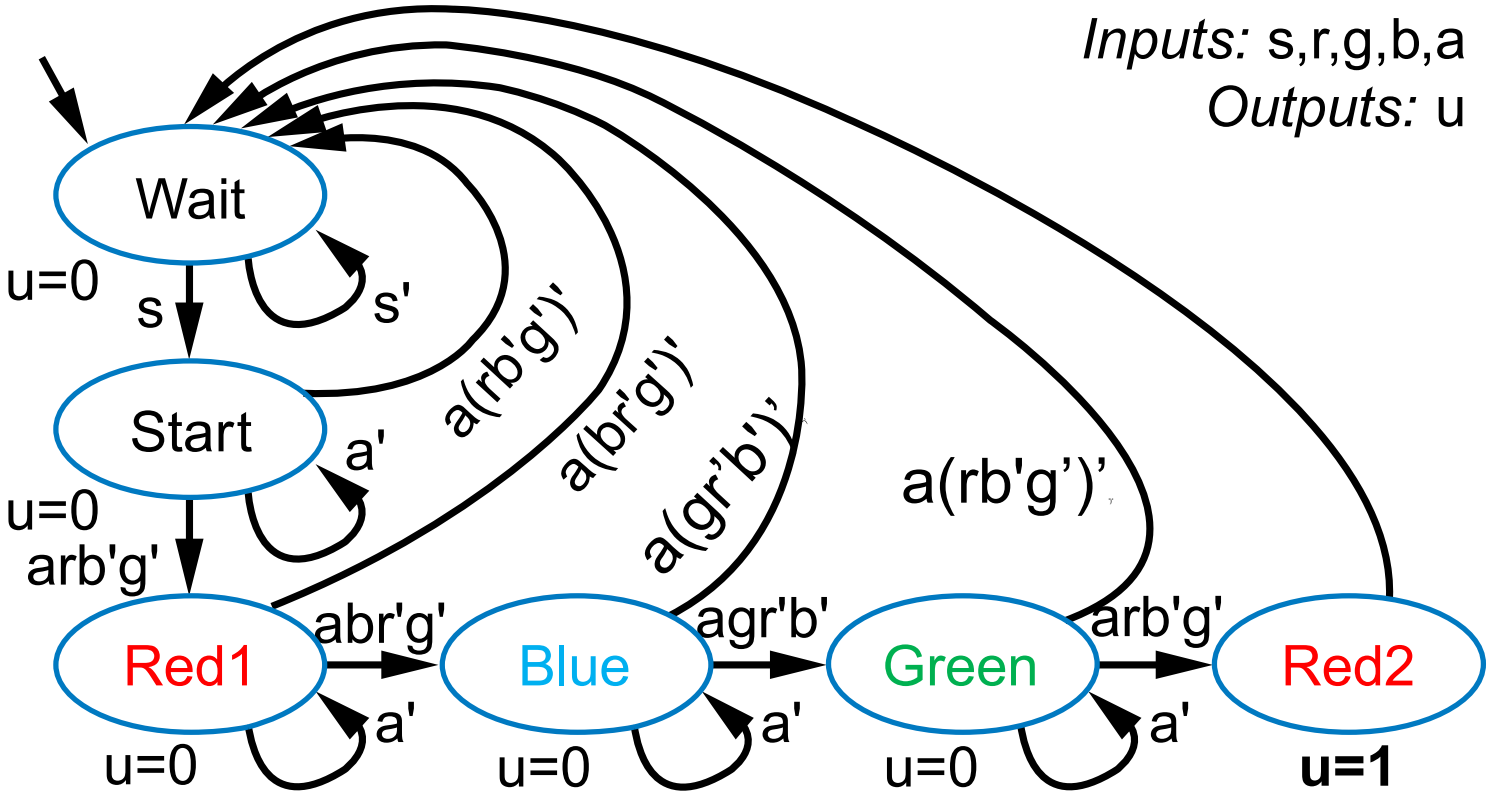
More Complex FSMs

NANYANG TECHNOLOGICAL UNIVERSITY

- The previous example had just four states
- In such cases, determining the transition logic for the individual states is easy, by looking at the transition table
- For more complex state machines, we can use Verilog's behavioral description to do this for us
- A combinational always block with a case statement can capture transitions in an easy to read manner
- We can implement the output logic separately to the transitions if that helps

A More Complex Example

- Consider the sequence detector from the last lecture:



Verilog Constants

NANYANG TECHNOLOGICAL UNIVERSITY

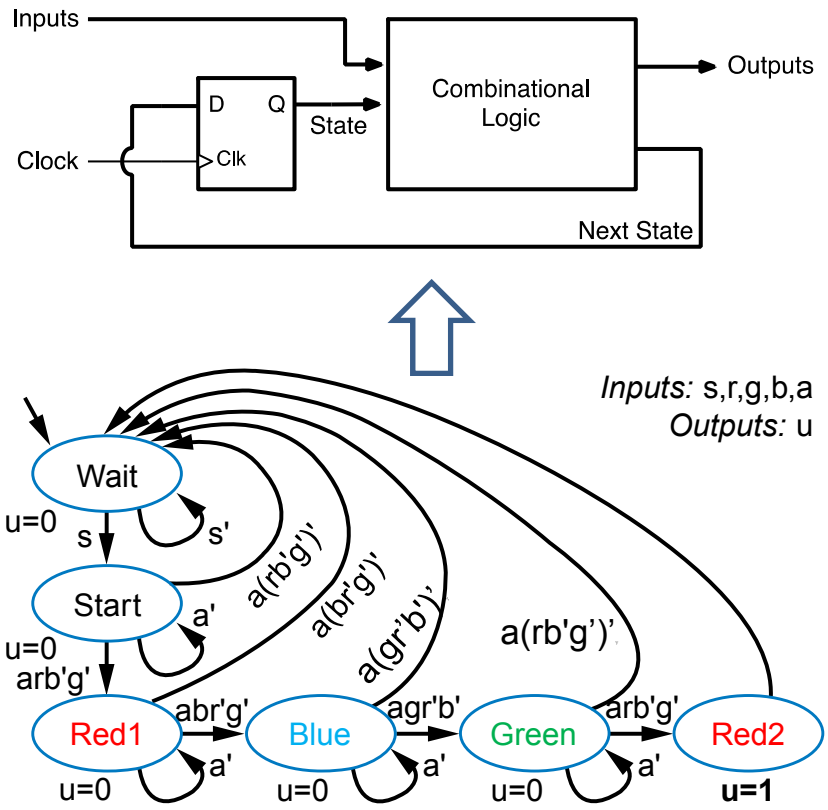
- First we need to encode the states:
  - Wait = 000, Start = 001, Red1 = 010, Blue = 011, Green = 100, Red2 = 101
- As we assign each state a binary value, we'd still rather not deal with binary numbers in our Verilog code.
- We can use the Verilog parameter keyword to declare named constants:

```
parameter st1 = 2'b00, st2 = 2'b01, st3 = 2'b10;
```

- We can then use these names in place of the values anywhere in the code.

```
nst = st2;
```

A More Complex Example




```
module seqdet (input s, r, g, b, a,
               input clk, rst,
               output u);

parameter waite=3'b000, start=3'b001,
red1=3'b010,  blue=3'b011,
green=3'b100, red2=3'b101;

reg [2:0] nst, st;

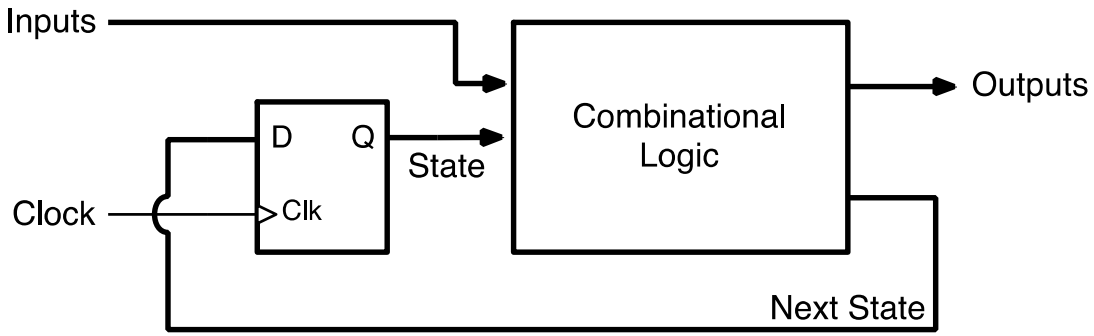
assign u = (st == red2);
   //u = (st == red2) ? 1'b1 : 1'b0;

always @ (posedge clk)
begin
    if(rst)
        st <= waite;
    else
        st <= nst;
end

/* continued... */
endmodule
```

A More Complex Example

- On the previous slide:
  - We declare two 3-bit reg signals (since both will be assigned from always blocks)
  - The output is high only when the current state is Red2 (101), so we can assign it directly
  - The state register process is always the same



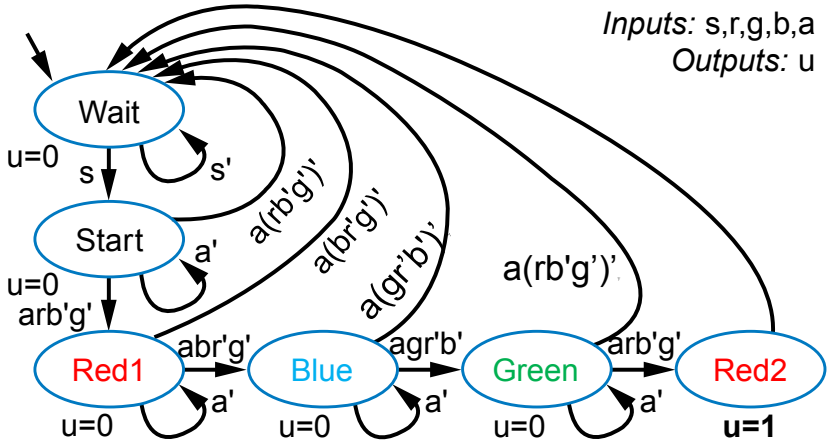
- We still need to describe the combinational logic to determine the next state (based on current state and inputs)

A More Complex Example

```
always @ *
begin
    nst = st;
    case(st)
        waite: if(s) nst = start;
        start: if(a)
                  if(r&~b&~g) nst = red1;
                  else nst = waite;
        red1: if(a)
                  if(b&~r&~g) nst = blue;
                  else nst = waite;
        blue: if(a)
                  if(g&~r&~b) nst = green;
                  else nst = waite;
        green: if(a)
                   if(r&~g&~b) nst = red2;
                   else nst = waite;
        red2: nst = waite;

        default: nst = waite;
    endcase

end
```

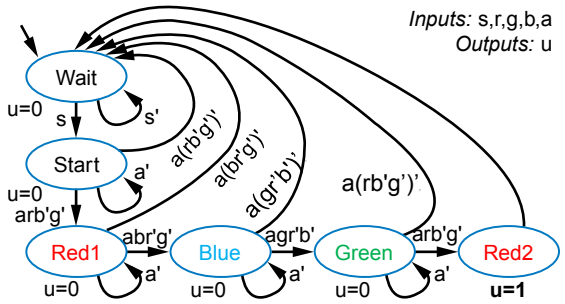


A More Complex Example

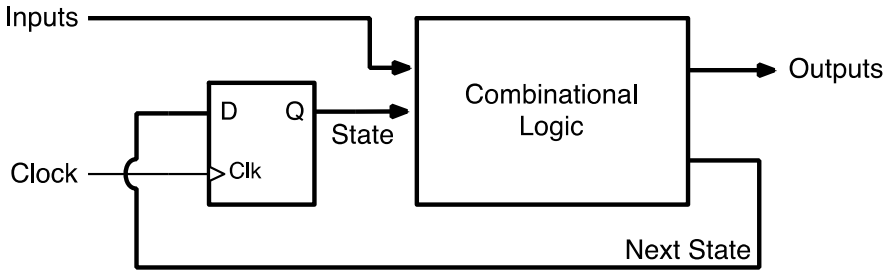
- For each state, we use the information in the diagram to determine the next state based on the inputs
- We can see that we check *a* (i.e. a button was pressed) then if the right single button was pressed, we move to the next state
- If any other or multiple buttons were pressed (else) we go to 000
  - The default case takes care of ensuring nst is assigned in all cases
- A default at the top says that if no assignment is made, we stay in the current state

```
always @ *
begin
    nst = st;
    case(st)
        waite: if(s) nst = start;
        start: if(a)
                  if(r&~b&~g) nst = red1;
                  else nst = waite;
        red1: if(a)
                  if(b&~r&~g) nst = blue;
                  else nst = waite;
        blue: if(a)
                  if(g&~r&~b) nst = green;
                  else nst = waite;
        green: if(a)
                   if(r&~g&~b) nst = red2;
                   else nst = waite;
        red2: nst = waite;

        default: nst = waite;
    endcase
end
```



A More Complex Example




```
module seqdet (input s, r, g, b, a,
               input clk, rst,
               output u);

parameter waite=3'b000, start=3'b001,
red1=3'b010,  blue=3'b011,
green=3'b100, red2=3'b101;

reg [2:0] nst, st;

assign u = (st == red2);
     //u = (st == red2) ? 1'b1 : 1'b0;

always @ (posedge clk)
begin
    if(rst)
        st <= waite;
    else
        st <= nst;
end

. . .

endmodule
```

```
always @ *
begin
    nst = st;
    case(st)
        waite: if(s) nst = start;
        start: if(a)
                  if(r&~b&~g) nst = red1;
                  else nst = waite;
        red1: if(a)
                  if(b&~r&~g) nst = blue;
                  else nst = waite;
        blue: if(a)
                  if(g&~r&~b) nst = green;
                  else nst = waite;
        green: if(a)
                   if(r&~g&~b) nst = red2;
                   else nst = waite;
        red2: nst = waite;

        default: nst = waite;
    endcase
end
```

A More Complex Example

- Note that nst is the next state value; it is only clocked into the state register at the next rising edge

```
module seqdet (input s, r, g, b, a,
               input clk, rst,
               output u);

parameter waite=3'b000, start=3'b001,
red1=3'b010,  blue=3'b011,
green=3'b100, red2=3'b101;

reg [2:0] nst, st;

assign u = (st == red2);
     //u = (st == red2) ? 1'b1 : 1'b0;

always @ (posedge clk)
begin
    if(rst)
        st <= waite;
    else
        st <= nst;
end

. . .

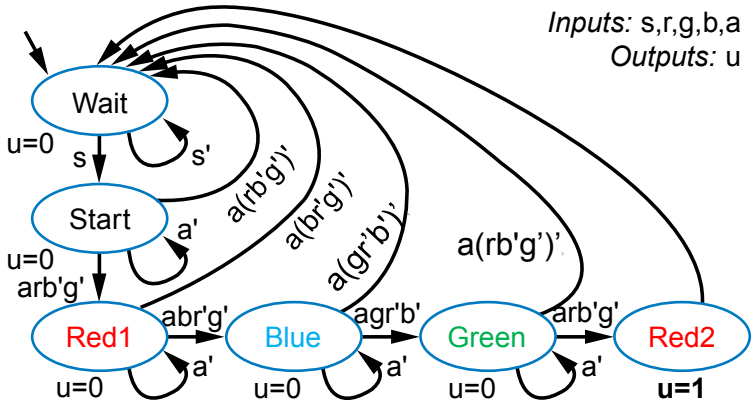
endmodule
```

```
always @ *
begin
    nst = st;
    case(st)
        waite: if(s) nst = start;
        start: if(a)
                  if(r&~b&~g) nst = red1;
                  else nst = waite;
        red1: if(a)
                  if(b&~r&~g) nst = blue;
                  else nst = waite;
        blue: if(a)
                  if(g&~r&~b) nst = green;
                  else nst = waite;
        green: if(a)
                   if(r&~g&~b) nst = red2;
                   else nst = waite;
        red2: nst = waite;

        default: nst = waite;
    endcase
end
```

A More Complex Example

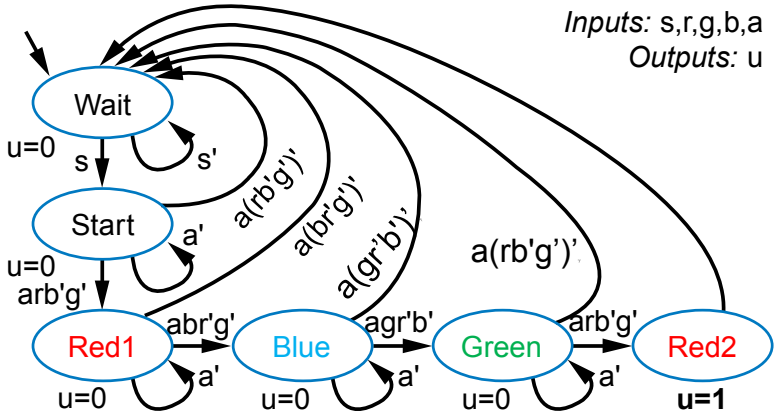
- We could also have added the output assignment to the same always block as the transitions
- Remember to change the u port to **reg**

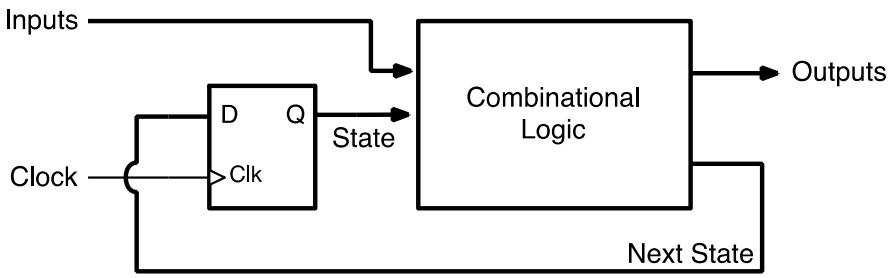



```
always @ *
begin
    nst = st; u = 1'b0;
    case(st)
        waite: if(s) nst = start;
        start: if(a)
                   if(r&~b&~g) nst = red1;
                   else nst = waite;
        red1: if(a)
                   if(b&~r&~g) nst = blue;
                   else nst = waite;
        blue: if(a)
                   if(g&~r&~b) nst = green;
                   else nst = waite;
        green: if(a)
                    if(r&~g&~b) nst = red2;
                    else nst = waite;
        red2: begin
                    nst = waite;
                    u = 1'b1;
              end

        default: nst = waite;
    endcase
end
```

A More Complex Example






```
module seqdet (input s, r, g, b, a,
               input clk, rst,
               output u);

parameter waite=3'b000, start=3'b001, red1=3'b010,
blue=3'b011, green=3'b100, red2=3'b101;

reg [2:0] nst, st;

assign u = (st == red2);

always @ (posedge clk)
begin
    if(rst)
        st <= waite;
    else
        st <= nst;
end

always @ *
begin
    nst = st;
    case(st)
        waite: if(s) nst = start;
        start: if(a)
                  if(r&~b&~g) nst = red1;
                  else nst = waite;
        red1: if(a)
                  if(b&~r&~g) nst = blue;
                  else nst = waite;
        blue: if(a)
                  if(g&~r&~b) nst = green;
                  else nst = waite;
        green: if(a)
                   if(r&~g&~b) nst = red2;
                   else nst = waite;
        red2:  nst = waite;

        default: nst = waite;
    endcase
end
endmodule
```

Logic to drive output

State register

Logic to determine next state

FSM Structure in Verilog

- So the generalized FSM structure looks like this:

```
module seqdet (input /* FSM inputs */
               input clk, rst,
               output /* FSM outputs */);

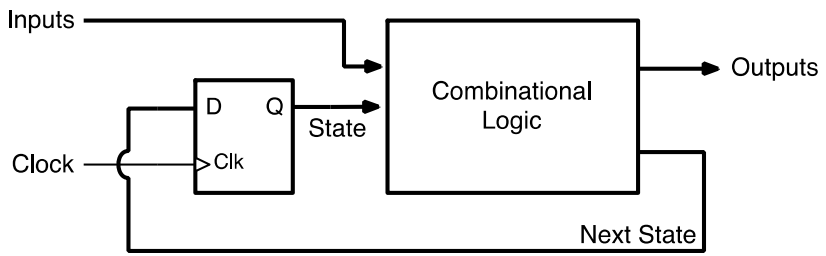
parameter st1 = … /* state names and assignments */

reg [2:0] nst, st;  /* next and current state signal
*/

/* state register always block
   always the same structure
   st <= nst, with a reset to intial */

/* state transition always block
   generally uses a case statement
   assigning to nst based on st and inputs */

/* output assignment can be separate using
   assign or always block, or combined in
   state transition always block */
endmodule
```



Notes on Implementing FSMs

NANYANG TECHNOLOGICAL UNIVERSITY

- It is possible to write a whole FSM in a single synchronous always block
  - but that is harder to verify
- Hence, always separate the state register: a simple block that assigns next state to current state at each clock edge
- Ensure you reset the state register to a valid state
- If the output and transitions can be combined cleanly, you can do that
- Otherwise, assign the output separately

Notes on Implementing FSMs

NANYANG TECHNOLOGICAL UNIVERSITY

- Use a default next state (and output) assignment at the top of the state transition block to minimize the number of statements – only other cases needed
- Always ensure next state is assigned in every case
- Add a default case that transitions other invalid states to the initial state

```
always @ *
begin
    nst = st;
    case(st)
        waite: if(s) nst = start;
        start: if(a)
                  if(r&~b&~g) nst = red1;
                  else nst = waite;
        red1: if(a)
                  if(b&~r&~g) nst = blue;
                  else nst = waite;
        blue: if(a)
                  if(g&~r&~b) nst = green;
                  else nst = waite;
        green: if(a)
                   if(r&~g&~b) nst = red2;
                   else nst = waite;
        red2: nst = waite;

        default: nst = waite;
    endcase
end
```

- Using a combinational always block, it becomes easy to test that it works, since we always know what outputs we should get, based on the state transition diagram

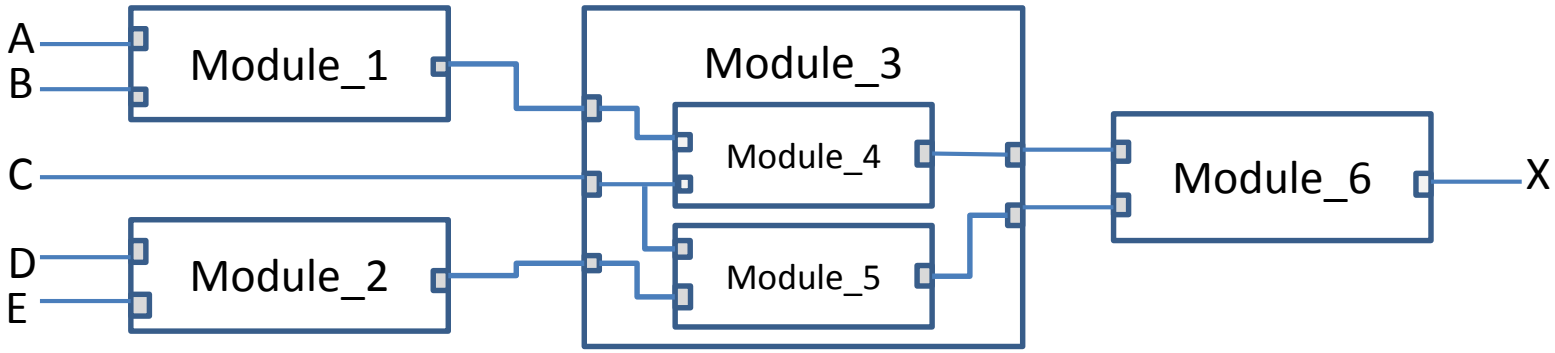
NANYANG TECHNOLOGICAL UNIVERSITY

CX1005
Digital Logic

Verilog Recap

Verilog Hardware Description Language

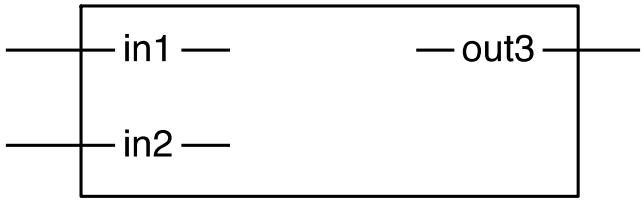
- Language with special constructs for describing hardware
- The most important rule when thinking about Verilog design, is to think *in hardware*
- In Verilog, designs are broken down into ***modules***
- Modules can contain code to describe hardware and also instances of other modules



Ports, Wires, Reg

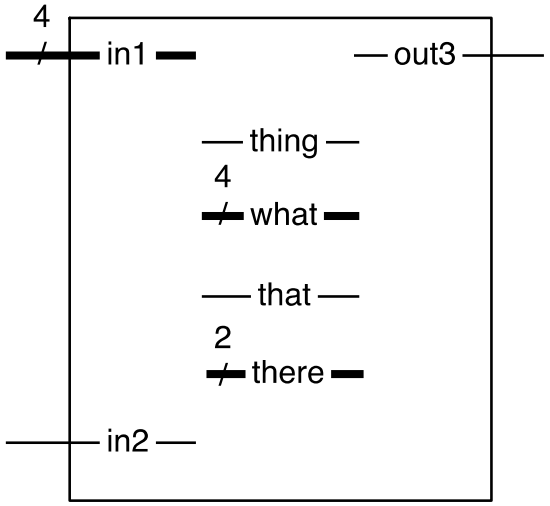
- Ports act exactly like wires:
  - Can "read" the value of an input or output port
  - But can only "assign"/"write" to an output port

```
module simple (input in1, in2,
               output out3);
endmodule
```



- Regs are exactly the same as wires, but can only be assigned to from within always blocks
  - Can be "read" anywhere

```
module simple (input [3:0] in1,
               input in2,
               output out3);
    wire thing;
    wire [3:0] what;
    reg that;
    reg [1:0] there;
endmodule
```



Ports, Wires, Regs

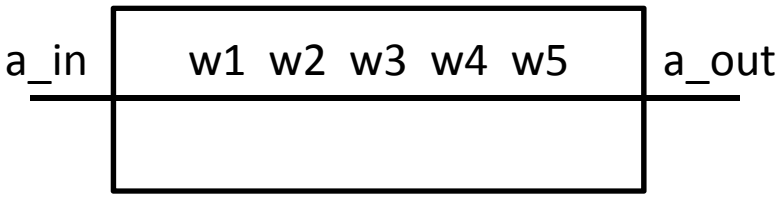
- Internal wires are just anchor points to which you can connect logic:
- What will this do?

```
module nothing (input a_in,
                output a_out);

wire w1, w2, w3, w4, w5;

assign w1 = a_in;
assign w2 = w1;
assign w3 = w2;
assign w4 = w3;
assign w5 = w4;
assign a_out = w5;

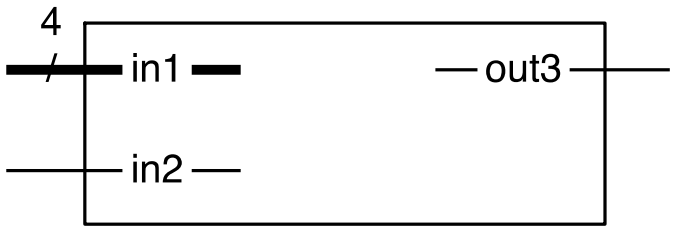
endmodule
```



Ports, Wires, Regs

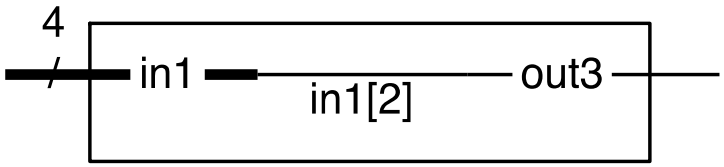
- Any port, wire, or reg can be declared multi-bit by preceding its name with an index range:

```
module simple (input [3:0] in1,
               input in2,
               output out3);
endmodule
```



- We can slice sections or single bits of any port, wire, or reg and use them as we do for single wires:

```
module simple (input [3:0] in1,
               output out3);
    assign out3 = in1[2];
endmodule
```



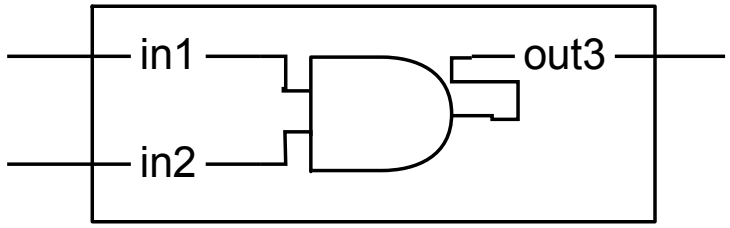
Instantiating Gate-Level Primitives

- We can connect ports/wires/reg by instantiating primitives or modules:

```
module simple (input in1, in2,
               output out3);

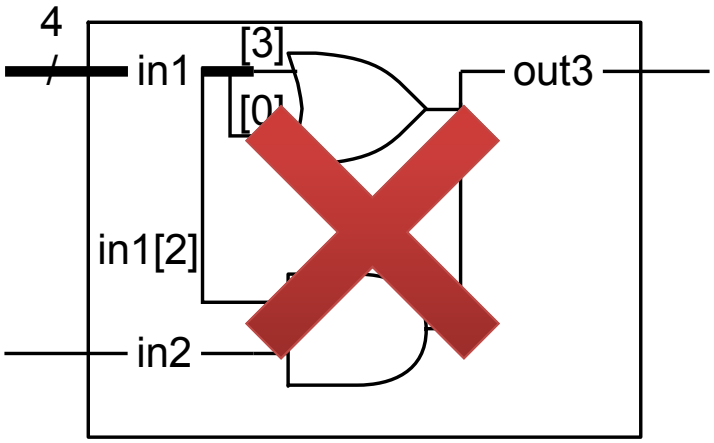
   and a1 (out3, in1, in2);

endmodule
```



- You never want to assign, by any means, to a wire, port, or reg multiple times:

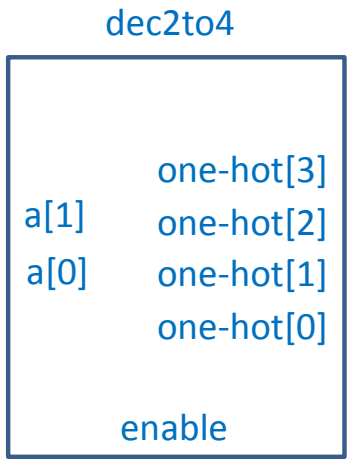
```
module simple (input [3:0] in1,
               input in2,
               output out3);
   and (out3, in1[2], in2);
   or  (out3, in1[3], in1[0]);
endmodule
```

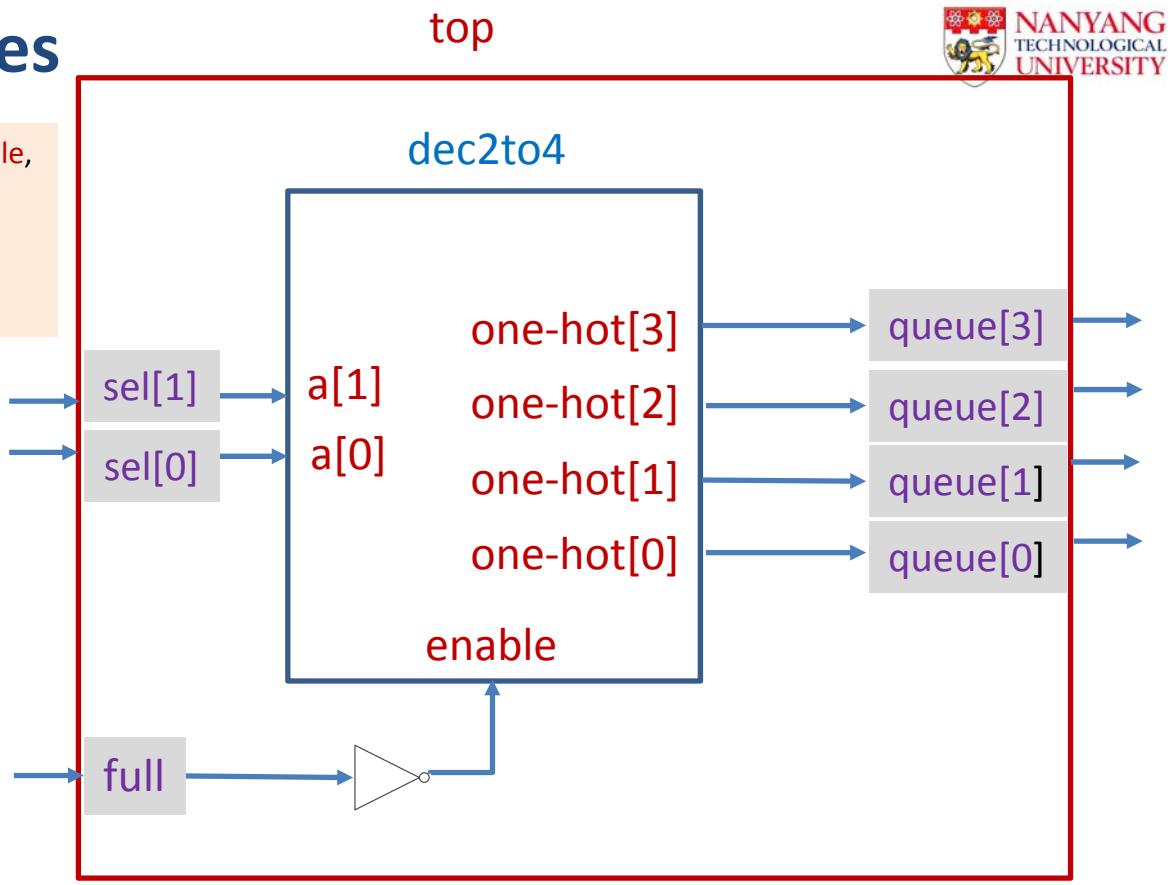


Instantiating Modules

```
module dec2to4 (input [1:0] a,  input enable,
                output [3:0] one-hot);

...
endmodule
```






```
module top (input [1:0]  sel,
            input        full,
            output [3:0] queue);

    dec2to4 (.a(sel), .enable(~full), .one-hot(queue));

endmodule
```

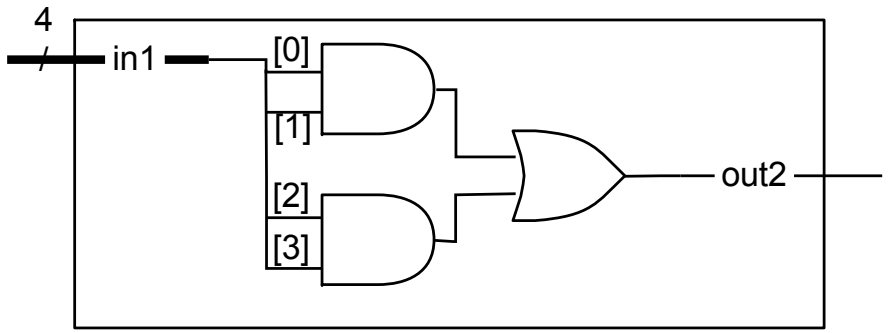
Assign Statements

- These are the same:

```
module simple (input [3:0] in1,
               output out2);

  and (w1, in1[0], in1[1]);
  and (w2, in1[2], in1[3]);
  or  (out2, w1, w2);

endmodule
```




```
module simple (input [3:0] in1,
               output out2);

  assign out2 = ((in1[0] & in1[1]) |
                 (in1[2] & in1[3]));

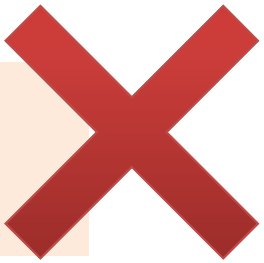
endmodule
```

Assign Statements

- Assign statements let us assign the result of an expression to a signal
- Allow us to use many more operators
  - Bitwise or Logical
  - Comparisons and equality
  - Arithmetic
- This is always preferred to instantiating gates or simple low-level modules
- Remember, you cannot assign to regs!
  - Reg can only be assigned within an always block

```
reg temp;

assign temp = 1'b0;
```



~	(bitwise NOT)
&	(bitwise AND)
|	(bitwise OR)
^	(bitwise XOR)
~^ or ^~	(bitwise XNOR)

!	(logical NOT)
&&	(logical AND)
||	(logical OR)

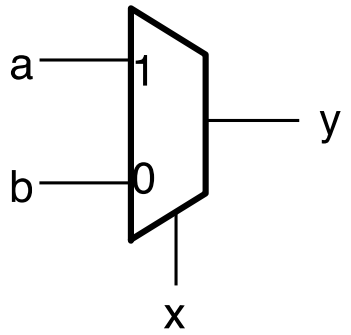
<	(less than)
<=	(less than or equal to)
>	(greater than)
>=	(greater than or equal to)
==	(equal to)
!=	(not equal to)

+	(addition)
-	(subtraction)
*	(multiplication)
/	(division)
%	(modulus)

Conditional Assignment

- One other trick with the assign statement is conditional assignment
- Creates a multiplexer, like an if/else statement:

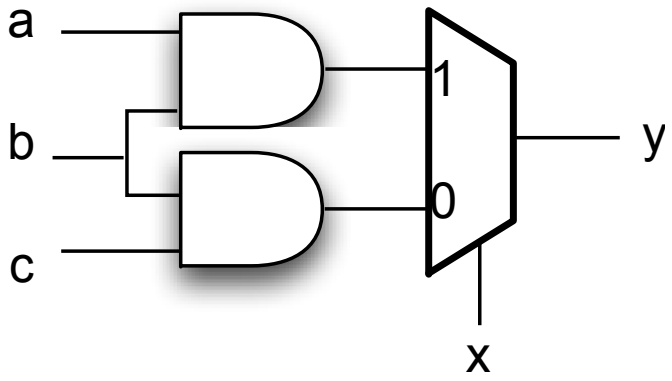
```
assign y = x ? a : b;
```





- Any expression in the body of the conditional assignment is also generated as hardware

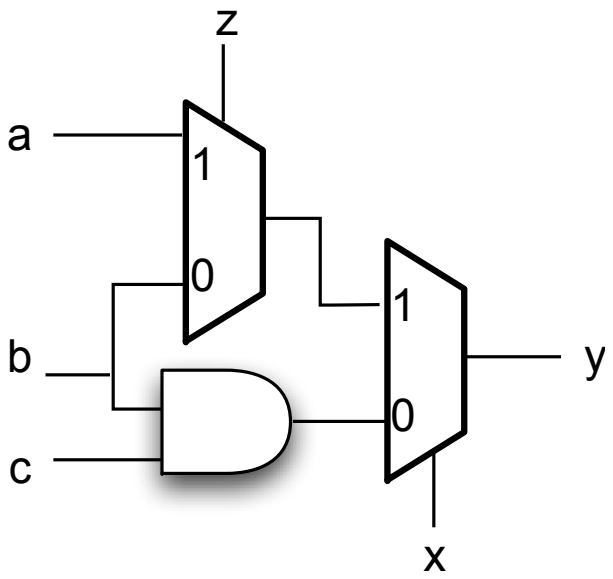
```
assign y = x ? a & b : b & c;
```



Conditional Assignment

- Those expressions can themselves be conditional assignments (Brackets make this easier to read)

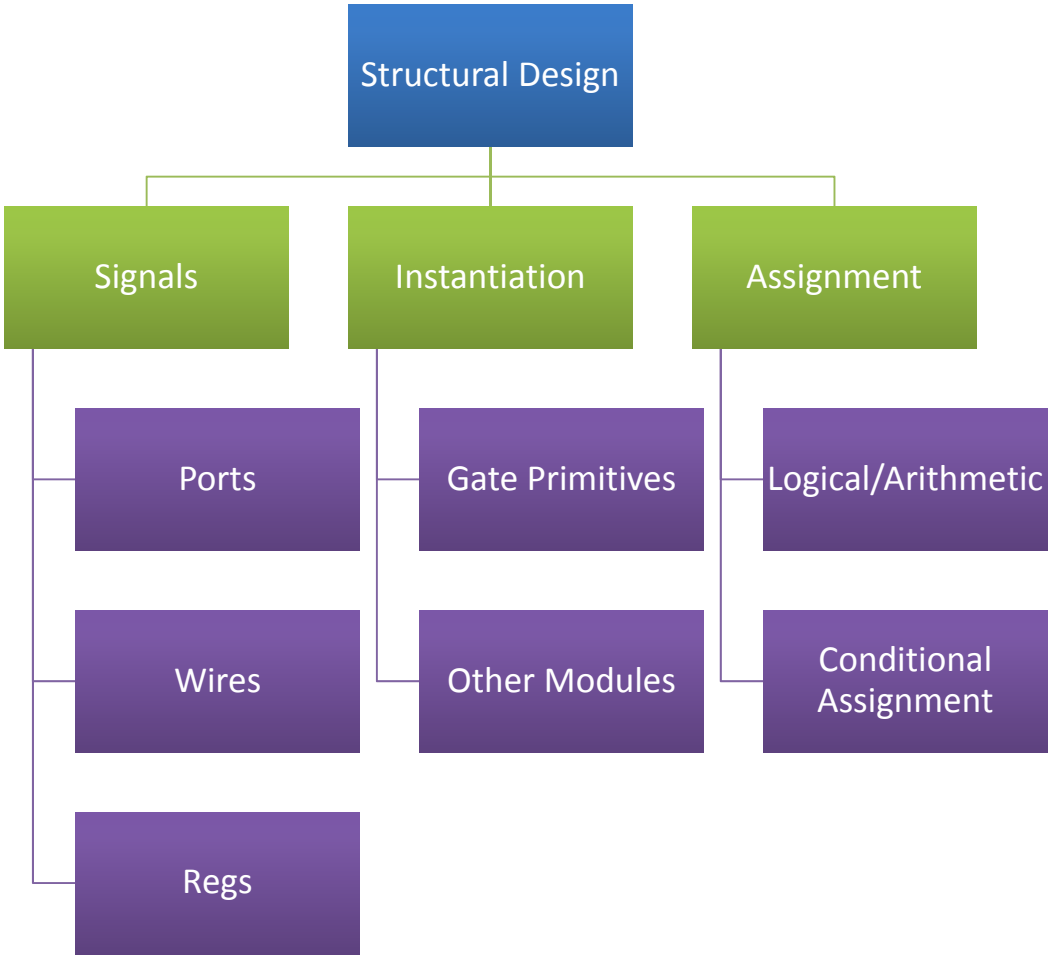
```
assign y = x ? (z ? a : b):
               b & c;
```



- Everything in an assign statement gets turned into a mini combinational circuit
- The signal being assigned to is the output of this mini circuit
- The tools take care of how best to design the circuit

- Conditional assignment simply adds a multiplexer to select which expression to finally route to the assigned signal

Overview of Structural Design



Behavioral Design

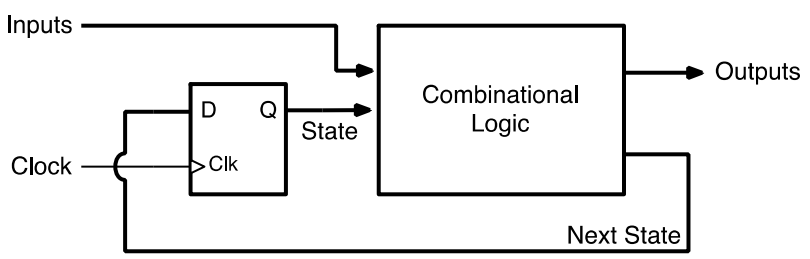
- In behavioral design, we raise the level of our description and rely on the synthesis tools to do more
- We use the always block to describe circuits behaviorally
- We can design both combinational and sequential circuits using always blocks
- Always block is effectively a super-mega assign statement
- There is no other way to build synchronous blocks

```
always @ *
begin
   a = … ;
   b = … ;
end

assign c = … ;

always @ (posedge clk)
begin
   y <= … ;
   x <= … ;
end

assign z = … ;
```



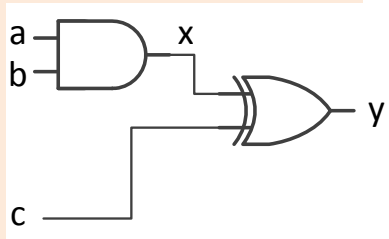
Combinational Always Blocks

- Has a list of signals in its sensitivity list, or better: *
- Can assign to one or more signals
- Must only assign to reg signals
- We use a blocking assignment (=)
- **Statement order** within the always block does **matter**
- Each assignment results in a combinational circuit
- If and case statements are very useful
- All signals to which it assigns must be assigned in all possible cases

```
module gate2 (input a, b, c,
              output reg y);
   reg x;

   always @*
   begin
     x = a & b;
     y = x ^ c;
   end

endmodule
```

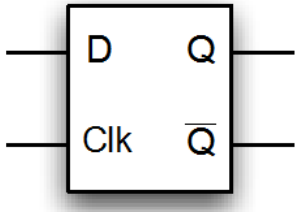



```
module mux4 (output reg q,
             input [3:0] d,
             input [1:0] sel);
   always @* begin
      case (sel)
         2'b00 : q = d[0];
         2'b01 : q = d[1];
         2'b10 : q = d[2];
         2'b11 : q = d[3];
      endcase
   end
endmodule
```

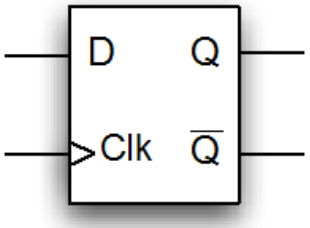
Sequential Circuits

- Sequential circuits are any circuits that have the concept of state: include latches, flip-flops, etc.
- Latches are level-sensitive, i.e., their outputs change when the control (enable) input is high
- A flip-flop/register is edge-sensitive, i.e. their outputs change at the rising/falling edge of the control (enable)
- Modern design is almost always synchronous (one shared clock)

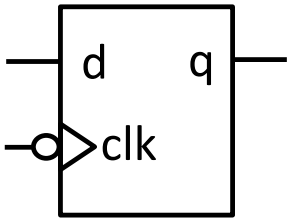
D Latch

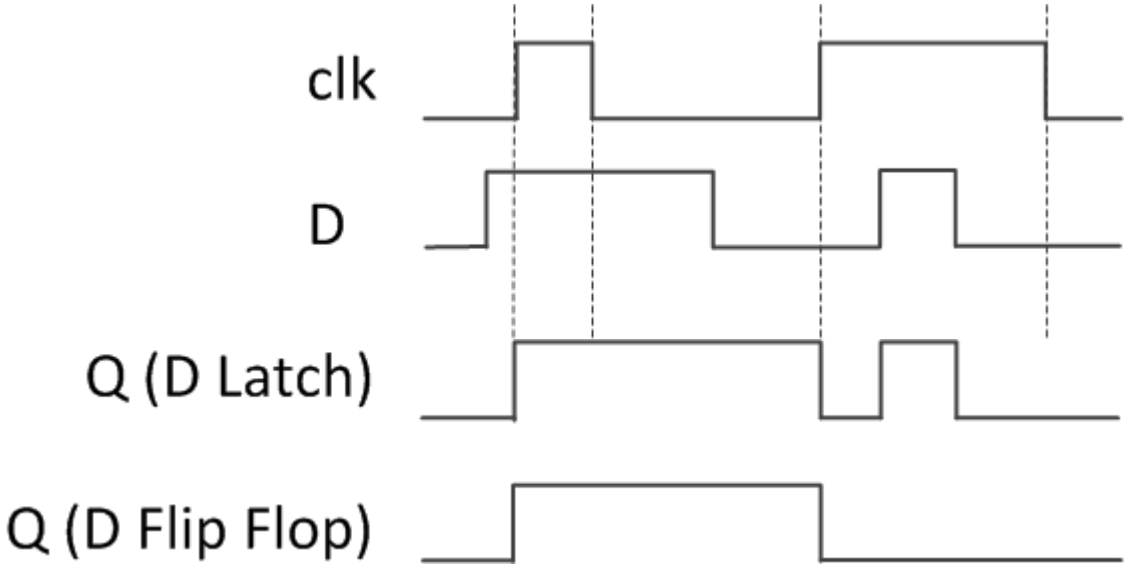


Positive edge triggered Flip-Flop



Negative edge triggered Flip-Flop



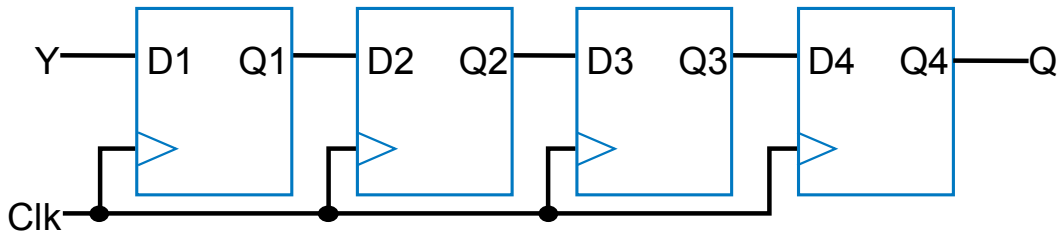


Synchronous Always Block

- Use ***posedge*** *clk* in its sensitivity list
- Can assign to one or more signals
- Must only assign to reg signals
- Each assigned signal becomes a register connected to any logic given in the expressions
- We use non-blocking assignment (<=)
- **Statement order does not matter** with non-blocking assignments
- We should ensure all registers respond to a reset

```
reg q1, q2, q3, q;

always@(posedge clk)
begin
  if (rst) begin
    q1 <= 1'b0;
    q2 <= 1'b0;
    q3 <= 1'b0;
    q  <= 1'b0;
  end
  else begin
    q1 <= y;
    q2 <= q1;
    q3 <= q2;
    q  <= q3;
  end
end
```

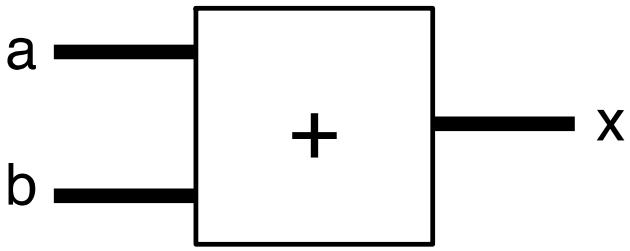


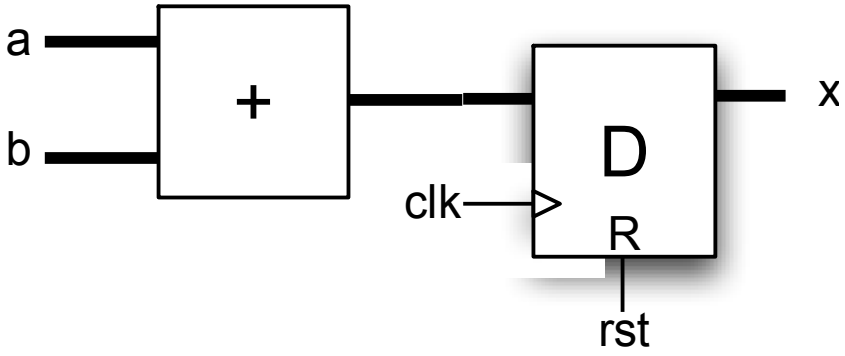
Combinational and Synchronous

- Identical statements produce the same circuit, but with a synchronous always, we get a register on the end:

```
always @ *
begin
    x = a + b;
end
```

```
always @ (posedge clk)
begin
    if (rst)
        x <= 6'b000000;
    else
        x <= a + b;
end
```

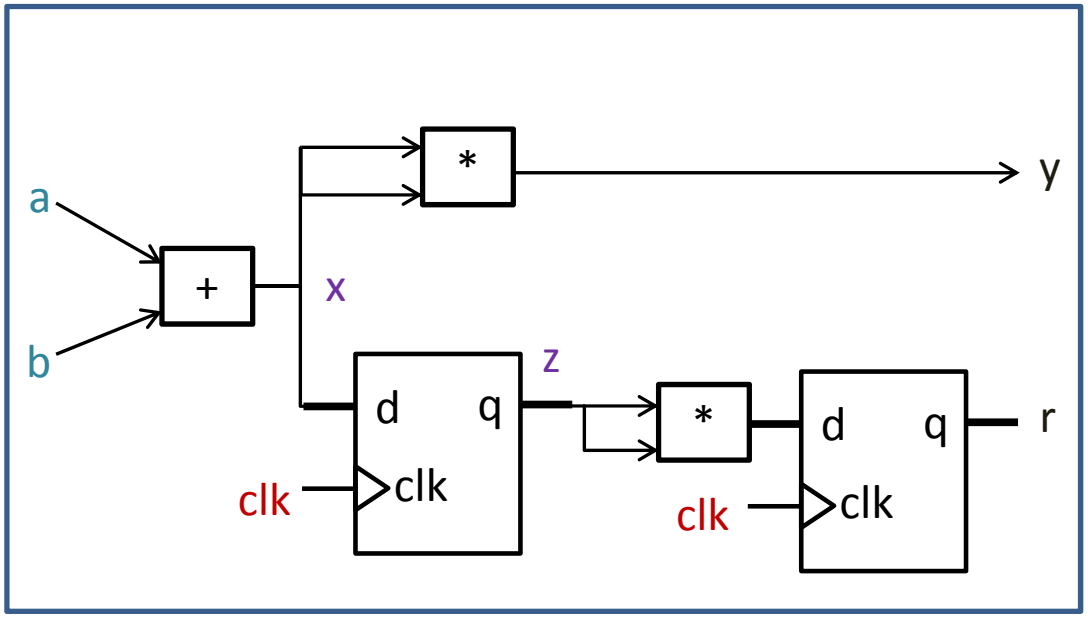


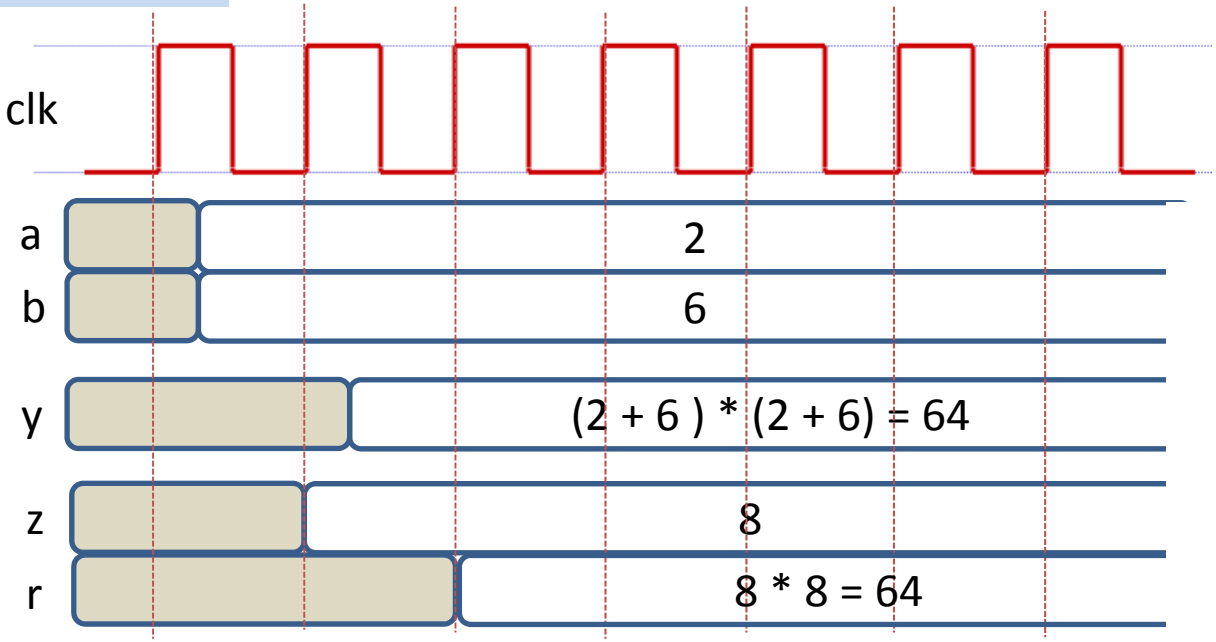


Combinational and Synchronous

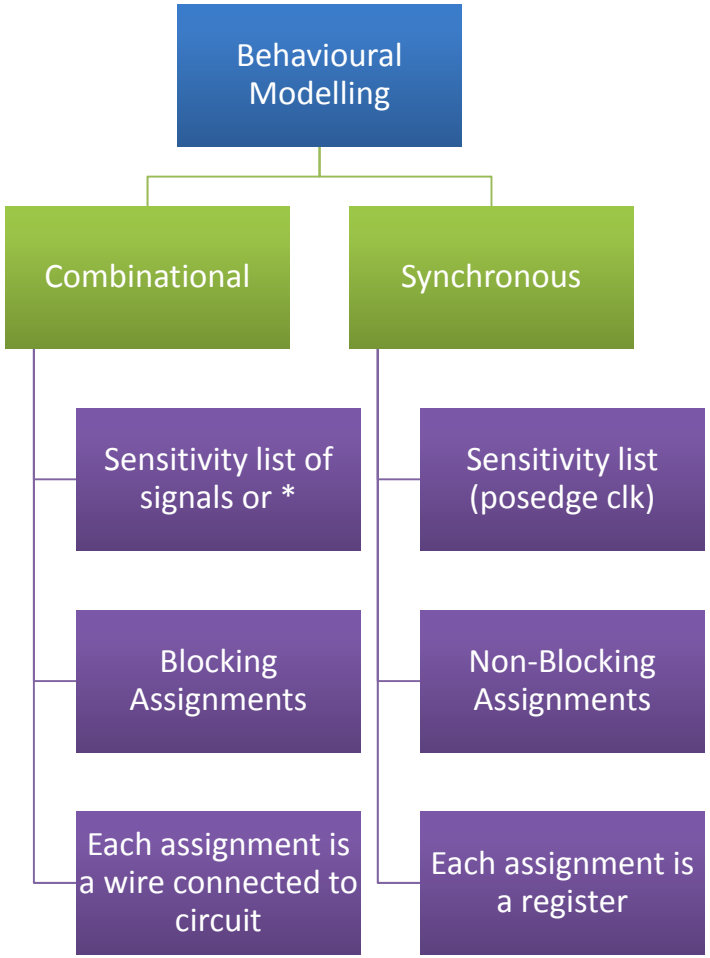
```
always @ *
begin
    x = a + b;
    y = x * x;
end

always@(posedge clk)
begin
    z <= x;
    r <= z * z;
end
```



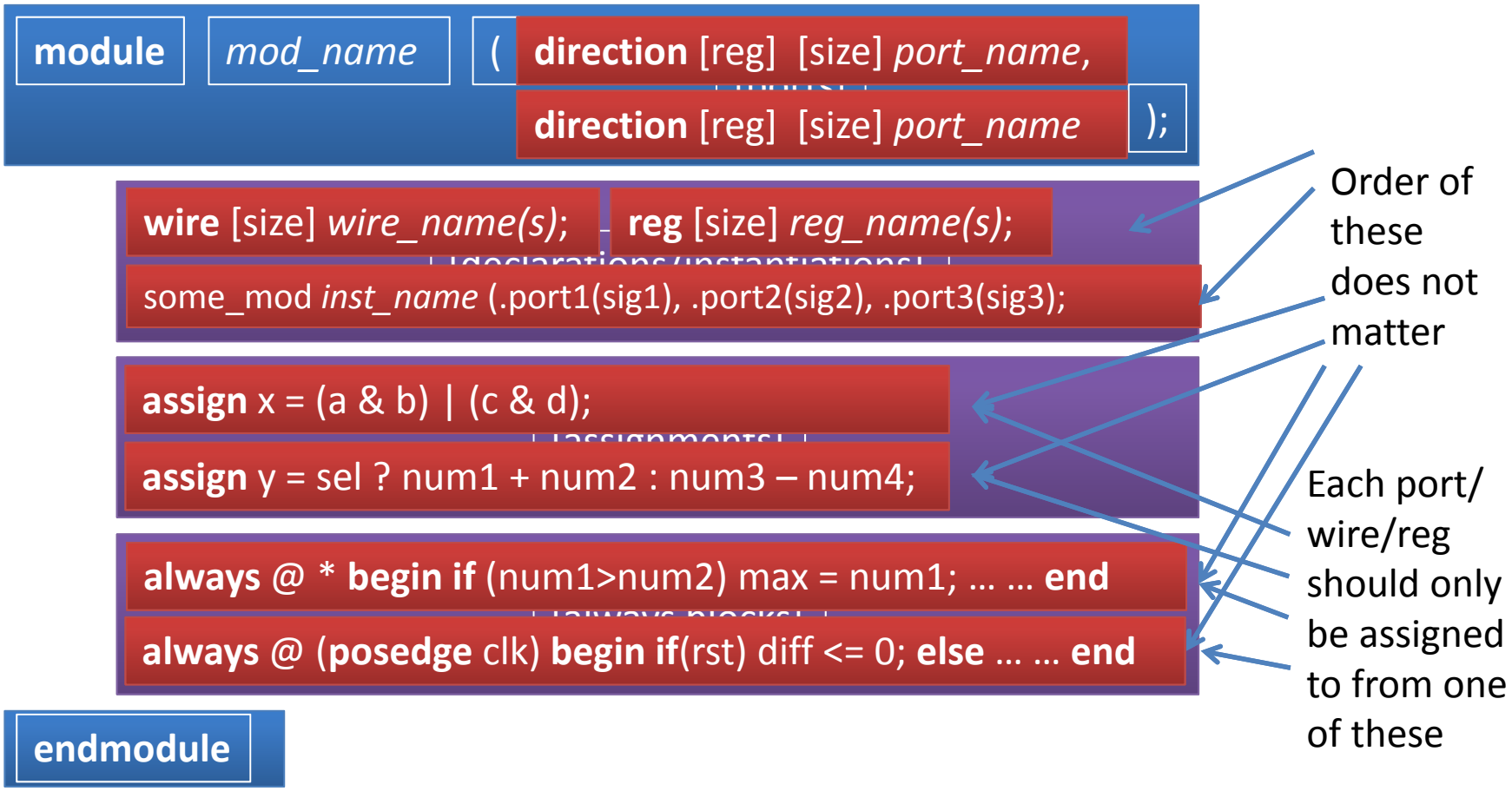


Behavioral Verilog



Code structure

- This general structure for a module should help:



Verilog Summary

NANYANG TECHNOLOGICAL UNIVERSITY

- Think in terms of hardware blocks, not instructions
- Think of block descriptions, not programs
- If you can't understand what the circuit will look like, likely it won't work
- There are many ways to achieve the same thing
  - Go for simplicity
  - Split things up when they get unwieldy
- We have not covered everything:
  - Simulation and verification
  - Non-synthesizable constructs